

Universitatea POLITEHNICA din București  
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Arhitectură Agentică pentru Asistență Inteligentă în  
Dezvoltarea Software**

**Lucrare de licență**

**Prezentată ca cerință parțială pentru obținerea  
titlului de *Inginer*  
în domeniul *Calculatoare și Tehnologia Informației*  
programul de studii *Ingineria Informației***

**Conducător științific**  
Ș.L. dr. ing. Mihai DOGARIU

**Absolvent**  
Eduard-Daniel NĂNESCU

**Anul 2026**



**TEMA PROIECTULUI DE DIPLOMĂ**  
a studentului **Eduard-Daniel NĂNESCU, 441A**

**1. Titlul temei:** Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software

**2. Descrierea temei:**

Contribuția originală a lucrării va consta în dezvoltarea unei arhitecturi software bazată pe Modele Mari de Limbaj (LLMs - Large Language Models), capabilă să asiste utilizatorul în scrierea, depanarea, documentarea și explicarea codului sursă. Aplicațiile vizate sunt eficientizarea procesului de dezvoltare software și automatizarea sarcinilor repetitive întâlnite în programare. Sistemul propus nu vizează antrenarea de la zero a modelelor, ci implementarea unei soluții agentice care utilizează modele pre-antrenate pentru a executa acțiuni complexe. Cercetarea aferentă va include: sinteza bibliografică a realizărilor actuale din domeniu, utilizarea unui mecanism care permite modelului să utilizeze instrumente pentru a interacționa cu mediul virtual, implementarea unor tehnici de inginerie a prompturilor și gestionarea contextului pentru a asigura relevanța răspunsurilor, validarea experimentală a prototipului dezvoltat prin scenarii de utilizare reale utilizând medii de dezvoltare precum PyCharm [1]/VS Code [2] și biblioteci specifice precum LangChain [3], LangGraph [4] sau GoogleADK [5], concluzionarea rezultatelor obținute și a contribuției originale, cât și enunțarea perspectivelor ulterioare de cercetare.

**3. Contribuția originală:**

Proiectarea și implementarea arhitecturii agentului, incluzând logica de selecție și utilizare a instrumentelor, gestionarea contextului și dezvoltarea unei interfețe sau integrarea cu interfețe existente care să permită interacțiunea facilă cu asistentul.

**4. Resurse și bibliografie:**

- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." In The eleventh international conference on learning representations. 2022.
- Bozkurt, Aras, and Ramesh C. Sharma. "Generative AI and prompt engineering: The art of whispering to let the genie out of the algorithmic world." Asian Journal of Distance Education 18, no. 2 (2023): i-vii.
- Alto, Valentina. Building LLM Powered Applications: Create intelligent apps and agents with large language models. Packt Publishing Ltd, 2024.
- Zhu, He, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao et al. "Oa-gents: An empirical study of building effective agents." arXiv preprint arXiv:2506.15741 (2025).

**5. Data înregistrării temei:** 2025-12-11 22:54:00

**Conducător(i) lucrare,**  
Ș.L. dr. ing. Mihai DOGARIU

**Student,**  
Eduard-Daniel NĂNESCU

**Director departament,**  
Conf. dr. ing Bogdan FLOREA

**Decan,**  
Prof. dr. ing. Mihnea UDREA

Cod Validare: **621d9dfde3**





## Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Eduard-Daniel NĂNESCU

.....

# Cuprins

|                                                                                                                                   |           |
|-----------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>LISTA FIGURILOR</b>                                                                                                            | <b>9</b>  |
| <b>LISTA TABELELOR</b>                                                                                                            | <b>11</b> |
| <b>LISTA DE ACRONIME</b>                                                                                                          | <b>12</b> |
| <b>INTRODUCERE</b>                                                                                                                | <b>13</b> |
| CONTEXT ȘI MOTIVAȚIE                                                                                                              | 13        |
| SCOPUL ȘI OBIECTIVELE LUCRĂRII                                                                                                    | 13        |
| METODOLOGIE                                                                                                                       | 14        |
| CONTRIBUȚII PROPRII ȘI REZULTATE                                                                                                  | 14        |
| STRUCTURA LUCRĂRII                                                                                                                | 15        |
| <b>1. CONTEXT, POZIȚIONARE ȘI DECIZII ARHITECTURALE</b>                                                                           | <b>16</b> |
| 1.1. Evoluția către modelele mari de limbaj (Large Language Model (LLM))                                                          | 16        |
| 1.2. Maturizarea modelelor mari de limbaj și cercetări actuale                                                                    | 17        |
| 1.3. Fundamentele arhitecturilor agentice și cercetări actuale                                                                    | 19        |
| 1.4. Limitări ale arhitecturilor agentice                                                                                         | 22        |
| 1.5. Analiză preliminară a peisajului actual în domeniul inteligenței artificiale concentrată pe aplicații de dezvoltare software | 23        |
| 1.6. Plasarea în industrie și oportunități identificate                                                                           | 24        |
| 1.6.1. Analiza ecosistemelor proprietare                                                                                          | 24        |
| 1.6.2. Analiza soluțiilor open-source                                                                                             | 25        |
| 1.6.3. Cerințe pentru o soluție agentică în dezvoltarea software                                                                  | 26        |
| 1.6.4. Nevoia de personalizare                                                                                                    | 27        |
| 1.6.5. Lipsa de observabilitate                                                                                                   | 27        |
| 1.6.6. Securitate și confidențialitate                                                                                            | 27        |
| 1.6.7. Costul de utilizare                                                                                                        | 27        |
| 1.7. Analiză proprie a necesităților și decizii arhitecturale                                                                     | 27        |
| 1.7.1. Adoptarea unei arhitecturi modulare                                                                                        | 28        |
| 1.7.2. Alegerea limbajului de programare și a cadrului de lucru agentic                                                           | 28        |
| 1.7.3. Adoptarea motorului pentru interfața cu utilizatorul                                                                       | 28        |
| 1.7.4. Tehnici proprii de gestionare a contextului și a apelurilor la unelte                                                      | 30        |
| 1.7.5. Suportul pentru date multi-modale                                                                                          | 30        |
| 1.7.6. Observabilitate în două straturi                                                                                           | 30        |
| 1.7.7. Cercetarea tehnicilor de utilizare optimă pentru arhitecturi locale în dezvoltarea software                                | 30        |
| 1.7.8. Sinteza recomandărilor extrase din experimente                                                                             | 31        |
| <b>2. IMPLEMENTARE</b>                                                                                                            | <b>32</b> |
| 2.1. Mediul de dezvoltare                                                                                                         | 32        |
| 2.2. Structura aplicației                                                                                                         | 32        |
| 2.3. Prezentarea procesului de configurare                                                                                        | 34        |
| 2.4. Interfața cu utilizatorul                                                                                                    | 34        |

|           |                                                                                                                                               |           |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 2.5.      | Funcționalități de nivel înalt . . . . .                                                                                                      | 36        |
| 2.6.      | Instalare și utilizare . . . . .                                                                                                              | 37        |
| 2.6.1.    | Cerințe preliminare . . . . .                                                                                                                 | 37        |
| 2.6.2.    | Instrucțiuni de instalare . . . . .                                                                                                           | 37        |
| 2.6.3.    | Configurare la prima rulare . . . . .                                                                                                         | 37        |
| 2.6.4.    | Utilizarea aplicației . . . . .                                                                                                               | 37        |
| 2.6.5.    | Modul neinteractiv . . . . .                                                                                                                  | 38        |
| 2.7.      | Gestionarea contextului și a memoriei . . . . .                                                                                               | 39        |
| 2.7.1.    | Memoria și istoricul conversației . . . . .                                                                                                   | 39        |
| 2.7.2.    | Contextul . . . . .                                                                                                                           | 39        |
| 2.7.3.    | Retrieval Augmented Generation (RAG) . . . . .                                                                                                | 40        |
| 2.7.4.    | Gestionarea datelor multi-modale . . . . .                                                                                                    | 40        |
| 2.7.5.    | Compactarea . . . . .                                                                                                                         | 42        |
| 2.7.6.    | Tehnici pentru eficientizarea ferestrei de context . . . . .                                                                                  | 43        |
| 2.8.      | Implementarea componentelor arhitecturilor agentice moderne . . . . .                                                                         | 43        |
| 2.8.1.    | Unelte . . . . .                                                                                                                              | 43        |
| 2.8.2.    | Modul de planificare . . . . .                                                                                                                | 46        |
| 2.8.3.    | Capabilitățile predefinite . . . . .                                                                                                          | 46        |
| 2.8.4.    | Subagenții . . . . .                                                                                                                          | 47        |
| 2.9.      | Extensibilitatea . . . . .                                                                                                                    | 49        |
| 2.9.1.    | Model Context Protocol (MCP) și suplimentarea uneltelor . . . . .                                                                             | 49        |
| 2.9.2.    | Extinderea echipei de subagenți . . . . .                                                                                                     | 49        |
| 2.9.3.    | Dezvoltarea capabilităților și personalizare . . . . .                                                                                        | 50        |
| 2.9.4.    | Extinderea tehnicilor de compactare . . . . .                                                                                                 | 50        |
| 2.10.     | Securitatea în mediul agentic . . . . .                                                                                                       | 51        |
| 2.10.1.   | Medii securizate de execuție . . . . .                                                                                                        | 51        |
| 2.10.2.   | Permisiuni pentru aprobarea umană a acțiunilor . . . . .                                                                                      | 52        |
| 2.10.3.   | Filtre, gestionarea intențiilor periculoase și cereri în afara scopului (Out of Scope (OOS)) . . . . .                                        | 52        |
| 2.11.     | Observabilitatea . . . . .                                                                                                                    | 52        |
| 2.11.1.   | Urmărirea execuției prin Langfuse . . . . .                                                                                                   | 52        |
| 2.11.2.   | Panoul web proprietar . . . . .                                                                                                               | 53        |
| <b>3.</b> | <b>EVALUAREA SOLUȚIEI ȘI REZULTATE EXPERIMENTALE . . . . .</b>                                                                                | <b>57</b> |
| 3.1.      | Poziționarea finală față de concurență . . . . .                                                                                              | 57        |
| 3.1.1.    | Funcționalități prezente în concordanță cu aplicațiile concurente proprietare și Open Source Software (OSS) . . . . .                         | 59        |
| 3.1.2.    | Funcționalități absente față de aplicațiile concurente . . . . .                                                                              | 60        |
| 3.2.      | Influența lungimii și a exactității instrucțiunilor asupra performanței. De ce un prompt prea exact nu garantează rezultate mai bune. . . . . | 60        |
| 3.3.      | Contextul în mediul agentic. Consecințele metodelor de compactare. . . . .                                                                    | 63        |
| 3.3.1.    | Metode care elimină date din context . . . . .                                                                                                | 64        |
| 3.3.2.    | Metode care parafrazează contextul . . . . .                                                                                                  | 64        |
| 3.3.3.    | Metode auxiliare . . . . .                                                                                                                    | 64        |
| 3.3.4.    | Secvență recomandată de metode de compactare . . . . .                                                                                        | 64        |
| 3.4.      | Modele optime pentru sarcini de programare. Influența datelor de antrenare și a mărimii modelului. . . . .                                    | 65        |
| 3.4.1.    | Rezultate . . . . .                                                                                                                           | 66        |
| 3.4.2.    | Analiza rezultatelor . . . . .                                                                                                                | 66        |
| 3.4.3.    | Modelele open-source: evaluarea valorii și a potențialului de utilizare . . . . .                                                             | 69        |

|                                                                                                              |           |
|--------------------------------------------------------------------------------------------------------------|-----------|
| <b>4. DEZVOLTĂRI VIITOARE</b>                                                                                | <b>70</b> |
| 4.1. Automatizări, integrări cu alte programe                                                                | 70        |
| 4.2. Integrarea cu mediile de dezvoltare                                                                     | 70        |
| 4.3. Construirea unui graf de cunoștințe pentru optimizarea înțelegerii proiectelor                          | 70        |
| 4.4. Suport pentru metode de compactare și sandboxing avansate. Software Development Kit (SDK) pentru agenți | 71        |
| <b>CONCLUZII</b>                                                                                             | <b>72</b> |
| <b>BIBLIOGRAFIE</b>                                                                                          | <b>73</b> |
| <b>ANEXE</b>                                                                                                 | <b>78</b> |
| Anexa 1: config/config.yml                                                                                   | 78        |
| Anexa 2: config/agent_bench.yml                                                                              | 78        |
| Anexa 3: src/artifacts.py                                                                                    | 80        |
| Anexa 4: src/attachments.py                                                                                  | 81        |
| Anexa 5: src/capabilities.py                                                                                 | 83        |
| Anexa 6: src/clipboard.py                                                                                    | 83        |
| Anexa 7: src/compaction/base.py                                                                              | 84        |
| Anexa 8: src/compaction/stages/bulk.py                                                                       | 84        |
| Anexa 9: src/compaction/stages/ref_substitute.py                                                             | 85        |
| Anexa 10: src/compaction/stages/tool_truncate.py                                                             | 85        |
| Anexa 11: src/main.py                                                                                        | 85        |
| Anexa 12: src/multimodal.py                                                                                  | 88        |
| Anexa 13: src/permissions.py                                                                                 | 89        |
| Anexa 14: src/providers.py                                                                                   | 89        |
| Anexa 15: src/sandbox.py                                                                                     | 91        |
| Anexa 16: src/subagents/base.py                                                                              | 92        |
| Anexa 18: src/subagents/types/web_research.py                                                                | 95        |
| Anexa 19: src/tool_guards.py                                                                                 | 95        |
| Anexa 20: src/tools.py                                                                                       | 96        |
| Anexa 21: src/tracing.py                                                                                     | 99        |
| Anexa 22: ui/skills.py                                                                                       | 100       |
| Anexa 23: ui/vibe_md.py                                                                                      | 102       |

# LISTA FIGURILOR

|                                                                                                                                                                                                                         |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1. Arhitectura transformer [6] . . . . .                                                                                                                                                                              | 16 |
| 1.2. Evoluția recentă a modelelor mari de limbaj [7] . . . . .                                                                                                                                                          | 17 |
| 1.3. Performanța arhitecturii Longformer: în timp ce atenția standard scalează exponențial, Longformer scalează liniar cu lungimea contextului [8] . . . . .                                                            | 17 |
| 1.4. Diferite configurații de atenție folosite în arhitectura Longformer în comparație cu mecanismul obișnuit de atenție [8] . . . . .                                                                                  | 17 |
| 1.5. Fine-tuning-ul se realizează doar pe A și B [9] . . . . .                                                                                                                                                          | 18 |
| 1.6. Rezultatele procesului de Simple Self Distillation [10] . . . . .                                                                                                                                                  | 18 |
| 1.7. Ilustrarea procedurii de Reinforcement Learning (RL) [11] . . . . .                                                                                                                                                | 19 |
| 1.8. Exemplu de Chain-of-Thought (CoT) [12] . . . . .                                                                                                                                                                   | 19 |
| 1.9. Paradigma ReAct [13] . . . . .                                                                                                                                                                                     | 20 |
| 1.10. Îmbunătățirile pe care le aduce ReAct [13] . . . . .                                                                                                                                                              | 20 |
| 1.11. Îmbunătățirea adusă de ReWOO: agentul are mai multă libertate, putând apela mai multe unelte până să producă un răspuns. P = Plan, E = Evidence, A = Answer, O = Observation, A = Act, T = Thought [14] . . . . . | 21 |
| 1.12. Modelele tind să aibă o acuratețe foarte slabă contra întrebărilor ce vizează informații care se află la mijlocul ferestrei curente de context [15] . . . . .                                                     | 21 |
| 1.13. Vizualizarea problemei și al rezultatului dorit la întrebare. De multe ori, modelele întâmpină dificultăți în accesarea acestor informații ascunse în mijlocul ferestrei de context [15] . . . . .                | 22 |
| 1.14. Reprezentarea ferestrei de context într-o conversație uzuală (Sursă: [16]) . . . . .                                                                                                                              | 22 |
| 1.15. Clasamentul modelelor în SWE-Bench Pro [17] la momentul actual în funcție de performanța rezolvării sarcinilor complexe în limba engleză. . . . .                                                                 | 24 |
| 1.16. Clasamentul după cost al modelelor în SWE-Bench Pro [17] la momentul actual. . . . .                                                                                                                              | 24 |
| 1.17. Reprezentarea ferestrei de context într-o conversație în Claude Code . . . . .                                                                                                                                    | 27 |
| 1.18. Reprezentarea utilizării în Claude (varianta web) (Sursă: [18]) . . . . .                                                                                                                                         | 27 |
| 2.1. Arhitectura modulară a aplicației <b>vibe-cli</b> . . . . .                                                                                                                                                        | 33 |
| 2.2. Structura directoarelor vitale ale proiectului . . . . .                                                                                                                                                           | 34 |
| 2.3. Prima versiune a interfeței, realizată cu ajutorul bibliotecii Textual . . . . .                                                                                                                                   | 35 |
| 2.4. Continuarea conversației cu prima versiune a interfeței, realizată cu ajutorul bibliotecii Textual . . . . .                                                                                                       | 35 |
| 2.5. A doua versiune a interfeței, realizată fără biblioteca Textual . . . . .                                                                                                                                          | 35 |
| 2.6. Continuarea conversației cu a doua versiune a interfeței, realizată fără biblioteca Textual . . . . .                                                                                                              | 35 |
| 2.7. Exemplu de conversație în interfața vibe-cli . . . . .                                                                                                                                                             | 36 |
| 2.9. Împărțirea ferestrei de context (Sursă: [19]) . . . . .                                                                                                                                                            | 39 |
| 2.10. Fluxul de prelucrare a datelor multi-modale în <b>vibe-cli</b> . . . . .                                                                                                                                          | 41 |
| 2.11. Schema componentelor care alcătuiesc fereastra de context și compactarea la prag . . . . .                                                                                                                        | 42 |
| 2.12. Compactarea după apelul uneltei . . . . .                                                                                                                                                                         | 42 |
| 2.13. Implementarea unei funcții pe care agentul o folosește pentru a lista directoriile și fișierele la o anumită cale pe sistem . . . . .                                                                             | 44 |
| 2.14. Suita de unelte expuse agentului în <b>vibe-cli</b> și relațiile dintre acestea . . . . .                                                                                                                         | 45 |
| 2.15. Implementarea unei capabilități de revizuire de Pull Request (PR) . . . . .                                                                                                                                       | 46 |
| 2.16. Arhitectura orchestrator-subagenți din <b>vibe-cli</b> . . . . .                                                                                                                                                  | 47 |
| 2.17. Implementarea unei funcții pe care agentul o folosește pentru a porni un subagent . . . . .                                                                                                                       | 48 |

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| 2.18. Meniul de adăugare a serverelor Model Context Protocol (MCP) . . . . .                     | 49 |
| 2.19. Înregistrarea unui nou subagent . . . . .                                                  | 50 |
| 2.20. Cererea de aprobare a permisiunii de a scrie un fișier . . . . .                           | 52 |
| 2.21. Interfața Langfuse . . . . .                                                               | 53 |
| 2.22. Distribuția contextului . . . . .                                                          | 54 |
| 2.23. Optimizarea adusă ferestrei de context a agentului principal de către subagenți . . . .    | 54 |
| 2.24. Imagini cu interfața web a aplicației . . . . .                                            | 55 |
| 2.25. Detalii ale panoului web: apelarea uneltelor și reconstrucția interacțiunilor . . . . .    | 55 |
| 2.26. Interacțiunea dintre subagenți și agentul principal . . . . .                              | 56 |
| 2.27. Detalii ale panoului web: distribuția costului și detaliile procesului de compactare . . . | 56 |
| 3.1. Poziționarea <b>vibe-cli</b> față de concurență. . . . .                                    | 57 |
| 3.2. Comparatie vizuală între prompt-ul detaliat și cel simplu . . . . .                         | 63 |
| 3.3. Corelația între numărul de parametri ai modelelor locale și scor . . . . .                  | 66 |

## LISTA TABELELOR

|                                                                                                                                                                                                                |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1. Selecție de modele și prețul de utilizare în comparație cu capabilitățile. . . . .                                                                                                                        | 23 |
| 1.2. Ecosistemul de produse cu Inteligență Artificială (IA) al principalilor furnizori de modele de limbaj . . . . .                                                                                           | 25 |
| 1.3. Top 5 asistenți open-source după numărul de "stele" pe GitHub . . . . .                                                                                                                                   | 26 |
| 1.4. Comparația cadrelor de lucru pentru întrebuințări agentice . . . . .                                                                                                                                      | 28 |
| 1.5. Comparația bibliotecilor pentru interfața cu utilizatorul . . . . .                                                                                                                                       | 29 |
| 2.1. Hardware recomandat pentru modelele open-source evaluate local (Sursa [20]). . . . .                                                                                                                      | 32 |
| 2.2. Comenzi disponibile în interfață . . . . .                                                                                                                                                                | 38 |
| 3.1. Comparație între soluția dezvoltată și principalele unelte agentice de programare pentru terminal disponibile pe piață . . . . .                                                                          | 58 |
| 3.2. Comparație între soluția propusă și principalii asistenți de programare Open Source Software (OSS) din ecosistem . . . . .                                                                                | 59 |
| 3.3. Rezultatele pentru prompt-ul detaliat: metrici per proiect și pe nivele de severitate . .                                                                                                                 | 62 |
| 3.4. Rezultatele pentru prompt-ul simplu: metrici per proiect și pe nivele de severitate . .                                                                                                                   | 62 |
| 3.5. Modelele testate: tip, număr de parametri, mărime pe disc și suport multi-modal. . . .                                                                                                                    | 65 |
| 3.6. Clasamentul modelelor după scor mediu, durata totală de rulare și cost. Modelele rulate local prin ollama au cost zero. Modelele proprietare nu au date publice referitoare la mărimea modelului. . . . . | 66 |

# LISTA DE ACRONIME

|             |                                                                            |               |                                                                                                                     |
|-------------|----------------------------------------------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------|
| <b>AML</b>  | Adversarial Machine Learning (Învățare Automată Adversarială)              | <b>NLP</b>    | Natural Language Processing (Procesarea Limbajului Natural)                                                         |
| <b>API</b>  | Application Programming Interface (Interfață de Programare a Aplicațiilor) | <b>OOS</b>    | Out of Scope (În Afara Domeniului de Aplicabilitate)                                                                |
| <b>BLEU</b> | Bilingual Evaluation Understudy (Substituent de Evaluare Bilingvă)         | <b>OSS</b>    | Open Source Software (Software cu Sursă Deschisă)                                                                   |
| <b>BTT</b>  | Backpropagation Through Time (Retropropagare prin Timp)                    | <b>PDF</b>    | Portable Document Format (Format Portabil de Document)                                                              |
| <b>CD</b>   | Continuous Deployment (Implementare Continuă)                              | <b>PR</b>     | Pull Request (Cerere de Integrare)                                                                                  |
| <b>CI</b>   | Continuous Integration (Integrare Continuă)                                | <b>RAG</b>    | Retrieval Augmented Generation (Generare Augmentată prin Recuperare)                                                |
| <b>CLI</b>  | Command Line Interface (Interfață în Linie de Comandă)                     | <b>RAM</b>    | Random Access Memory (Memorie cu Acces Aleatoriu)                                                                   |
| <b>CNN</b>  | Convolutional Neural Network (Rețea Neuronală Convoluțională)              | <b>RL</b>     | Reinforcement Learning (Învățare prin Recompensă)                                                                   |
| <b>CoT</b>  | Chain-of-Thought (Lanț de Gândire)                                         | <b>RNN</b>    | Recurrent Neural Network (Rețea Neuronală Recurentă)                                                                |
| <b>CPU</b>  | Central Processing Unit (Unitate Centrală de Procesare)                    | <b>ROUGE</b>  | Recall-Oriented Understudy for Gisting Evaluation (Substituent Orientat spre Recuperare pentru Evaluarea Rezumării) |
| <b>E2E</b>  | End to End (De la Capăt la Capăt)                                          | <b>SDD</b>    | Spec-Driven Development (Dezvoltare Ghidată de Specificații)                                                        |
| <b>FN</b>   | False Negative (Fals Negativ)                                              | <b>SDK</b>    | Software Development Kit (Kit de Dezvoltare Software)                                                               |
| <b>FP</b>   | False Positive (Fals Pozitiv)                                              | <b>SHA</b>    | Secure Hash Algorithms (Algoritmi de Hash Securizat)                                                                |
| <b>GPT</b>  | Generative Pre-trained Transformer (Transformator Pre-trenat Generativ)    | <b>SSE</b>    | Server-Side Events (Evenimente pe Partea Serverului)                                                                |
| <b>GPU</b>  | Graphics Processing Unit (Unitate de Procesare Grafică)                    | <b>STDIO</b>  | Standard Input-Output (Intrare-Ieșire Standard)                                                                     |
| <b>HTML</b> | Hyper-Text Markup Language (Limbaj de Marcare Hipertext)                   | <b>STDOUT</b> | Standard Output (Ieșire Standard)                                                                                   |
| <b>HTTP</b> | Hyper-Text Transfer Protocol (Protocol de Transfer Hipertext)              | <b>TDD</b>    | Test-Driven Development (Dezvoltare Ghidată de Teste)                                                               |
| <b>IA</b>   | Inteligență Artificială                                                    | <b>TN</b>     | True Negative (Negativ Adevărat)                                                                                    |
| <b>IDE</b>  | Integrated Development Environment (Mediu de Dezvoltare Integrat)          | <b>ToT</b>    | Tree-of-Thought (Arbore de Gândire)                                                                                 |
| <b>JSON</b> | Java Script Object Notation (Notatie de Obiecte JavaScript)                | <b>TP</b>     | True Positive (Pozitiv Adevărat)                                                                                    |
| <b>KV</b>   | Key-Value (Cheie-Valoare)                                                  | <b>TUI</b>    | Terminal User Interface (Interfață de Utilizator în Terminal)                                                       |
| <b>LLM</b>  | Large Language Model (Model de Limbaj de Dimensiuni Mari)                  | <b>URL</b>    | Uniform Resource Locator (Localizator Uniform de Resurse)                                                           |
| <b>LSTM</b> | Long Short Term Memory (Memorie pe Termen Lung-Scurt)                      | <b>UUID</b>   | Universally Unique Identifier (Identificator Universal Unic)                                                        |
| <b>MCP</b>  | Model Context Protocol (Protocol de Context al Modelului)                  | <b>VRAM</b>   | Video Random Access Memory (Memorie Video cu Acces Aleatoriu)                                                       |
|             |                                                                            | <b>YAML</b>   | YAML Ain't Markup Language (YAML Nu Este un Limbaj de Marcare)                                                      |



# INTRODUCERE

## CONTEXT ȘI MOTIVAȚIE

Motivația acestei lucrări izvorăște din experiența personală în dezvoltarea software. Atribuțiile unui inginer sunt numeroase și diverse: pe lângă scrierea codului, dezvoltatorul investește timp în planificare, documentație, revizuire, teste unitare, de integrare și End to End (E2E), instrumentare și monitorizare, procese de Continuous Integration (CI)/Continuous Deployment (CD), administrarea infrastructurii și, când apar probleme, depanare și analiza cauzelor principale. Concomitent trebuie să rămână la curent cu cele mai noi practici, securitate cibernetică, rețelistică, precum și să coopereze cu alți colegi.

Evoluția recentă a sistemelor bazate pe Inteligență Artificială (IA) ce folosesc un Large Language Model (LLM) a deschis calea către un proces în care multe sarcini sunt delegate sistemelor automate, atenția inginerului mutându-se de la scrierea de cod la supravegherea acestor instrumente și asumarea deciziilor arhitecturale. Trecerea de la o experiență pur conversațională la integrarea cu mediile de dezvoltare și terminalul sistemului de operare a fost facilitată de competențele avansate de prelucrare și generare de text ale modelelor.

În prezent, furnizorii mari de modele de limbaj controlează o bună parte din cota de piață, fortificând ecosistemul prin produse proprii (asistenți de cercetare, programare, automatizări) și restricționând sau majorând prețul accesului în afara acestuia. Există modele de tip Open Source Software (OSS), dar ecosistemul uneltelor gratuite construite peste ele este încă în dezvoltare ceea ce creează un mediu propice pentru un asistent independent de furnizor, optimizat pentru costuri reduse, ce poate rula local, precum cel propus în această lucrare.

În ciuda impactului pe care acest val îl are asupra industriei, din soluțiile disponibile pe piață lipsește libertatea utilizatorului de a personaliza modul de lucru prin alegerea obiectivă a celor mai bune modele și echipe de agenți pentru diverse sarcini, a tacticilor de compactare, de a vedea concret distribuția costului dar și impactul asupra contextului și a performanței modelelor în fluxul său de lucru.

## SCOPUL ȘI OBIECTIVELE LUCRĂRII

Scopul este realizarea unui **asistent inteligent extensibil și transparent** bazat pe IA, **agnostic față de furnizorii de modele de limbaj**, cu modele ce rulează local sau în cloud și care poate interacționa cu mediul virtual (sistem de operare, mediu de dezvoltare, internet) prin tool-uri<sup>1</sup>, realizat de un flux de lucru agentic<sup>2</sup>.

Obiectivul principal este un produs funcțional ce poate ajuta un inginer software, atingând următoarele necesități: interfață versatilă prin terminal (Terminal User Interface (TUI)), set de unelte pentru sarcini de complexitate ridicată, gestionarea conversației, contextului și memoriei, observabilitate și siguranță prin medii sigure de execuție.

---

<sup>1</sup>Tool-urile sunt funcții cu descrieri detaliate pe care agentul le populează cu parametri, le execută și recepționează răspunsul.

<sup>2</sup>Agentii sunt aplicații ce folosesc IA pentru a interacționa cu mediul înconjurător.

## METODOLOGIE

Realizarea lucrării a urmat o metodologie iterativă în patru etape:

1. Studiul literaturii și al uneltelor concurente (Claude Code, Codex Command Line Interface (CLI), Copilot CLI, Aider), din care au rezultat cerințele și justificarea alegerilor tehnologice;
2. Definirea modulelor ce urmează a fi implementate: graf de execuție, sistem de unelte, proces de compactare, subagenți, sandboxing, permisiuni, observabilitate, privilegiind agnosticismul față de furnizor și extensibilitatea prin fișierele de configurare YAML Ain't Markup Language (YAML);
3. Dezvoltarea incrementală a aplicației **vibe-cli** în Python folosind LangGraph, `prompt_toolkit`/`Rich` pentru TUI și Flask pentru panoul web local;
4. Validarea prin trei protocoale experimentale: evaluare de revizuire de Pull Request (PR) pe 50 de revizuire umane din 7 proiecte OSS, evaluarea stadiilor procesului de compactare și o comparație a 16 modele de limbaj pe 6 sarcini de programare.

## CONTRIBUȚII PROPRII ȘI REZULTATE

Contribuția principală a lucrării o reprezintă proiectarea și implementarea aplicației **vibe-cli**, un asistent inteligent de programare, agnostic față de furnizorul de modele de limbaj, transparent și extensibil. Pe lângă produsul în sine, contribuțiile originale se pot grupa astfel:

- **Proces configurabil de compactare a contextului**, structurat în stadii independente (de-duplicare, trunchierea ieșirilor uneltelor, sumarizare, substituie prin referințe, ștergere bazată pe importanță), configurabil integral prin YAML și extensibil;
- **Sistem de filtre deterministe peste apelurile de unelte**, ce impune protocolul *read-before-write* prin verificarea unui hash Secure Hash Algorithms (SHA) al fișierului înainte de modificare, prevenind inconsistența contextului.
- **Mecanism de integrare automată a capabilităților multi-modale** ale modelelor de limbaj cu suport pentru fișiere text, Portable Document Format (PDF) sau imagini;
- **Arhitectura echipelor de agenți și capabilități** (cercetare web, căutare în cod, sarcini generale, revizuire de PR), configurabile în întregime prin YAML;
- **Sistem local și distribuit de observabilitate**, ce expune utilizatorului metrice detaliate despre consumul de tokeni, costuri, latențe și comportamentul agentului, informații frecvent ascunse în produsele comerciale studiate;
- Trei **protocoale experimentale proprii** pentru validarea soluției și extragerea unor recomandări transferabile la ecosistemul mai larg de unelte similare.

Rezultatele obținute confirmă atingerea obiectivelor propuse. Aplicația finală oferă un set complet de capabilități agentice (sandboxing, permisiuni granulare, suport Model Context Protocol (MCP) pe toate protocoalele de transport uzuale, intrări multi-modale, subagenți și proces de compactare), comparabil din punct de vedere funcțional cu uneltele comerciale dominante, dar diferențiat prin transparență, configurabilitate și absența dependenței de un furnizor unic. Evaluarea experimentală a arătat că pe testele de evaluare proprii de revizuire de PR soluția livrează rezultate comparabile cu cele ale concurenței, că procesul de compactare poate reduce semnificativ consumul de tokeni fără degradarea calității răspunsurilor în conversațiile lungi, și că modele OSS de dimensiuni medii rulate local pot atinge un raport cost-performanță competitiv pe sarcini specifice de programare. Aceste rezultate sunt detaliate în capitolul *Rezultate experimentale* și sintetizate în *Concluzii*.

## STRUCTURA LUCRĂRII

Lucrarea este organizată în cinci capitole. Capitolul *Context și poziționare* prezintă evoluția modelelor de limbaj, stadiul actual al arhitecturilor agentice și analiza comparativă a soluțiilor existente, încheindu-se cu justificarea alegerilor de implementare. Capitolul *Implementare* descrie în detaliu arhitectura aplicației *vibe-cli*, componentele majore și modul lor de interacțiune. Capitolul *Rezultate experimentale* prezintă cele trei protocoale de validare și interpretarea datelor obținute. Capitolul *Dezvoltări viitoare* schițează direcțiile de evoluție identificate, iar capitolul *Concluzii* sintetizează contribuțiile și rezultatele lucrării.

# Capitolul 1

## CONTEXT, POZIȚIONARE ȘI DECIZII ARHITECTURALE

O bună înțelegere a tehnologiilor, standardelor și a obstacolelor actuale din industrie necesită studierea literaturii de specialitate. În a doua parte a capitolului analizăm locul pe care îl ocupă soluția propusă în peisajul industrial actual, prin comparație cu aplicațiile concurente atât proprietare cât și open-source.

### 1.1 Evoluția către modelele mari de limbaj (LLM)

Drumul către modelele mari de limbaj a fost marcat de câteva contribuții fundamentale: perceptronul [21] a demonstrat în 1958 fezabilitatea învățării din date pentru clasificarea binară a vectorilor liniar separabili; rețelele convoluționale (Convolutional Neural Network (CNN)) [22] au introdus extragerea ierarhică de trăsături locale prin filtre de convoluție și *pooling*, devenind fundamentul pentru *computer vision*; pentru date secvențiale au fost propuse rețelele recurente (Recurrent Neural Network (RNN)), iar pentru a depăși problema dispariției gradientului în Backpropagation Through Time (BTT), arhitectura Long Short Term Memory (LSTM) [23] a introdus stări de celulă controlate prin porți de uitare, intrare și ieșire, devenind multă vreme standardul în Natural Language Processing (NLP).

**Punctul de cotitură** a fost reprezentat de arhitectura *transformer* [6] (Figura 1.1), care înlocuiește procesarea secvențială cu mecanismul de *self-attention*: toate elementele unei secvențe sunt analizate simultan, iar fiecare token primește informații contextuale din întreaga secvență indiferent de distanță, deblocând paralelizarea pe Graphics Processing Unit (GPU). Mecanismul de *scaled dot-product attention* este definit pentru matricile de interogări  $\mathbf{Q}$ , chei  $\mathbf{K}$  și valori  $\mathbf{V}$  prin  $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}) \mathbf{V}$ , iar extensia *multi-head attention* folosește mai multe capete în paralel, fiecare specializându-se pe un alt tip de dependență între date.

Odată eliminată limitarea computațională, cercetătorii au putut antrena modele pe seturi de date fără precedent, deschizând drumul către modelele mari de limbaj (LLM). Echipa OpenAI [24] a dezvoltat GPT-3, cu aproximativ **175 miliarde de parametri** [25] care, prin furnizarea câtorva exemple sau a unei simple instrucțiuni în limbaj natural, putea genera cod sursă, rezuma texte, traduce sau crea conținut creativ, marcând tranziția de la sisteme specializate la modele cu competențe generale.

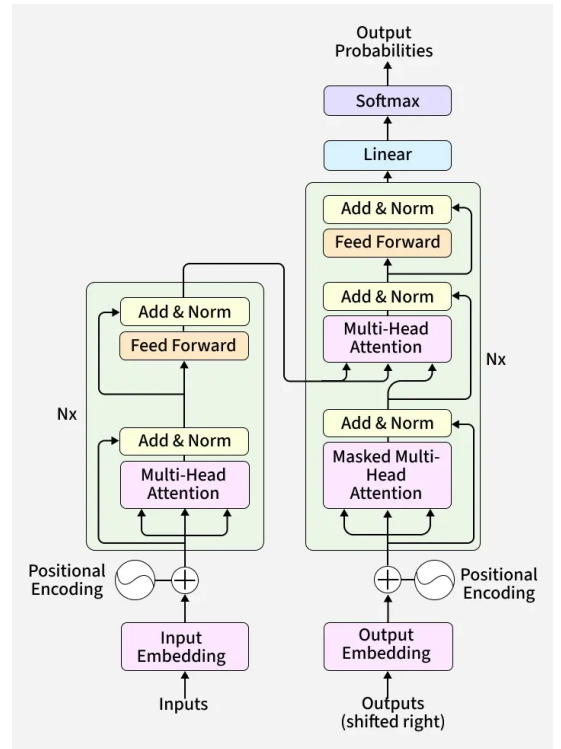


Figura 1.1: Arhitectura transformer [6]

Odată cu apariția primelor *foundation models* (modele mari de uz general antrenate pe date diverse) precum Generative Pre-trained Transformer (GPT)-4, GPT-5, Sonnet sau Opus, industria a cunoscut o creștere exponențială (Figura 1.2). Aceste modele rezolvă probleme din domenii diverse, iar fiecare generație aduce îmbunătățiri în raționament, înțelegerea limbajului și rezolvarea problemelor complexe.

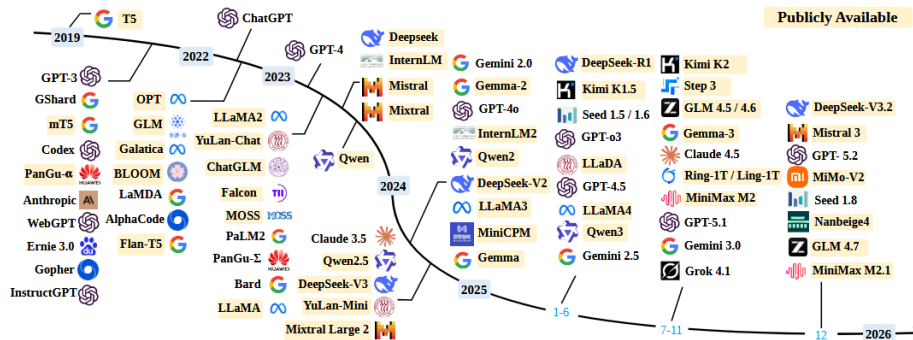


Figura 1.2: Evoluția recentă a modelelor mari de limbaj [7]

O direcție importantă de optimizare a fost procesarea secvențelor lungi. “Longformer: The Long-Document Transformer” [8] reformulează mecanismul de atenție astfel încât consumul de memorie să crească liniar cu lungimea contextului, iar abordări mai recente folosesc serii trigonometrice pentru a stabiliza compresia cache-ului Key-Value (KV) [26], îmbunătățind raționamentul pe termen lung [27].

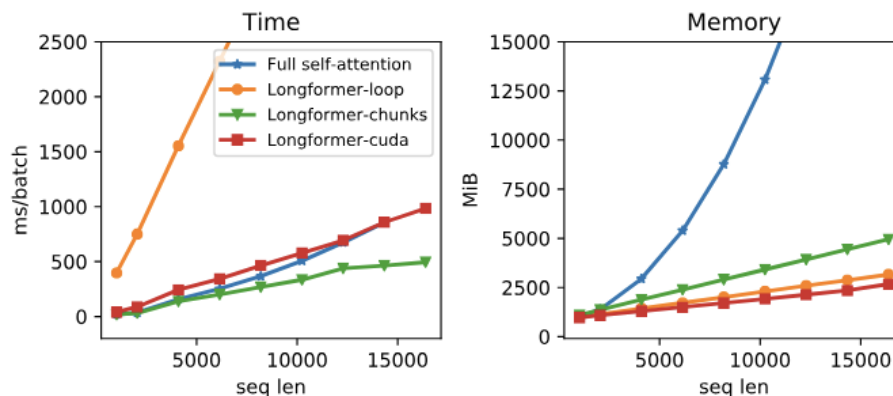


Figura 1.3: Performanța arhitecturii Longformer: în timp ce atenția standard scalează exponențial, Longformer scalează liniar cu lungimea contextului [8]

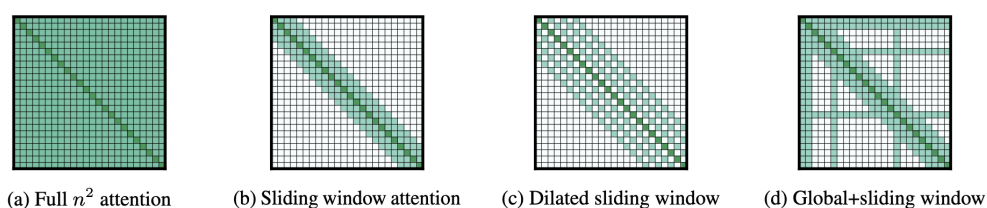


Figura 1.4: Diferite configurații de atenție folosite în arhitectura Longformer în comparație cu mecanismul obișnuit de atenție [8]

Pentru *embedding*-uri (vectori multi-dimensionalai ce reprezintă token-ul plus poziția acestuia în secvență), “RoFormer” [28] codifică poziția printr-o matrice de rotație ce încorporează direct dependența relativă în formula atenției.

Pentru *fine-tuning* (ajustarea parametrilor unui model pentru un domeniu specific), LoRA [9] îngheață ponderile pre-antrenate  $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$  și injectează o descompunere de rang mic  $\Delta \mathbf{W} = \mathbf{B}\mathbf{A}$  ( $\mathbf{B} \in \mathbb{R}^{d \times r}$ ,  $\mathbf{A} \in \mathbb{R}^{r \times k}$ ,  $r \ll \min(d, k)$ ), reducând numărul de parametri antrenabili de până la 10.000 de ori.

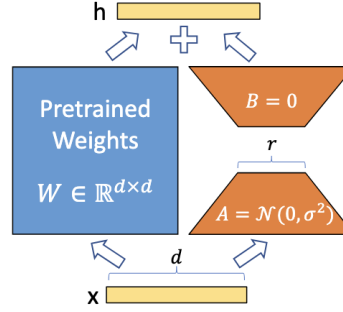


Figura 1.5: Fine-tuning-ul se realizează doar pe A și B [9]

Viteza de inferență a beneficiat de optimizări precum decodarea speculativă [29], care livrează mai mulți tokeni în paralel pornind de la aproximările unui model mai mic, fără a modifica distribuția. În paralel, procedeele de *cuantizare* (reducerea preciziei numerice a parametrilor) și *distilare* (antrenarea unui model mic să reproducă răspunsurile unuia mai mare) micșorează modelele cu pierderi minime de performanță. Pentru distilare, în studiul [10] propune o alternativă proprie ce folosește eșantioane generate cu diverse temperaturi pentru *fine-tuning* supervizat.

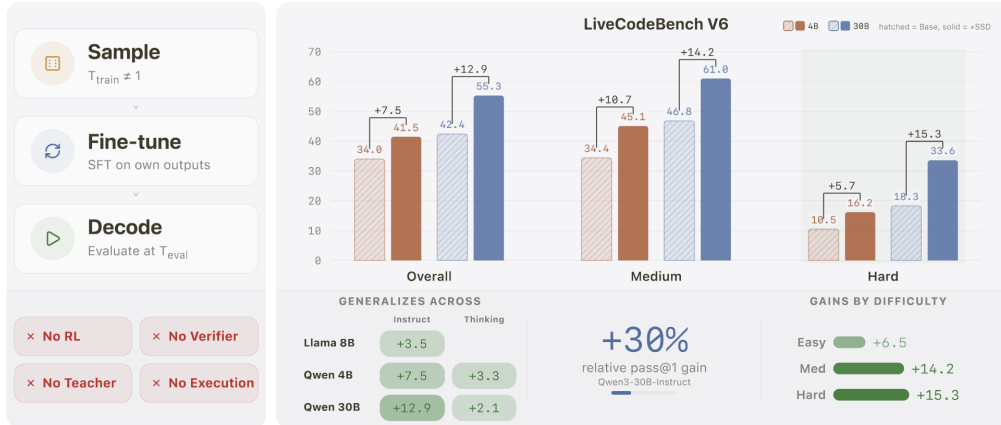


Figura 1.6: Rezultatele procesului de Simple Self Distillation [10]

Cuantizarea este tratată în lucrarea [30] prin transformări (rotații aleatoare) ce induc o distribuție "Beta", obținând reduceri semnificative de memorie, timp de inferență și lățime de bandă atât pentru ponderi cât și pentru cache-ul KV, esențial pentru ferestre de context lungi. Aceste avansuri sunt combinate cu LoRA în QLoRA [31].

Evoluția raționamentului profund se bazează în mare măsură pe Reinforcement Learning (RL): modelul este recompensat pentru ieșiri care manifestă gândire explicită (Chain-of-Thought (CoT) [12]), iar răspunsurile greșite sau nefundamentate sunt penalizate. Publicația [32] discută importanța unor semnale de recompensă precise pentru aplicații de programare, prin generarea de cazuri de test complexe.

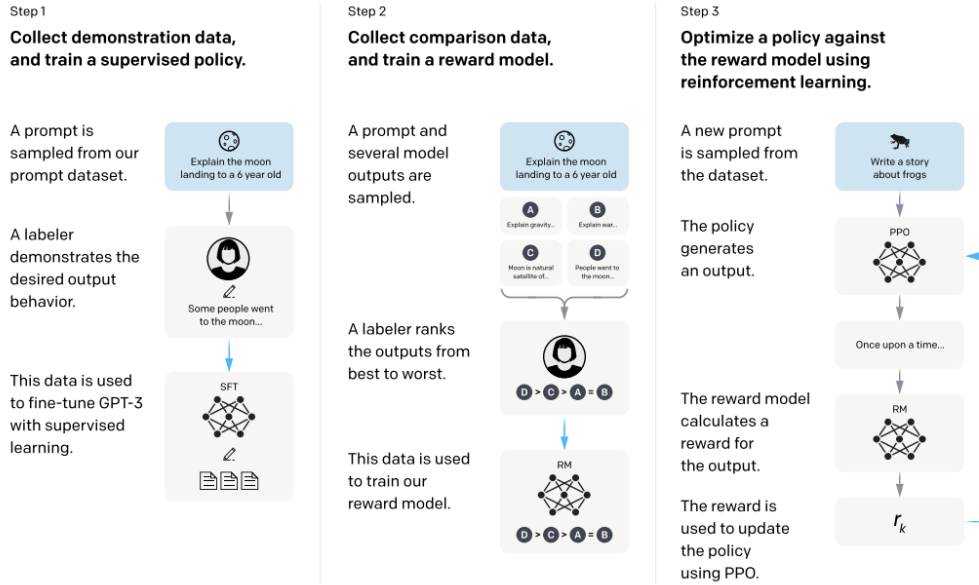


Figura 1.7: Ilustrarea procedurii de RL [11]

În final, și studiile asupra instrucțiunilor oferite modelului au fost de impact: fiind auto-regresive (generează un token în funcție de token-ul precedent), modelele depind puternic de un *prompt* bine construit, ceea ce a dat naștere disciplinei de *Prompt Engineering*, sistematizată în materialul [33]. Pentru rezultate consistente și versionabile, DSPy [34] abstractizează prompt-urile prin grafuri de transformări însoțite de un compilator ce produce instrucțiuni optimizate.

### 1.3 Fundamentele arhitecturilor agentice și cercetări actuale

Arhitecturile agentice presupun capacități de raționament combinate cu înzestrarea modelelor mari de limbaj cu uneltele necesare interacționării cu ecosistemul în care rulează. Fundamentul a fost pus de CoT [12], precedat ulterior de Tree-of-Thought (ToT) [35], care a demonstrat că un model poate naviga probleme complexe prin descompunerea raționamentului în pași intermediari expliciți. Metoda Plan-and-Solve [36] extinde această idee instruind modelul să conceapă mai întâi un plan ce divide sarcina în sub-obiective, eliminând erorile de pași lipsă sau răspunsurile nefondate frecvente în CoT [12] clasic.

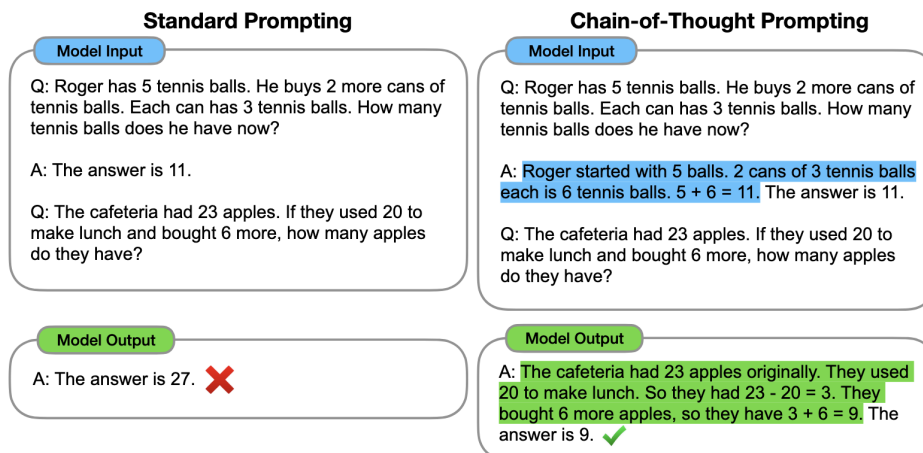


Figura 1.8: Exemplu de CoT [12]

Mai târziu, abordarea ReAct [13] a combinat raționamentul cu acțiunea, permițând modelului să alterneze între generarea de gânduri intermediare și execuția de acțiuni concrete în mediul extern, după tiparul *Thought* → *Action* → *Observation*, repetat până la producerea răspunsului final. Această sinergie a încercat să combată fenomenul *halucinațiilor* (răspunsuri neancorate în fapte reale) și inconsistența în răspunsuri, însă presupunea o intercalare costisitoare între raționament și observații: modelul genera un gând, apela o unealtă, aștepta răspunsul, apoi decidea următoarea acțiune pe baza tuturor datelor anterioare.



Figura 1.9: Paradigma ReAct [13]

|         | Type                | Definition                                                                 | ReAct | CoT |
|---------|---------------------|----------------------------------------------------------------------------|-------|-----|
| Success | True positive       | Correct reasoning trace and facts                                          | 94%   | 86% |
|         | False positive      | Hallucinated reasoning trace or facts                                      | 6%    | 14% |
| Failure | Reasoning error     | Wrong reasoning trace (including failing to recover from repetitive steps) | 47%   | 16% |
|         | Search result error | Search return empty or does not contain useful information                 | 23%   | -   |
|         | Hallucination       | Hallucinated reasoning trace or facts                                      | 0%    | 56% |
|         | Label ambiguity     | Right prediction but did not match the label precisely                     | 29%   | 28% |

Figura 1.10: Îmbunătățirile pe care le aduce ReAct [13]

Pentru a elimina această ineficiență, arhitectura ReWOO [14] decuplează complet procesul de raționament de observațiile externe: modelul generează întregul plan dintr-o singură trecere, iar observațiile sunt colectate independent și integrate ulterior. Această separare reduce consumul de tokeni de până la 5 ori și îmbunătățește acuratețea cu 4% pe HotpotQA.



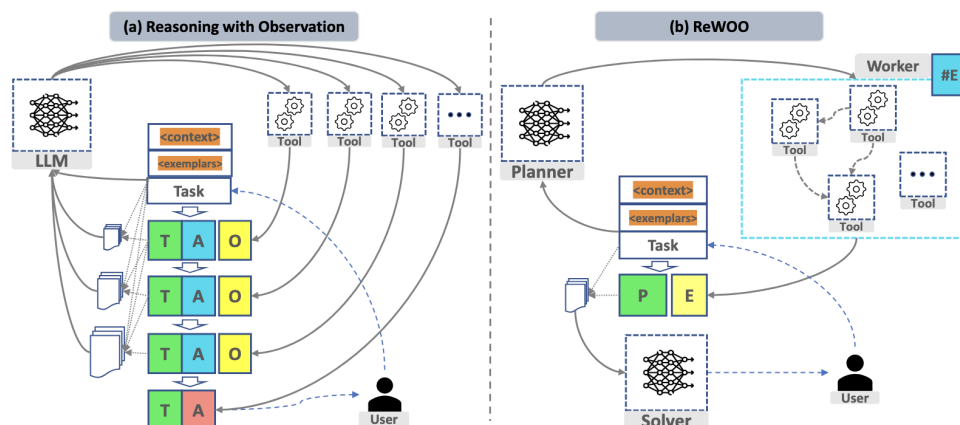


Figura 1.11: Îmbunătățirea adusă de ReWOO: agentul are mai multă libertate, putând apela mai multe unelte până să producă un răspuns. P = Plan, E = Evidence, A = Answer, O = Observation, A = Act, T = Thought [14]

Pe măsură ce agenții au fost înzestrați cu unelte tot mai numeroase (căutare web, terminal, control browser), fereastra de context a devenit o limitare critică. Deși modele precum Opus 4.8 (Anthropic) sau GPT-5.5 (OpenAI) anunță o fereastră de 1 milion de tokeni, “Lost in the Middle” [15] demonstrează că performanța modelelor se degradează semnificativ când informația relevantă se află în mijlocul unui context lung (Figura 1.12, Figura 1.13). Acest lucru aduce implicații directe asupra agenților care acumulează istoric, apeluri la unelte și observații.

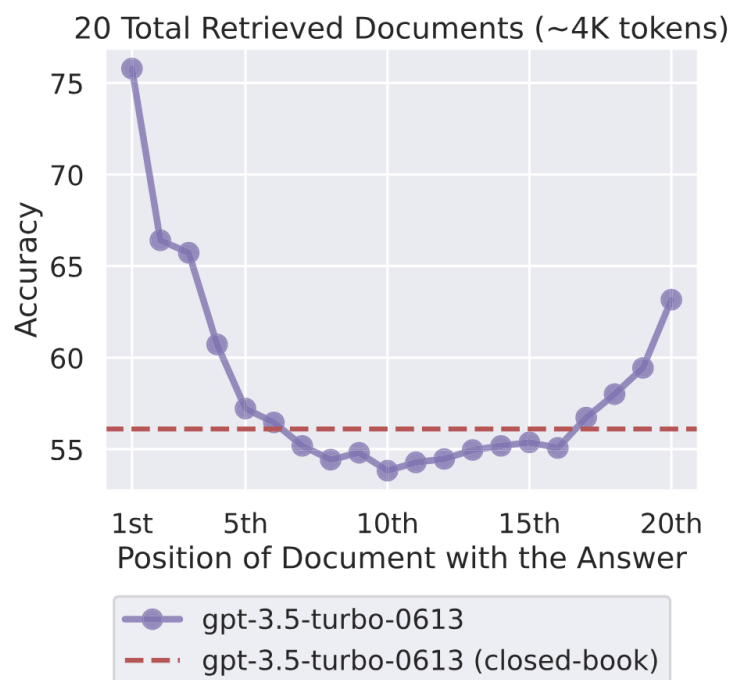


Figura 1.12: Modelele tind să aibă o acuratețe foarte slabă contra întrebărilor ce vizează informații care se află la mijlocul ferestrei curente de context [15]

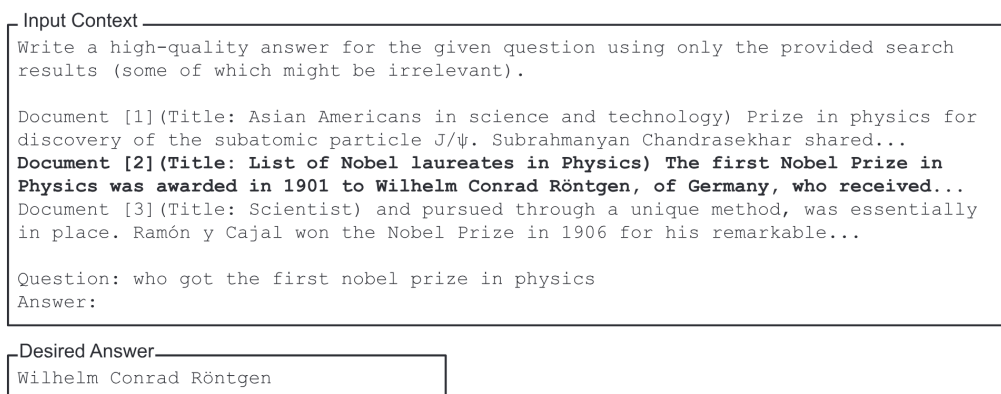


Figura 1.13: Vizualizarea problemei și al rezultatului dorit la întrebare. De multe ori, modelele întâmpină dificultăți în accesarea acestor informații ascunse în mijlocul ferestrei de context [15]

Cadrul de lucru Michelangelo [37] extinde testele clasice de căutare în date numeroase, testând abilitatea modelului de a sintetiza informații dispersate, și confirmă un spațiu semnificativ de îmbunătățire chiar și pentru ultimele generații. Pentru eficiența cache-ului KV, LycheeCluster [38] fragmentează contextul pe baza delimitatorilor și construiește un index ierarhic ce transformă căutarea în cache într-un proces logaritm, obținând o accelerare **de până la 3.6 ori**.

## 1.4 Limitări ale arhitecturilor agentice

Bariera principală în avansarea acestor arhitecturi și a aplicațiilor care se bazează pe ele este **ferestra de context**. Pentru sisteme cu numeroase linii de cod sau proiecte distribuite în mai multe servicii, contextul limitează masiv capabilitățile agenților. Deși există tehnici de optimizare, acestea nu sunt suficiente în conversații lungi: multe funcționalități care fac un agent performant, precum uneltele, consumă rapid contextul ajungând în final să degradeze performanța agentului. De asemenea, în general, aceste tehnici de optimizare sunt impuse de producător, fără posibilitatea de configurare. Acest lucru limitează performanțele aplicațiilor din pricina faptului că o configurație unică nu poate fi optimă pentru toate scenariile de utilizare.

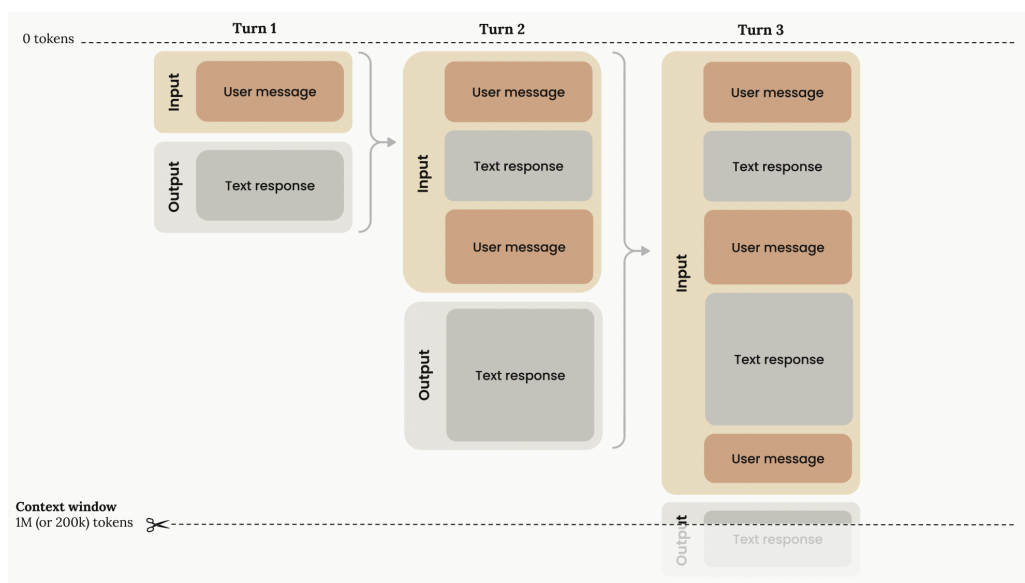


Figura 1.14: Reprezentarea ferestrei de context într-o conversație uzuală (Sursă: [16])

În plus, modelele curente tind să genereze date de ieșire mai detaliate decât este necesar (cantitate peste calitate), iar în sarcini de revizuire de PR se grăbesc să relateze multe probleme simple sau de

formatare, ratând probleme importante de logică. În final, agenții nu au o bună înțelegere legată de când ar trebui să oprească investigațiile, rămânând adesea blocați în bucle de lungă durată.

## 1.5 Analiză preliminară a peisajului actual în domeniul inteligenței artificiale concentrată pe aplicații de dezvoltare software

Industria dezvoltării software are probabil una din cele mai profitabile întrebuintări ale acestor arhitecturi agentice, datorită volumului masiv de muncă existent în domeniu, așa că nu a durat mult până ce marii furnizori de pe piața modelelor de limbaj au început să ofere produse care să asiste programatorii în sarcinile lor. La momentul actual există mai multe soluții competitive: Codex [39] de la OpenAI, Gemini CLI [40] de la Google, Claude Code [41] de la Anthropic, Cursor [42] de la Anysphere sau Kiro [43] de la Amazon. Toate acestea sunt soluții proprietare care sunt făcute să ruleze cu o gamă restrânsă de modele și pentru care utilizatorul trebuie să plătească. Există și soluții open-source precum OpenHands, Cline sau Aider cu care deținătorii de modele se luptă pentru a domina piața, deși lupta nu este strânsă din cauza monopolului soluțiilor proprietare.

| Companie               | Model                 | Preț [\$/milion de tokeni]                                                  | Capabilități                                                                                                                                                                 |
|------------------------|-----------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Anthropic [44]</b>  | Claude Opus 4.8       | \$5/input MTok, \$25/output MTok                                            | Cel mai capabil model disponibil, multi-modal, îmbunătățire semnificativă pentru programare agentică față de 4.6 și 4.7, gândire adaptivă, 1MTok context, 128kTok la ieșire  |
|                        | Claude Sonnet 4.6     | \$3/input MTok, \$15/output MTok                                            | Mai rapid, compromite inteligența pentru viteză, gândire adaptivă, 1MTok context, 64kTok max. la ieșire                                                                      |
|                        | Claude Haiku 4.5      | \$1/input MTok, \$5/output MTok                                             | Cel mai rapid, fără gândire adaptivă, 200kTok context, 64kTok max. la ieșire                                                                                                 |
| <b>OpenAI [45]</b>     | gpt-5.5               | \$5/input MTok, \$30/output MTok                                            | Noua clasă de inteligență pentru programare și muncă profesională, raționament profund cu niveluri multiple de efort, suport multi-modal, 1MTok context și 128kTok la ieșire |
|                        | gpt-5.4-mini          | \$0.75/input MTok, \$4.5/output MTok                                        | Model mai mic, rapid dar puternic pentru programare și subagenți. 400kTok context și 128kTok max. la ieșire                                                                  |
|                        | gpt-5.4-nano          | \$0.20/input MTok, \$1.25/output MTok                                       | Model mic și foarte rapid pentru sarcini ușoare în volum mare. 200kTok context și 128kTok max. la ieșire                                                                     |
| <b>Google [46]</b>     | Gemini 3.1 Pro        | \$2/in MTok, \$12/out MTok < 200kTok, \$4/in MTok, \$18/out MTok >= 200kTok | Model pentru sarcini complexe ce necesită cunoștințe dezvoltate. 1MTok context in, 64k out                                                                                   |
|                        | Gemini 3.1 Flash      | \$0.5/input MTok, \$3/output MTok                                           | Inteligență comparabilă cu modelul Pro la o viteză mai mare. Context 1M in, 64k out                                                                                          |
|                        | Gemini 3.1 Flash-Lite | \$0.25/input MTok, \$1.5/output MTok                                        | Model construit pentru costuri reduse și volum mare de sarcini. Context 1M in, 64k out                                                                                       |
|                        | Gemma 4               | Open Source                                                                 | Familie de modele open-source pentru rulare locală, cost redus și personalizare                                                                                              |
| <b>DeepSeek [47]</b>   | DeepSeek R1           | \$0.55/input MTok, \$2.19/output MTok                                       | Model specializat în raționament și gândire adâncă pentru probleme complexe, 128kTok context, cache hit: \$0.14/input MTok                                                   |
| <b>Qwen [48]</b>       | Qwen3-Coder           | Open Source                                                                 | Model specializat pe cod din familia Qwen3, optimizat pentru generare și înțelegere de cod, arhitectură Mixture of Experts, gândire adaptivă, 1MTok context.                 |
| <b>Mistral AI [49]</b> | Mixtral 8x22B         | Open Source                                                                 | Model Mixture of Experts open-source european. 66kTok fereastră de context                                                                                                   |
|                        | Devstral Small 2      | \$0.10/input MTok, \$0.30/output MTok                                       | Model open-source (Apache 2.0) specializat pe cod, bazat pe arhitectura Mistral Small 3.1, optimizat pentru agenți de programare și completare de cod. 128kTok context       |

Tabelul 1.1: Selecție de modele și prețul de utilizare în comparație cu capabilitățile.

Din tabelul 1.1 se poate deduce că utilizarea frecventă sau inefficientă a unui model de ultimă generație într-una din aplicațiile enumerate mai sus poate ajunge să coste o mică avere. Într-o conversație lungă, fiecare pereche răspuns-întrebare din istoric este re-introdusă ca date de intrare pentru următoarea interacțiune, iar fiecare apel către unelte rămâne și el în context, crescând exponențial

costul de întreținere a conversației, lucru des întâlnit în aplicațiile din domeniul dezvoltării software.

Din punct de vedere al capabilității de rezolvare a sarcinilor reale, clasamentul celor mai bune modele în sarcini de programare se schimbă frecvent. Utilizând SWE-Bench Pro [17] putem măsura performanța contra unor sarcini complexe (Figura 1.15) și raportul între performanță și cost (Figura 1.16).

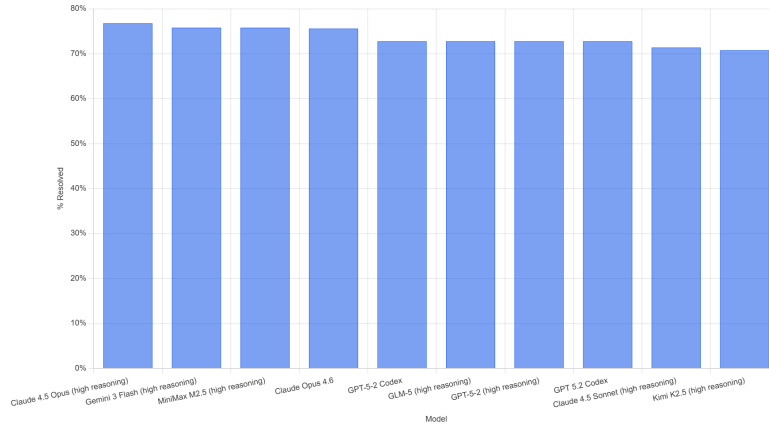


Figura 1.15: Clasamentul modelelor în SWE-Bench Pro [17] la momentul actual în funcție de performanța rezolvării sarcinilor complexe în limba engleză.

Cele mai puternice modele necesită resurse semnificative pentru antrenare și inferență, ce se reflectă asupra utilizatorului final (cu strategii de taxare per-token sau abonament). Modelele cu gândire adâncă au un cost mai mare per-token și consumă mai mulți tokeni. Fiecare deținător de modele oferă astfel variante de complexități diferite (de exemplu gama Gemini cu versiunile "Flash", "Thinking", "Pro"), iar aplicațiile au algoritmi ce rutează instrucțiunile către modelul potrivit în funcție de complexitate, pentru a echilibra costul.

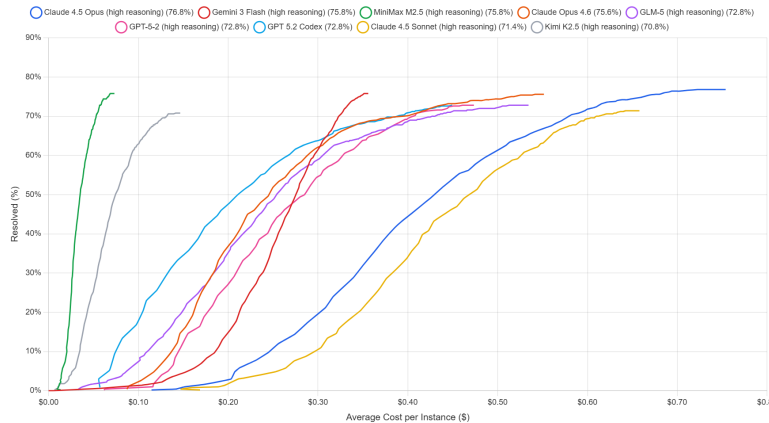


Figura 1.16: Clasamentul după cost al modelelor în SWE-Bench Pro [17] la momentul actual.

## 1.6 Plasarea în industrie și oportunități identificate

Având o imagine clară asupra stadiului actual și a alegerilor arhitecturale, abordăm plasarea proiectului în industrie și deciziile prin care propunem un produs competitiv.

### 1.6.1 Analiza ecosistemelor proprietare

Momentan, cea mai mare cotă de piață o au furnizorii de modele mari de limbaj precum Anthropic, Google sau OpenAI. Aceștia, pe lângă aplicațiile pentru dezvoltarea software, lansează aplicații având ca obiectiv blocarea utilizatorilor în ecosistemul lor, prin unelte ce urmăresc domenii diverse, fenomen observabil în tabelul 1.2.

| Companie                          | Produs                    | Descriere                                                                                                                                                | Audiență                                     | Preț                                                                                                                                                                  |
|-----------------------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Anthropic [50]</b>             | Claude [51]<br>(web/app)  | Asistent IA multi-modal pentru analiză, scriere și programare                                                                                            | Utilizatori generali, profesioniști          | Freemium (Pro: \$20/lună, Max: \$100–200/lună)                                                                                                                        |
|                                   | Claude Code [41]          | Agent de cod în terminal/Integrated Development Environment (IDE); navighează baze de cod, creează PR-uri și rulează teste                               | Dezvoltatori software                        | Inclus în Pro/Max (Application Programming Interface (API): cost per token)                                                                                           |
|                                   | Claude Cowork             | Automatizează sarcini recurente în fișiere locale și aplicații cloud (Slack, Drive)                                                                      | Profesioniști, echipe                        | Inclus în Pro/Max                                                                                                                                                     |
|                                   | Claude Design             | Generează interfețe Graphical User Interface și mockup-uri din descrieri în limbaj natural                                                               | Designeri, dezvoltatori                      | Inclus în Pro/Max                                                                                                                                                     |
| <b>Google [52]</b>                | Gemini [53]<br>(web/app)  | Asistent IA multi-modal cu căutare Google și generare de imagini                                                                                         | Utilizatori generali                         | Freemium (Google AI Pro: \$19.99/lună)                                                                                                                                |
|                                   | Antigravity CLI [54]      | Agent TUI în terminal cu subagenți concurenți, suport MCP, editare multi-fișier și comenzi slash; optimizat pentru fluxuri keyboard-first și sesiuni SSH | Dezvoltatori software                        | Freemium (Individual: gratuit cu limite săptămânale de rată; Google AI Pro și Ultra: credite flexibile și limite mai generoase; Enterprise: Google Cloud, per consum) |
|                                   | NotebookLM [55]           | Analizează documente încărcate (PDF, site-uri, video-uri) și generează rezumate, podcast-uri și conexiuni între surse                                    | Cercetători, studenți                        | Gratuit (Plus: \$7.99/lună, Business: contact)                                                                                                                        |
|                                   | Vertex AI [56]            | Platformă enterprise Machine Learning cu 200+ modele și instrumente Machine Learning Operations                                                          | Echipe enterprise, ingineri Machine Learning | Plătit (pay-as-you-go, de la \$0.0001 per operație)                                                                                                                   |
|                                   | Firebase Studio [57]      | Integrated Development Environment (IDE) cloud cu agenți IA pentru aplicații full-stack direct din browser                                               | Dezvoltatori software                        | Gratuit în previzualizare (3 spații de lucru; se încheie în 2027)                                                                                                     |
| <b>OpenAI [24]</b>                | ChatGPT [58]<br>(web/app) | Asistent IA multi-modal cu căutare web, generare imagini și analiză date                                                                                 | Utilizatori generali, profesioniști          | Freemium (Plus: \$20/lună, Pro: \$200/lună)                                                                                                                           |
|                                   | Codex [39]                | Agent cloud pentru programare paralelă în medii izolate preîncărcate cu codul din GitHub [59]                                                            | Dezvoltatori software, echipe                | Inclus în abonamentul ChatGPT                                                                                                                                         |
| <b>Microsoft [60]/<br/>GitHub</b> | GitHub Copilot [61]       | Agent în GitHub/Integrated Development Environment (IDE) (VS Code [2], JetBrains [62]); completare cod, chat, revizuire PR-uri și rezolvare tichete      | Dezvoltatori software, echipe                | Freemium (plan gratuit: 2.000 completări/lună; Pro: \$10/lună, Business: \$19/lună, Pro+: \$39/lună; Enterprise: tarif separat)                                       |

Tabelul 1.2: Ecosistemul de produse cu IA al principalilor furnizori de modele de limbaj

Alternativele open-source se lovesc de probleme de natură concurențială: tarife diferențiale pentru cheile Application Programming Interface (API) (uneori cu risc de restricționare), lipsa fondurilor și a contribuitorilor, precum și a publicității. În ciuda acestor dificultăți, accesul gratuit, suportul pentru modele open-source rulate local și natura agnostică de furnizor sunt avantaje majore în fața costurilor ridicate.

### 1.6.2 Analiza soluțiilor open-source

Pe lângă aplicațiile furnizorilor există și câteva aplicații open-source care s-au bucurat de o oarecare adopție, regăsite în tabelul 1.3.

| Aplicație                                           | Audiență și scop                                                                                                                  | Puncte forte                                                                                                                                                                                                                                                                                                                                       | Puncte slabe                                                                                                                                                                                                                                                               | Funcționalități cheie                                                                                                                                |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>OpenClaw</b><br>(GitHub [63])<br>Licență MIT     | Utilizatori care doresc un asistent IA personal, mereu activ, accesibil pe orice canal de comunicare                              | <ul style="list-style-type: none"> <li>• Suport multi-canal masiv (WhatsApp [64], Telegram [65], Slack [66], Discord [67])</li> <li>• Sandboxing cu Docker [68], Secure Shell și OpenShell</li> <li>• Aplicații native macOS [69], iOS, Android [70]</li> <li>• Agnostic față de furnizor</li> </ul>                                               | <ul style="list-style-type: none"> <li>• Orientat spre asistent personal, nu exclusiv programare</li> <li>• Configurare complexă pentru multi-canal</li> <li>• Nu are integrare directă în Integrated Development Environment (IDE)</li> </ul>                             | Multi-provider, multi-canal, sandboxing, control vocal, suprafață interactivă, sarcini programabile, capabilități, multi-agent                       |
| <b>OpenHands</b><br>(GitHub [71])<br>Licență MIT    | Dezvoltatori și echipe enterprise care necesită un agent autonom complet cu izolare securizată                                    | <ul style="list-style-type: none"> <li>• Sandboxing în containere Docker</li> <li>• Graphical User Interface web și CLI disponibile</li> <li>• Software Development Kit (SDK) Python pentru construcția de agenți personalizați</li> <li>• Suport Kubernetes [72]</li> <li>• Agnostic față de furnizor</li> </ul>                                  | <ul style="list-style-type: none"> <li>• Necesită Docker</li> <li>• Configurare inițială complexă</li> <li>• Nu are integrare directă în Integrated Development Environment (IDE)</li> <li>• Curbă de învățare mai abruptă</li> </ul>                                      | Sandboxing, Software Development Kit (SDK), Graphical User Interface web, CLI, Kubernetes, integrări Slack [66]/Jira [73]                            |
| <b>Cline</b><br>(GitHub [74])<br>Licență Apache 2.0 | Dezvoltatori care preferă lucrul direct din Integrated Development Environment (IDE) cu control granular și automatizări avansate | <ul style="list-style-type: none"> <li>• Extensie VS Code [2] și extensie JetBrains [62]</li> <li>• Software Development Kit (SDK) Node.js [75] și CLI neinteractiv pentru CI/CD</li> <li>• Echipe multi-agent și agenți programați</li> <li>• Suport MCP și extensii</li> <li>• Agnostic față de furnizor</li> </ul>                              | <ul style="list-style-type: none"> <li>• Consum ridicat de tokeni datorită contextului mare</li> <li>• Fără sandboxing nativ incorporat</li> <li>• Complexitate ridicată, multe funcționalități</li> <li>• Dependent de ecosistemul VS Code/JetBrains</li> </ul>           | Extensie Integrated Development Environment (IDE), Software Development Kit (SDK), CLI, multi-agent, MCP, agenți programați, integrări Slack/Discord |
| <b>Goose</b><br>(GitHub [76])<br>Licență Apache 2.0 | Utilizatori generali și dezvoltatori care doresc un agent IA nativ, performant și extensibil                                      | <ul style="list-style-type: none"> <li>• Aplicație nativă desktop (macOS, Linux, Windows)</li> <li>• Construit în Rust [77] deci are o performanță ridicată</li> <li>• Suport MCP (70+ extensii)</li> <li>• Agent general, nu doar pentru cod</li> <li>• Agnostic față de furnizor (15+ furnizori, suport Agent Communication Protocol)</li> </ul> | <ul style="list-style-type: none"> <li>• Mai puțin specializat pe programare</li> <li>• Ecosistem de extensii mai tânăr</li> <li>• Nu are integrare directă în Integrated Development Environment (IDE)</li> <li>• Documentație în curs de maturizare</li> </ul>           | Aplicație nativă, CLI, API, MCP, Agent Communication Protocol, Rust, multi-platformă, Linux Foundation                                               |
| <b>Aider</b><br>(GitHub [78])<br>Apache 2.0         | Dezvoltatori individuali care preferă un flux de lucru minimalist din terminal cu integrare Git [79]                              | <ul style="list-style-type: none"> <li>• Mapare automată a codului sursă</li> <li>• Integrare Git nativă cu commit-uri automate</li> <li>• Suport 100+ limbaje de programare</li> <li>• Verificare cod și testare automată</li> <li>• Agnostic față de furnizor</li> </ul>                                                                         | <ul style="list-style-type: none"> <li>• Fără sandboxing</li> <li>• Exclusiv terminal, fără Graphical User Interface</li> <li>• Capabilități agentice limitate față de concurență</li> <li>• Nu are extensie nativă de Integrated Development Environment (IDE)</li> </ul> | Terminal, Git auto-commit, repo map, voice-to-code, verificare statică, modele locale                                                                |

Tabelul 1.3: Top 5 asistenți open-source după numărul de "stele" pe GitHub

În urma documentării în privința industriei la momentul actual, voi enumera în continuare câteva arii ce merită abordate pentru o soluție competitivă.

### 1.6.3 Cerințe pentru o soluție agentică în dezvoltarea software

Pentru a realiza un asistent inteligent în dezvoltarea software trebuie atinse câteva obiective principale: o interfață accesibilă din diferite medii de dezvoltare, unelte ce pot rula în terminalul oricărui sistem de operare, posibilitatea de a alege dintr-o gamă largă de modele, comenzi pentru a controla parametrii aplicației, suport pentru rutine predefinite și echipe de agenți.

Agenții funcționează, pe lângă procedeele expuse anterior, pe una dintre filozofiile Spec-Driven

Development (SDD) (programare pe baza documentelor de specificație) sau Test-Driven Development (TDD) (programare după cerințe sumare și executarea testelor la final). Pe lângă acestea, trebuie să ne asigurăm că modelul răspunde doar instrucțiunilor cu scop legitim și nu poate rula comenzi care pun în pericol integritatea sistemului, aspecte tratate în capitolele ce urmează.

### 1.6.4 Nevoia de personalizare

Aplicația trebuie să ofere o multitudine de oportunități de personalizare pentru a face produsul să se potrivească necesităților unui eșantion cât mai mare de utilizatori și să ofere un sentiment de atașament iar în final să intre în rutina acestora. De asemenea, transparența joacă un rol important în personalizare deoarece pentru a personaliza eficient funcționarea aplicației, utilizatorul trebuie să poată face alegeri informate și să încerce diferite configurații, analizând rezultatele.

### 1.6.5 Lipsa de observabilitate

Majoritatea panourilor de informații ale furnizorilor sunt sumare. Aplicațiile în terminal oferă mai multe date, dar nu detaliază printre altele ce unelte consumă cel mai mult context sau dacă agentul le utilizează eficient.

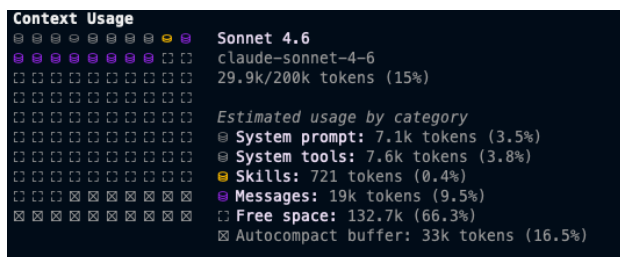


Figura 1.17: Reprezentarea ferestrei de context într-o conversație în Claude Code



Figura 1.18: Reprezentarea utilizării în Claude (varianta web) (Sursă: [18])

Astfel, observabilitatea joacă un rol important pentru transparență: utilizatorul poate interoga orice metrică relevantă în timp real folosind Langfuse și panoul web inclus cu aplicația.

### 1.6.6 Securitate și confidențialitate

Confidențialitatea datelor trebuie să fie totală (excepție: modelele furnizorilor sau Langfuse cloud, opționale). Pentru securitate, aplicația trebuie să includă: sandboxing prin containere (execuție de comenzi în medii sigure), filtrarea logică a apelării uneltelor (de exemplu, *read-before-write*) și cererea de aprobare a utilizatorului pentru acțiuni periculoase.

### 1.6.7 Costul de utilizare

Utilizarea trebuie să fie gratuită. Pentru modelele open-source rulate local, singurele costuri suplimentare sunt cele de hardware, curent electric și internet. Aplicația este optimizată pentru a reduce consumul de tokeni, deci și pentru modele furnizate de terți, costul rămâne minim.

## 1.7 Analiză proprie a necesităților și decizii arhitecturale

Pornind de la oportunitățile identificate, precum suportul pentru o gamă largă de modele de limbaj, personalizarea extensivă, observabilitatea în timp real, securitatea și confidențialitatea, respectiv cost minim, prezentăm în continuare deciziile arhitecturale și necesitățile care conturează implementarea aplicației.

### 1.7.1 Adoptarea unei arhitecturi modulare

Aplicația trebuie să fie structurată în patru subsisteme cu responsabilități separate (`src/`, `ui/`, `web/`, `eval/`), personalizabile prin fișiere de configurație YAML și prin preferințe persistate local, ceea ce va permite evoluția independentă a modulelor și testarea izolată.

### 1.7.2 Alegerea limbajului de programare și a cadrului de lucru agentic

Modelele disponibile inițial vor fi din gama Gemini, Anthropic, OpenAI și modele open-source rulate local prin Ollama sau în cloud. În materie de limbaj, **Python** [80] a fost alegerea evidentă: fiind un standard în industrie cu un ecosistem matur și portabilitate pe multiple platforme. Pentru cadrul de lucru agentic, am căutat o soluție gândită pentru utilizarea uneltelor, compatibilitate multi-model, ramuri de execuție, management al stărilor și memoriei și fluxuri de execuție robuste. Astfel, comparația din tabelul 1.4 a stat la baza alegerii făcute.

| Criteriu                     | LangChain [3]                                 | LangGraph [4]                                             | Vertex [56]                                          | LiteLLM [81]                                               | OpenAI SDK [82]                                    |
|------------------------------|-----------------------------------------------|-----------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------|
| <b>Rol</b>                   | Orchestrator general de LLM-uri și integrări. | Cadru de lucru pentru agenți cu stare și fluxuri ciclice. | Platformă integrată în Google Cloud.                 | Intermediar universal pentru 100+ LLM-uri.                 | Client oficial pentru modelele OpenAI.             |
| <b>Capabilități Agentice</b> | Limitate (predominant fluxuri liniare).       | <b>Excelent</b> (bucle, evaluări și multi-agent).         | Bun (suport nativ pentru instrumente).               | N/A (doar rutare de mesaje).                               | Depinde de utilizator (manual sau Assistants API). |
| <b>Suport Multi-Model</b>    | Da (agnostic).                                | Da (prin ecosistemul LangChain).                          | Optimizat pentru Gemini.                             | <b>Absolut</b> (scopul principal).                         | Nu (doar OpenAI).                                  |
| <b>Management Stare</b>      | Încorporat (tamponare de memorie).            | Superior (persistență și "time travel").                  | Gestionat de platformă.                              | N/A (fără stare).                                          | Threads (doar în Assistants API).                  |
| <b>Învățare</b>              | Medie (ecosistem foarte mare).                | Abruptă (necesită logica grafurilor).                     | Ușoară (Graphical User Interface intuitiv în cloud). | Ușoară (identic cu OpenAI Software Development Kit (SDK)). | Foarte Ușoară, documentat                          |

Tabelul 1.4: Comparația cadrelor de lucru pentru întrebări agentice

La nivel conceptual, LangGraph modelează un flux de lucru agentic ca un *graf orientat* cu noduri de calcul (apeluri LLM, unelte, funcții) și muchii *condiționale* sau *ciclice* esențiale pentru bucle *reason-act-observe*. Graful operează pe o *stare* ce este partajată tuturor nodurilor, iar mecanismul de *puncte de control* permite serializarea stării după fiecare pas pentru reluare sau *inspecția stărilor precedente*, oferind persistență între sesiuni și puncte pentru interacțiunea cu utilizatorul (*human in the loop*).

În final am ales **LangGraph** datorită suportului pentru o gamă largă de furnizori de modele de limbaj, a controlului fin asupra fluxului de lucru agentic, al contextului și al memoriei, a bibliotecilor disponibile pentru dezvoltarea uneltelor, a agenților și a integrării native cu Langfuse [83].

### 1.7.3 Adoptarea motorului pentru interfața cu utilizatorul

Interfața cu utilizatorul trebuie să ofere o bară de derulare reală, latență redusă în conversații lungi, copiere facilă a conținutului și extensibilitate, precum soluțiile consacrate (Claude Code, Codex CLI, Aider). Pe lângă aceste cerințe de bază trebuie să proiectăm un set propriu de elemente vizuale pentru transparența activității agentului: indicator de activitate cu starea curentă și cronometru, fișiere jurnal



persistente ale uneltelor, panouri de diferențe la scrierea în fișiere, *împachetarea lipirii* pentru texte lungi și antet de stare cu metrice de sesiune. Tabelul 1.5 compară opțiunile pentru implementare.

| Bibliotecă                              | Arhitectură                                                                                                | Avantaje                                                                                                                                                                                                                                          | Dezavantaje                                                                                                                                                                                                                                          |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Textual [84]</b>                     | Motor TUI pe ecran complet. Arbore de elemente bazat pe Document Object Model                              | <ul style="list-style-type: none"> <li>• Sistem de așezare similar cu Cascading Style Sheets</li> <li>• Formatare de text încorporată nativ</li> <li>• Complet asincron și bazat pe evenimente</li> </ul>                                         | <ul style="list-style-type: none"> <li>• Nu funcționează derularea nativă</li> <li>• Latență dacă mesajele vechi nu sunt șterse</li> <li>• Dificultate la copierea/lipirea textului</li> </ul>                                                       |
| <b>Prompt Toolkit [85] și Rich [86]</b> | Experiență interactivă folosind tamponul principal al terminalului. Afișare prin Standard Output (STDOUT). | <ul style="list-style-type: none"> <li>• Derulare perfectă prin bara de derulare nativă a terminalului</li> <li>• Zero latență sau probleme de memorie pe măsură ce istoricul crește</li> <li>• Selectare, copiere și lipire eficientă</li> </ul> | <ul style="list-style-type: none"> <li>• Nu este un motor real de Graphical User Interface (doar flux liniar)</li> <li>• Dificil de implementat panouri laterale sau complexe</li> <li>• Limitat strict la input ancorat în partea de jos</li> </ul> |
| <b>Urwid [87]</b>                       | TUI pe ecran complet prin tamponul alternativ al terminalului. Ierarhie strictă de componente.             | <ul style="list-style-type: none"> <li>• Extrem de matur, performant, stabil și testat în producție</li> <li>• Gestionarea intrărilor</li> </ul>                                                                                                  | <ul style="list-style-type: none"> <li>• API învechit și curbă de învățare abruptă</li> <li>• Sincron implicit (necesită integrare suplimentară)</li> </ul>                                                                                          |

Tabelul 1.5: Comparația bibliotecilor pentru interfața cu utilizatorul

Inițial am ales librăria Textual datorită sistemului său puternic de amplasare a componentelor, dar am abandonat ideea în timpul implementării din cauza problemelor cu derularea conversației, copierea și lipirea de text și cu elementele animate, în favoarea opțiunii cu "Prompt Toolkit" și "Rich".

### 1.7.4 Tehnici proprii de gestionare a contextului și a apelurilor la unelte

Pentru a aduce un aport pozitiv capabilităților agentice, propunem un număr de 4 funcționalități care facilitează împreună utilizarea eficientă și consistentă a ferestrei de context a modelelor și execuția corectă a uneltelor:

1. Un proces de compactare configurabil integral din fișiere YAML, structurat ca registru de strategii extensibile prin subclase. Spre deosebire de produsele din industrie, fiecare stadiu trebuie expus utilizatorului pentru a permite studiul efectului acestor metode asupra coerenței agentului;
2. Filtre deterministe ce verifică precondiții logice între apelurile de unelte, dintre care *read-before-write* (citire înainte de scriere) folosind un hash SHA pentru integritatea contextului fișierului;
3. Echipe de subagenți specializați pentru conservarea contextului agentului principal;
4. Politică de sandboxing, cu suport pentru subagenți prin contextul părintelui pentru izolare uniformă.

### 1.7.5 Suportul pentru date multi-modale

Arhitectura agnostică de furnizor impune ca aplicația să trateze uniform modele cu capabilități eterogene, fără a presupune disponibilitatea unor metadate consistente despre tipurile de date acceptate. De aceea, suportul multi-modal trebuie construit pe un mecanism propriu de detectare automată a capabilităților fiecărui model, care să determine suportul prin probare efectivă.

Gestionarea contextului trebuie să adopte o schemă hibridă, adaptată tipului de date și capabilităților modelului. Fișierele textuale și PDF trebuie să rămână accesibile în format text prin uneltele uzuale (zone de interes, trunchiere), iar atunci când modelul susține nativ tipul de date, agentul trebuie să poată cere o vedere completă prin citiri succesive. Imaginile trebuie injectate direct în context, iar fișierele atașate trebuie persistate automat ca artefacte.

### 1.7.6 Observabilitate în două straturi

Observabilitatea trebuie oferită pe două nivele complementare, dezactivate implicit:

1. Tracing extern prin Langfuse Cloud care pentru fiecare invocare a grafului, generează un *trace* (urmă a execuției) ierarhic cu *span*-uri (compartimentări) per apel LLM, unealtă și nod, etichetate cu identificatori;
2. Panou web local, care vizualizează direct fișierele de sesiune fără bază de date intermediară. Panoul, dezvoltat propriu, expune metrici care sunt absente în produsele comerciale studiate: consum de tokeni pe categorii de evenimente, evoluția dimensiunii contextului raportată la limita modelului, optimizările aduse de subagenți, latența uneltelor, debitul de generare, costul estimat per apel pe baza unui tabel de prețuri configurabil și istoricul evenimentelor de compactare cu diferențele aferente.

### 1.7.7 Cercetarea tehnicilor de utilizare optimă pentru arhitecturi locale în dezvoltarea software

Este necesară construcția unor suite de evaluare proprii pentru extragerea de concluzii în privința utilizării sistemelor inteligente în dezvoltarea software. Astfel, acestea trebuie să abordeze câteva arii de interes:

1. Influența lungimii și specificității instrucțiunilor de sistem asupra performanței, prin agentul rulat în sarcina de revizuire de PR pe 50 de revizuiți umane ideale extrase din 7 proiecte OSS, cu metrici de precizie, recunoaștere și  $F_1$ ;

2. Interacțiunea dintre stadiile procesului de compactare, observând efectele asupra integrității contextului și recunoașterii;
3. Performanța modelelor în sarcini de dezvoltare software prin compararea modelelor (proprietary și open-source) ce rulează pe un proiect real, cu notare hibridă (similaritate semantică, *LLM-as-judge*, teste deterministe ascunse).

### 1.7.8 Sinteza recomandărilor extrase din experimente

Pe baza experimentelor prezentate anterior, vom sintetiza un set de recomandări și observații practice legate de utilizarea aplicației în contexte reale de dezvoltare software:

1. Alegerea modelului optim pentru o anumită sarcină;
2. Performanțele modelelor sub 20 de miliarde de parametri în medii cu apel intensiv de unelte;
3. Preferințele modelelor pentru limba de intrare, sau pentru anumite limbaje de programare;
4. Configurarea procesului de compactare în funcție de sarcină;
5. Utilizarea modelelor open-source ca alternativă viabilă când confidențialitatea sau costul sunt constrângeri principale.

Detalierea fiecărei recomandări, ancorată în date, se găsește în capitolul de rezultate experimentale.

## Capitolul 2

# IMPLEMENTARE

### 2.1 Mediul de dezvoltare

Cerințele hardware pentru dezvoltarea și rularea acestei aplicații sunt minime: pentru modelele din cloud este suficient un laptop ce poate rula Python 3 cu conexiune la internet, întrucât inferența nu este realizată local. Pentru rularea unui model local, cerințele cresc semnificativ, depinzând de modelul ales.

| Model                | Parametri | GPU Recomandat             | Video Random Access Memory (VRAM) | Random Access Memory (RAM) |
|----------------------|-----------|----------------------------|-----------------------------------|----------------------------|
| Llama 3.2 (3B)       | 3.21B     | NVIDIA RTX 3060 8GB        | 2–7 GB (Q4 vs FP16)               | 8–16 GB                    |
| Qwen2.5-Coder (7B)   | 7.62B     | NVIDIA RTX 3060 / RTX 4070 | 5–15 GB (Q4 vs FP16)              | 16 GB                      |
| Devstral Small (24B) | 23.6B     | NVIDIA RTX 4090 24GB       | 14–48 GB (Q4 vs FP16)             | 32 GB                      |
| Gemma 4 (26B)        | 25.2B     | NVIDIA RTX 4090 / RTX 5090 | 18–50 GB (Q4 vs FP16)             | 32–64 GB                   |
| Qwen3-Coder (30B)    | 30.5B     | NVIDIA RTX 4090 / RTX 5090 | 19–61 GB (Q4 vs FP16)             | 32–64 GB                   |

Tabelul 2.1: Hardware recomandat pentru modelele open-source evaluate local (Sursa [20]).

Pentru dezvoltare am folosit un laptop MacBook Pro cu chip Apple M5 Max și 48 GB RAM, pe macOS Tahoe 26.4.1 cu Python 3.10.

### 2.2 Structura aplicației

Am structurat aplicația în mai multe directoare specializate:

- Directorul `src` conține logica agentului: graful LangGraph, uneltele, sistemul de permisiuni, clientul MCP, procesul de compactare, subagenții, mediul de execuție Docker și urmărirea execuției;
- Directorul `ui` conține interfața cu utilizatorul: bucla principală, comenzile slash, sistemul de capabilități, gestionarea sesiunilor și a preferințelor;
- Directorul `web` conține panoul analitic care vizualizează, printre altele, sesiunile, contextul, consumul de tokeni și activitatea uneltelor;

- Directorul `eval` conține suita de evaluări cu metrici precum Bilingual Evaluation Understudy (BLEU), Recall-Oriented Understudy for Gisting Evaluation (ROUGE) și potrivire semantică.
- Documentația detaliată per subsistem se găsește în `docs`, iar configurarea implicită în `config/config.yml`.

Vederea de ansamblu asupra modulelor aplicației și a interacțiunilor dintre ele este prezentată în figura 2.1.

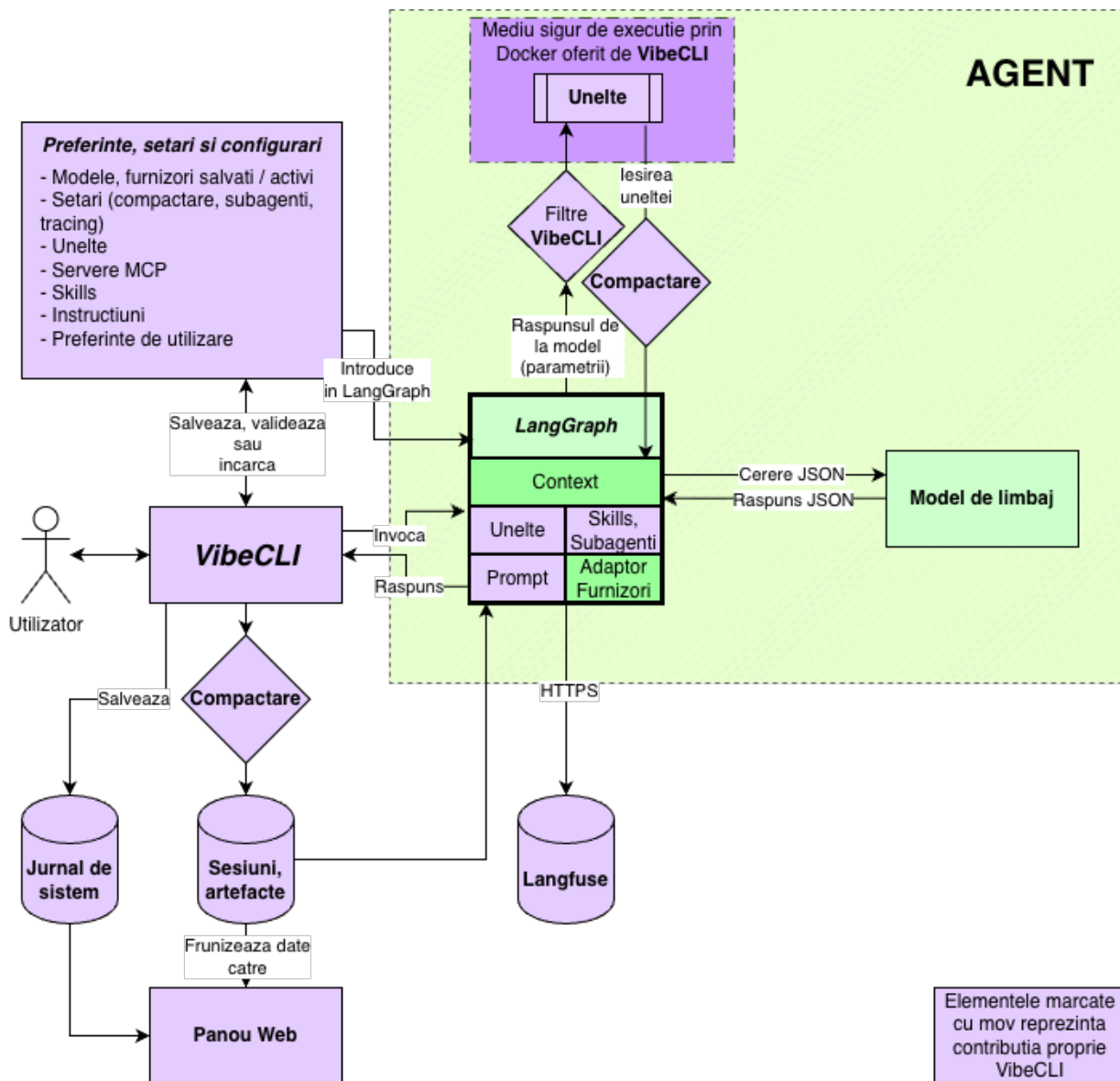


Figura 2.1: Arhitectura modulară a aplicației `vibe-cli`

|                     |                    |                       |
|---------------------|--------------------|-----------------------|
| vibe-cli/           | src/               | config/               |
| - config/           | - main.py          | - config.yml          |
| - docs/             | - tools.py         | \- eval.yml           |
| - eval/             | - tool_guards.py   |                       |
| - examples/         | - providers.py     | ui/                   |
| - scripts/          | - permissions.py   | - app.py              |
| - src/              | - sandbox.py       | - ui.py               |
| - tests/            | - mcp_client.py    | - commands.py         |
| - ui/               | - artifacts.py     | - skills.py           |
| - web/              | - attachments.py   | - sessions.py         |
| - ARCHITECTURE.md   | - capabilities.py  | - preferences.py      |
| - FIRST_START.md    | - clipboard.py     | - provider_handler.py |
| - HEADLESS_USAGE.md | - log_context.py   | - pastes.py           |
| - Makefile          | - logging_setup.py | - paths.py            |
| - pyproject.toml    | - multimodal.py    | - profile.py          |
| - README.md         | - tracing.py       | - config_init.py      |
| - UPDATING.md       | - compaction/      | - vibe_md.py          |
| \- uv.lock          | \- subagents/      | \- updater.py         |

Figura 2.2: Structura directoarelor vitale ale proiectului

## 2.3 Prezentarea procesului de configurare

Configurarea aplicației se face prin intermediul fișierului `config.yml` care conține catalogul de furnizori (Ollama, OpenAI, Claude, Amazon Bedrock [88], Gemini), modelele disponibile pentru fiecare, procesul de compactare, setările de urmărire a execuției, configurația mediului de execuție sau a subagenților și a uneltelor. Cheile API sunt declarate ca variabile de mediu sau în fișierul de preferințe și sunt încărcate la pornire. Dacă nici o cheie nu este setată și Ollama nu este accesibil, aplicația solicită interactiv introducerea unei chei la pornire prin comanda `/model` sau `/key`.

De asemenea pentru procesul de evaluare al capabilităților folosit pentru teste de sanitate și regresie, se pot modifica setările din fișierul `~/vibe-cli/eval/eval.yml`

Preferințele utilizatorului (furnizorul implicit, modelul selectat per furnizor, nivelul de efort, serve-rele MCP, listele de permisiuni) sunt persistate în `~/vibe-cli/preferences.json` și supraviețuiesc între sesiuni. Sesiunile de conversație sunt salvate ca jurnale de evenimente în `~/vibe-cli/sessions/`, sortate per proiect (directorul de lucru).

## 2.4 Interfața cu utilizatorul

Interfața este mediul de legătură între utilizator și agent, prin care acesta poate consulta starea aplicației și totodată spațiu de configurare pentru diferitele componente ce o alcătuiesc.

Primul prototip a folosit librăria Textual, dar pe măsură ce conversația se dezvoltă, mișcarea ferestrei se comporta inconsistent iar compatibilitatea cu funcțiile de copiere și lipire lăsa de dorit. Soluții similare (Claude Code, Codex) nu folosesc un TUI propriu-zis, ci scriu în terminal prin Standard Output (STDOUT), lăsând derularea nativă să gestioneze istoricul. Am abandonat astfel Textual în favoarea unei interfețe bazate pe **prompt\_toolkit** și **Rich**, obținând o experiență fluidă, fără probleme de derulare sau latență.

Astfel, interfața a fost regândită să rămână discretă, dar funcțională. Întreaga stare a sesiunii, consistând printre altele din furnizorul și modelul activ, nivelul de efort, conexiunile MCP, gradul de ocupare al contextului și directorul de lucru, este condensată într-o bară de subsol mereu prezentă. De asemenea, textul de mari dimensiuni ce este lipit în bara de instrucțiuni nu ocupă tot ecranul, ci este înlocuit printr-un marcaj specific, iar o cale de fișier lipită este recunoscută, fișierul fiind transformat automat în atașament, scutind agentul de o căutare în plus.

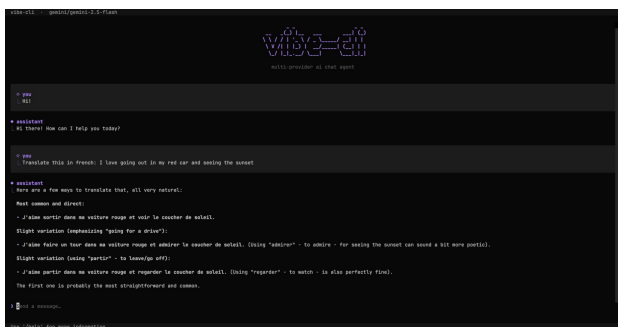


Figura 2.3: Prima versiune a interfeței, realizată cu ajutorul bibliotecii Textual

Schimbarea s-a produs încă de la primele prototipuri, fără a întârzia implementarea, iar soluția finală este estetică, simplă și performantă.

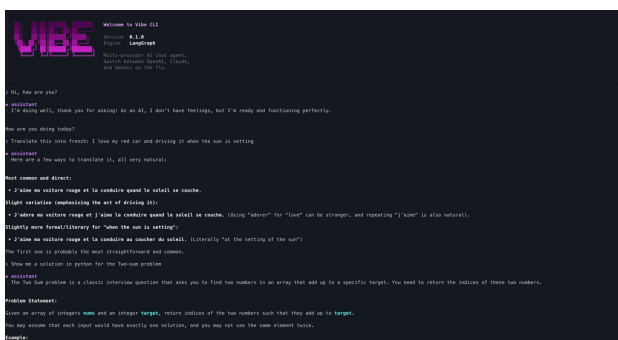


Figura 2.5: A doua versiune a interfeței, realizată fără biblioteca Textual

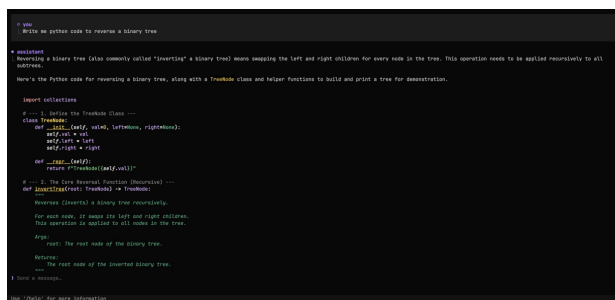


Figura 2.4: Continuarea conversației cu prima versiune a interfeței, realizată cu ajutorul bibliotecii Textual

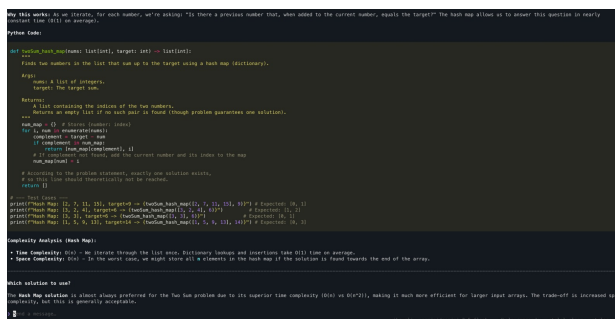


Figura 2.6: Continuarea conversației cu a doua versiune a interfeței, realizată fără biblioteca Textual

Interfața oferă mai multe elemente vizuale menite să informeze utilizatorul în timp real:

- **Indicator de activitate animat cu starea curentă și cronometru** (de exemplu  $\diamond$  thinking... 1.2s).
- **Jurnal persistent de unelte:** fiecare apel lasă o linie  $\hookrightarrow$  Tool(arg), astfel încât execuția uneltelor este vizibilă.
- **Panouri de diferențe pentru write\_file**, redat unificat cu numărul de linii adăugate/șterse.
- **Împachetare lipire:** textele lipite mai lungi de 200 de caractere sunt înlocuite cu un substituent [Paste #N <len> chars].
- **Câmp de stare:** durata interacțiunii, setări active, atașamente, informații despre context.
- **Auto-completare** pentru comenzile slash și actualizare automată a versiunii.

```

> Hi, write me a poem in french.

+ assistant
  | Voici un poème en français pour vous :

-----

L'Écho du Soir

Le vent murmure aux feuilles tombantes, La lune dore les collines lointaines. Les étoiles dansent au-dessus des champs silencieux, Comme si le ciel répondait à nos prières muettes.
Je marche seul sur ce chemin de terre battue, Pensant à toi qui es déjà bien loin. Ton rire résonne encore en mon cœur fidèle, Un souvenir doux que je porte sans trépidation.
Les saisons passent comme des vagues d'automne, Emportant avec elles nos rêves d'enfance. Mais l'amour reste ancré dans le temps qui court, Comme une rivière pure et ininterrompue.
L'écho du soir nous réunit encore, Dans la danse éternelle des ombres et de la lumière.

+ Sorted in 3.7s

> Send a message...

ollama/qwen3.5 effort:medium mcp:0/0 trace:langfuse:vibe-code
~/stuff/vibe-cli

```

Figura 2.7: Exemplu de conversație în interfața vibe-cli

```

Edited: poem.md +19 -0
@@ -0,0 +1,19 @@
+# The Whispering Willow
+
+## Verse 1
++Beneath the willow's silver bough,
++The moonlight dances soft and slow,
++Where shadows play in gentle flow,
++And quiet secrets softly grow.
+
+## Verse 2
++The river winds through ancient stones,
++Reflecting dreams of skies above,
++While time has carved its silent throne,
++In moments we can never love.
+
+## Verse 3
++But hope remains where light breaks through,
++Like morning dew on petals new,
++A promise kept by stars and you,
++That beauty finds a way to anew.

```

(a) Crearea unui fișier nou prin interfața vibe-cli

```

Edited: poem.md +4 -4
@@ -8,8 +8,8 @@
## Verse 2
-The river winds through ancient stones,
-Reflecting dreams of skies above,
-While time has carved its silent throne,
-In moments we can never love.
+The river flows with gentle grace,
+Through valleys deep in quiet place,
+Where ancient trees their roots embrace,
+And nature's song begins its chase.

## Verse 3

```

(b) Editarea unui fișier existent prin interfața vibe-cli, cu afișarea diferențelor

## 2.5 Funcționalități de nivel înalt

Funcționalitățile principale ale aplicației includ:

- Unelte inteligente de interacțiune (citire/editare fișiere, căutare web, rulare în terminal);
- Controlul fluxului logic de apel al uneltelor (de exemplu, `edit_file` nu poate fi apelat fără un `read_file` prealabil; citirea de fișiere mari este partiționată pentru a nu umple contextul);
- Alegerea modelului și a efortului de gândire;
- Suport multi-modal (4 tipuri: imagini, PDF, text, documente) prin cale locală sau Uniform Resource Locator (URL), când modelul suportă tipul de date (vezi tabelul 3.5);
- Gestionarea sesiunilor per-proiect, cu salvare, ștergere, listare și continuare.
- Sistem de permisiuni per-unealtă cu liste de preferințe persistente;
- Conectarea cu servere MCP prin Standard Input-Output (STDIO), Hyper-Text Transfer Protocol (HTTP) sau Server-Side Events (SSE);
- Capabilități bazate pe fișiere Markdown cu încărcare progresivă;
- Subagenți specializați extensibili (general, code search, web research), executați în pa-



- ralel cu modele configurabile per-agent și recursivitate;
- Proces configurabil de compactare a contextului (trunchiere, sumarizare, deduplicare).
- Mediu de execuție Docker opțional pentru rularea izolată a comenzilor.
- Urmărire a execuției prin Langfuse Cloud și panou web pentru observabilitate.

## 2.6 Instalare și utilizare

Această secțiune descrie cerințele sistemului, pașii de instalare, configurarea inițială, precum și modul de utilizare a aplicației.

### 2.6.1 Cerințe preliminare

Aplicația necesită Python 3.10 sau o versiune ulterioară și este disponibilă pe sistemele de operare Linux, macOS și cu suport limitat pentru Windows. Pentru accesul la modelele de limbaj este necesar fie un server Ollama local pornit prin `ollama serve`, fie cel puțin o cheie API validă pentru unul dintre furnizorii cloud. În lipsa oricărei configurații prealabile, aplicația oferă la pornire un flux interactiv pentru selecția furnizorului și introducerea cheii.

### 2.6.2 Instrucțiuni de instalare

Metoda recomandată este executarea unei simple comenzi de terminal care obține aplicația din proiectul găzduit pe GitHub și o instalează:

```
curl -fsSL https://raw.githubusercontent.com/Moshulika/vibe-cli/main/scripts/install.sh | bash
```

Alternativ, instalarea se poate efectua manual prin `uv tool install` sau `pipx install`, indicând proiectul GitHub ca sursă. Dezinstalarea se face prin `uv tool uninstall vibe-cli`. Pentru dezvoltare, după clonarea proiectului, comanda `make dev-setup` instalează dependențele și interceptoarele `pre-commit`, iar `make ui` lansează interfața de chat. Sunt expuse de asemenea ținte `make` pentru rularea suitei de teste (`make test`), formatarea codului (`make format`) și pornirea panoului analitic web (`make web`).

### 2.6.3 Configurare la prima rulare

După instalare, comanda `vibe` este disponibilă global din orice director. La prima pornire, aplicația creează arborele de configurație sub `~/.vibe-cli/`, ce conține fișierul de preferințe `preferences.json` (ce conține printre altele: furnizorul implicit, modelul selectat per furnizor, nivelul de efort, cheile API și permisiunile per unealtă), un identificator anonim `profile.json`, directorul cu **3** capabilități exemplificative și jurnale structurate. Furnizorii configurați anterior sunt validați automat printr-un apel către punctul de acces al fiecăruia, fără a consuma tokeni; cheile invalide sunt șterse și semnalate utilizatorului. În absența unei configurații valide, un flux interactiv solicită selecția furnizorului și introducerea cheii API (cu mascarea intrării), validând-o înainte de salvare.

### 2.6.4 Utilizarea aplicației

Aplicația suportă **2** moduri de operare: sesiunea interactivă TUI, descrisă în continuare, și modul neinteractiv, prezentat în secțiunea următoare. După configurare, utilizatorul interacționează cu modelul printr-o sesiune persistentă de chat, salvată automat în evenimente granulare sub directorul corespunzător proiectului curent. Cele **22** comenzi disponibile suportă auto-completare. Sunt disponibile și scurtături de tastatură precum `CTRL+L` pentru a goli conversația și `CTRL+C` pentru închiderea aplicației

cu salvarea automată a sesiunii. Pe lângă intrarea textuală, utilizatorul poate utiliza și alte tipuri de date, atunci când modelul dispune de capabilități multi-modale. Panoul web analitic poate fi pornit separat prin comanda **vibe web** și oferă vizualizări pentru consumul de tokeni, costuri, latență, evenimente de compactare și activitatea subagenților.

| Comanda      | Descriere                                                                               |
|--------------|-----------------------------------------------------------------------------------------|
| /help        | Listarea tuturor comenzilor și a scurtăturilor de tastatură                             |
| /model       | Selectarea furnizorului și modelului, sau setarea cheii API                             |
| /key         | Setarea sau actualizarea cheii API per furnizor                                         |
| /effort      | Selectarea nivelului de efort al modelului                                              |
| /mcp         | Configurarea serverelor MCP (adăugare/ștergere/activare)                                |
| /skills      | Listarea, activarea sau reîncărcarea capabilităților                                    |
| /subagents   | Listarea tipurilor de subagenți și configurarea modelelor per tip                       |
| /plan        | Activarea/dezactivarea modului de planificare                                           |
| /agent       | Activarea/dezactivarea modului agentic                                                  |
| /permissions | Vizualizarea și editarea listelor de permisiuni per unealtă                             |
| /clear       | Ștergerea conversației curente și schimbarea la o sesiune nouă                          |
| /sessions    | Listarea sesiunilor salvate pentru proiectul curent                                     |
| /resume      | Continuarea ultimei sesiuni sau a uneia specifice prin identificator                    |
| /compaction  | Inspectarea procesului de compactare și selectarea modelului                            |
| /compact     | Forțarea unei compactări imediate a contextului                                         |
| /tracing     | Configurarea și activarea urmăririi execuției Langfuse                                  |
| /score       | Evaluarea ultimei interacțiuni pentru Langfuse (good/bad)                               |
| /attach      | Atașarea datelor multi-modale de tip imagini, PDF sau fișiere text ca și cale de sistem |
| /paste       | Atașarea datelor multi-modale de tip imagini, PDF sau fișiere text din clipboard        |
| /dataset     | Adăugarea ultimei interacțiuni la un set de date Langfuse                               |
| /update      | Actualizarea la ultima versiune disponibilă în GitHub                                   |
| /exit        | Salvarea conversației și închiderea aplicației                                          |

Tabelul 2.2: Comenzi disponibile în interfață

## 2.6.5 Modul neinteractiv

Pe lângă sesiunea interactivă, aplicația expune un mod *headless*, neinteractiv, activat prin parametrul **-p/-prompt** (cu suport pentru citirea promptului din **stdin** prin **vibe -p -**). Acesta execută o singură tură întrebare-răspuns (un apel către model urmat de buclă de procesare a uneltelor) și tipărește rezultatul final, în format text sau Java Script Object Notation (JSON) (**-o text|json**), după care returnează un cod conform convențiilor: 0 la succes, 1 la eroare de execuție, 2 la argumente invalide și 130 la întreruperea prin **SIGINT**. Modul este *hermetic* în mod implicit: dezactivează MCP, salvarea sesiunii, tracing-ul și interacțiunile cu utilizatorul, cu opțiunea de a salva sesiunea prin **-save-session**. Autentificarea se realizează automat prin detectarea variabilelor de mediu corespunzătoare, scurtcircuitată de fanionul **-provider**.

Sunt expuse de asemenea opțiuni pentru controlul fin al execuției: selecția modelului (**-model**), reglarea efortului de gândire prin **-effort**, modificarea promptului de sistem (**-system** pentru înlocuire, exclusiv cu **-append-system-prompt** pentru extindere), restricționarea uneltelor disponibile (**-allowedTools**) și limitarea bugetului de iterații (**-max-turns**). Ieșirea **json** include furnizorul, modelul, răspunsul final, lista apelurilor de unelte, capabilitățile utilizate, statistici de consum, numărul de iterații și durata execuției, facilitând integrarea în procese automatizate de tip CI/CD sau cu aplicații de tip wrapper <sup>1</sup>.

<sup>1</sup>O aplicație de tip wrapper este o aplicație care înglobează o altă aplicație și folosește ieșirea acesteia pentru

## 2.7 Gestionarea contextului și a memoriei

Aplicațiile ce utilizează principii agentice au nevoie de o logică complexă pentru gestionarea contextului și a memoriei din două motive diferite: personalizare și consistența conversației.

### 2.7.1 Memoria și istoricul conversației

Aplicațiile concurente folosesc un tipar comun pentru memoria persistentă: un folder ascuns per proiect (`.claude`, `.copilot`, `.codex`) cu preferințe locale, un folder global pentru preferințe ce țin de utilizator, și un istoric al conversațiilor (compactat și integral) per proiect cu identificator unic, cel compactat fiind reintrodus integral în context la reluarea conversației. Memoria se separă în două straturi: *memoria contextuală* (limitată de fereastra de context a modelului, necesitând compactare pentru a fi conformă cu specificațiile modelului) și *memoria integrală* (stocată separat, păstrând istoricul textual și artefactele neafectate).

În implementare am preluat aceste convenții și le-am adaptat: un singur fișier de sesiune cu două câmpuri pentru fiecare eveniment (integral și compactat), preferințe locale și globale completate progresiv pe baza interacțiunilor de către modelul de limbaj (pentru personalizarea răspunsurilor și a acțiunilor pe placul utilizatorului, date extrase din conversațiile cu acesta), și stocare exclusiv locală pentru păstrarea intimității.

### 2.7.2 Contextul

Contextul este unul dintre punctele de interes pentru dezvoltarea sistemelor agentice datorită implicațiilor pe care le are în executarea sarcinilor complexe și de lungă durată sau cu cantități mari de date. O practică curentă în industrie este aceea de a folosi doar prima parte a ferestrei de context a agentului pentru a beneficia de performanțe maxime ale modelului. În cazul depășirii acestui prag, se recomandă uzual începerea unei noi conversații cu un context curat.

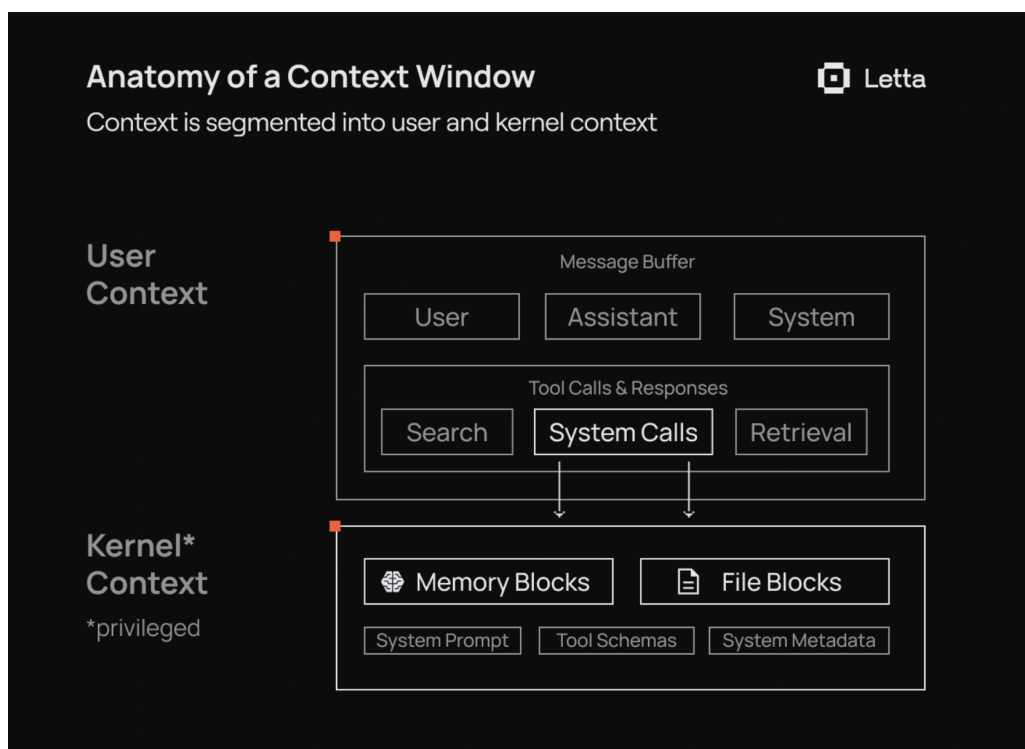


Figura 2.9: Împărțirea ferestrei de context (Sursă: [19])

Mai multe elemente împart fereastra de context concomitent:

- Instrucțiunile de sistem: instrucțiuni pentru comportamentul agentului. Conține printre altele rolul pe care acesta îl joacă, restricții comportamentale, exemple de răspunsuri și capacități disponibile.
- Mesajele utilizatorului: cererea pe care utilizatorul o trimite către agent.
- Răspunsurile asistentului: răspunsul oferit către utilizator la cererea acestuia. Întotdeauna contextul declarat (spre ex. 200.000 tokeni) va avea mărimea adevărată  $C = C_{\max} - O_{\max}$ , unde  $C$  este contextul adevărat,  $C_{\max}$  contextul declarat, și  $O_{\max}$  numărul maxim de tokeni de ieșire suportați de model. Din această valoare se scad și alte valori rezervate de sistem, precum spațiul tampon pentru compactare.
- Atașamente: documente, imagini, audio și videoclipuri
- Capabilitățile sau alte instrucțiuni: descrierea și titlul tuturor rutinelor salvate care pot fi folosite de către agent sunt injectate în context pentru ca agentul să poată alege în cunoștință de cauză o rutină din cele disponibile.
- Unelte: similar capacităților, semnătura uneltelor, descrierea acestora, datele de ieșire și instrucțiunile lor sunt disponibile modelului prin injectarea în context. Se aplică atât uneltelor locale sau celor expuse prin MCP.
- Ieșirea uneltelor: datele de ieșire ale uneltelor, precum conținutul unui fișier rezultat din utilizarea unei unelte pentru citirea fișierelor pot fi injectate direct în context. În cazul fișierelor mari, acest lucru poate însemna o problemă pentru fereastra de context.
- Căutări pe internet: căutările pe internet returnează răspunsuri JSON sau Hyper-Text Markup Language (HTML) care uzual nu conțin strict date punctuale, ci întregul conținut al paginii. Acest lucru ajută pentru poziționarea elementelor în pagină dar adaugă multe caractere ce umplu fereastra de context.

### 2.7.3 Retrieval Augmented Generation (RAG)

Pentru conservarea contextului în lucrul cu documente de dimensiuni mari sau cu colecții de documente, a fost adoptată tehnica RAG [89]. Acesta se bazează pe un proces independent care ingestează documentele de interes prin vectorizare și partiționare semantică. În acest mod, agentul poate căuta date cu ușurință ajungând să extragă dintr-un document de dimensiuni mari doar bucățile relevante.

Dezavantajul acestei tehnici este complexitatea infrastructurii necesare. În primul rând, este nevoie de un serviciu separat care să proceseze și să stocheze aceste date, iar în final modelul trebuie să poată accesa aceste date într-un mod eficient și sigur.

Odată cu progresele aduse mărimii ferestrei de context, modelele de vârf ajung să aibă până la unul sau două milioane de tokeni ca fereastră de context. Astfel, nevoia pentru RAG scade. Pentru majoritatea întrebunțărilor, această mărime a ferestrei este mai mult decât suficientă, chiar și pentru conversații mai complexe, în ciuda limitărilor cunoscute precum scăderea performanțelor pe date ce se află în mijlocul ferestrei de context [15].

În implementarea actuală *nu am folosit tehnici RAG*, atât din cauza complexității de implementare și a infrastructurii necesare, cât și a beneficiilor estompate de performanțele curente ale modelelor de limbaj cu ferestre mari de context.

### 2.7.4 Gestionarea datelor multi-modale

Dată fiind alegerea de abandonare a tehnicii RAG, în implementare avem nevoie de o modalitate inteligentă de a prelucra datele multi-modale pentru conservarea contextului. Arhitectura acestei capacități poate fi consultată în figura 2.10.

Prima problemă în lucrul cu date multi-modale apare din cauza suportului pentru o plajă foarte largă de modele de limbaj. Din cauza arhitecturii agnostice de furnizor care permite folosirea oricărui model, avem nevoie să detectăm capabilitățile de prelucrare a tipurilor de fișiere pentru fiecare model în parte. Din păcate, furnizorii de modele sau creatorii care le includ în Ollama, nu specifică nicăieri consistent în metadatele modelului dacă acesta prezintă capabilități pentru lucrul cu anumite tipuri de fișiere.

Din acest motiv, am avut de ales între două soluții de implementare:

- Punerea utilizatorului în ipostaza în care trebuie să se documenteze și să declare în configurație capabilitățile modelului pe care dorește să îl folosească;
- Încercarea automată a trimiterii de date multi-modale către model, iar în cazul în care acesta refuză datele, să bifăm în fișierul de preferințe faptul că acest model nu suportă respectivul tip de date pentru utilizări viitoare, anunțând utilizatorul fără a avea de a face cu erori.

Din motive legate de experiența utilizatorului am ales să continuăm cu a doua soluție.

După decizia legată de detectarea capabilităților am trecut la gestionarea contextului pentru aceste date. Am proiectat o abordare hibridă, compusă din două ramuri pe care agentul poate alege să le apeleze. În prima fază, pentru fișierele de tip text sau PDF, agentul poate citi fișierele în format pur textual folosindu-se de unelte uzuale de citit fișiere, beneficiind de optimizările acestora, precum folosirea punctuală în zone de interes sau trunchierea datelor de ieșire. Dacă modelul consideră că are nevoie de o vedere completă, atunci când suportă fișierul PDF ca tip de date pentru intrare, se poate uita direct pe întregul fișier prin mai multe citiri succesive de maxim 20 de pagini (configurabil). Imaginile sunt introduse direct în context dacă modelul suportă acest tip de date și sunt limitate la o dimensiune de 10 MB.

Compactarea acestor date este realizată prin metode ce rulează imediat după folosirea uneltelor (trunchiere, referențiere sau sumarizare), fișierele fiind stocate automat ca artefacte recuperabile.

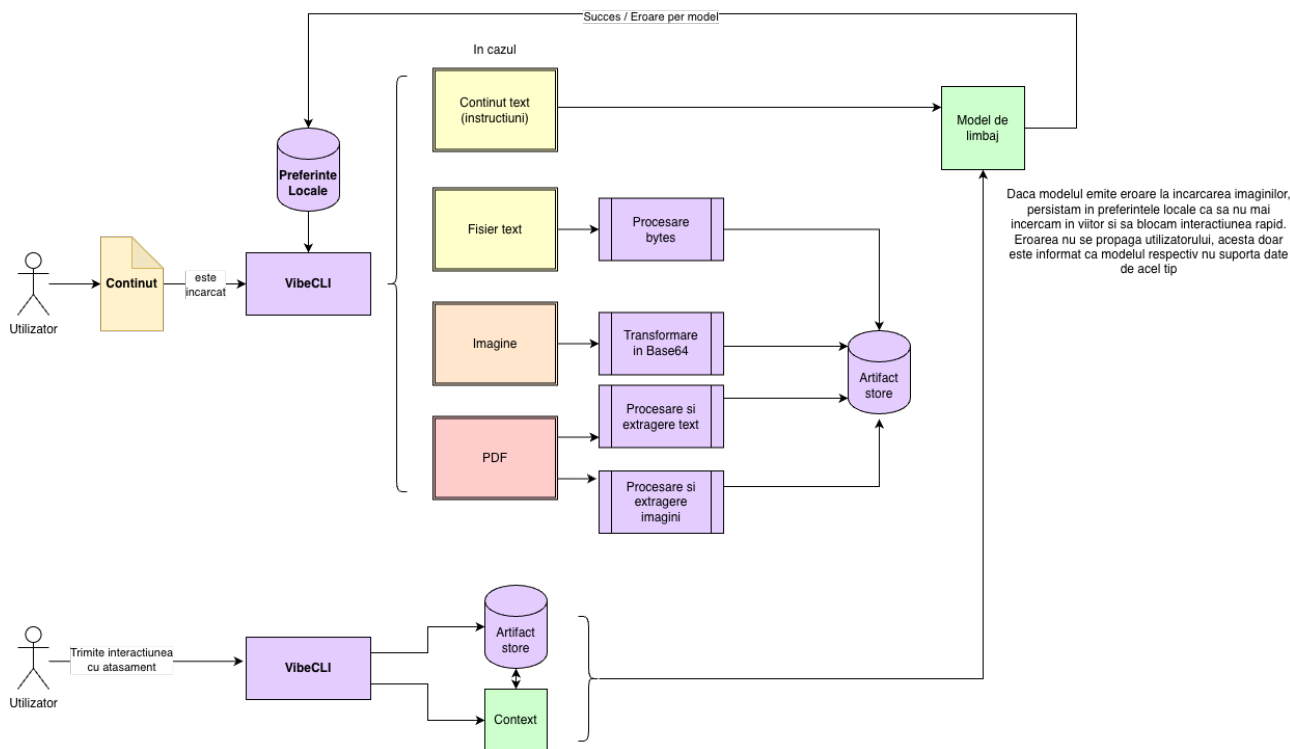


Figura 2.10: Fluxul de prelucrare a datelor multi-modale în `vibe-cli`

## 2.7.5 Compactarea

Compactarea conversației este un cumul de tehnici ce asigură compatibilitatea conversației curente cu restricțiile de intrare ale modelului. Aceasta este folosită predominant în sistemele agentice, deoarece aici se pot genera cantități mari de date ce epuizează rapid fereastra de context.

La produsele existente în piață, compactarea are loc în mod automat când conversația atinge în jur de 80% din mărimea maximă a ferestrei de context pentru modelul respectiv (figura 2.11). Acest lucru oferă o margine de eroare pentru a evita pornirea procesului de compactare până nu se termină interacțiunea curentă <sup>2</sup>.

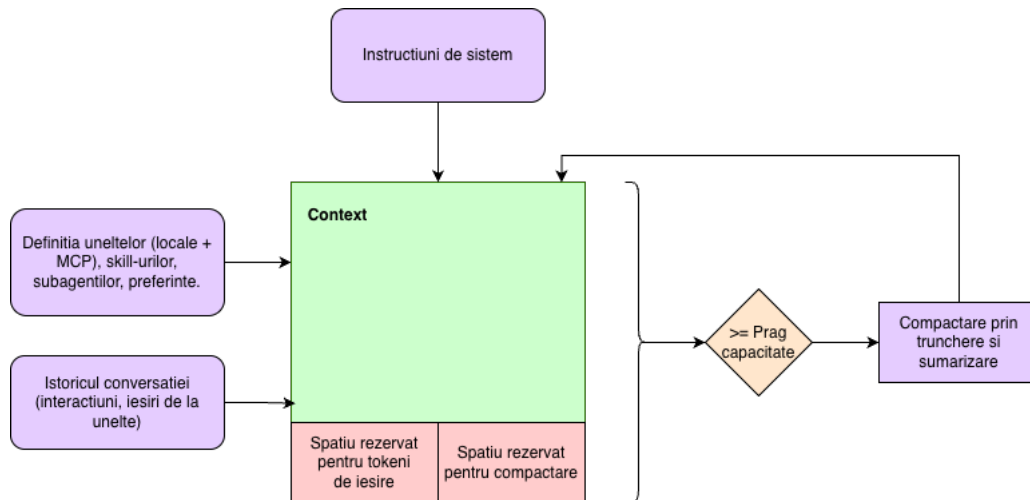


Figura 2.11: Schema componentelor care alcătuiesc fereastra de context și compactarea la prag

Tehnicile uzuale se împart în transformări locale pe ieșirea uneltelor (formatare deterministă, trunchiere cap-coadă, sumarizare prin LLM, substituția cu referințe în artefacte externe) și transformări globale pe întreaga conversație (deduplicare prin scor de similaritate, eliminare de mesaje în funcție de tip, sumarizarea sau trunchierea integrală a istoricului). Primele se execută imediat după rularea uneltei (figura 2.12), ultimele doar la atingerea pragului de compactare (figura 2.11).

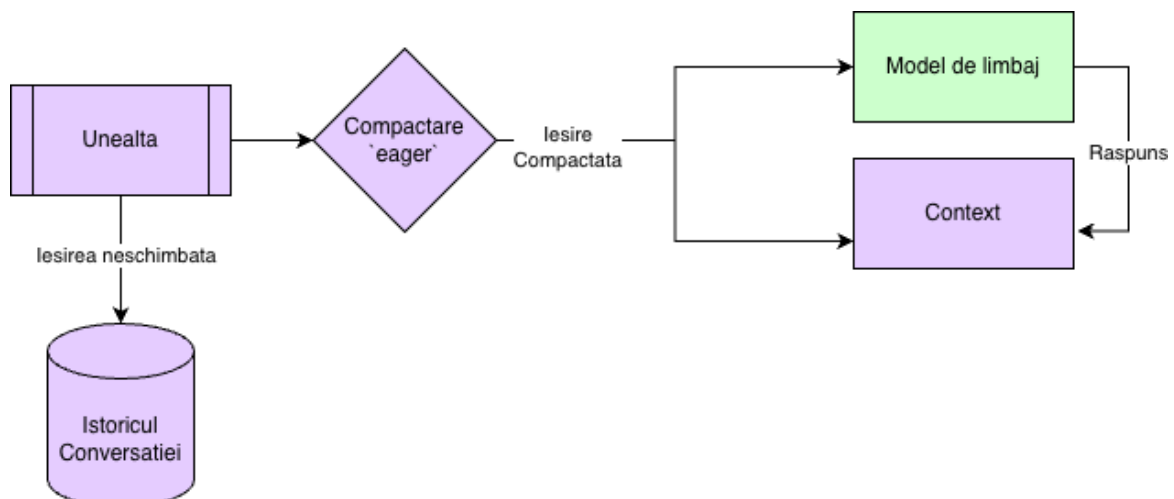


Figura 2.12: Compactarea după apelul uneltei

<sup>2</sup>O interacțiune reprezintă întrebarea utilizatorului și toate procesele pe care le execută agentul până la producerea răspunsului final de către modelul de limbaj

Spre deosebire de produsele existente, în care aceste setări sunt fixe, am ales **configurabilitatea completă** pentru studierea efectului fiecărei strategii asupra coerenței agentului; configurarea completă este furnizată în anexa 4.4.

Procesul de compactare este pornit la atingerea valorii de prag sau după fiecare apel de unealtă, iar pașii activați execută succesiv transformări locale sau globale; pașii ce necesită LLM pot folosi modelul principal sau unul mai mic. Cele 7 moduri expuse sunt: `tool_truncate` (trunchiere cu `max_tool_tokeni`, capete configurabile), `tool_summarize` (sumarizare în loc de trunchiere), `format_compress` (formatare per unealtă cu prag), `ref_substitute` (extragere în artefact extern peste un prag de tokeni), `dedupe` (prag de similaritate), `importance_prune` (liste de tipuri păstrate/șterse) și `bulk` (sumarizare globală cu mai multe strategii). Stadiile sunt fie *eager*, fie declanșate la prag, iar unele perechi sunt incompatibile.

### 2.7.6 Tehnici pentru eficientizarea ferestrei de context

Pe lângă compactare, două tehnici complementare reduc consumul de context: utilizarea inteligentă a uneltelor (comenzi terminal cu `head/tail`, sau căutări punctuale prin `grep/find`) și pornirea de subagenți care preiau cercetarea sau executarea unor sub-sarcini și returnează doar un rezumat, păstrând curat contextul orchestratorului. Ambele tehnici sunt folosite în implementare și detaliate în secțiunea următoare.

## 2.8 Implementarea componentelor arhitecturilor agentice moderne

Pentru a avea un flux de lucru agentic, modelele de limbaj sunt adaptate pentru a urma instrucțiuni ce îl ghidează în a folosi diverse instrumente, capabilități predefinite sau pentru interacțiunea cu o echipă de agenți. În această secțiune vom discuta implementarea acestor capabilități consacrate în cadrul de lucru agentic.

### 2.8.1 Uneltele

**Inima arhitecturii agentice este reprezentată de uneltele** pe care modelele de limbaj învață să le utilizeze. Acestea nu reprezintă nimic mai mult decât metode uzuale ale limbajului de programare cu precizarea că modelul de limbaj are acces la semnătura acestora (gruparea de nume al funcției, parametrii de intrare, tip de date returnate) împreună cu o descriere detaliată de utilizare, un exemplu fiind prezent în figura 2.13.

```

1 @tool
2 def list_directory(path: str = ".", show_hidden: bool = False) -> str:
3     """List files and directories at the given path.
4     Args:
5         path: Directory path to list (default: current directory).
6         show_hidden: Whether to include hidden files (default: false).
7     """
8     try:
9         p = Path(path).expanduser().resolve()
10        if not p.exists():
11            return f"Directory not found: {path}"
12        if not p.is_dir():
13            return f"Not a directory: {path}"
14        entries = sorted(p.iterdir(), key=lambda e: (not e.is_dir(), e.name.lower()))
15        lines = [f"Contents of {p}:", ""]
16        for entry in entries:
17            if not show_hidden and entry.name.startswith("."):
18                continue
19            prefix = "[Folder] " if entry.is_dir() else "[File] "
20            size = ""
21            if entry.is_file():
22                s = entry.stat().st_size
23                if s < 1024:
24                    size = f" ({s}B)"
25                elif s < 1_048_576:
26                    size = f" ({s / 1024:.1f}KB)"
27                else:
28                    size = f" ({s / 1_048_576:.1f}MB)"
29            lines.append(f" {prefix}{entry.name}{size}")
30        if len(lines) == 2:
31            lines.append(" (empty)")
32        return "\n".join(lines)
33    except PermissionError:
34        return f"Permission denied: {path}"
35    except Exception as e:
36        return f"Error listing directory: {e}"

```

Figura 2.13: Implementarea unei funcții pe care agentul o folosește pentru a lista directoriile și fișierele la o anumită cale pe sistem

Aceste unelte sunt alese cu grijă pentru a menține un echilibru între numărul total de unelte și suita de capacități necesare agentului.

*Numărul de unelte afectează direct modul de gândire al modelului* și performanța în execuția sarcinilor complexe. Cu prea puține, acesta poate întâmpina probleme în execuția problemelor cu adevărat sofisticate deoarece îi poate lipsi o capacitate necesară. Pe de altă parte, dacă are prea multe acesta nu se va putea decide definitiv asupra unei singure unelte, iar în lipsa unei descrieri detaliate crește riscul de folosire greșită sau inefficientă.

De asemenea, pentru o eficiență sporită, scopul acestora nu trebuie să se suprapună. Unele trebuie să fie unice și să aibă un scop clar. Mai mult, este recomandată exemplificarea cazurilor în care o unealtă ar trebui folosită, evitată sau preferată în detrimentul alteia. Câteva dintre capacitățile utilizate frecvent de agenți sunt:

- Listarea directoarelor și fișierelor
- Citirea fișierelor
- Crearea fișierelor cu sau fără conținut
- Editarea fișierelor
- Căutarea pe internet
- Execuția comenzilor în terminal
- Utilizarea capacităților
- Pornirea subagenților

Utilizarea acestor instrumente trebuie îngrădită totuși de noțiuni logice. Execuția acestora este determinată de șiruri logice de evenimente care trebuie îndeplinite anterior execuției. Spre exemplu, un agent nu ar trebui să scrie într-un fișier dacă nu l-a citit înainte, deoarece acesta nu are contextul acelui fișier și poate returna lucruri care nu sunt ancorate în realitate (Figura 2.14). Acest lucru



este implementat cu ajutorul LangGraph printr-un interceptor <sup>3</sup> ce este apelat înaintea sau la finalul execuției unei unealte. Rezumându-ne la exemplul cu unealte de citire și scriere din fișiere, aceste interceptoare salvează un SHA<sup>4</sup> al fișierului citit în memorie, pe care mai apoi unealta de scriere a fișierului îl compară cu SHA-ul calculat individual. Dacă aceste două coduri coincid, rezultă că fișierul citit este exact cel care urmează să fie scris, astfel este garantată integritatea contextului înaintea scrierii. Configurarea interceptoarelor de protecție este detaliată în anexa 4.4. Agentul are acces doar la umplerea parametrilor funcției cu valori arbitrare, dar nu are control asupra logicii efective a unealtei.

Opțiunea de a configura aceste moduri de protecție revine utilizatorului ce poate porni sau opri această funcționalitate. Fiind o aplicație OSS, dacă utilizatorul este și dezvoltator software, acesta poate modifica codul sursă și extinde logica de protecție cu mai multe reguli ce pot fi oprite sau pornite din această secțiune.

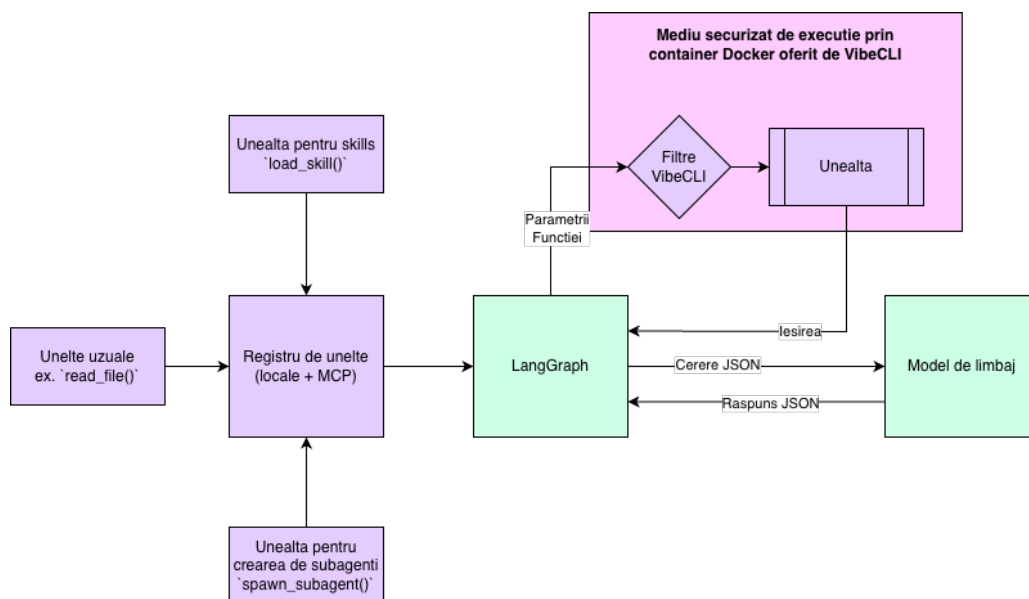


Figura 2.14: Suita de unealte expuse agentului în `vibe-cli` și relațiile dintre acestea

Agentul trebuie să utilizeze în mod inteligent aceste unealte. Pe lângă rularea logică a secvenței de instrumente, acesta trebuie să știe cum se poate folosi de ele la potențialul maxim. Un bun exemplu este execuția de cod scris de agenți în terminal. Pentru sarcini repetitive care ar consuma o bună parte din context, modelele sunt antrenate să scrie porțiuni mici de cod, uzual în limbajul Python sau Bash datorită simplității și a disponibilității crescute. Astfel, agentul va scrie aceste bucăți de cod și le va executa în terminal pentru a realiza acțiuni complexe în doar câteva linii de cod, sporind eficiența cercetării și a testării. Această utilizare este fructificată de promptul de sistem care instruește agentul, dar și din procesul de antrenare.

Antrenarea pentru scrierea, folosirea și execuția de cod a devenit o prioritate pentru agenți, modelele de limbaj fiind antrenate explicit folosind RL să folosească secvențe de cod pentru facilitarea rezolvării problemelor care ar beneficia după urma unei astfel de abordări. Această logică este utilizată și pentru testarea de bază a ideilor și explorarea acestora înainte de implementarea propriu-zisă, agentul rulând un mic script demonstrativ în terminal pentru a dovedi că principiul pe care se bazează ideea ce urmează a fi implementată este valid.

<sup>3</sup>Punct de interceptare a execuției unei funcții care permite inserarea altor blocuri logice ce sunt rulate în momentul apelării acestuia

<sup>4</sup>Codarea SHA este cea mai comună metodă de verificare a integrității unui fișier: orice mică modificare a fișierului schimbă complet codul, fără a fi necesară citirea integrală a acestuia.

## 2.8.2 Modul de planificare

Planificarea sarcinilor complexe folosește principiul CoT [12]. Astfel, execuția unei sarcini complexe este împărțită în două părți: planificare și execuție.

În primă fază, agentul realizează cercetarea necesară pentru implementare și documentează totul într-un fișier. Mai apoi, se va folosi de contextul din acel fișier pentru a ține minte pașii ce urmează a fi abordați. Multe implementări împart planul în secțiuni bine delimitate printre care se regăsesc: ideea și descrierea planului, fișiere afectate și modificări punctuale, împărțindu-și o execuție lungă în mai multe sub-sarcini individuale și de complexitate redusă.

Acest mod de planificare nu este nimic mai mult decât o schimbare a instrucțiunilor de sistem într-o combinație cu o antrenare de tip RL care recompensează crearea de planuri detaliate și consistente în situații care beneficiază de pe urma acestui procedeu.

Modul de planificare reprezintă materializarea SDD în contextul dezvoltării codului sursă de către sisteme agentice. O altă abordare folosită în aceste sisteme este aceea de TDD, care permite mai întâi experimentarea și scrierea de cod fără o documentație bine pusă la punct în prealabil, dar implică testarea amănunțită la final.

## 2.8.3 Capabilitățile predefinite

Skill-urile, rutinele sau capabilitățile nu sunt decât un set predefinit de instrucțiuni pentru realizarea punctuală a unei sarcini repetitive și sunt compuse din două componente care se află într-un fișier de tip Markdown pentru formatare augmentată.

Prima componentă este în format YAML, și reprezintă date referitoare la capabilitate precum numele, descrierea, o recomandare pentru cazurile în care utilizarea acestuia este dorită, și un fanion ce setează activarea acestei capabilități. Aceste date sunt disponibile agentului și sunt inserate în contextul tuturor conversațiilor pentru ca modelul să aibă acces la lista din care poate alege.

A doua componentă este reprezentată de capabilitatea propriu-zisă. Aceasta este formată din instrucțiuni scrise în limbaj natural. De obicei aceste instrucțiuni sunt structurate în pași bine definiți, cu reguli și exemple.

```
1 ---
2 name: pr-review
3 description: Structured pull-request review - risk surface, testing, naming, error handling.
4 when_to_use: When the user asks for a code review, PR review, or feedback on a diff.
5 enabled: false
6 ---
7 You are now performing a pull-request review. Walk through the diff and respond with these
  sections, in this order. Skip a section only if there's truly nothing to say there.
8
9 1. Summary - one short paragraph: what does this change do, in plain language.
10 2. Risk surface -- what could break in production. Concurrency, data migrations, error
  paths, rollback story, blast radius.
11 3. Correctness -- bugs, off-by-ones, race conditions, mishandled None/empty cases, wrong
  types, security issues (injection, auth, secrets).
12 4. Tests -- is the change covered. Are the tests testing the right thing. Any missing edge
  cases.
13 5. Style & naming -- readability nits only if they actually hurt comprehension. Don't
  bikeshed.
14 6. Suggestions -- concrete proposed changes, ideally with a snippet.
15
16 Rules of engagement:
17 - Be specific: cite file paths and line numbers.
18 - Distinguish blockers from nits - say so explicitly.
19 - If the diff is too large to review thoroughly, say so and pick the riskiest files first.
20 - No fluff, no praise sandwich. The reader is technical and time-constrained.
```

Figura 2.15: Implementarea unei capabilități de revizuire de PR

Aceste capabilități permit execuția consistentă și reproductibilă (deoarece agentul primește aceeași instrucțiune de fiecare dată) a unui flux de execuție dorit în rezolvarea unei sarcini.

## 2.8.4 Subagenții

Echipa de agenți constă dintr-un agent principal denumit orchestrator, ce planifică și delegă sarcini, împreună cu mai mulți subagenți specializați cu context limitat la o sub-sarcină, care returnează doar deznodământul acesteia (figura 2.16). Un subagent este doar un alt apel LLM cu personalitate diferită, set restrâns de unelte și instrucțiuni de sistem dedicate ce specifică și formatul de răspuns așteptat de agentul principal. Subagenții folosesc uzual modele mai mici care execută sarcini de explorare sau testare, deci compactarea contextului este de asemenea necesară. Orchestratorul îi pornește printr-o unealtă LangGraph (figura 2.17) și centralizează răspunsurile acestora pentru a menține doar o vedere de nivel înalt asupra sarcinii și a putea lua decizii informate fără a-și umple fereastra proprie de context.

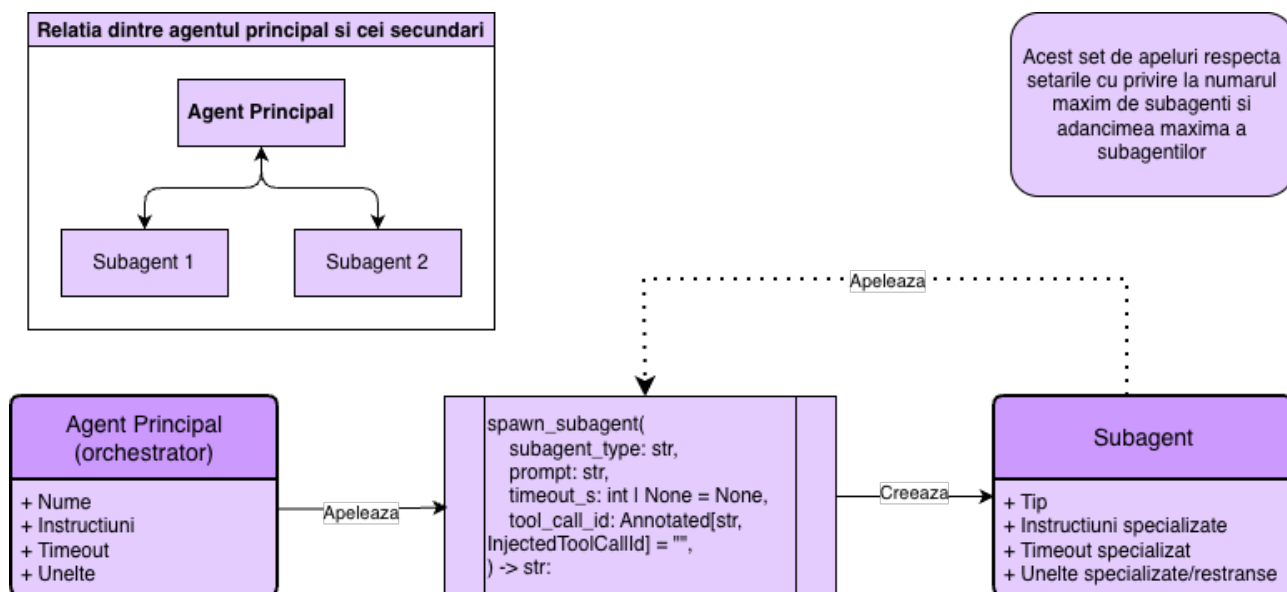


Figura 2.16: Arhitectura orchestrator-subagenți din `vibe-cli`

```

1 @tool
2 async def spawn_subagent(
3     subagent_type: str, prompt: str, timeout_s: int | None = None,
4     tool_call_id: Annotated[str, InjectedToolCallId] = "",
5 ) -> str:
6     """Spawn a specialised subagent to handle a focused sub-task in isolation.
7     The subagent runs its own tool-calling, then returns a single final report.
8     You only see that report. Use this to to delegate a deep dive without
9     polluting your own context.
10    Pick a 'subagent_type' that matches the work: 'general' - open-ended research,
11    'code_search' - read-only code lookups, 'web_research'
12    Args:
13        subagent_type: One of the registered types above.
14        prompt: The task description for the subagent. Be specific: it has
15            none of your conversation context.
16        timeout_s: Optional hard timeout in seconds. Defaults to the type's
17            built-in default; capped by config.
18    """
19    parent_ctx = _ctx()
20    result = await run_subagent(
21        subagent_type=(subagent_type or "").strip(),
22        prompt=(prompt or "").strip(),
23        parent_ctx=parent_ctx,
24        timeout_s=timeout_s,
25        parent_tool_call_id=tool_call_id or None,
26    )
27    if tool_call_id:
28        parent_ctx.setdefault("pending_subagent_results", {})[tool_call_id] = {
29            "subagent_type": (subagent_type or "").strip(),
30            "prompt": (prompt or "").strip(),
31            "text": result.text,
32            "iterations": result.iterations,
33            "tool_call_count": result.tool_call_count,
34            "timed_out": result.timed_out,
35            "error": result.error,
36            "duration_s": result.duration_s,
37            "usage": result.usage,
38        }
39    return result.text

```

Figura 2.17: Implementarea unei funcții pe care agentul o folosește pentru a porni un subagent

Niciun produs comercial nu permite configurarea în detaliu a subagenților; implementarea noastră permite **configurarea completă** per subagent: model, prompt, posibilitatea de a porni subagenți proprii, număr maxim de iterații, adâncime maximă și timp maxim de așteptare (anexa 4.4). Aplicația furnizează trei subagenți predefiniți (general, cercetare cod, cercetare web), executați de obicei în paralel (cu limită de execuție configurabilă), fiecare cu setul propriu de unelte, instrucțiuni și setări proprii pentru gestionarea răspunsurilor de la acestea.

## 2.9 Extensibilitatea

Deoarece aplicația este gândită pentru a fi open-source, arhitectura aplicației trebuie să fie ușor adaptabilă de utilizatori ce doresc să aducă modificări în modul de funcționare, în special în punctele de extensie. Câteva din aceste puncte gândite de la bun început sunt: logica de compactare, setul de unelte, setul de subagenți, setul de furnizori și setul de capabilități.

### 2.9.1 MCP și suplimentarea uneltelor

Aplicația conține un client MCP pentru adăugarea uneltelor externe și integrarea cu servicii terțe. Acesta acceptă cele **3** metode de comunicare comune în industrie: STDIO (capturează ieșirea proceselor locale), HTTP și SSE (comunicare cu servere). Pentru a configura un server MCP și a accesa instrumentele pe care le pune la dispoziție în aplicație, utilizatorul poate folosi comanda `/mcp` în interiorul interfeței, sau poate adăuga personal serverul în fișierul `/.vibe-cli/preferences.json` și să repornească aplicația.

Serverele MCP pot expune atât unelte cât și capabilități sau instrucțiuni. Acestea pot fi reîncărcate sau gestionate prin comanda `/mcp`

Adăugarea de unelte poate fi făcută cu ușurință, fiind o aplicație open-source. Dacă utilizatorul este dezvoltator software și a clonat proiectul local, acesta poate adăuga un instrument nou după modelul din figura 2.13 și să își implementeze propria logică. Uneltele sunt definite la calea `src/tools.py`. După adăugarea uneltelor, aplicația trebuie reinstalată/repornită.

```
● system
MCP Servers
jira-corp (sse) - http://localhost:3000/sse disabled

Usage:
/mcp add stdio <name> <command>      - add stdio server
/mcp add http  <name> <url>           - add streamable_http server
/mcp add sse   <name> <url>           - add SSE server
/mcp remove <name>                     - remove
/mcp toggle <name>                     - enable/disable
/mcp reload                                - reconnect
/mcp tools                                - list tools
```

Figura 2.18: Meniul de adăugare a serverelor MCP

### 2.9.2 Extinderea echipei de subagenți

Adăugarea unui subagent presupune crearea unui fișier în `src/subagents/types` conform modelului din figura 2.19, extinderea clasei `Subagent` și completarea câmpurilor de bază (nume, descriere, rol), urmate de înregistrarea în `config.yml` și repornirea aplicației.

```

1 @register
2 class CodeSearchSubagent(Subagent):
3     name = "code_search"
4     description = (
5         "Fast read-only code search. Finds files/functions/symbols and "
6         "reports paths + line ranges + relevant snippets. Cannot write or "
7         "execute side-effectful commands."
8     )
9     default_timeout_s = 90
10    max_iterations = 15
11    # Read-only toolset. 'run_command' is included for grep/rg/find use.
12    allowed_tools = ["read_file", "list_directory", "run_command", "fetch_artifact"]
13
14    def role_prompt(self, prompt: str, parent_ctx: dict) -> str:
15        return (
16            "ROLE: read-only code-search subagent.\n\n"
17            "Your job is to LOCATE code that matches the orchestrator's "
18            "request and report findings concisely. You are an advisor only "
19            "- even though 'run_command' is available, it is for read-only "
20            "searches (rg / grep / find / ls / cat / wc). NEVER invoke "
21            "installers, builds, test runs, package managers, or any "
22            "command that mutates state.\n\n"
23            "Stop searching once you have enough to answer; do not enumerate "
24            "exhaustively.\n\n"
25            "Final report format:\n"
26            " - For each match: 'path:line' + a one-line description.\n"
27            " - When useful, include a short snippet (<=6 lines).\n"
28            " - End with a one-sentence takeaway for the orchestrator."
29        )

```

Figura 2.19: Înregistrarea unui nou subagent

Înregistrarea se face prin decoratorul `@register`, expunând puncte de extensie precum: `role_prompt` (fragment de rol cu acces la contextul părintelui), `allowed_tools/excluded_tools` (intersectate cu uneltele active), interceptoare pentru recalcularea uneltelor și a prompt-ului per invocare, `max_iterations` și `default_timeout_s` (suprascrise din `config.yml`), și model dedicat configurabil prin `/subagents`.

La execuție, fiecare subagent rulează o buclă completă de apelare de unelte cu același flux de permisiuni și `tool_guards` ca părintele (de ex. *read-before-write* prin partajarea `ctx["file_reads"]`). De asemenea, beneficiază de compactare locală a răspunsurilor de la instrumente, telemetrie Langfuse în span-ul părintelui, agregarea tokenilor și respectarea limitelor `max_depth/max_parallel`; dacă acesta iese din buclă fără raport final, este reapelat o singură dată pentru emiterea raportului.

Modelul actual nu suportă definiții declarative bazate pe fișiere Markdown, delimitarea serverelor MCP per subagent, flux continuu intermediar al ieșirii și nici politici de *reîncercare/replanificare* dincolo de `max_iterations`.

### 2.9.3 Dezvoltarea capabilităților și personalizare

Crearea de capabilități este reprezentată de scrierea unui fișier de tip Markdown și de plasarea acestuia fie în containerul global aflat la `/vibe-cli/skills`, fie în folderul `.vibe` din cadrul proiectului în care se lucrează curent pentru capabilitățile locale. Pentru aplicarea schimbărilor sau gestionarea acestora, utilizatorul poate folosi comanda `/skills`.

Fișierele de personalizare disponibile local, per proiect sau global pot fi scrise atât de agenți (pentru a nota preferințele utilizatorului extrase din conversație) sau pot fi modificate chiar de utilizator. Un exemplu bun de astfel de preferință utilizată în sarcinile de dezvoltare software ar fi specificarea modului de testare: "La sfârșitul implementării rulează întotdeauna testele unitare și de integrare".

### 2.9.4 Extinderea tehnicilor de compactare

Procesul de compactare este bazat pe un sistem de extensii înmagazinate într-un registru. Fiecare strategie este o subclasă a `Stage` (*eager*, per eveniment) sau `ThresholdStage` (la atingerea pragului, cu vizibilitate globală), declarată în `src/compaction/stages/` și înregistrată în `src/compaction/__init__.py`.

Adăugarea unei strategii presupune: implementarea metodelor clasei pe care o extinde, adăugarea în registru, opțional înscrierea în lista de excludere mutuală (dacă e incompatibilă cu alta) și introducerea în `config.yml`.

Strategiile asincrone bazate pe LLM folosesc un protocol în două etape: o rulare sincronă produce un rezultat sigur (`truncated_pending`), iar o sarcină de fundal îl înlocuiește ulterior cu varianta rezumată de modelul de limbaj; în caz de eșec, starea trece în `truncated_fallback`. Câmpul `event["content"]` rămâne imutabil, iar fișierul de sesiune este copiat înainte de fiecare trecere, garantând reversibilitatea.

Pe baza acestui mecanism, candidații naturali pentru extindere sunt: memorarea apelurilor de unealtă prin perechi (nume, argumente normalizate); proiecția diferențială a citirilor repetate (dacă un fișier este citit de 2 ori, se salvează doar diferența între cele 2 stări observate); fereastră glisantă de fapte ancorate persistente; proiectori structurați per unealtă (extractori declarativi care păstrează doar câmpurile consumate de agent). Strategiile bazate pe notare cu LLM ca arbitru nu au constituit o prioritate momentan, fiind acoperite determinist de `importance_prune` și `bulk.hybrid`.

## 2.10 Securitatea în mediul agentic

Încredințarea controlului unui agent cu acces la comenzi periculoase necesită măsuri de siguranță împotriva acțiunilor nedorite. Studiul recent al amenințărilor specifice sistemelor agentice [90] le grupează în trei clase:

1. *Prompt injection* indirect prin conținut ingestat din fișiere, pagini web sau ieșirea uneltelor, acestea fiind tehnici de tipul Adversarial Machine Learning (AML) [91];
2. *Utilizarea greșită a uneltelor*, prin halucinarea argumentelor sau compunerea unor secvențe inofensive individual dar distructive în ansamblu (`rm -rf` pe căi eronate);
3. *Data poisoning* la nivelul memoriei persistente, prin plantarea de fapte false reluate ulterior ca autoritare.

La aceste amenințări se adaugă până și frustrarea agentului, care în unele cazuri decide că cel mai ușor mod de a scăpa de o eroare este de a șterge teste unitare sau funcționalitatea ce provoacă eroarea cu totul.

Întrucât aliniamentul modelului [92] și filtrele de tip *guardrail* [93], [94] nu oferă garanții suficiente în fața acestor vectori, am proiectat o tehnică de apărare pe trei niveluri: izolarea execuției prin sandboxing, restricționarea acțiunilor prin permisiuni explicite și delegarea către subagenți cu privilegii reduse. De asemenea, furnizorii de model includ protecții pentru intrarea și ieșirea modelelor (pentru a limita ingestia sau returnarea de date nocive), dar și modelele sunt antrenate (în unele cazuri) contra acestor lucruri.

### 2.10.1 Medii securizate de execuție

Mediul securizat de execuție izolează comenzile de terminal ale agentului într-un container Docker dedicat sesiunii, cu directorul utilizatorului montat prin `bind-mount` la `/workspace` pentru propagarea directă a modificărilor. Funcționalitatea este opțională și utilizează automat execuția pe gazdă când Docker nu este disponibil.

Configurația (anexa 4.4) acoperă imaginea de bază, modul de rețea, limitele de memorie/Central Processing Unit (CPU), timpul maxim, căi interzise și prag pentru non-proiecte. La pornire, aplicația aplică o politică de validare pe trei niveluri: refuz pentru căi sensibile (`/`, `/tmp`, `$HOME` și subdirectoarele directe), acceptare pentru directoare cu marcaje de proiect (`.git`, `pyproject.toml`) și validare pe bază de dimensiune/număr de fișiere pentru ambiguă, suprascrisă de `VIBE_SANDBOX_FORCE`.

Containerul rulează cu `docker run -d -rm --cap-drop ALL --user <uid>:<gid>`, trăiește pe toată durata sesiunii și servește comenzile prin `docker exec`. În caz de eșec, modulul nu propagă excepția, iar execuția continuă pe gazdă. Rutarea apelurilor se face înapoi prin `run_command`; uneltele de fișiere

rămân pe gazdă (echivalente prin `bind-mount`, cu latență redusă). Când mediul de execuție e activ, este expusă unealta și `run_command_host` ca metodă condiționată de permisiuni de a executa comenzi pe mașina gazdă. Subagenții moștenesc aceste setări prin context, beneficiind uniform de izolare.

## 2.10.2 Permisii pentru aprobarea umană a acțiunilor

Aprobarea execuției uneltelor de către utilizator este o măsură de apărare necesară: unelte de citire sunt aprobate din oficiu, dar pentru restul utilizatorul are întotdeauna ultimul cuvânt și poate aproba fiecare comandă în parte, toate comenzile de un tip pentru sesiunea curentă sau permanent, primind ca informație unealta și argumentele acesteia (figura 2.20).



Figura 2.20: Cererea de aprobare a permisiunii de a scrie un fișier

Restricțiile se aplică atât agentului orchestrator cât și subagenților.

## 2.10.3 Filtre, gestionarea intențiilor periculoase și cereri în afara scopului (Out of Scope (OOS))

O aplicație în producție trebuie să răspundă doar la cereri din scopul declarat, lucru greu de garantat cu un sistem nedeterminist. Tehnicile comune combină instrucțiuni de prompt, antrenament dedicat, liste de cuvinte interzise sau apel către un model extern de validare. În aplicație nu am implementat astfel de filtre, lăsând acest aspect în sarcina furnizorilor de modele de limbaj care au deja implementate astfel de soluții.

## 2.11 Observabilitatea

Am proiectat observabilitatea aplicației pe două niveluri complementare: urmărire externă a execuției, prin integrarea cu Langfuse Cloud, și introspecție locală, prin panoul web analitic livrat împreună cu aplicația. Ambele componente sunt opționale, dezactivate implicit, și nu transmit date către alte servicii în afara celor configurate explicit de utilizator.

### 2.11.1 Urmărirea execuției prin Langfuse

Pentru urmărirea externă a execuției, aplicația integrează Langfuse Cloud printr-o interceptare a grafului agentului. Activarea se face din `config.yml` sau prin comanda `/tracing setup`, care persistă cheile API în `preferences.json`. Precedența de rezolvare: preferințe locale, configurație, variabile de mediu `LANGFUSE_*`.

La fiecare invocare, evenimentele sunt publicate asincron către Langfuse sub forma unui trace<sup>5</sup>

---

<sup>5</sup>Un trace este un mesaj JSON folosit în monitorizarea distribuită; conține detalii și poate îngloba mai multe span-uri.



principal cu span-uri<sup>6</sup> ierarhice per apel LLM, unealtă și nod de graf. Pentru fiecare span se înregistrează numele modelului, intrările/ieșirile, consumul de tokeni și latența. Trace-urile sunt etichetate cu identificador de sesiune (Universally Unique Identifier (UUID)v7), utilizator anonim și tag-uri (mod activ, furnizor, model), permițând agregarea și filtrarea ulterioară.

Pentru bucla de feedback, două comenzi operează pe ultimul trace: `/score` atașează scoruri numerice sau calitative, iar `/dataset` adaugă interacțiunea într-un set de date persistent pentru evaluare. Combinate cu mecanismul *LLM-as-judge* oferit de Langfuse, ele formează fundamentul unei suite de monitorizare a calității.

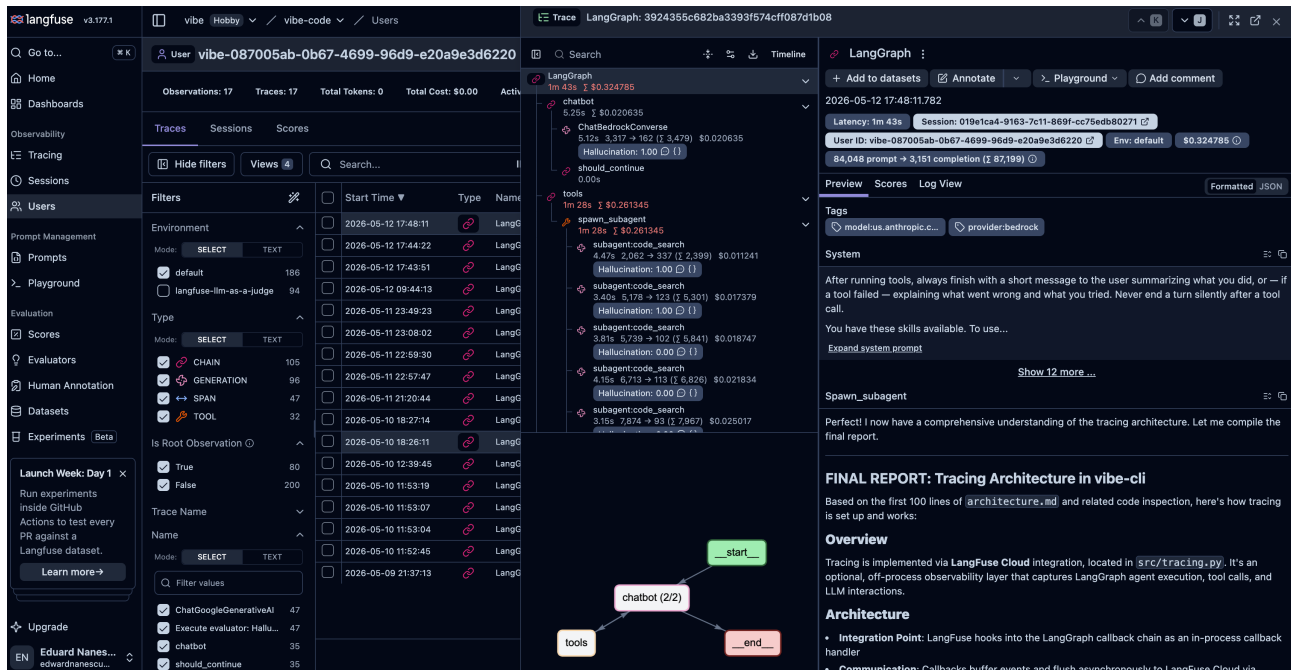


Figura 2.21: Interfața Langfuse

Figura 2.21 prezintă interfața Langfuse: **stânga** conține navigarea (trace-uri per sesiune și utilizator, managementul versionat al prompt-urilor, rubrica de calitate cu evaluatori automați și verificare umană); **centrul** arată meniul *Users* cu sesiunile și trace-urile filtrabile; **dreapta** expune trace-ul deschis, cu graful de execuție pas cu pas, apelurile uneltelor și răspunsurile, gândirea modelului, istoricul augmentat și metricile per tokeni, plus evaluările automate configurate.

## 2.11.2 Panoul web proprietar

Panoul web este o aplicație Flask locală, *read-only*, care vizualizează direct fișierele de sesiune din `~/vibe-cli/sessions/`, fără bază de date intermediară. Se lansează prin `vibe web`, expusă implicit pe `http://127.0.0.1:5050`, cu adresa și portul configurabile.

Interfața (o singură pagină) oferă selectoare pentru proiect și sesiune și afișează sumarul consumului de tokeni pe categorii de evenimente, evoluția dimensiunii contextului raportată la limita modelului, frecvența și latența apelurilor către unelte (cu  $p_{50}$ ,  $p_{95}$ ), debitul de generare în tokeni/secundă, costul estimat per apel pe baza unui tabel de prețuri configurabil și istoricul compactărilor cu diferențele aferente. Componente dedicate permit inspectarea granulară a fiecărei unelte și reconstrucția turului conversațional eveniment cu eveniment.

La nivel înalt, panoul deschide fiecare sesiune cu o fișă de sinteză ce reunește, într-un singur ecran, furnizorul și modelul folosite, numărul de interacțiuni și de evenimente, totalul de tokeni consumați, utilizarea de vârf a contextului raportată la limita modelului, durata sesiunii și costul estimat alături

<sup>6</sup>Un span reprezintă o unitate de observare a unei singure acțiuni.

de preț de referință (\$/1M tokeni) (figurile 2.24, 2.27, 2.25). Această privire de ansamblu este completată de o reprezentare în cerc (*doughnut*) a ponderii tokenilor pe categorii de evenimente și de o defalcare a apelurilor între uneltele native și cele provenite din servere MCP. Scopul acestor vizualizări agregate este de a oferi utilizatorului, dintr-o singură privire, răspunsul la întrebările esențiale legate de propria utilizare: cât a costat o sesiune, unde s-au consumat de fapt tokenii (intrare, ieșire, rezultate ale uneltelor, gândirea modelului), cât de aproape de saturarea contextului a ajuns interacțiunea și ce unelte domină activitatea agentului. Astfel, în loc de a inspecta fișierele de sesiune brute, utilizatorul își poate urmări tiparul de consum și ajusta configurația (model, unelte, prag de compactare) pe baza unor metrice comparabile între sesiuni.

Sincronizarea folosește SSE pe o singură conexiune persistentă; detecția modificărilor compară metadatele de fișier la 1.5s, iar reîncărcările sunt spațiate la 250 ms. Un panou inferior afișează ultimele 1000 de linii din jurnalul aplicației cu auto-derulare. Când urmărirea execuției Langfuse este activă, un buton dedicat deschide proiectul în interfața externă.

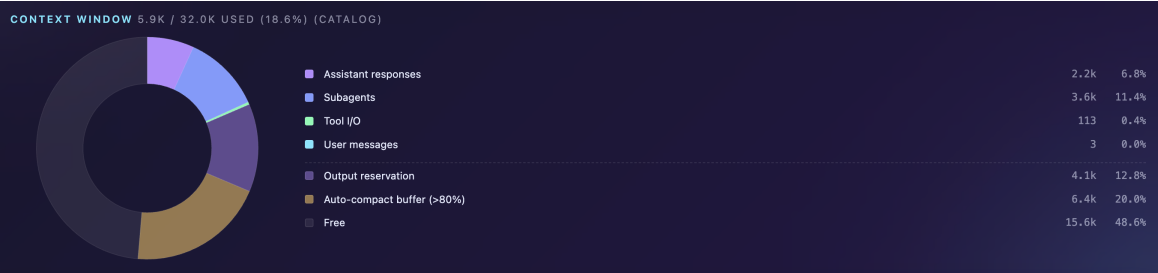


Figura 2.22: Distribuția contextului

| SUBAGENTS (2, 1 TIMED OUT, \$0) |             |                   |                                                                                    |                  |                 |   |
|---------------------------------|-------------|-------------------|------------------------------------------------------------------------------------|------------------|-----------------|---|
| BY TYPE                         |             | COUNT · TIMED-OUT | MEAN DUR                                                                           | INPUT · OUTPUT   | COST            |   |
| AGENT                           | code_search | 1 · 1 TO          | 1.5m                                                                               | 0 · 0            | \$0             |   |
| AGENT                           | general     | 1                 | 19.12s                                                                             | 45015 · 1394     | \$0             |   |
| #                               | TYPE        | PROMPT PREVIEW    | ITER · TOOLS · DUR                                                                 | IN · OUT TOK     |                 |   |
| 1                               | AGENT       | code_search       | Research this repository and provide a high-level overview of what it's about. PL_ | 01 · 0t · 1.5m   | TO 0 · 0        | > |
| 2                               | AGENT       | general           | You are researching a code repository. Do the following steps in order, keeping e_ | 71 · 9t · 19.12s | 45015 · 1394    | > |
| SUBAGENT CONTEXT                |             |                   | RETURNED TO ORCHESTRATOR                                                           |                  | TOKENS SAVED    |   |
| 46.4k tok                       |             |                   | 749 tok                                                                            |                  | 45.7k tok · 98% |   |

Figura 2.23: Optimizarea adusă ferestrei de context a agentului principal de către subagenți

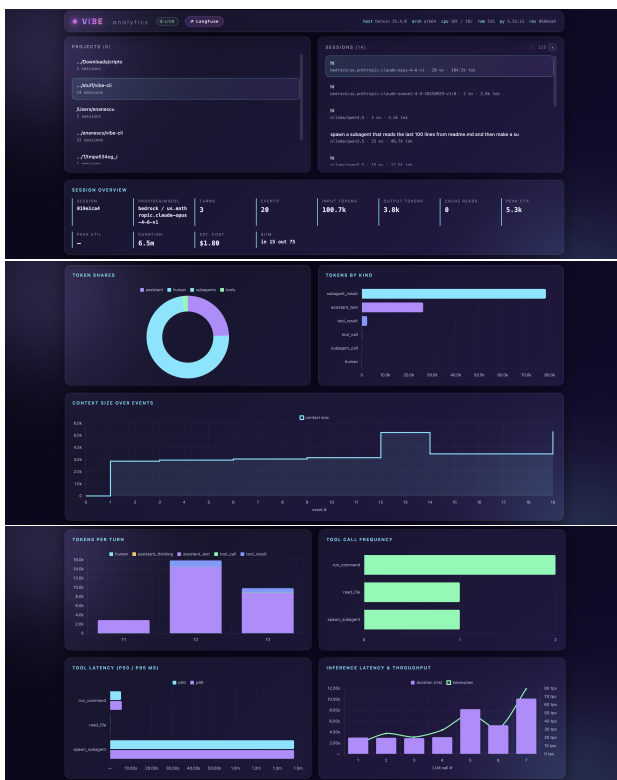


Figura 2.24: Imagini cu interfața web a aplicației

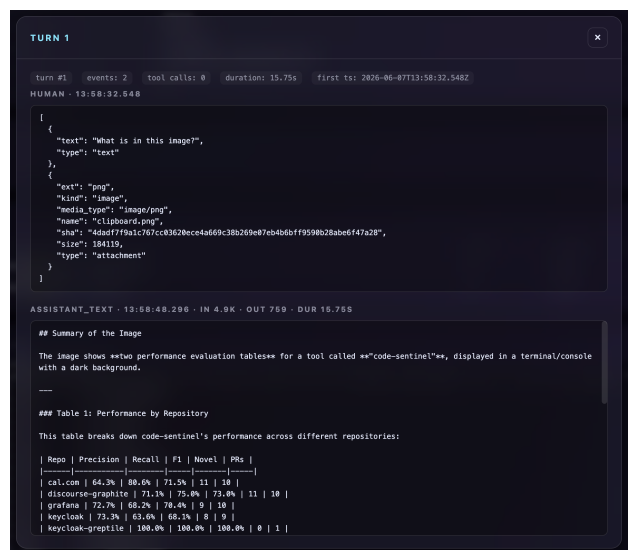
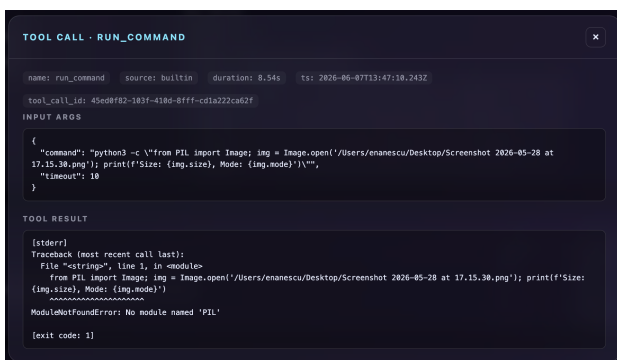


Figura 2.25: Detalii ale panoului web: apelarea uneltelor și reconstrucția interacțiunilor

```
SUBAGENT · GENERAL

type: general  depth: 1  iterations: 7  tool calls: 9  duration: 19.12s  ts: 2026-06-19T08:44:50.110Z
tool_call_id: tooluse_suZwYcafaz0MakATYhWPYi

PROMPT (ORCHESTRATOR → SUBAGENT)

You are researching a code repository. Do the following steps in order, keeping each tool call minimal:

1. List the root directory (path=".")
2. Read any README file you find (e.g. README.md)
3. Read any top-level config file that reveals the tech stack (package.json, pyproject.toml, Cargo.toml, go.mod, etc.)
4. List the main source directory if one exists (src/, lib/, app/, etc.)
5. Skim 1-2 key source files to understand what the code does.

Then write a concise high-level summary covering:
- What the project does
- Tech stack / languages
- Architecture style (library, CLI, web app, monorepo, etc.)
- Main components / modules
- Any notable conventions

Keep the final report under 400 words.

FINAL REPORT (SUBAGENT → ORCHESTRATOR)

Perfect. I have enough information now. Let me write the final report.

---

## Final Report

**Code Sentinel** is an automated pull request reviewer powered by Claude AI (via AWS Bedrock) that integrates with GitHub to provide thorough, actionable code review feedback with inline comments on PRs.

### What It Does
- Automatically reviews pull requests when requested via GitHub's "Request Review" feature
- Posts inline comments and overall review decisions (APPROVE, COMMENT, REQUEST_CHANGES) directly on PRs
- Supports both git.corp.adobe.com (GHE) and GitHub Enterprise Cloud (GHEC)
- Includes a self-improvement mechanism to learn from past reviews
- Provides benchmarking and evaluation tools for review quality

### Tech Stack
- **Language**: Python 3.10+
```

Figura 2.26: Interacțiunea dintre subagenți și agentul principal

| COST                                    |        |          |
|-----------------------------------------|--------|----------|
| BEDROCK/US.ANTHROPIC.CLAUDE-OPUS-4-6-V1 |        |          |
| BUCKET                                  | TOKENS | COST     |
| input                                   | 10.8k  | \$0.1625 |
| output                                  | 1.2k   | \$0.0919 |
| cache read                              | 0      | \$0      |
| total                                   | 12.1k  | \$0.2543 |

| COMPACTION                                   |                            |              |
|----------------------------------------------|----------------------------|--------------|
| Eager stages                                 |                            |              |
| format_compress: 5x saved 457                |                            |              |
| Pre-compaction backups                       |                            |              |
| FILE                                         | EVENTS FREED               | TOKENS FREED |
| pre-compaction-2026-06-19T08-46-29-267Z.json | 12 events (pass was no-op) | 0            |
| pre-compaction-2026-06-19T08-49-30-030Z.json | 24 events (pass was no-op) | 0            |

Figura 2.27: Detalii ale panoului web: distribuția costului și detaliile procesului de compactare

Scopul panoului web este acela de a aduce transparență pentru un produs care are acces la sistemul utilizatorului și care face parte din rutina zilnică de muncă. Astfel, toate acțiunile și costurile sunt contorizate și explicate, spre deosebire de restul aplicațiilor disponibile pe piață care ascund aceste numere.

## Capitolul 3

# EVALUAREA SOLUȚIEI ȘI REZULTATE EXPERIMENTALE

Având în vedere finalizarea implementării, am condus un set de analize și experimente din utilizarea asistentului de programare cu diferite variații ale configurației pentru extragerea unui mod optim de operare și strângerea de date legate de mai multe aspecte ale folosirii acesteia, care influențează performanțele aplicației.

Data fiind natura nedeterministă a răspunsurilor modelelor de limbaj, schimbările de configurație precum modelul utilizat sau promptul de sistem, cât și uneltele puse la dispoziție, dar și alte aspecte pot avea un impact consistent asupra datelor de ieșire și acțiunile întreprinse de agent.

### 3.1 Poziționarea finală față de concurență

La finalul implementării, rezultatul este comparabil din punct de vedere al funcționalităților, dar și al performanței cu aplicațiile existente pe piață în momentul de față.

Calitățile ce diferențiază abordarea noastră de restul competiției sunt: integrarea ușoară cu modele OSS ce pot rula local pe mașina utilizatorului, scăpând de dependența de un furnizor de modele de limbaj și de costurile asociate lor, transparența interacțiunii cu agentul prin panoul web și integrarea cu Langfuse dar și prin opțiunile exhaustive de configurație prin care utilizatorul poate controla majoritatea funcționalităților vitale.

Din figura 3.1 se observă că **vibe-cli** este singura soluție care îmbină deschiderea ecosistemului OSS cu o transparență și un set de funcționalități comparabile cu ale uneltelor comerciale.

Pentru a susține cantitativ această diferențiere, în tabelul 3.1 am rezumat principalele caracteristici ale celor mai populare unelte agentice de programare pentru terminal în raport cu soluția propusă.

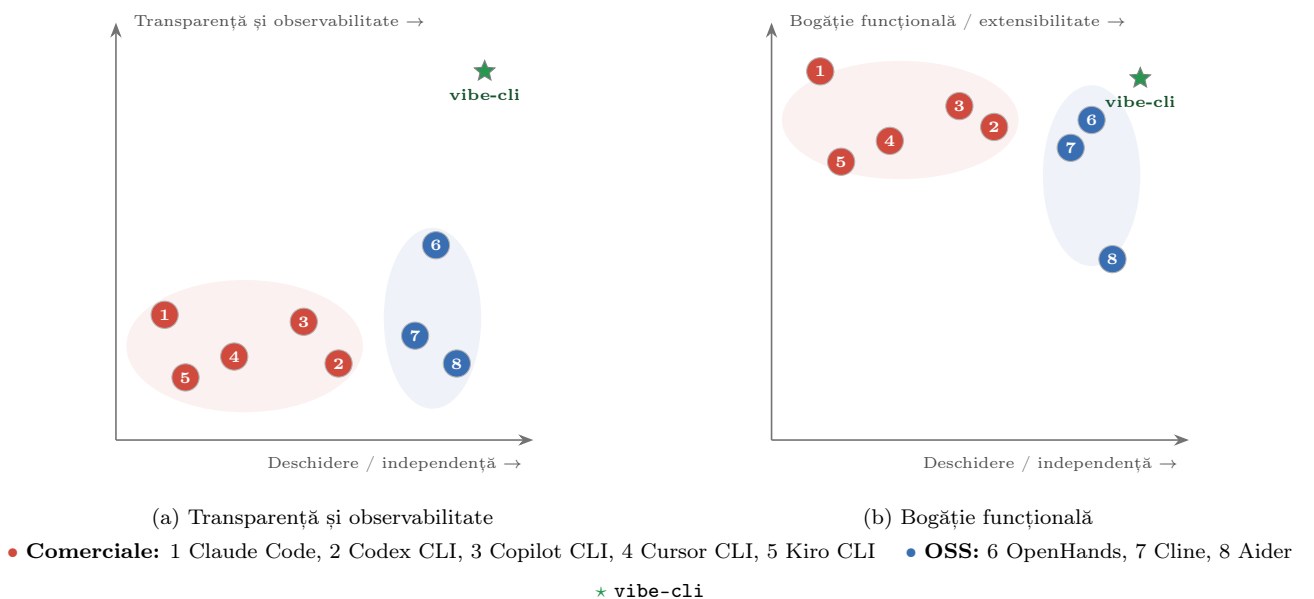


Figura 3.1: Poziționarea **vibe-cli** față de concurență.

| Aplicație                                                                | Furnizori și model de cost                                                                                                                                                                                                                                      | Puncte forte                                                                                                                                                                                                                                                                                                                                                                                        | Puncte slabe (raportat la soluția propusă)                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Claude Code</b> (Anthropic [41])<br>Sursă închisă, proprietară        | Doar modele Anthropic Claude (Amazon Web Services Bedrock și Vertex AI doar în ediția enterprise).<br>Abonament Pro/Max sau cheie API cu plata per-token.                                                                                                       | <ul style="list-style-type: none"> <li>• Maturitate și integrare nativă cu modelele Claude</li> <li>• Extensibilitate prin MCP, subagenți, capabilități, interceptări și extensii</li> <li>• Permisuni granulare, sandboxing și suprafețe multiple (terminal, Integrated Development Environment (IDE), web)</li> </ul>                                                                             | <ul style="list-style-type: none"> <li>• Cod sursă închis, dependență totală de un singur furnizor; fără modele OSS locale</li> <li>• Cost obligatoriu (abonament sau per-token), fără regim gratuit</li> <li>• Fără panou web propriu; urmărire nativă minimă a execuției (OpenTelemetry parțial [95])</li> </ul> |
| <b>Codex CLI</b> (GitHub [39])<br>Licență Apache-2.0                     | Doar modele OpenAI.<br>Inclus în plan ChatGPT Plus/Pro/Business sau cheie API cu plata per-token.                                                                                                                                                               | <ul style="list-style-type: none"> <li>• Cod sursă complet deschis (Apache-2.0), implementare în Rust și executabil unic</li> <li>• Suport pentru MCP, subagenți, capabilități, reguli, interceptări și sandboxing</li> <li>• Autentificare directă cu un cont ChatGPT existent</li> </ul>                                                                                                          | <ul style="list-style-type: none"> <li>• <i>Furnizor unic</i> (OpenAI), fără modele OSS locale (Ollama, llama.cpp)</li> <li>• Cost obligatoriu (abonament ChatGPT sau cheie API)</li> <li>• Transparență minimă a utilizării; fără integrare Langfuse sau alt sistem de urmărire a execuției</li> </ul>            |
| <b>Copilot CLI</b> (GitHub Docs [96])<br>Sursă închisă, proprietară      | Modele găzduite de GitHub (Claude, GPT); rutare opțională către puncte OpenAI-compatibile (Ollama, vLLM, Azure OpenAI [97], Anthropic) prin variabile de mediu.<br>Abonament Copilot Pro/Business/Enterprise + credite per interacțiune.                        | <ul style="list-style-type: none"> <li>• Rutare explicită către Ollama și puncte de acces locale</li> <li>• Configurabilitate avansată (agenți per utilizator/repo/organizație, MCP, capabilități, interceptări, memorie persistentă)</li> <li>• Integrare nativă cu fluxurile GitHub (PR-uri, tichete, context de repo) și comenzi de inspecție a sesiunii (/usage, /context, /compact)</li> </ul> | <ul style="list-style-type: none"> <li>• Cod sursă închis, abonament obligatoriu, preț opac (credite); status de previzualizare publică</li> <li>• Fără panou web sau interfață grafică (analiză numai în terminal)</li> <li>• Fără integrare cu Langfuse sau OpenTelemetry [95]</li> </ul>                        |
| <b>Cursor CLI</b> (Anysphere [42])<br>Sursă închisă, proprietară         | Multi-model în regim cloud: OpenAI, Anthropic, Google și modelul propriu Composer, comutabile prin -model. Abonament Pro/Pro+/Ultra cu plata per utilizare (API); plan gratuit limitat.                                                                         | <ul style="list-style-type: none"> <li>• Suport multi-provider cu moduri Agent/Plan/Ask și sandboxing configurabil</li> <li>• MCP, capabilități, interceptări, agenți cloud și sesiuni reluabile</li> <li>• Integrare strânsă cu editorul Cursor (terminal și Integrated Development Environment (IDE), inclusiv macOS/Linux/Windows)</li> </ul>                                                    | <ul style="list-style-type: none"> <li>• Cod sursă închis; fără modele OSS rulate local (doar găzduite)</li> <li>• Cost obligatoriu peste pragul gratuit, preț bazat pe utilizare/API</li> <li>• Fără panou web local de transparență per sesiune și fără integrare Langfuse</li> </ul>                            |
| <b>Kiro CLI</b> (Amazon Web Services [43])<br>Sursă închisă, proprietară | Modele găzduite de Amazon Web Services (Claude Sonnet/Opus și modele open-weight precum Qwen3 Codex, DeepSeek), neconfigurabile către furnizori externi.<br>Plan gratuit cu credite limitate și abonamente Pro/Pro+/Max/Power, cu plata per credit la depășire. | <ul style="list-style-type: none"> <li>• MCP, agenți personalizați, interceptări inteligente și <i>steering</i></li> <li>• Dezvoltare bazată pe specificații (SDD) și integrare cu ecosistemul Kiro (Integrated Development Environment (IDE))</li> <li>• Mod headless pentru automatizări CI/CD; suport macOS/Linux/Windows</li> </ul>                                                             | <ul style="list-style-type: none"> <li>• Cod sursă închis; fără rulare locală a modelelor OSS (doar găzduite Amazon Web Services)</li> <li>• Cost pe credite, model de preț opac; status de previzualizare</li> <li>• Fără panou web local de transparență per sesiune sau integrare Langfuse</li> </ul>           |

Tabelul 3.1: Comparatie între soluția dezvoltată și principalele unelte agentice de programare pentru terminal disponibile pe piață

În continuare, în tabelul 3.2 prezentăm o comparație cu cei mai relevanți asistenți de programare OSS din ecosistem (*OpenHands*, *Cline* și *Aider*), care împart cu soluția propusă avantajul codului sursă deschis și al independenței față de un furnizor unic de modele. Menționăm că *OpenClaw*, deși este cel mai popular proiect OSS din tabelul 1.3, nu a fost inclus în această comparație deoarece este poziționat ca asistent personal multi-canal (mesagerie, voce, automatizări), nu ca asistent dedicat dezvoltării software.

| Aplicație                                           | Furnizori și model de cost                                                                                                                                         | Puncte forte                                                                                                                                                                                                                                                                                                                                                                  | Puncte slabe (raportat la soluția propusă)                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>OpenHands</b><br>(GitHub [71])<br>Licență MIT    | Agnostic față de furnizor (Claude, GPT, Ollama și altele).<br>Aplicație gratuită; cost limitat la tariful furnizorului ales.                                       | <ul style="list-style-type: none"> <li>• Sandboxing robust în containere Docker</li> <li>• Interfață grafică web și cli disponibile în paralel</li> <li>• Software Development Kit (SDK) Python pentru construirea de agenți personalizați</li> <li>• Suport Kubernetes și scenarii enterprise</li> <li>• Integrări Slack, Jira, Linear</li> </ul>                            | <ul style="list-style-type: none"> <li>• Cerințe de resurse ridicate, Docker obligatoriu pentru fluxul principal</li> <li>• Configurare inițială complexă, curbă de învățare abruptă</li> <li>• Fără integrare nativă Langfuse pentru urmărirea execuției și evaluări automate</li> <li>• Fără panou de transparență cost și tokeni la nivel de sesiune</li> <li>• Fără suport explicit pentru subagenți tipizați cu model dedicat per tip</li> </ul> |
| <b>Cline</b><br>(GitHub [74])<br>Licență Apache 2.0 | Agnostic față de furnizor, cu rutare prin OpenRouter [98] (200+ modele: Anthropic, OpenAI, Ollama).<br>Aplicație gratuită; cost limitat la tariful modelului ales. | <ul style="list-style-type: none"> <li>• Extensie nativă pentru VS Code și extensie JetBrains</li> <li>• Software Development Kit (SDK) Node.js și cli neinteractiv pentru CI/CD</li> <li>• Echipe multi-agent și agenți programați</li> <li>• Suport MCP și extensii</li> <li>• Acces la peste 200 de modele prin OpenRouter</li> </ul>                                      | <ul style="list-style-type: none"> <li>• Consum ridicat de tokeni datorită contextului mare încărcat permanent</li> <li>• Fără sandboxing nativ incorporat</li> <li>• Dependent de ecosistemul VS Code/JetBrains, fără aplicație autonomă</li> <li>• Fără panou web propriu de transparență tokeni și cost</li> <li>• Fără integrare Langfuse sau alt sistem de urmărire a execuției</li> </ul>                                                       |
| <b>Aider</b><br>(GitHub [78])<br>Licență Apache 2.0 | Agnostic față de furnizor (Claude, GPT, DeepSeek [99], Ollama).<br>Aplicație gratuită; cost limitat la tariful modelului ales.                                     | <ul style="list-style-type: none"> <li>• Mapare automată a codului sursă (<i>repo map</i>) pentru context relevant</li> <li>• Integrare Git nativă cu commit-uri automate per modificare</li> <li>• Suport pentru peste 100 de limbaje de programare</li> <li>• Linting și testare automată cu reîncercare la eșec</li> <li>• Suport explicit pentru modele locale</li> </ul> | <ul style="list-style-type: none"> <li>• Fără sandboxing</li> <li>• Capabilități agentice limitate, fără subagenți tipizați paraleli</li> <li>• Fără suport MCP pentru extinderea uneltelor</li> <li>• Fără panou web propriu de transparență</li> <li>• Fără integrare Langfuse pentru urmărirea execuției și evaluări automate</li> </ul>                                                                                                           |

Tabelul 3.2: Comparatie între soluția propusă și principalii asistenți de programare OSS din ecosistem

Pentru a sintetiza poziționarea **vibe-cli** față de întregul ansamblu al concurenței (atât unelte comerciale cât și OSS), în continuare listăm succint funcționalitățile pe care soluția propusă le preia de la celelalte aplicații (demonstrând caracterul de înlocuitor complet) și simetric, funcționalitățile prezente la concurență dar neacoperite direct în implementarea curentă, clasificate în trei categorii: funcționalități în afara scopului proiectului, funcționalități care există indirect prin mecanisme deja prezente și funcționalități total absente.

### 3.1.1 Funcționalități prezente în concordanță cu aplicațiile concurențe proprietare și OSS

1. Suport multi-provider;
2. Modele OSS locale prin Ollama;
3. Suport MCP;
4. Subagenți specializați paraleli;
5. Skills;
6. Interceptări per tură și *filtre* deterministe;
7. Permisuni de aprobare sau respingere per unealtă;
8. Sandboxing opțional în container Docker;
9. Comenzi cu auto-completare;
10. Sesiuni persistente cu posibilitatea de continuare a conversației;
11. Compactare automată a istoricului conversației;
12. Căutare web integrată;
13. Citire, scriere și editare de fișiere;
14. Execuție de comenzi în terminal;
15. Persistarea preferințelor utilizatorului;
16. Intrări multi-modale (imagini, PDF, fișiere text și documente) când modelul selectat le suportă;

### 3.1.2 Funcționalități absente față de aplicațiile concurente

#### 1. Existente indirect, prin mecanisme deja prezente:

- (a) Extensia pentru mediile de dezvoltare este acoperită prin terminalul integrat al oricărui Integrated Development Environment (IDE) sau sistem de operare;
- (b) Integrarea cu GitHub pentru PR-uri sau tichetele GitHub este realizată prin servere MCP sau invocarea utilitarului `gh`;
- (c) Rutare prin agregatoare externe de tip OpenRouter este ușor extensibilă prin editarea `PROVIDER_REGISTRY` din codul sursă;
- (d) Nu există un Software Development Kit (SDK) extern pentru terți, dar modulele Python pot fi importate direct;
- (e) Suport pentru operațiuni de commit Git per modificare nu este suportat nativ dar este realizabil prin capabilități și prin execuție de comenzi în terminal;
- (f) Maparea codului sursă e acoperită parțial de subagentul `code_search`.

#### 2. În afara scopului proiectului:

- (a) Aplicație nativă desktop, mobilă sau extensii IDE;
- (b) Control prin vorbire;
- (c) Suport multi-canal (Slack, Discord, WhatsApp);
- (d) Artefacte interactive;
- (e) Mediu securizat de execuție în cloud și execuție la distanță;
- (f) Suport Kubernetes și orchestrare enterprise;
- (g) Magazin propriu de extensii.

#### 3. Absente:

- (a) Execuție de acțiuni la intervale predefinite sau la momente declarate de timp.

Așadar, prin acoperirea funcționalităților centrale ale concurenței la care se adaugă caracterul open-source multi-provider, panoul web local și integrarea Langfuse, subagenții configurabili și capabilitățile de personalizare, `vibe-cli` se poziționează ca un înlocuitor funcțional complet pentru sarcinile uzuale de programare asistată din terminal, în timp ce diferențele în raport cu concurența țin în principal de suprafețele de utilizare (extensii IDE, aplicații desktop, canale de mesagerie) și de maturitatea ecosistemului din jurul aplicației.

### 3.2 Influența lungimii și a exactității instrucțiunilor asupra performanței. De ce un prompt prea exact nu garantează rezultate mai bune.

Unul dintre experimentele rulate a constatat în evaluarea performanțelor agentului în sarcini de revizuire de PR pe 7 proiecte open-source scrise în limbaje de programare diferite cu două tipuri de prompt de sistem:

- 1. Un prompt sumar, de tipul "Review these PRs, output your findings in this format". Promptul complet este disponibil în sursa [100].
- 2. Un prompt detaliat, de aproximativ 300 de linii care încearcă să maximizeze găsirea tuturor problemelor și îndeamnă agentul să își dea silința să găsească doar probleme reale și să evite problemele triviale. Promptul complet este disponibil în sursa [101]

Experimentul s-a desfășurat în următoarele condiții:



- Baza de date pe care o folosim conține 50 de revizuiți umane considerate ideale. Dorim să testăm dacă agentul nostru reușește să găsească toate problemele pe care le-au găsit inginerii umani în aceste proiecte.
- Baza de date conține o listă cu fiecare revizuire umană, linia la care acesta a adăugat comentariul dar și întregul proiect de la momentul producerii revizuirii pentru context.
- A fost folosită o arhitectură ce pornește mai multe procese paralele care rulează agentul cu acces complet la unelte și subagenți, pentru a procesa mai repede cele 50 de revizuiți.
- La finalul procesului de revizuire, un alt model de limbaj mai mic se uită la revizuirile umane și la cele generate de agent și verifică dacă acestea relatează aceleași probleme.
- Derivăm scorurile de precizie, recunoaștere și  $F_1$  din aceste metrici emise de modelul judecător.
- Modelele folosite au fost Claude Opus 4.7 pentru revizuirea efectivă și Claude Haiku 4.5 pentru agentul judecător și deduplicare.

Pentru a cuantifica performanța agentului, fiecare comentariu produs în raport cu setul de referință uman este clasificat în una dintre următoarele categorii:

- **True Positive (TP):** o problemă semnalată de agent care corespunde unei probleme identificate și în revizuirea umană de referință.
- **False Positive (FP):** o problemă semnalată de agent căreia nu i se asociază nicio observație din revizuirea umană (zgomot sau alertă greșită).
- **False Negative (FN):** o problemă prezentă în revizuirea umană pe care agentul nu a reușit să o identifice (omisiune).
- **True Negative (TN):** o linie de cod pentru care nici agentul, nici recenzentul uman nu au formulat o observație; în contextul revizuirii de PR acest număr este dominant și dificil de definit riguros, motiv pentru care nu intervine în metricile raportate.

Pornind de la aceste categorii, definim *precizia* ca fracțiunea de alerte corecte din totalul alertelor emise de agent, *recunoașterea* (engl. *recall*) ca fracțiunea de probleme reale efectiv detectate, iar scorul  $F_1$  ca media armonică a celor două, oferind o singură valoare scalară ce penalizează dezechilibrul între precizie și recunoaștere:

$$\begin{aligned}
 \text{Precizie} &= \frac{TP}{TP + FP}, \\
 \text{Recunoaștere} &= \frac{TP}{TP + FN}, \\
 F_1 &= 2 \cdot \frac{\text{Precizie} \cdot \text{Recunoaștere}}{\text{Precizie} + \text{Recunoaștere}} = \frac{2 TP}{2 TP + FP + FN},
 \end{aligned} \tag{3.1}$$

unde Precizie, Recunoaștere,  $F_1 \in [0, 1]$ . Valori apropiate de 1 corespund unei performanțe superioare. Un agent care raportează foarte puține probleme poate atinge o precizie ridicată cu o recunoaștere scăzută, în timp ce un agent care raportează agresiv obține recunoaștere mare în detrimentul preciziei; scorul  $F_1$  surprinde acest compromis într-o singură metrică.

Problemele noi sunt probleme pe care modelul judecător le-a considerat ca fiind valide în ciuda faptului că nu se regăseau prin revizuirile umane.

Rezultatele experimentului pot fi regăsite în tabelele 3.3 și 3.4.

| Prompt   | Proiect            | Precizie     | Recunoaștere | F1           | Probleme noi | PRs       |
|----------|--------------------|--------------|--------------|--------------|--------------|-----------|
| Detaliat | cal.com            | 64.3%        | 80.6%        | 71.5%        | 11           | 10        |
| Detaliat | discourse-graphite | 71.1%        | 75.0%        | 73.0%        | 11           | 10        |
| Detaliat | grafana            | 72.7%        | 68.2%        | 70.4%        | 9            | 10        |
| Detaliat | keycloak           | 73.3%        | 63.6%        | 68.1%        | 8            | 9         |
| Detaliat | keycloak-greptile  | 100.0%       | 100.0%       | 100.0%       | 0            | 1         |
| Detaliat | sentry             | 81.2%        | 52.6%        | 63.9%        | 3            | 6         |
| Detaliat | sentry-greptile    | 37.0%        | 75.0%        | 49.6%        | 1            | 4         |
| Detaliat | <b>TOTAL</b>       | <b>66.5%</b> | <b>70.6%</b> | <b>68.5%</b> | <b>43</b>    | <b>50</b> |

(a) Metrici per proiect

| Severitate | Precizie | Recunoaștere | F1    | TP | FP | FN | Probleme noi |
|------------|----------|--------------|-------|----|----|----|--------------|
| critical   | 58.8%    | 88.9%        | 70.8% | 8  | 7  | 1  | 2            |
| high       | 75.7%    | 80.5%        | 78.0% | 33 | 18 | 8  | 23           |
| medium     | 59.5%    | 72.3%        | 65.3% | 34 | 34 | 13 | 16           |
| low        | 67.6%    | 53.8%        | 60.0% | 21 | 11 | 18 | 2            |

(b) Metrici pe nivele de severitate

Tabelul 3.3: Rezultatele pentru prompt-ul detaliat: metrici per proiect și pe nivele de severitate

| Prompt | Proiect            | Precizie     | Recunoaștere | F1           | Probleme noi | PRs       |
|--------|--------------------|--------------|--------------|--------------|--------------|-----------|
| Simplu | cal.com            | 18.2%        | 83.9%        | 30.0%        | 1            | 10        |
| Simplu | discourse-graphite | 24.4%        | 100.0%       | 39.2%        | 2            | 10        |
| Simplu | grafana            | 16.2%        | 90.0%        | 27.5%        | 0            | 10        |
| Simplu | keycloak           | 25.8%        | 95.5%        | 40.7%        | 2            | 9         |
| Simplu | keycloak-greptile  | 33.3%        | 100.0%       | 50.0%        | 0            | 1         |
| Simplu | sentry             | 27.8%        | 73.7%        | 40.3%        | 1            | 6         |
| Simplu | sentry-greptile    | 15.1%        | 83.3%        | 25.5%        | 1            | 4         |
| Simplu | <b>TOTAL</b>       | <b>20.9%</b> | <b>88.8%</b> | <b>33.8%</b> | <b>7</b>     | <b>50</b> |

(a) Metrici per proiect

| Severitate | Precizie | Recunoaștere | F1    | TP | FP  | FN | Probleme noi |
|------------|----------|--------------|-------|----|-----|----|--------------|
| critical   | 17.3%    | 100.0%       | 29.5% | 9  | 43  | 0  | 0            |
| high       | 25.0%    | 95.0%        | 39.6% | 38 | 120 | 2  | 2            |
| medium     | 22.6%    | 89.4%        | 36.1% | 42 | 161 | 5  | 5            |
| low        | 16.3%    | 78.9%        | 27.0% | 30 | 154 | 8  | 0            |

(b) Metrici pe nivele de severitate

Tabelul 3.4: Rezultatele pentru prompt-ul simplu: metrici per proiect și pe nivele de severitate

Analizând datele din tabelele 3.3 și 3.4, putem observa un rezultat neașteptat. Având în vedere tehnica CoT [12], ne-am fi așteptat ca un agent căruia i se spune pas cu pas cum trebuie să opereze să aibă performanțe mai bune decât unul care este lăsat liber.

Totuși, uitându-ne mai atent la datele fals pozitive, putem intui care este cauza acestor rezultate. Din cauza restricțiilor impuse de prompt, modelul cu indicațiile detaliate este reticent în a returna sau cerceta probleme din afara ariei pe care o acoperă instrucțiunile de sistem. Astfel face revizuirii despre care este sigur că sunt corecte și urmează pașii indicați de prompt, având astfel un scor F1 mare și o precizie bună, dar o recunoaștere mai mică decât cu indicațiile simple. Numeric, acest tip de prompt a produs în total 70 de rezultate fals pozitive.

Pe de altă parte, folosind instrucțiunile simple, agentul a fost lăsat să relateze orice problemă, verificată sau nu. Acesta a scris orice pseudo-probleme (reale sau nu) care ar fi putut fi posibile. Astfel, semnalând mai multe probleme (fictive sau nu) a reușit să acopere o suprafață mai mare de probleme, iar probabilitatea ca o problemă reală să se afle prin cele sugerate crește. Acest lucru este clar uitându-ne la secțiunea de fals pozitive unde, în loc de 70 de revizuiți ca și în cazul precedent, aici avem în total 478 de comentarii marcate ca fiind fals pozitive.

Acest studiu reflectă o problemă dominantă în lumea IA: alegerea între precizie și recunoaștere. Cum realizăm un agent care are o recunoaștere perfectă și identifică corect toate instanțele ce trebuie recunoscute fără a returna rezultate nedorite. În domenii sensibile precum cel medical sau militar, acest subiect este foarte dezbătut, deoarece în general dacă încercăm să mărim recunoașterea, precizia va scădea. O alternativă este folosirea scorului F1, dar nu ajută când singura modificare disponibilă este cea asupra instrucțiunilor.

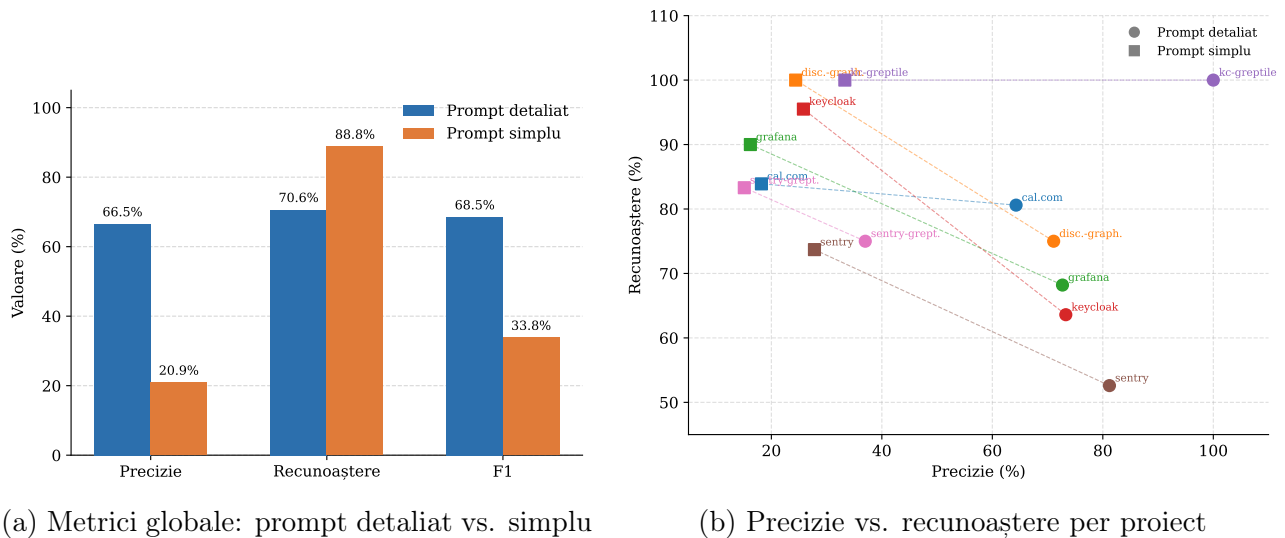


Figura 3.2: Comparație vizuală între prompt-ul detaliat și cel simplu

În experimentul de față, se ridică aceeași problemă: preferăm un agent care găsește toate problemele critice dintr-un sistem, dar sunt ascunse printr-o sumedenie de raportări de severitate mică sau false, ajungând să fie ignorate de ingineri? Sau ne bazăm că inginerul este judecătorul final și că agentul este doar o unealtă, în final tot inginerul ajungând să facă o revizuire completă, bazându-se cât mai puțin pe IA din cauza că poate rata probleme majore?

Răspunsul este cel mai probabil, undeva la mijloc. Proiectantul sistemului agentic trebuie să găsească un echilibru bun între precizie și recunoaștere astfel încât produsul să aibă sens.

**Agentul depistează probleme pe care revizorii umani le-au omis.** Un rezultat semnificativ al experimentului, parțial eclipsat de discuția precizie-recunoaștere, este că agentul a identificat probleme *valide* care nu se regăseau în nicio revizuire umană de referință: **43 de probleme noi** cu prompt-ul detaliat și 7 cu cel simplu, validate ulterior de modelul judecător. Raportat la cele 50 de PR-uri analizate, înseamnă că agentul a adus în medie aproape o problemă nouă per PR față de setul uman. Constatarea este relevantă în practică: chiar dacă agentul nu înlocuiește complet revizuirea umană, el acționează ca un al doilea verficator care identifică omisiuni reale. Valoarea unei astfel de unelte nu se limitează la acoperirea problemelor umane cunoscute, ci și la lărgirea suprafeței de inspecție dincolo de acestea.

### 3.3 Contextul în mediul agentic. Consecințele metodelor de compactare.

Compactarea este un procedeu necesar, după cum am menționat și în capitolele anterioare. Dar, în practică, aceasta nu este perfectă, având consecințe în funcție de metodă sau setul de metode de

compactare alese. Din utilizarea soluției agentice dezvoltate cu mai multe combinații de astfel de metode, am extras un set de repercusiuni ale folosirii acestora pe categorii, dar și un set recomandat pentru performanțe optime în raport cu integritatea informațiilor din fereastra de context.

Cea mai utilă clasificare nu este în funcție de momentul în care rulează compactarea (imediat după apelul uneltelor sau când acesta depășește o limită), ci după natura pierderii pe care o introduce în context. Stadiile care taie literal blocuri din rezultatele uneltelor (`tool_truncate` și `ref_substitute`) pierd date întregi, dar lasă o urmă vizibilă în conversație: un marker de trunchiere sau o referință `@artifact:<sha>`. Stadiile care parafrazează prin LLM (`tool_summarize` pe rezultatele individuale ale uneltelor, `bulk` cu strategie `summarize` sau `hybrid` pe întregul istoric) păstrează întinderea conversației, dar pierde nuanțe fără nicio urmă vizibilă pentru agent. Toate trei stadiile care rescriu rezultatele uneltelor (`tool_truncate`, `tool_summarize`, `ref_substitute`) operează asupra aceluiași slot proiectat și sunt declarate mutual exclusive în procesul `vibe-cli`; oricare configurație validă conține cel mult unul dintre ele.

### 3.3.1 Metode care elimină date din context

Spre exemplu, `tool_truncate` păstrează un cap și o coadă peste `max_tool_tokeni` și externalizează restul în artefacte externe. `ref_substitute` merge mai departe: lasă doar un extras de `excerpt_tokeni` și o referință. În ambele cazuri, în sesiunile testate, agentul ajunge în scurt timp să **reapeleze unealta cu argumente mai înguste** (`grep` după un identificator, `read_file` pe un interval mai mic) pentru a recupera fragmentul dispărut, în loc să cheme `fetch_artifact`. Costul real al acestor stadii nu este deci pierderea de informație, ci tura suplimentară pe care agentul o consumă căutând-o, vizibilă în creșterea numărului de apeluri de unealtă per sarcină.

### 3.3.2 Metode care parafrazează contextul

În contrast, `tool_summarize` este, în cazul de succes, strict superior trunchierii pure: rulează în două faze (o trunchiere sincronă, apoi o sumarizare LLM asincronă care înlocuiește forma trunchiată când termină), iar în caz de eșec rămâne la forma trunchiată. `bulk` face același tip de transformare la nivelul întregului istoric. Diferența față de metodele care elimină date din context este că **pierderea nu lasă urmă**: rezultatul are aceeași formă textuală, dar afirmații cantitative (numere de linii, nume de fișiere, identificatori) pot dispărea silențios. Efectul practic observat este pierderea de *recunoaștere*: agentul nu mai cere date pe care nu le mai are, pentru că rezumatul îi spune că le-a văzut deja. Când modelul de compactare este unul mai mic decât cel principal, același mecanism produce ocazional afirmații incorecte în rezumat, care apoi se propagă ca premisă în turele următoare.

### 3.3.3 Metode auxiliare

Restul stadiilor operează cu alte principii și pot însoți orice combinație de mai sus. Metoda "format compress" reformatează JSON-ul fără apel LLM și fără pierdere semantică: în practică nu am observat efecte negative și poate fi ținut activat necondiționat. `dedupe` elimină apeluri identice și rezultate similare peste un prag (implicit 0,92); pragurile mai mici au șters în sesiunile noastre rezultate de teste cu output similar dar semnificație diferită. `importance_prune` este robust cu setările implicite: pierderea evenimentelor `assistant_thinking` reduce vizibil contextul fără consecințe observabile, iar păstrarea ultimului `tool_result` per tură funcționează pentru că rezultatele intermediare sunt subsumate de cel final.

### 3.3.4 Secvență recomandată de metode de compactare

Configurația care a oferit cel mai bun compromis între reamintire și ocupare a ferestrei de context în experimentele rulate, are următoarea formă:

1. `format_compress` activat aduce câștig de spațiu garantat, fără pierderi;

2. `tool_summarize` ca singur reprezentant al grupeii mutual exclusive de rescriere a rezultatelor uneltelor (`tool_truncate` rămâne soluția sa de rezervă sincronă, deci este implicit prezentă);
3. `dedupe` cu pragul 0,92, poziționat după `format_compress` pentru că normalizarea formataării crește rata de detecție;
4. `importance_prune` cu configurarea implicită;
5. `bulk` cu strategia `hybrid` și `min_turns_kept` = 4, suficient pentru a acoperi o iterație completă cerere-investigare-răspuns.

Pentru sarcini scurte de tip întrebare și răspuns rularea cu funcționalitatea de compactare complet dezactivată nu aduce probleme. Pentru sarcini cu citiri exhaustive de cod, înlocuirea `tool_summarize` cu `ref_substitute` reduce semnificativ ocuparea, cu condiția ca promptul de sistem să instruiască explicit agentul când să apeleze `fetch_artifact`. În modul `tool_persistence: true`, în care întregul istoric al uneltelor trăiește între interacțiuni, prezența a cel puțin unui stadiu ce rulează imediat devine obligatorie: în lipsa lui, fereastra se poate umple într-un număr relativ mic de ture cu volum normal de unelte, înainte ca pragul `threshold` să apuce să declanșeze `bulk`.

### 3.4 Modele optime pentru sarcini de programare. Influența datelor de antrenare și a mărimii modelului.

Nu toate modelele au capacități similare când vine vorba despre sarcini de dezvoltare software. Pe lângă necesitatea funcționalităților de gândire rațională adâncă, acestea trebuie să fie antrenate pentru a apela funcții în mod eficient și pentru a respecta dogme de programare precum SDD sau TDD.

Pentru a cuantiza exact cât de importantă este alegerea unui tip de model pentru sarcini de dezvoltare software, am încercat rularea programului cu mai multe tipuri de modele, cu capacități diferite și mărime diferită (ca număr de parametri).

Modelele folosite, împreună cu caracteristicile lor relevante, sunt prezentate în tabelul 3.5. Selecția acoperă atât modele accesate prin API (familiile Claude și Gemini), cât și modele open-weight rulate local prin `ollama`, într-o plajă largă de dimensiuni (de la 1.5 miliarde la 32 miliarde de parametri).

| Model              | Tip                                | Parametri  | Mărime pe disc | Multi-modal |
|--------------------|------------------------------------|------------|----------------|-------------|
| Claude Opus 4.7    | LLM raționament                    | nepublicat | nepublicat     | da          |
| Claude Sonnet 4.6  | LLM uz general                     | nepublicat | nepublicat     | da          |
| Claude Haiku 4.5   | LLM compact, uz general            | nepublicat | nepublicat     | da          |
| Gemini 2.5 Flash   | LLM uz general                     | nepublicat | nepublicat     | da          |
| Gemini 2.5 Pro     | LLM raționament                    | nepublicat | nepublicat     | da          |
| Gemini 2.0 Flash   | LLM uz general                     | nepublicat | nepublicat     | da          |
| deepseek-r1:32b    | LLM, raționament                   | 32 B       | 19 GB          | nu          |
| gemma4:26b         | LLM, uz general                    | 26 B       | 17 GB          | da          |
| gemma4:8b          | LLM, uz general                    | 8 B        | 5.1 GB         | da          |
| phi4:14b           | LLM, uz general                    | 14 B       | 9.1 GB         | nu          |
| llama3.2:3b        | Small Language Model, uz general   | 3 B        | 2.0 GB         | nu          |
| devstral-small-2   | LLM, generare cod                  | 24 B       | 15 GB          | nu          |
| qwen3-coder:30b    | LLM, generare cod                  | 30 B       | 18 GB          | nu          |
| qwen3.5            | LLM, uz general                    | 9.7 B      | 6.2 GB         | nu          |
| qwen2.5-coder:7b   | LLM, generare cod                  | 7 B        | 4.7 GB         | nu          |
| qwen2.5-coder:1.5b | Small Language Model, generare cod | 1.5 B      | 986 MB         | nu          |

Tabelul 3.5: Modelele testate: tip, număr de parametri, mărime pe disc și suport multi-modal.

Pentru rezultate comparabile, toate modelele au avut de realizat aceleași operații. Testul a constatat din 6 sarcini de intensitate diferită, executate pe un proiect real. Fiecare etapa a avut propriul mod de recompensare: de la metrici de similaritate, la LLM-as-judge, la teste ascunse. Toate acestea testează corectitudinea soluției, unelte sau capacitățile pe care le-a apelat și dacă în final implementarea trece suita de teste. Aceste metrici iau forma unor procentaje, valori binare (admis/respins) sau scoruri, reflectând atât progresul înregistrat în rezolvarea sarcinii, cât și calitatea raționamentului modelului.

Configurația testului împreună cu sarcinile pe care agenții le-au avut de rezolvat pot fi consultate în sursa [102].

### 3.4.1 Rezultate

Rezultatele agregate ale rulărilor sunt prezentate în tabelul 3.6, sortate după scorul mediu obținut pe ansamblul cazurilor de test.

| #  | Furnizor  | Model             | Param. (Mld.) | Scor   | Durata      | Cost (USD) | Tokeni totali |
|----|-----------|-------------------|---------------|--------|-------------|------------|---------------|
| 1  | anthropic | claude-opus-4-7   | —             | 0.9062 | 7 min 27 s  | 13.40      | 803 585       |
| 2  | anthropic | claude-sonnet-4-6 | —             | 0.8743 | 10 min 4 s  | 0.06       | 17 606        |
| 3  | anthropic | claude-haiku-4-5  | —             | 0.8081 | 7 min 0 s   | 0.02       | 17 361        |
| 4  | gemini    | gemini-2.5-flash  | —             | 0.7871 | 3 min 6 s   | 0.16       | 400 841       |
| 5  | ollama    | devstral-small-2  | 24            | 0.7051 | 14 min 22 s | 0.00       | 14 620        |
| 6  | ollama    | gemma4            | 26            | 0.6803 | 9 min 49 s  | 0.00       | 6 241         |
| 7  | gemini    | gemini-2.5-pro    | —             | 0.6431 | 4 min 14 s  | 0.67       | 435 404       |
| 8  | ollama    | qwen3-coder       | 30            | 0.6256 | 4 min 33 s  | 0.00       | 13 926        |
| 9  | ollama    | gemma4            | 8             | 0.3803 | 3 min 26 s  | 0.00       | 6 852         |
| 10 | ollama    | qwen3.5           | 9.7           | 0.2963 | 7 min 28 s  | 0.00       | 15 031        |
| 11 | ollama    | deepseek-r1       | 32            | 0.0747 | 4 min 15 s  | 0.00       | 1 484         |
| 12 | ollama    | llama3.2:3b       | 3             | 0.0590 | 35 s        | 0.00       | 5 380         |
| 13 | ollama    | qwen2.5-coder     | 7             | 0.0442 | 32 s        | 0.00       | 834           |
| 14 | ollama    | qwen2.5-coder     | 1.5           | 0.0298 | 21 s        | 0.00       | 828           |
| 15 | ollama    | phi4              | 14            | 0.0000 | Eroare      | 0.00       | —             |
| 16 | gemini    | gemini-2.0-flash  | —             | 0.0000 | Eroare      | 0.00       | —             |

Tabelul 3.6: Clasamentul modelelor după scor mediu, durata totală de rulare și cost. Modelele rulate local prin ollama au cost zero. Modelele proprietate nu au date publice referitoare la mărimea modelului.

### 3.4.2 Analiza rezultatelor

Interpretând rezultatele obținute putem face mai multe observații:

1. Numărul de parametri nu este întotdeauna direct proporțional cu capabilitățile. O bună parte din capacitatea de rezolvare a problemelor în dezvoltarea software este extrasă din specializarea modelelor pe sarcini de acest tip. De aceea modelele care au fost pregătite pentru așa ceva au performanțe mai bune în ciuda numărului de parametri. Un viitor experiment ar fi încercarea modelelor de la Anthropic care aici au dominat tabela pe alte domenii de activitate;

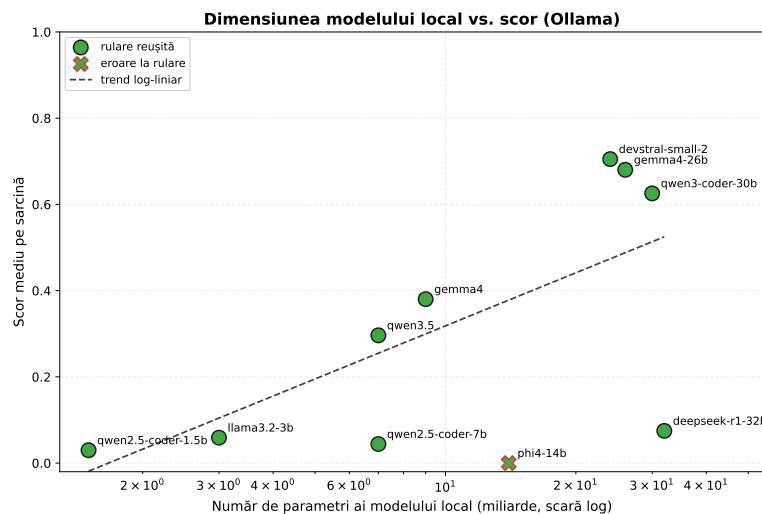
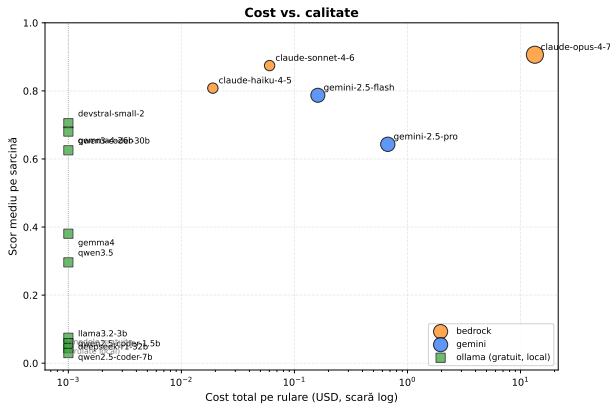


Figura 3.3: Corelația între numărul de parametri ai modelelor locale și scor

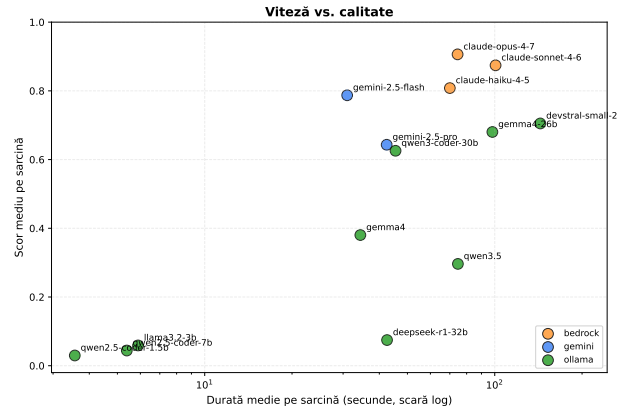
2. Putem vedea influența competențelor de gândire adâncă asupra numărului de tokeni folosiți (și automat a costului). Prin tehnica CoT [12] aceste modele reușesc să își formeze o logică superioară care ajută la rezolvarea problemelor complexe. Dar, acest procedeu vine cu un cost. Iar în cazul Opus 4.7, acest cost este mare. Pentru 6 sarcini de programare acesta a reușit să producă prin gândire și cercetarea fișierelor 800.000 de tokeni (în valoare de aproximativ 13 dolari): de **≈45 de ori** mai mulți tokeni față de Sonnet 4.6 (17.606) și Haiku 4.5 (17.361), diferență atribuibilă aproape în întregime gândirii extinse (CoT [12]). Un cost enorm comparat cu celelalte modele, chiar și de la Anthropic;
3. Modelele prea mici fie nu suportă apelarea de unelte din cauza depășirii ferestrei de context, fie nu sunt antrenate pentru așa ceva și întorc o eroare sau halucinează. Gemini 2.0 Flash și Phi4 întorc erori care denotă incompatibilitatea cu sarcini agentice. Restul modelelor cu scoruri sub 10%, au halucinat rezolvarea problemei și nu au apelat nici o unealtă cu adevărat. Unele dintre ele, chiar dacă ar fi reușit să apeleze uneltele, nu ar fi putut să ducă o conversație prea complexă din cauza ferestrei mici de context;
4. Modelele locale necesită hardware performant pentru a rezolva probleme reale. Deși rezultatele sunt bune și comparabile cu cele ale modelelor de top la o fracțiune din numărul de parametri, timpul de execuție încă creează o experiență a utilizatorului neoptimă pe sisteme neperformante;
5. Variante ale modelelor antrenate special pentru sarcini de dezvoltare software arata o diferență semnificativă în performanță, precum Qwen3.5 și Qwen3-Coder.

### Anomalii în rezultate:

1. Modele care dețin în teorie capabilități de gândire (Sonnet, DeepSeek R1) nu au exercitat în timpul testului aceste capabilități la potențialul maxim, ceea ce sugerează setări implicite de efort ale modelului reduse;
2. Modelul Haiku a avut o performanță surprinzătoare având în vedere că acesta este considerat cel mai slab din flota Anthropic. Această aparență se datorează în mare antrenamentului pentru acest tip de sarcini, dar și discrepantei de concurență din cadrul testului. Nu am avut acces la modele de aceeași generație OpenAI sau Gemini cu care să putem face o comparație corectă, iar experimentul urmarea în principal relația dintre cost și performanță;



(a) Cost vs. calitate.



(b) Viteză vs. calitate.

3. Modele cu mai mulți parametri au avut un scor mai slab decât modelele cu număr mai mic de parametri. Spre exemplu DeepSeek R1 nu a fost antrenat pentru apelarea uneltelor și deși acesta știa teoretic cum ar rezolva problema, nu a apelat nici o unealtă. Pe scurt, a fabricat o narațiune, o poveste despre cum ar rezolva el problema, iar la final a declarat-o rezolvată;
4. Gemini 2.5 Pro, considerat în mod uzual mai capabil decât Flash pentru sarcini complexe, a obținut un scor mai mic (0.6431, locul 7) față de Gemini 2.5 Flash (0.7871, locul 4). O explicație posibilă este că modelele Pro sunt optimizate pentru raționament profund pe context lung, avantaj care nu se manifestă în sarcini agentice scurte.

### Recomandări extrase din acest experiment:

1. Pentru rularea modelelor locale, este necesar un hardware bun. Timpul de inferență poate fi mai crescut decât la modelele furnizorilor, sau mai mic pentru unele modele în funcție de hardware;
2. Costul modelelor crește odată cu capabilitățile de gândire și exponențial cu mărimea contextului;
3. În general, modelele prea mici (mai mici de 20 de miliarde de parametri) nu reușesc să se descurce în sarcini de programare. Pot răspunde la întrebări punctuale, dar nu pot lucra într-un mediu agentic, având un risc mare de halucinații. Prin urmare, alegerea dimensiunii modelului trebuie calibrată în funcție de complexitatea sarcinii: pentru întrebări simple sau operații izolate sunt suficiente modele compacte, în timp ce sarcinile complexe, multi-pas, care implică planificare și apel de unelte necesită modele de dimensiuni mai mari;
4. Alegerea unui model în funcție de sarcină este benefică și pentru optimizarea costurilor de utilizare, modelele mari având capabilități de gândire mai adâncă dar și un preț mai mare, nejustificat pentru abordarea problemelor simple.
5. Modelele neantrenate pe sarcini agentice (apel de unelte, gândire) nu au performanțe în mediul agentic pentru dezvoltarea software având un risc mare de halucinații;



6. Modelele mai mici ale furnizorilor de modele de limbaj sunt mai avantajoase din punct de vedere al raportului calitate-preț;
7. Gândirea crește și ea timpul de inferență. O comparație bună ar fi între Devstral Small 2 și Gemma4:26b. Deși Devstral este un model mai mic, din numărul de tokeni utilizați putem deduce că a avut o gândire mai adâncă ce a dus la timpul ridicat de rezolvare. În același timp, Gemma a avut rezultate similare cu un timp mai scăzut de inferență (deși Gemma are și o arhitectură diferită);
8. Modele open-source recomandate: Gemma4 pentru echilibrul dintre scor și performanță, Devstral pentru scorul bun, cu dezavantajul timpului mare de inferență și Qwen3-Coder pentru timpul foarte scurt de inferență și scor bun;
9. Din punctul de vedere al modelelor proprietare, Sonnet și Haiku au avut performanțe foarte bune, dar și Gemini 2.5 Flash a surprins cu performanțele sale (având în vedere că nu mai este de ultimă generație în familia Gemini);
10. Folosirea limbii engleze pentru discuțiile cu modelele de limbaj. În special modelele de mărimi mici sunt antrenate preponderent folosind limba engleză. Performanța acestora poate scădea folosind limbi diferite;
11. Similar, pentru limbajul de programare, unele modele vor avea performanțe mai ridicate pe anumite limbaje (precum Python) și mai slabe în sarcini pe alte limbaje, în funcție de datele de antrenament.

### 3.4.3 Modelele open-source: evaluarea valorii și a potențialului de utilizare

La finalul analizei rezultatelor, putem rezuma performanța modelelor open-source într-un singur cuvânt: surprinzătoare.

A fost neașteptat ca 6 din cele 16 modele să aibă scor aproximativ 0 în cadrul experimentului, dar nu imposibil. Era de așteptat ca modelele sub 10 miliarde de parametri să întâmpine probleme, motiv pentru care le-am și ales. Dar cele între 10 și 20 de miliarde, cât și DeepSeek R1 au surprins prin neajunsurile lor.

Pe de altă parte am fost impresionat de rezultatele Devstral Small 2, Qwen3-Coder și Gemma 4. Acestea au un potențial imens, mai ales cele două din urmă care oferă un timp de inferență foarte bun. Gemma4 este interesant și deoarece are o arhitectură specială de tipul Mixture of Experts.

În final, consider că folosirea modelelor open-source locale pentru sarcini de dezvoltare software (considerând deținerea unui sistem performant) este motivată de costurile reduse, controlul asupra datelor, timpul scurt de inferență dar și performanțele bune (care pe măsură ce domeniul se dezvoltă vor crește).

## Capitolul 4

# DEZVOLTĂRI VIITOARE

Finalizând analiza post-implementare, împreună cu experimentele rulate, putem extrage un set de potențiale îmbunătățiri pe care le putem aduce aplicației în viitor.

### 4.1 Automatizări, integrări cu alte programe

O zonă potențială de dezvoltare este suportul pentru rularea de sarcini agentice la un interval de timp, sau la un moment în timp predefinit. Astfel de implementări sunt deja regăsite în unelte precum Claude Code, prin comanda `/loop`. Această funcționalitate permite rularea unui prompt în mod automat la un moment determinat. Un exemplu bun pentru această utilizare în domeniul dezvoltării software ar fi un prompt de tipul:

```
Realizează un rezumat al celor mai importante tichete sau PR-uri apărute în ultimele  
24 de ore pe GitHub, și analizează canalele de Slack, Discord, Teams la care  
ai acces. Fă-mi un rezumat complet cu ceea ce găsești.
```

Acest prompt ar fi foarte util pentru a fi la curent cu starea proiectelor la care se lucrează și la deciziile ce se întâmplă în afara programului.

### 4.2 Integrarea cu mediile de dezvoltare

Deși interfața de tip CLI este foarte versatilă, compatibilitatea cu mediile de dezvoltare aduce o granularitate mai mare asupra interacțiunii cu agentul. Acesta poate sugera schimbări restrânse (linie cu linie), pe care utilizatorul le poate accepta sau respinge individual.

### 4.3 Construirea unui graf de cunoștințe pentru optimizarea înțelegerii proiectelor

O metodă inovativă care are potențialul de a îmbunătăți contextul unui agent din punctul de vedere al fabricării legăturilor între componentele unui proiect, dar mai ales din punctul de vedere al numărului de tokeni de intrare și al reducerii apelului de unelte este aceea de a construi grafuri ale legăturilor între diverse părți ale proiectului.

Una dintre soluțiile pentru implementarea acestei facilități este utilizarea Graphify [103]. Acest program construiește grafurile necesare înțelegerii proiectului și folosește modele de limbaj pentru a detalia grafurile din punct de vedere semantic. Acest lucru reduce numărul de apeluri la unelte pentru citirea fișierelor, îmbunătățind astfel utilizarea contextului și sporind cunoașterea amănunțită a proiectului pe care se lucrează.

Integrarea este una simplă, graful se construiește o dată (și ulterior automat dacă apar schimbări în cod) iar aplicațiile agentice precum Claude Code, pot folosi integrarea oficială sau pot folosi un server MCP pentru a interacționa cu acesta.

## 4.4 Suport pentru metode de compactare și sandboxing avansate. SDK pentru agenți

Tehnicile avansate de compactare bazate pe analiza similarității semantice a frazelor ce urmează a fi șterse sau a importanței reduse sunt o completare bună la setul de metode deja existente. Aceste tehnici pot cimenta consistența informațiilor importante, protejându-le de compactare.

Din punctul de vedere al securității, containerele Docker nu sunt impenetrabile. O îmbunătățire ce poate fi adusa în zona de sandboxing ar fi introducerea suportului pentru platforme cloud specializate în acest domeniu precum E2B.

O ultimă schimbare care ar ușura munca potențialilor contributori la dezvoltarea programului ar fi dezvoltarea unui SDK prin care aceștia ar putea apela cu ușurință metodele programului aducând astfel un plus de valoare.

## CONCLUZII

Lucrarea a urmărit proiectarea și realizarea unui asistent inteligent de programare, agnostic față de furnizorul de modele de limbaj, transparent și extensibil, capabil să ruleze cu modele locale prin Ollama, sau în cloud. Obiectivul a fost atins prin aplicația `vibe-cli`, livrată cu un set complet de capabilități agentice precum sandboxing, permisiuni granulare, subagenți paraleli complet configurabili, suport MCP cu toate metodele de comunicare uzuale, intrări multi-modale și proces configurabil de compactare, dar și cu tehnologii proprii de observabilitate pe două niveluri, ce expune utilizatorului metrice detaliate despre consumul de tokeni, cost, latență și comportamentul agentului.

Ca și contribuții proprii, am realizat un proces de compactare a contextului configurabil și extensibil, organizat în transformări locale aplicate imediat după fiecare unealtă și transformări globale declanșate la prag și am proiectat o arhitectură de echipe de agenți, cu un agent orchestrator ce planifică și delegă sarcini către subagenți specializați rulați în paralel, complet configurabili, precum și un sistem de capabilități și preferințe injectate în context, gestionabile prin comenzi interne și definibile atât de utilizator, cât și de agent.

La acestea se adaugă un sistem de filtre deterministe peste apelurile de unelte, ancorat pe un protocol *read-before-write* cu verificare de hash, ce completează izolarea prin sandboxing și permisiunile granulare, un mecanism de detectare automată a capabilităților multi-modale cu abordare hibridă pentru documente și persistarea atașamentelor ca artefacte recuperabile, suport MCP cu toate metodele de transport uzuale.

De asemenea, am implementat tehnici de observabilitate: urmărire externă a execuției prin Langfuse cu bucle de feedback și un panou web local care expune metrice precum consumul de tokeni pe categorii, evoluția contextului față de limita modelului, de latența și debitul uneltelor, costul estimat și istoricul compactărilor dar și interogarea interacțiunilor, a delegărilor către subagenți și unelte și a economiilor de context aduse agentului orchestrator prin distribuirea activităților către subagenți specializați.

În final am implementat și rulat o suită de experimente proprii: evaluarea de revizuire de PR pe 50 de referințe umane din 7 proiecte OSS, investigarea stadiilor de compactare și compararea a 16 modele pe 6 sarcini de programare din care am extras un set de recomandări transferabile la ecosistemul mai larg de unelte similare.

În ansamblu, lucrarea arată că o soluție agentică transparentă ce rulează local, agnostică față de furnizorii de modele de limbaj este nu doar fezabilă, ci poate ocupa un loc distinct în peisajul actual, dominat de produse strâns legate de ecosistemele distribuitorilor mari. Direcțiile de evoluție identificate în capitolul *Dezvoltări viitoare* (integrări IDE, grafuri de cunoștințe, sandboxing avansat, SDK pentru terți) pot transforma prototipul actual într-o platformă matură, capabilă să acompanieze evoluția rapidă a domeniului.

# BIBLIOGRAFIE

- [1] JetBrains. Pycharm: The python ide for professional developers. disponibil la: <https://www.jetbrains.com/pycharm/>, 2026. accesat la data: 08.06.2026.
- [2] Microsoft. Visual studio code. disponibil la: <https://code.visualstudio.com>, 2026. accesat la data: 12.04.2026.
- [3] LangChain. Langchain: Applications that can reason. powered by langchain. disponibil la: <https://www.langchain.com>, 2026. accesat la data: 02.05.2026.
- [4] LangChain. Langgraph: Build resilient language agents as graphs. disponibil la: <https://www.langchain.com/langgraph>, 2026. accesat la data: 21.04.2026.
- [5] Google. Agent development kit (adk) documentation. disponibil la: <https://google.github.io/adk-docs/>, 2026. accesat la data: 12.04.2026.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- [7] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models.
- [8] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer.
- [9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. version: 1.
- [10] Ruixiang Zhang, Richard He Bai, Huangjie Zheng, Navdeep Jaitly, Ronan Collobert, and Yizhe Zhang. Embarrassingly simple self-distillation improves code generation.
- [11] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.
- [12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. 35:24824–24837.
- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models.
- [14] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. ReWOO: Decoupling reasoning from observations for efficient augmented language models.
- [15] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. 12:157–173.
- [16] Anthropic. Build with claude: Context windows. disponibil la: <https://platform.claude.com/docs/en/build-with-claude/context-windows>, 2026. accesat la data: 17.05.2026.
- [17] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?
- [18] Anthropic. Claude.ai settings: Usage dashboard. disponibil la: <https://claude.ai/settings/usage>, 2026. accesat la data: 14.04.2026.

- [19] Letta. Guide to context engineering. disponibil la: <https://www.letta.com/blog/guide-to-context-engineering>, 2026. accesat la data: 30.05.2026.
- [20] Ollama. Ollama: Get up and running with large language models locally. disponibil la: <https://ollama.com>, 2026. accesat la data: 16.04.2026.
- [21] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain.
- [22] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. 86(11):2278–2324.
- [23] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. 9:1735–1780.
- [24] OpenAI. Openai. disponibil la: <https://openai.com>, 2026. accesat la data: 19.05.2026.
- [25] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners.
- [26] Yichun Xu, Navjot K. Khaira, and Tejinder Singh. KV cache optimization strategies for scalable and efficient LLM inference. version: 1.
- [27] Weian Mao, Xi Lin, Wei Huang, Yuxin Xie, Tianfu Fu, Bohan Zhuang, Song Han, and Yukang Chen. TriAttention: Efficient long reasoning with trigonometric KV compression.
- [28] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding.
- [29] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding.
- [30] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate.
- [31] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. 36:10088–10115.
- [32] Zhongmou He, Yee Man Choi, Kexun Zhang, Ivan Bercovich, Jiabao Ji, Juntong Zhou, Dejia Xu, Aidan Zhang, Yixiao Zeng, and Lei Li. HARDTESTGEN: A high-quality RL verifier generation pipeline for LLM algorithmic coding.
- [33] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, Hyojung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidya-dhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques.
- [34] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines.
- [35] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc.
- [36] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634. Association for Computational Linguistics.
- [37] Kiran Vodrahalli, Santiago Ontanon, Nilesch Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopal Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, Orhan Firat, Angeliki Lazaridou, Jean-Baptiste Lespiau, Nithya Attaluri, and Kate Olszewska. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. version: 2.

- [38] Dongfang Li, Zixuan Liu, Gang Lin, Baotian Hu, and Min Zhang. LycheeCluster: Efficient long-context inference with structure-aware chunking and hierarchical KV indexing.
- [39] OpenAI. Codex cli: Lightweight coding agent that runs in the terminal, github. disponibil la: <https://github.com/openai/codex>, 2026. accesat la data: 28.04.2026.
- [40] Google. Gemini cli: An open-source ai agent, github. disponibil la: <https://github.com/google-gemini/gemini-cli>, 2026. accesat la data: 20.04.2026.
- [41] Anthropic. Claude code documentation. disponibil la: <https://docs.anthropic.com/en/docs/claude-code/overview>, 2026. accesat la data: 04.04.2026.
- [42] Anysphere Inc. Cursor: The ai code editor. disponibil la: <https://cursor.com>, 2026. accesat la data: 18.06.2026.
- [43] Amazon Web Services. Kiro: The ai ide for prototype to production. disponibil la: <https://kiro.dev>, 2026. accesat la data: 18.06.2026.
- [44] Anthropic. Anthropic api: Models overview. disponibil la: <https://platform.claude.com/docs/en/about-claude/models/overview>, 2026. accesat la data: 08.05.2026.
- [45] OpenAI. Openai api: Models documentation. disponibil la: <https://developers.openai.com/api/docs/models>, 2026. accesat la data: 25.05.2026.
- [46] Google. Gemini api: Models documentation. disponibil la: <https://ai.google.dev/gemini-api/docs/models>, 2026. accesat la data: 15.04.2026.
- [47] DeepSeek. Deepseek api: Pricing. disponibil la: [https://api-docs.deepseek.com/quick\\_start/pricing](https://api-docs.deepseek.com/quick_start/pricing), 2026. accesat la data: 04.04.2026.
- [48] Alibaba Cloud. Qwen model family on hugging face. disponibil la: <https://huggingface.co/Qwen>, 2026. accesat la data: 13.04.2026.
- [49] Mistral AI. Mistral ai models overview. disponibil la: [https://docs.mistral.ai/getting-started/models/models\\_overview/](https://docs.mistral.ai/getting-started/models/models_overview/), 2026. accesat la data: 28.04.2026.
- [50] Anthropic. Anthropic. disponibil la: <https://www.anthropic.com>, 2026. accesat la data: 19.05.2026.
- [51] Anthropic. Claude.ai: Web application. disponibil la: <https://claude.ai>, 2026. accesat la data: 18.04.2026.
- [52] Google. Google. disponibil la: <https://www.google.com>, 2026. accesat la data: 10.04.2026.
- [53] Google. Gemini: Ai assistant by google. disponibil la: <https://gemini.google.com>, 2026. accesat la data: 04.04.2026.
- [54] Google. Antigravity cli: Terminal-first surface for antigravity agents, github. disponibil la: <https://github.com/google-antigravity/antigravity-cli>, 2026. accesat la data: 21.06.2026.
- [55] Google. Notebooklm: Ai research assistant. disponibil la: <https://notebooklm.google.com>, 2026. accesat la data: 08.06.2026.
- [56] Google Cloud. Vertex ai: Build, deploy, and scale ml models. disponibil la: <https://cloud.google.com/vertex-ai>, 2026. accesat la data: 09.06.2026.
- [57] Google. Firebase studio: Cloud-based development environment. disponibil la: <https://firebase.google.com/products/studio>, 2026. accesat la data: 30.04.2026.
- [58] OpenAI. Chatgpt. disponibil la: <https://chatgpt.com>, 2026. accesat la data: 29.04.2026.
- [59] GitHub Inc. Github: Where the world builds software. disponibil la: <https://github.com>, 2026. accesat la data: 28.05.2026.
- [60] Microsoft. Microsoft corporation. disponibil la: <https://www.microsoft.com>, 2026. accesat la data: 29.04.2026.
- [61] GitHub Inc. Github copilot: Ai pair programmer. disponibil la: <https://github.com/features/copilot>, 2026. accesat la data: 06.05.2026.
- [62] JetBrains. JetBrains: Developer tools. disponibil la: <https://www.jetbrains.com>, 2026. accesat la data: 22.04.2026.
- [63] OpenClaw Contributors. Openclaw: Open-source ai assistant platform, github. disponibil la: <https://github.com/openclaw/openclaw>, 2026. accesat la data: 14.04.2026.

- [64] Meta Platforms. Whatsapp messenger. disponibil la: <https://www.whatsapp.com>, 2026. accesat la data: 30.04.2026.
- [65] Telegram FZ-LLC. Telegram messenger. disponibil la: <https://telegram.org>, 2026. accesat la data: 08.05.2026.
- [66] Slack Technologies. Slack: Channel-based messaging platform. disponibil la: <https://slack.com>, 2026. accesat la data: 30.04.2026.
- [67] Discord Inc. Discord: Group chat platform. disponibil la: <https://discord.com>, 2026. accesat la data: 05.04.2026.
- [68] Docker Inc. Docker: Accelerated container application development. disponibil la: <https://www.docker.com>, 2026. accesat la data: 16.05.2026.
- [69] Apple Inc. macos operating system. disponibil la: <https://www.apple.com/macos/>, 2026. accesat la data: 29.05.2026.
- [70] Google. Android operating system. disponibil la: <https://www.android.com>, 2026. accesat la data: 13.04.2026.
- [71] OpenHands Contributors. Openhands: Code less, make more, github. disponibil la: <https://github.com/OpenHands/OpenHands>, 2026. accesat la data: 21.04.2026.
- [72] Cloud Native Computing Foundation. Kubernetes: Production-grade container orchestration. disponibil la: <https://kubernetes.io>, 2026. accesat la data: 15.05.2026.
- [73] Atlassian. Jira: Issue & project tracking software. disponibil la: <https://www.atlassian.com/software/jira>, 2026. accesat la data: 21.04.2026.
- [74] Cline Contributors. Cline: Autonomous coding agent for vs code, github. disponibil la: <https://github.com/cline/cline>, 2026. accesat la data: 14.05.2026.
- [75] OpenJS Foundation. Node.js: Javascript runtime built on chrome's v8 engine. disponibil la: <https://nodejs.org>, 2026. accesat la data: 29.05.2026.
- [76] Block Inc. Goose: Local, extensible, open-source ai agent, github. disponibil la: <https://github.com/aaif-goose/goose>, 2026. accesat la data: 25.05.2026.
- [77] Rust Foundation. Rust programming language official website. disponibil la: <https://www.rust-lang.org>, 2026. accesat la data: 08.04.2026.
- [78] Aider Contributors. Aider: Ai pair programming in your terminal, github. disponibil la: <https://github.com/Aider-AI/aider>, 2026. accesat la data: 06.05.2026.
- [79] Git Project. Git: Distributed version control system. disponibil la: <https://git-scm.com>, 2026. accesat la data: 29.04.2026.
- [80] Python Software Foundation. Python programming language official website. disponibil la: <https://www.python.org>, 2026. accesat la data: 12.05.2026.
- [81] BerriAI. Litellm: Call all llm apis using the openai format. disponibil la: <https://www.litellm.ai>, 2026. accesat la data: 19.05.2026.
- [82] OpenAI. Openai python sdk, github. disponibil la: <https://github.com/openai/openai-python>, 2026. accesat la data: 14.05.2026.
- [83] Langfuse. Langfuse: Open-source llm engineering platform. disponibil la: <https://langfuse.com>, 2026. accesat la data: 04.05.2026.
- [84] Textualize. Textual: Tui framework for python documentation. disponibil la: <https://textual.textualize.io>, 2026. accesat la data: 11.04.2026.
- [85] Jonathan Slenders. Python prompt toolkit documentation. disponibil la: <https://python-prompt-toolkit.readthedocs.io>, 2026. accesat la data: 25.04.2026.
- [86] Will McGugan. Rich: Python library for rich text and beautiful formatting in the terminal. disponibil la: <https://rich.readthedocs.io>, 2026. accesat la data: 09.04.2026.
- [87] Urwid Project. Urwid: Console user interface library for python. disponibil la: <https://urwid.org>, 2026. accesat la data: 30.04.2026.
- [88] Amazon Web Services. Amazon bedrock: The easiest way to build and scale generative ai applications. disponibil la: <https://aws.amazon.com/bedrock/>, 2026. accesat la data: 06.05.2026.



- [89] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
- [90] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. A survey on agentic security: Applications, threats and defenses.
- [91] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In Leon Derczynski, Jekaterina Novikova, and Muhao Chen, editors, *Proceedings of the The First Workshop on LLM Security (LLMSEC)*, pages 101–114. Association for Computational Linguistics.
- [92] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback.
- [93] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In Yansong Feng and Els Lefever, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445. Association for Computational Linguistics.
- [94] Yue Liu, Hongcheng Gao, Shengfang Zhai, Yufei He, Jun Xia, Zhengyu Hu, Yulin Chen, Xihong Yang, Jiaheng Zhang, Stan Z. Li, Hui Xiong, and Bryan Hooi. GuardReasoner: Towards reasoning-based LLM safeguards.
- [95] Cloud Native Computing Foundation. Opentelemetry: High-quality, ubiquitous, and portable telemetry. disponibil la: <https://opentelemetry.io>, 2026. accesat la data: 10.06.2026.
- [96] GitHub Inc. About copilot cli documentation. disponibil la: <https://docs.github.com/en/copilot/concepts/agents/about-copilot-cli>, 2026. accesat la data: 12.04.2026.
- [97] Microsoft. Azure openai service documentation. disponibil la: <https://azure.microsoft.com/en-us/products/ai-services/openai-service>, 2026. accesat la data: 02.05.2026.
- [98] OpenRouter. Openrouter: A unified interface for llms. disponibil la: <https://openrouter.ai>, 2026. accesat la data: 11.04.2026.
- [99] DeepSeek. Deepseek. disponibil la: <https://www.deepseek.com>, 2026. accesat la data: 18.05.2026.
- [100] Eduard-Daniel Nănescu. Prompt sumar pentru revizuirea pr-urilor, github gist. disponibil la: <https://gist.github.com/Moshulika/b3f1d85fa269bae8c06b216d8e2e8776>, 2026. accesat la data: 24.05.2026.
- [101] Eduard-Daniel Nănescu. Prompt complet pentru revizuirea pr-urilor, github gist. disponibil la: <https://gist.github.com/Moshulika/6f2859da2291ec38544f6deaa209613a>, 2026. accesat la data: 26.04.2026.
- [102] Eduard-Daniel Nănescu. Configurația testului de evaluare agentică, github gist. disponibil la: <https://gist.github.com/Moshulika/7c5aecdc01c3e7756c1b5584f39613cd>, 2026. accesat la data: 09.06.2026.
- [103] Graphify. Graphify: Code knowledge graphs for ai agents. disponibil la: <https://graphify.net>, 2026. accesat la data: 19.05.2026.

# ANEXE

## Anexa 1: config/config.yml

```
1 providers:
2   ollama:
3     base_url: ${OLLAMA_URL}
4     models:
5       - qwen3.5
6       - gemma4
7       - deepseek-r1:32b
8       - gemma4:26b
9       - phi4:14b
10      - llama3.2:3b
11      - devstral-small-2
12      - qwen3-coder:30b
13      - qwen2.5-coder:7b
14      - qwen2.5-coder:1.5b
15      temperature: 0.6
16   openai:
17     api_key: ${OPENAI_API_KEY}
18     models:
19       - gpt-4.1
20       - gpt-5.5
21       - gpt-5.4
22     temperature: 0
23   claude:
24     api_key: ${ANTHROPIC_API_KEY}
25     models:
26       - claude-opus-4-7
27       - claude-sonnet-4-6
28       - claude-haiku-4-5-20251001
29     temperature: 0
30   bedrock:
31     api_key: ${BEDROCK_API_KEY}
32     region: us-west-2
33     models:
34       - us.anthropic.claude-sonnet-4-6
35       - us.anthropic.claude-opus-4-7
36       - us.anthropic.claude-haiku-4-5-20251001-v1:0
37     temperature: 0
38   gemini:
39     api_key: ${GOOGLE_API_KEY}
40     models:
41       - gemini-2.5-flash
42       - gemini-2.5-pro
43       - gemini-2.0-flash
44     temperature: 0
45   multimodal:
46     pdf:
47       small_threshold_pages: 15
48       render_dpi: 144
49       max_pages_per_call: 20
50     enforce_session_model: true
51     trigger:
52       threshold: 0.80
53       min_turns_kept: 4
54     pipeline:
55       - tool_truncate:
56         enabled: false
57         max_tool_tokens: 2000
58         head_tail: [800, 400]
59       - tool_summarize:
60         enabled: false
61         max_tool_tokens: 2000
62         head_tail: [800, 400]
63         timeout_s: 30
64         max_summary_tokens: 500
65       model: null
66       - format_compress:
67         enabled: false
68       per_tool: {}
69       - ref_substitute:
70         enabled: false
71         min_ref_tokens: 1500
72         excerpt_tokens: 200
73       - dedupe:
74         enabled: false
75         similarity: 0.92
76         backend: string
77         - importance_prune:
78           enabled: false
79           keep_kinds: [human, system_prompt, assistant_text,
80             tool_result_last]
81           drop_kinds: [assistant_thinking]
82         - bulk:
83           enabled: false
84           strategy: hybrid # trim / summarize / hybrid
85           summarize:
86             model: ${COMPACTION_MODEL}
87             max_summary_tokens: 1500
88         storage:
89           backup: true
90           keep_backups: 3
91           artifacts: true
92         sandboxing:
93           enabled: false
94           image: python:3.13-slim
95           network: bridge
96           memory: 2g
97           cpus: 2
98           timeout_default: 30
99           blocked_paths:
100            - "/"
101           ambiguous_max_mb: 500
102           ambiguous_max_files: 10000
103         ui:
104           pastes:
105             threshold: 200
106             max_per_turn: 5
107           skills:
108             max_active: 10
109           vibe_md:
110             enabled: true
111             max_bytes_per_file: 8192
112           mcp:
113             max_active: 10
114             logging:
115               enabled: true
116               level: INFO
117               dir: ~/.vibe-cli/logs
118               text_max_bytes: 5242880
119               text_backups: 10
120               jsonl_retention_days: 30
121             tool_guards:
122               enabled: true
123               read_before_write:
124                 enabled: true
125                 check_freshness: true
126             subagents:
127               enabled: true
128               default_timeout_s: 120
129               max_timeout_s: 600
130               max_depth: 2
131               max_parallel: 3
132               child_context:
133                 max_tool_tokens: 4000
134                 head_tail: [800, 400]
135             types:
136               general:
137                 max_iterations: 25
138                 default_timeout_s: 180
139                 code_search:
140                   max_iterations: 15
141                   default_timeout_s: 90
142                 web_research:
143                   max_iterations: 18
144                   default_timeout_s: 120
145             tracing:
146               enabled: true
147               project: vibe-code
148               public_key: ${LANGFUSE_PUBLIC_KEY}
149               secret_key: ${LANGFUSE_SECRET_KEY}
150               host: ${LANGFUSE_HOST}
```

## Anexa 2: config/agentbench.yml

```
1 name: agentbench
2 description: >
3   Five-stage agentbench software-development benchmark targeting the
4   vibe-cli repository. Each task stresses one additional capability:
5   Task 1 (read-only comprehension) -> Task 2 (multi-file navigation +
6   surgical edit) -> Task 3 (cross-file refactor + tests) -> Task 4
7   (subtle bug hunt) -> Task 5 (greenfield feature design). Stages 2-5
8   run in throwaway git worktrees so the source repo is never modified.
9 cases:
10   - id: task_01_comprehension
11     workspace: null # read-only; runs in the source tree directly
12     allowed_tools: "read_file,list_directory"
13     max_turns: 6
14     input: |
15       Read the file 'src/tools.py' in the current working directory and
16       produce a numbered list of every Python function decorated with
```

```

17     'tool' (the langchain decorator). For each one, give a one-sentence
18     description of what it does.
19
20     Do not modify any files. Reply with only the numbered list - no
21     preamble.
22     expected_output: |
23     1. web_search - performs a DuckDuckGo web search
24     2. read_file - reads a file from disk
25     3. write_file - writes content to a file
26     4. edit_file - edits a file by replacing a string
27     5. list_directory - lists files in a directory
28     6. run_command - executes a shell command
29     7. run_skill - loads and applies a skill
30     8. remember - appends a learning to VIBE.md
31     9. fetch_artifact - retrieves a stored artifact by handle
32     expected_tools: [read_file]
33     tags: [baseline, comprehension]
34     checks: []
35     - id: task_02_focused_edit
36     workspace: fresh_worktree
37     allowed_tools: "read_file,list_directory,write_file,edit_file,
38         run_command"
39     max_turns: 15
40     input: |
41     The vibe-cli project (you are in the project root) has a slash-
42     registry. Locate it yourself - do not ask. Add a new command named
43     '/health' that, when invoked, prints exactly 'vibe-cli OK' to the
44     console.
45
46     Requirements:
47     - The command must be registered using the same pattern as existing
48     commands (look at e.g. '/help' or '/preferences' for reference).
49     - Importing 'ui.commands' must place the string "health" as a key in
50     the 'COMMANDS' dict (no leading slash).
51     - Do not break any existing command.
52
53     When done, reply with the file path and line where you registered
54     expected_output: ""
55     expected_tools: [read_file, write_file]
56     tags: [navigation, edit]
57     checks:
58     - name: command_registered_in_source
59     type: file_check
60     paths:
61     - path: ui/commands.py
62     must_contain: "health"
63     - name: command_runs
64     type: command_check
65     cmd: |
66     $VIBE_PYTHON -c "import sys; sys.path.insert(0, '.'); from ui.
67     commands import COMMANDS; assert 'health' in COMMANDS, 'health not
68     in COMMANDS'; print('OK!)"
69     timeout: 20
70     - id: task_03_passthrough_stage
71     workspace: fresh_worktree
72     allowed_tools: "read_file,list_directory,write_file,edit_file,
73         run_command"
74     max_turns: 25
75     input: |
76     Add a new no-op compaction stage called 'passthrough' to the
77     vibe-cli project (you are in the project root).
78
79     Requirements:
80     - File: 'src/compaction/stages/passthrough.py'.
81     - It must define a class that inherits from 'src.compaction.base.
82     Stage'
83     with 'mode = "eager"'. The class name is up to you.
84     - The class's 'apply_eager(event, *, store)' must leave the event
85     byte-for-byte unchanged and return False.
86     - Register it under the key "passthrough" in 'EAGER_REGISTRY'
87     in 'src/compaction/_init_.py'.
88     - Honor the existing 'enabled' config convention (the base class
89     already handles this - don't reinvent it).
90
91     Look at 'src/compaction/stages/format_compress.py' for a working
92     example of how an eager stage is structured.
93
94     When done, reply with a one-line summary.
95     expected_output: ""
96     expected_tools: [read_file, write_file]
97     tags: [refactor, registry]
98     inject_files:
99     - src: eval/examples/hidden_tests/task_03_passthrough_test.py
100     dst: tests/hidden/task_03_passthrough_test.py
101     checks:
102     - name: stage_file_exists
103     type: file_check
104     paths:
105     - path: src/compaction/stages/passthrough.py
106     importable: true
107     - path: src/compaction/_init_.py
108     must_contain: "passthrough"
109     - name: hidden_tests_pass
110     type: command_check
111     cmd: "$VIBE_PYTHON -m pytest tests/hidden/task_03_passthrough_test
112     .py -q --no-header 2>&1"
113     timeout: 90
114     - id: task_04_bug_hunt
115     workspace: fresh_worktree
116     allowed_tools: "read_file,list_directory,write_file,edit_file,
117         run_command"
118     max_turns: 20
119     input: |
120     In 'ui/app.py' of the vibe-cli project (you are in the project root)
121     ,
122     read the function '_resolve_compaction_llm' carefully.
123
124     Question: what happens on the next threshold-pass compaction if a
125     user previously set 'preferences.compaction_model' to a provider
126     whose API key has since been removed from
127     'preferences.api_keys' - but the cached provider config still
128     reports a (stale) value for that key?
129
130     Identify the failure mode by reading the code only - do not try
131     to reproduce. Then implement a minimal fix that:
132     - falls back to the chat model when the stale-key path would have
133     led to a downstream failure,
134     - logs a single-line warning at WARNING level via
135
136     'emit_event(get_logger("compaction"), "compaction.llm_resolve",
137     ...)'
138     (consistent with the existing pattern in that function),
139     - does NOT silently swallow unrelated errors.
140
141     State your hypothesis BEFORE you edit. After editing, explain in
142     one paragraph why your fix doesn't mask other failure modes.
143     expected_output: ""
144     expected_tools: [read_file]
145     tags: [bug_hunt, reasoning]
146     checks:
147     - name: function_modified
148     type: command_check
149     cmd: |
150     $VIBE_PYTHON -c "import re, pathlib; t = pathlib.Path('ui/app.py
151     ').read_text(); n = len(re.findall(r'compaction\\.llm_resolve', t));
152     assert n >= 3, f'expected >=3 emissions of compaction.llm_resolve (
153     baseline=2; fix must add one), got {n}'; print('OK!)"
154     timeout: 20
155     - name: hypothesis_quality
156     type: llm_judge
157     provider: bedrock
158     model: us.anthropic.claude-sonnet-4-5-20250929-v1:0
159     rubric_path: eval/examples/judge_rubrics/task_04_bug_hunt.md
160     max_score: 3
161     - id: task_05_compliance_evaluator
162     workspace: fresh_worktree
163     allowed_tools: "read_file,list_directory,write_file,edit_file,
164         run_command"
165     max_turns: 40
166     timeout: 1200
167     input: |
168     Design and implement an instruction-compliance evaluator for
169     vibe-cli (you are in the project root).
170
171     Requirements:
172     1. Create 'src/evals/instruction_compliance.py' (creating
173     'src/evals/_init_.py' if it doesn't exist).
174     2. Expose a public function with signature
175     score_turn(session_event: dict, instructions: str) -> dict
176     returning {'score': float, "rationale": str} where score is in
177     [0, 1].
178     3. Internally, resolve the judge LLM by reusing the same logic
179     that '_resolve_compaction_llm' in 'ui/app.py' uses - IMPORT it,
180     do NOT re-implement model selection.
181     4. If the resolver returns 'None', your function must return
182     {'score': 0.0, "rationale": "no LLM available"} and not raise.
183     5. Add a '/eval-last' slash command that scores the most recent
184     'assistant_text' event in the current session and prints the dict
185     (use the '@command' decorator from 'ui/commands.py').
186     6. When 'ctx["last_trace_id"]' is set, post the score to LangFuse
187     via 'post_score()' from 'src/tracing.py'.
188     7. Include one unit test under 'tests/test_instruction_compliance.py'
189     that mocks the LLM (no network call).
190
191     When done, reply with: design choices, any deviations from the
192     spec, and one tradeoff you accepted.
193     expected_output: ""
194     expected_tools: [read_file, write_file]
195     tags: [feature, design]
196     inject_files:
197     - src: eval/examples/hidden_tests/task_05_compliance_test.py
198     dst: tests/hidden/task_05_compliance_test.py
199     checks:
200     - name: module_exists
201     type: file_check
202     paths:
203     - path: src/evals/instruction_compliance.py
204     must_contain_symbol: score_turn
205     - name: command_registered
206     type: file_check
207     paths:
208     - path: ui/commands.py
209     must_contain: "eval-last"
210     - name: hidden_tests_pass
211     type: command_check
212     cmd: "$VIBE_PYTHON -m pytest tests/hidden/task_05_compliance_test.
213     py -q --no-header 2>&1"
214     timeout: 90
215     - name: design_quality
216     type: llm_judge
217     provider: bedrock
218     model: us.anthropic.claude-sonnet-4-6
219     rubric_path: eval/examples/judge_rubrics/task_05_feature.md
220     max_score: 3
221     - id: task_06_recency_window_stage
222     workspace: fresh_worktree
223     allowed_tools: "read_file,list_directory,write_file,edit_file,
224         run_command"
225     max_turns: 40
226     timeout: 1200
227     input: |
228     Add a new *threshold-mode* compaction stage called 'recency_window'
229     to the vibe-cli project (you are in the project root). It trims
230     the session log down to the N most-recent complete turns when
231     context utilisation crosses the threshold.
232
233     Requirements:
234     - File: 'src/compaction/stages/recency_window.py'.
235     - Define a class inheriting from
236     'src.compaction.base.ThresholdStage' (class name is up to you;
237     e.g. 'RecencyWindow'). It must have 'mode = "threshold"'.
238     - Implement 'async def apply_threshold(self, tctx: ThresholdContext)
239     -> ThresholdResult'. The stage must:
240     * Group 'tctx.events' into "turns": a turn starts at a
241     'human' event ('KIND_HUMAN') and contains every event up
242     to (but not including) the next 'human' event. Any events
243     *before* the first 'human' ('system_prompt', etc.) belong
244     to a synthetic "turn 0" that the stage MUST always keep
245     (those are session-scoped, not turn-scoped).
246     * Keep the most-recent 'keep_turns' turns
247     ('self.config["keep_turns"]', default 6).
248     * NEVER drop below 'tctx.min_turns_kept' - that floor wins
249     even if 'keep_turns' is smaller.
250     * Return 'ThresholdResult(removed_event_ids=[...])' listing
251     every event ID in the dropped older turns. The orchestrator
252     handles freeing-estimate accounting and the
253     'KIND_COMPACT' event - your stage just declares which
254     IDs go.

```

```

239 - Register the stage under the key "recency_window" in
240 'THRESHOLD_REGISTRY' in 'src/compaction/_init_.py'.
241 - Honor 'self.enabled' (the base class already gates this - don't
242 re-implement).
243
244 Look at 'src/compaction/stages/dedupe.py' for a working threshold
245 stage and at 'src/compaction/_init_.py' (around '
THRESHOLD_REGISTRY')
for the registry wiring.
246
247 Edge cases the implementation must handle correctly:
248 1. Fewer than 'keep_turns' total turns -> return empty
249 'removed_event_ids' (the stage is a no-op).
250 2. 'keep_turns < tctx.min_turns_kept' -> the floor wins; behave as
251 if 'keep_turns = tctx.min_turns_kept'.
252 3. The synthetic "turn 0" (events before the first 'KIND_HUMAN')
253 is never dropped, even if it pushes the kept-count above
254 'keep_turns'.
255 4. An event with no 'id' field - defensive: skip it (don't crash,
256 don't add 'None' to 'removed_event_ids').
257
258 When done, reply with: design choices, the turn-grouping
259 invariant in one sentence, and one tradeoff you accepted.
260 expected_output: ""
261 expected_tools: [read_file, write_file]
262
263 tags: [refactor, registry, design]
264 inject_files:
265 - src: eval/examples/hidden_tests/task_06_recency_test.py
266 dst: tests/hidden/task_06_recency_test.py
267 checks:
268 - name: stage_file_exists
269 type: file_check
270 paths:
271 - path: src/compaction/stages/recency_window.py
272 importable: true
273 - path: src/compaction/_init_.py
274 must_contain: "recency_window"
275 - name: hidden_tests_pass
276 type: command_check
277 cmd: "$VIBE_PYTHON -m pytest tests/hidden/task_06_recency_test.py
-q --no-header 2>&1"
278 timeout: 90
279 - name: design_quality
280 type: llm_judge
281 provider: bedrock
282 model: us.anthropic.claude-sonnet-4-6
283 rubric_path: eval/examples/judge_rubrics/task_06_threshold_stage.
md
284 max_score: 3

```

## Anexa 3: src/artifacts.py

```

1 from __future__ import annotations
2 import hashlib
3 import json
4 import shutil
5 import time
6 from dataclasses import dataclass
7 from pathlib import Path
8 from typing import Literal
9 from langchain_core.tools import tool
10 from ui.paths import ARTIFACTS_DIR, session_artifacts_dir
11
12 ARTIFACT_TTL_SECONDS = 30 * 24 * 60 * 60
13
14 HANDLE_PREFIX = "@artifact:"
15 SHA_PREFIX_LEN = 12
16
17 @dataclass(frozen=True)
18 class ArtifactRef:
19     session_id: str
20     sha256: str
21     kind: str
22     bytes_: int
23     ext: str
24
25     @property
26     def short(self) -> str:
27         return self.sha256[:SHA_PREFIX_LEN]
28
29     @property
30     def handle(self) -> str:
31         return f"{HANDLE_PREFIX}{self.short}"
32
33     def to_meta(self) -> dict:
34         return {
35             "sha256": self.sha256,
36             "kind": self.kind,
37             "bytes": self.bytes_,
38             "ext": self.ext,
39             "ts": time.time(),
40         }
41
42 class ArtifactStore:
43     def __init__(self, session_id: str):
44         self.session_id = session_id
45         self.root = session_artifacts_dir(session_id)
46
47     def _payload_path(self, sha: str, ext: str) -> Path:
48         return self.root / f"{sha}.{ext}"
49
50     def _meta_path(self, sha: str) -> Path:
51         return self.root / f"{sha}.meta.json"
52
53     def put(
54         self,
55         payload: str | bytes,
56         *,
57         kind: str = "text",
58         ext: str = "txt",
59     ) -> ArtifactRef:
60         data = payload.encode("utf-8") if isinstance(payload, str) else
payload
61         sha = hashlib.sha256(data).hexdigest()
62         self.root.mkdir(parents=True, exist_ok=True)
63
64         ppath = self._payload_path(sha, ext)
65         if not ppath.exists():
66             tmp = ppath.with_suffix(ppath.suffix + ".tmp")
67             tmp.write_bytes(data)
68             tmp.replace(ppath)
69
70         ref = ArtifactRef(
71             session_id=self.session_id,
72             sha256=sha,
73             kind=kind,
74             bytes_=len(data),
75             ext=ext,
76         )
77         mpath = self._meta_path(sha)
78         if not mpath.exists():
79             mpath.write_text(json.dumps(ref.to_meta(), indent=2))
80         return ref
81
82     def resolve(self, short_or_full: str) -> ArtifactRef | None:
83         if not self.root.exists():
84             return None
85         prefix = short_or_full
86         for meta_file in self.root.glob("*.meta.json"):
87             sha = meta_file.name[: -len(".meta.json")]
88             if sha.startswith(prefix):
89                 meta = json.loads(meta_file.read_text())
90                 return ArtifactRef(
91                     session_id=self.session_id,
92                     sha256=meta["sha256"],
93                     kind=meta.get("kind", "text"),
94                     bytes_=meta.get("bytes", 0),
95                     ext=meta.get("ext", "txt"),
96                 )
97         return None
98
99     def read(
100         self,
101         short_or_full: str,
102         *,
103         mode: Literal["text", "bytes"] = "text",
104         byte_range: tuple[int, int] | None = None,
105     ) -> str | bytes | None:
106         ref = self.resolve(short_or_full)
107         if ref is None:
108             return None
109         ppath = self._payload_path(ref.sha256, ref.ext)
110         if not ppath.exists():
111             return None
112         if byte_range is None:
113             data = ppath.read_bytes()
114         else:
115             start, end = byte_range
116             with ppath.open("rb") as fh:
117                 fh.seek(max(0, start))
118                 data = fh.read(max(0, end - start))
119             return data.decode("utf-8", errors="replace") if mode == "text"
120         else data
121
122     def purge_session(session_id: str) -> None:
123         d = session_artifacts_dir(session_id)
124         if d.exists():
125             shutil.rmtree(d, ignore_errors=True)
126
127     def gc_expired(now: float | None = None) -> int:
128         if not ARTIFACTS_DIR.exists():
129             return 0
130         cutoff = (now if now is not None else time.time()) -
ARTIFACT_TTL_SECONDS
131         removed = 0
132         for sess_dir in ARTIFACTS_DIR.iterdir():
133             if not sess_dir.is_dir():
134                 continue
135             try:
136                 mtime = sess_dir.stat().st_mtime
137             except OSError:
138                 continue
139             if mtime < cutoff:
140                 shutil.rmtree(sess_dir, ignore_errors=True)
141                 removed += 1
142         return removed
143
144     def make_fetch_artifact_tool(session_id: str):
145         store = ArtifactStore(session_id)
146
147         @tool
148         def fetch_artifact(handle: str, start: int = 0, length: int = 0) ->
str:
149             """Re-fetch a previously stored artifact by its '@artifact:<sha>'
handle.
150
151             Args:
152                 handle: The handle string (with or without the '@artifact:'
prefix).
153                 start: Byte offset to start reading from (default 0).
154                 length: Number of bytes to read (default 0 = full payload).
155             """
156             h = handle.removeprefix(HANDLE_PREFIX).strip()
157             if not h:
158                 return "Error: empty artifact handle."
159             byte_range = (start, start + length) if length > 0 else None
160             data = store.read(h, mode="text", byte_range=byte_range)
161             if data is None:

```

```

163         return f"Artifact not found: {handle}"
164     return data
165
166     return fetch_artifact
167
168

```

```

169 __all__ = [
170     "ArtifactRef", "ArtifactStore", "HANDLE_PREFIX",
171     "SHA_PREFIX_LEN", "ARTIFACT_TTL_SECONDS", "purge_session",
172     "gc_expired", "make_fetch_artifact_tool",
173 ]

```

## Anexa 4: src/attachments.py

```

1 from __future__ import annotations
2 import hashlib
3 import logging
4 import mimetypes
5 import re as _re
6 from dataclasses import dataclass, field
7 from pathlib import Path
8 from typing import Literal
9 from src.artifacts import ArtifactStore
10
11 logger = logging.getLogger(__name__)
12 Kind = Literal["image", "pdf", "text"]
13
14 MAX_ATTACHMENT_BYTES = 10 * 1024 * 1024 # 10 MB
15 MAX_TURN_TOTAL_BYTES = 20 * 1024 * 1024 # 20 MB
16
17 IMAGE_EXTS = {".png", ".jpg", ".jpeg", ".webp", ".gif"}
18 PDF_EXTS = {".pdf"}
19 TEXT_EXTS = {
20     ".txt", ".md", ".csv", ".json", ".log",
21     ".yaml", ".yml", ".toml", ".ini", ".cfg", ".py",
22 }
23
24
25 class AttachmentError(Exception):
26     ...
27
28 @dataclass
29 class Attachment:
30     path: str
31     name: str
32     kind: Kind
33     mime_type: str
34     size: int
35     sha: str
36     ext: str
37     text_content: str | None = None
38     pdf_manifest: str | None = None
39     pdf_page_count: int | None = None
40     pdf_render_strategy: str = "manifest"
41     rendered_page_shas: list[str] = field(default_factory=list)
42     pdf_full_text: str | None = None
43     extra: dict = field(default_factory=dict)
44
45     def to_block(self) -> dict:
46         out: dict = {
47             "type": "attachment",
48             "kind": self.kind,
49             "name": self.name,
50             "media_type": self.mime_type,
51             "sha": self.sha,
52             "ext": self.ext,
53             "size": self.size,
54         }
55         if self.kind == "text" and self.text_content is not None:
56             out["text"] = self.text_content
57         if self.kind == "pdf":
58             if self.pdf_manifest is not None:
59                 out["pdf_manifest"] = self.pdf_manifest
60             if self.pdf_page_count is not None:
61                 out["pdf_page_count"] = self.pdf_page_count
62             if self.pdf_render_strategy != "manifest":
63                 out["pdf_render_strategy"] = self.pdf_render_strategy
64             if self.rendered_page_shas:
65                 out["rendered_page_shas"] = list(self.rendered_page_shas)
66             if self.pdf_full_text is not None:
67                 out["pdf_full_text"] = self.pdf_full_text
68         return out
69
70     @classmethod
71     def from_block(cls, block: dict) -> "Attachment":
72         return cls(
73             path="",
74             name=block.get("name", ""),
75             kind=block.get("kind", "text"),
76             mime_type=block.get("media_type", "application/octet-stream"),
77             size=int(block.get("size", 0)),
78             sha=block.get("sha", ""),
79             ext=block.get("ext", ""),
80             text_content=block.get("text"),
81             pdf_manifest=block.get("pdf_manifest"),
82             pdf_page_count=block.get("pdf_page_count"),
83             pdf_render_strategy=block.get("pdf_render_strategy", "manifest"),
84             rendered_page_shas=list(block.get("rendered_page_shas", [])),
85             pdf_full_text=block.get("pdf_full_text"),
86         )
87
88     def detect_kind(path: Path | str) -> Kind | None:
89         p = Path(path)
90         ext = p.suffix.lower()
91         if ext in IMAGE_EXTS:
92             return "image"
93         if ext in PDF_EXTS:
94             return "pdf"
95         if ext in TEXT_EXTS:
96             return "text"
97         mt, _ = mimetypes.guess_type(str(p))
98         if not mt:
99             return None
100         if mt.startswith("image/"):
101             return "image"

```

```

102         if mt == "application/pdf":
103             return "pdf"
104         if mt.startswith("text/"):
105             return "text"
106         return None
107
108     def mime_for(path: Path, kind: Kind) -> str:
109         mt, _ = mimetypes.guess_type(str(path))
110         if mt:
111             return mt
112         if kind == "image":
113             return f"image/{path.suffix.lstrip('.').lower() or 'png'}"
114         if kind == "pdf":
115             return "application/pdf"
116         return "text/plain"
117
118     def load_attachment(
119         path: Path | str,
120         *,
121         session_id: str,
122         image_capable: bool = True,
123         small_threshold_pages: int = 15,
124         render_dpi: int = 144,
125     ) -> Attachment:
126         p = Path(path).expanduser()
127         if not p.exists():
128             raise AttachmentError(f"file not found: {p}")
129         if not p.is_file():
130             raise AttachmentError(f"not a regular file: {p}")
131         kind = detect_kind(p)
132         if kind is None:
133             raise AttachmentError(
134                 f"unsupported file type: {p.name} "
135                 f"({supported: PNG/JPG/JPEG/WebP/GIF, PDF, common text/code "
136                 f"files}")
137             )
138         size = p.stat().st_size
139         if size > MAX_ATTACHMENT_BYTES:
140             mb = MAX_ATTACHMENT_BYTES // (1024 * 1024)
141             raise AttachmentError(
142                 f"{p.name} is {size / (1024 * 1024):.1f} MB; per-file cap is { "
143                 f"mb} MB"
144             )
145         if size == 0:
146             raise AttachmentError(f"{p.name} is empty")
147
148         data = p.read_bytes()
149         sha = hashlib.sha256(data).hexdigest()
150         ext = p.suffix.lstrip(".").lower() or "bin"
151         mt = mime_for(p, kind)
152
153         store = ArtifactStore(session_id)
154         store.put(data, kind=f"attachment.{kind}", ext=ext)
155
156         text_content: str | None = None
157         pdf_manifest: str | None = None
158         pdf_page_count: int | None = None
159
160         if kind == "text":
161             try:
162                 text_content = data.decode("utf-8")
163             except UnicodeDecodeError:
164                 # Best-effort decode - show what we can, but warn.
165                 text_content = data.decode("utf-8", errors="replace")
166                 logger.info("attachment.text_decode_replace path=%s", p)
167         elif kind == "pdf":
168             try:
169                 pdf_manifest, pdf_page_count = _build_pdf_manifest(data, name=
170                 p.name)
171             except _PdfMissingError as exc:
172                 raise AttachmentError(str(exc)) from exc
173             except Exception as exc: # pypdf-side failure
174                 raise AttachmentError(f"failed to parse {p.name}: {exc}") from
175                 exc
176
177         pdf_render_strategy = "manifest"
178         rendered_page_shas: list[str] = []
179         pdf_full_text: str | None = None
180         if (
181             kind == "pdf"
182             and pdf_page_count is not None
183             and pdf_page_count <= small_threshold_pages
184             and image_capable
185             and _pymupdf_available()
186         ):
187             try:
188                 rendered_page_shas = _render_pdf_pages_to_store(
189                     data, session_id=session_id, dpi=render_dpi
190                 )
191                 pdf_full_text = _extract_pdf_full_text(data)
192                 pdf_render_strategy = "full"
193             except Exception as exc:
194                 logger.info(
195                     "attachment.pdf_render_failed path=%s error=%s - falling
196                     back to manifest mode",
197                     p,
198                     exc,
199                 )
200                 rendered_page_shas = []
201                 pdf_full_text = None
202                 pdf_render_strategy = "manifest"

```

```

200
201     return Attachment(
202         path=str(p),
203         name=p.name,
204         kind=kind,
205         mime_type=mt,
206         size=size,
207         sha=sha,
208         ext=ext,
209         text_content=text_content,
210         pdf_manifest=pdf_manifest,
211         pdf_page_count=pdf_page_count,
212         pdf_render_strategy=pdf_render_strategy,
213         rendered_page_shas=rendered_page_shas,
214         pdf_full_text=pdf_full_text,
215     )
216
217
218 def total_size(attachments: list[Attachment]) -> int:
219     return sum(a.size for a in attachments)
220
221
222 def check_turn_budget(
223     new_size: int, pending: list[Attachment]
224 ) -> None:
225     if total_size(pending) + new_size > MAX_TURN_TOTAL_BYTES:
226         mb = MAX_TURN_TOTAL_BYTES // (1024 * 1024)
227         raise AttachmentError(
228             f"total attachments would exceed the per-turn cap of {mb} MB"
229         )
230
231 class _PdfMissingError(Exception):
232     pass
233
234 _PDF_PREVIEW_CHARS = 200
235 _PDF_MANIFEST_HEAD_PAGES = 10
236 _PDF_MANIFEST_TAIL_PAGES = 5
237
238 def _build_pdf_manifest(data: bytes, *, name: str) -> tuple[str, int]:
239     try:
240         from pypdf import PdfReader
241     except ImportError as exc:
242         raise _PdfMissingError(
243             "PDF attachments need the 'pypdf' package, which should be "
244             "installed by default. Refresh deps with: 'make setup' "
245             "(or 'uv pip install pypdf')."
246         ) from exc
247
248     import io
249     reader = PdfReader(io.BytesIO(data))
250     n_pages = len(reader.pages)
251     title = ""
252     author = ""
253     try:
254         meta = reader.metadata or {}
255         title = (getattr(meta, "title", None) or meta.get("/Title") or ""
256                 or "")
257         author = (getattr(meta, "author", None) or meta.get("/Author") or ""
258                  or "")
259     except Exception:
260         pass
261     toc_lines: list[str] = []
262     try:
263         outline = getattr(reader, "outline", None) or []
264         toc_lines = _flatten_outline(reader, outline)
265     except Exception:
266         toc_lines = []
267
268 def _preview(page_idx: int) -> str:
269     try:
270         txt = reader.pages[page_idx].extract_text() or ""
271     except Exception:
272         txt = ""
273     txt = " ".join(txt.split())
274     return txt[:_PDF_PREVIEW_CHARS]
275
276 preview_lines: list[str] = []
277 show_full = n_pages <= (
278     _PDF_MANIFEST_HEAD_PAGES + _PDF_MANIFEST_TAIL_PAGES + 5
279 )
280 if show_full:
281     for i in range(n_pages):
282         preview_lines.append(f" p{i + 1}: {_preview(i)}r")
283 else:
284     for i in range(_PDF_MANIFEST_HEAD_PAGES):
285         preview_lines.append(f" p{i + 1}: {_preview(i)}r")
286     skipped = n_pages - _PDF_MANIFEST_HEAD_PAGES -
287     _PDF_MANIFEST_TAIL_PAGES
288     preview_lines.append(f" ... ({skipped} more pages) ...")
289     for i in range(n_pages - _PDF_MANIFEST_TAIL_PAGES, n_pages):
290         preview_lines.append(f" p{i + 1}: {_preview(i)}r")
291
292 parts = [f"# attachment: {name} ({n_pages} pages)"]
293 if title:
294     parts.append(f', "{title}"')
295 if author:
296     parts.append(f' by {author}')
297 parts.append("(")
298 header = ".join(parts)
299
300 body = [header, ""]
301 if toc_lines:
302     body.append("TOC:")
303     body.extend(f" {line}" for line in toc_lines[:80])
304     if len(toc_lines) > 80:
305         body.append(f" ... ({len(toc_lines) - 80} more entries) ...")
306     body.append("")
307 body.append("Page previews:")
308 body.extend(preview_lines)
309 body.append("")
310 body.append(
311     f'Use read_pdf_pages("{name}", start, end) to read specific pages
312     "(1-indexed). Pass as_image=True on vision-capable models for "
313     "figure-heavy pages."
314 )
315
316 def _flatten_outline(reader, outline, depth: int = 0) -> list[str]:
317     lines: list[str] = []
318     indent = " " * depth
319     for item in outline:
320         if isinstance(item, list):
321             lines.extend(_flatten_outline(reader, item, depth + 1))
322             continue
323         title = getattr(item, "title", None) or str(item)
324         page_num = ""
325         try:
326             page_idx = reader.get_destination_page_number(item)
327             if page_idx is not None:
328                 page_num = f" ({page_idx + 1})"
329         except Exception:
330             pass
331         lines.append(f"{indent}{title}{page_num}")
332     return lines
333
334 _pymupdf_probe_done: bool = False
335 _pymupdf_present: bool = False
336
337 def _pymupdf_available() -> bool:
338     global _pymupdf_probe_done, _pymupdf_present
339     if _pymupdf_probe_done:
340         return _pymupdf_present
341     try:
342         import fitz
343         _pymupdf_present = True
344     except ImportError:
345         logger.info(
346             "attachment.pymupdf_missing - small-PDF full mode disabled "
347             "(install with: uv pip install pymupdf)"
348         )
349     _pymupdf_present = False
350     _pymupdf_probe_done = True
351     return _pymupdf_present
352
353 def _render_pdf_pages_to_store(
354     data: bytes, *, session_id: str, dpi: int
355 ) -> list[str]:
356     import fitz
357
358     store = ArtifactStore(session_id)
359     doc = fitz.open(stream=data, filetype="pdf")
360     try:
361         shas: list[str] = []
362         for i in range(doc.page_count):
363             page = doc.load_page(i)
364             pix = page.get_pixmap(dpi=dpi)
365             png = pix.tobytes("png")
366             ref = store.put(png, kind="attachment.image", ext="png")
367             shas.append(ref.sha256)
368     finally:
369         doc.close()
370     return shas
371
372 def _extract_pdf_full_text(data: bytes) -> str:
373     try:
374         from pypdf import PdfReader
375     except ImportError:
376         return ""
377     import io
378
379     reader = PdfReader(io.BytesIO(data))
380     parts: list[str] = []
381     for i, page in enumerate(reader.pages, 1):
382         try:
383             txt = page.extract_text() or ""
384         except Exception:
385             txt = ""
386         parts.append(f"--- page {i} ---\n{txt}")
387     return "\n\n".join(parts)
388
389 def fetch_bytes(session_id: str, sha: str) -> bytes | None:
390     store = ArtifactStore(session_id)
391     return store.read(sha, mode="bytes")
392
393 _PATH_ATOM = r"(?:[\\w./-+@%~!~|\\s])"
394
395 if __name__ == "__main__":
396     # not in the middle of another path/word
397     (
398         file://[\\s]"<>]+ # explicit URI
399         | ~/[_PATH_ATOM]+ # home-relative
400         | /[_PATH_ATOM]+ # absolute
401     )
402     """
403     _re.VERBOSE,
404 )
405
406 _BACKTICK_BLOCK = _re.compile(r'`.*?`', _re.DOTALL)
407 _INLINE_CODE = _re.compile(r'`[^\s]*`')
408
409 def _normalize_path(token: str) -> str:
410     if token.startswith("file:///"):
411         token = token[len("file:///"):].replace("\\t", "\t")
412     return token.replace("\\ ", " ").replace("\\\\t", "\t")
413
414 def _strip_trailing_punct(token: str) -> str:
415     while token and token[-1] in ".,:;!?]>\"'":
416         token = token[:-1]
417     return token
418
419 def extract_existing_file_paths(text: str) -> list[tuple[str, Path]]:
420     if not text:
421         return []
422     redacted = _BACKTICK_BLOCK.sub(
423         lambda m: " " * len(m.group(0)), text
424     )
425     redacted = _INLINE_CODE.sub(lambda m: " " * len(m.group(0)), redacted)
426     seen: set[str] = set()
427     out: list[tuple[str, Path]] = []
428     for m in _PATH_PATTERN.finditer(redacted):
429         raw = _strip_trailing_punct(m.group(1))

```

```

436     normalised = _normalize_path(raw)
437     try:
438         p = Path(normalised).expanduser()
439     except (OSError, ValueError):
440         continue
441     try:
442         if not p.is_file():
443             continue
444     except OSError:
445         continue
446     key = str(p.resolve())
447     if key in seen:

```

```

448         continue
449     seen.add(key)
450     out.append((raw, p))
451     return out
452
453 __all__ = [
454     "Attachment", "AttachmentError", "IMAGE_EXTS",
455     "Kind", "MAX_ATTACHMENT_BYTES", "MAX_TURN_TOTAL_BYTES",
456     "PDF_EXTS", "TEXT_EXTS", "check_turn_budget",
457     "detect_kind", "extract_existing_file_paths", "fetch_bytes",
458     "load_attachment", "mime_for", "total_size",
459 ]

```

## Anexa 5: src/capabilities.py

```

1 from __future__ import annotations
2 import logging
3 from dataclasses import dataclass
4 from typing import Literal
5 from ui import preferences
6 logger = logging.getLogger(__name__)
7 Kind = Literal["image", "pdf", "text"]
8 KNOWN_KINDS: tuple[Kind, ...] = ("image", "pdf", "text")
9
10 @dataclass(frozen=True)
11 class Capabilities:
12     provider: str
13     model: str
14
15     def supports(self, kind: str) -> bool:
16         if kind == "text":
17             return True
18         return (
19             preferences.get_model_capability(self.provider, self.model,
20             kind) != "no"
21         )
22     def recorded(self, kind: str) -> str | None:
23         return preferences.get_model_capability(self.provider, self.model,
24         kind)
25
26 def get_capabilities(provider: str, model: str) -> Capabilities:
27     return Capabilities(provider=provider, model=model)
28
29 def record_unsupported(
30     provider: str,
31     model: str,
32     kind: str,
33     *,
34     reason: str = "",
35 ) -> None:
36     if kind not in ("image", "pdf"):
37         return
38     preferences.set_model_capability(provider, model, kind, "no")

```

```

38     logger.warning(
39         "model_capabilities.learned: provider=%s model=%s kind=%s value=no
40         reason=%s",
41         provider,
42         model,
43         kind,
44         reason[:200],
45     )
46
47 def record_supported(provider: str, model: str, kind: str) -> None:
48     if kind not in ("image", "pdf"):
49         return
50     current = preferences.get_model_capability(provider, model, kind)
51     if current is None:
52         preferences.set_model_capability(provider, model, kind, "yes")
53
54 def clear(provider: str, model: str | None = None) -> bool:
55     return preferences.clear_model_capabilities(provider, model)
56
57 def peers_supporting(
58     provider: str, kind: str, provider_configs: dict
59 ) -> list[str]:
60     catalog = provider_configs.get(provider, {}).get("models") or []
61     out: list[str] = []
62     for entry in catalog:
63         name = entry if isinstance(entry, str) else entry.get("name")
64         if not name:
65             continue
66         if preferences.get_model_capability(provider, name, kind) == "no":
67             continue
68         out.append(name)
69     return out
70
71 __all__ = [
72     "Capabilities", "Kind", "KNOWN_KINDS",
73     "clear", "get_capabilities", "peers_supporting",
74     "record_supported", "record_unsupported",
75 ]

```

## Anexa 6: src/clipboard.py

```

1 from __future__ import annotations
2 import logging
3 import platform
4 import shutil
5 import subprocess
6 logger = logging.getLogger(__name__)
7
8 def read_clipboard_image() -> bytes | None:
9     system = platform.system()
10     try:
11         if system == "Darwin":
12             return _read_macos()
13         if system == "Linux":
14             return _read_linux()
15         if system == "Windows":
16             return _read_windows()
17     except Exception as exc:
18         logger.info("clipboard.read_failed system=%s error=%s", system,
19         exc)
20     return None
21
22 def clipboard_help_hint() -> str:
23     system = platform.system()
24     if system == "Darwin":
25         return (
26             "install with: 'brew install pngpaste' (recommended), "
27             "or rely on the built-in osascript fallback"
28         )
29     if system == "Linux":
30         return (
31             "install one of: 'wl-clipboard' (Wayland) or 'xclip' (X11)"
32         )
33     if system == "Windows":
34         return "install Pillow: 'uv pip install Pillow'"
35     return f"unsupported platform: {system}"
36
37 def _read_macos() -> bytes | None:
38     if shutil.which("pngpaste"):
39         result = subprocess.run(
40             ["pngpaste", "-"],
41             capture_output=True,
42             timeout=5,
43         )
44         if result.returncode == 0 and result.stdout:
45             return result.stdout
46         logger.info("clipboard.pngpaste_empty returncode=%s", result.
47         returncode)

```

```

46         return None
47     import tempfile
48     with tempfile.NamedTemporaryFile(suffix=".png", delete=False) as tmp:
49         tmp_path = tmp.name
50         apple_script = f"""
51         try
52             set png_data to (the clipboard as « class PNGf »)
53             set f to open for access POSIX file "{tmp_path}" with write
54             permission
55             write png_data to f
56             close access f
57             return "ok"
58         on error errMsg
59             try
60                 close access POSIX file "{tmp_path}"
61             end try
62             return "err:" & errMsg
63         end try
64         """
65     result = subprocess.run(
66         ["osascript", "-e", apple_script],
67         capture_output=True,
68         timeout=5,
69         text=True,
70     )
71     if result.returncode != 0 or not (result.stdout or "").startswith(
72     "ok"):
73         logger.info("clipboard.osascript_no_image out=%r", result.
74         stdout[:80])
75         return None
76     from pathlib import Path
77     data = Path(tmp_path).read_bytes()
78     return data if data else None
79 finally:
80     try:
81         from pathlib import Path
82         Path(tmp_path).unlink(missing_ok=True)
83     except Exception:
84         pass
85
86 def _read_linux() -> bytes | None:
87     if shutil.which("wl-paste"):
88         result = subprocess.run(
89             ["wl-paste", "--type", "image/png"],

```



```

90         capture_output=True,
91         timeout=5,
92     )
93     if result.returncode == 0 and result.stdout:
94         return result.stdout
95     if shutil.which("xclip"):
96         result = subprocess.run(
97             ["xclip", "-selection", "clipboard", "-t", "image/png", "-o"]
98             capture_output=True,
99             timeout=5,
100         )
101     if result.returncode == 0 and result.stdout:
102         return result.stdout
103     return None
104
105 def _read_windows() -> bytes | None:
106     try:
107         from PIL import ImageGrab
108     except ImportError:
109         return None
110     img = ImageGrab.grabclipboard()
111     if img is None:
112         return None
113     if isinstance(img, list):
114         return None
115     import io
116     buf = io.BytesIO()
117     img.save(buf, format="PNG")
118     return buf.getvalue()
119
120 __all__ = ["clipboard_help_hint", "read_clipboard_image"]

```

## Anexa 7: src/compaction/base.py

```

1 from __future__ import annotations
2 import asyncio
3 import json
4 from abc import ABC, abstractmethod
5 from dataclasses import dataclass, field
6 from typing import Any, Callable
7 from src.artifacts import ArtifactStore
8
9 class CompactionConfigError(ValueError):
10     ...
11
12 class Stage(ABC):
13     ...
14
15     name: str = "stage"
16     mode: str = "eager"
17
18     def __init__(self, config: dict | None = None) -> None:
19         self.config: dict = config or {}
20         self.enabled: bool = bool(self.config.get("enabled", False))
21
22     @abstractmethod
23     def apply_eager(self, event: dict, *, store: ArtifactStore) -> bool:
24         ...
25
26     async def apply_eager_async(self, ectx: "EagerContext") -> bool:
27         return self.apply_eager(ectx.event, store=ectx.store)
28
29 @dataclass
30 class EagerContext:
31     event: dict
32     store: ArtifactStore
33     session_id: str | None = None
34     llm: Any | None = None
35     background_tasks: list[asyncio.Task] = field(default_factory=list)
36     on_event_updated: Callable[[dict], None] | None = None
37
38     def spawn(self, coro) -> asyncio.Task:
39         task = asyncio.create_task(coro)
40         self.background_tasks.append(task)
41         return task
42
43 def stringify(content: Any) -> str:
44     if content is None:
45         return ""
46     if isinstance(content, str):
47         return content
48     if isinstance(content, list):
49         out: list[str] = []
50         for block in content:
51             if isinstance(block, dict):
52                 out.append(block.get("text") or block.get("input") or "")
53             else:
54                 out.append(str(block))
55         return "".join(out)
56     if isinstance(content, dict):
57         try:
58             return json.dumps(content)
59         except (TypeError, ValueError):
60             return str(content)
61         return str(content)
62
63 def approx_tokens(text: str) -> int:
64     if not text:
65         return 0
66     return max(1, len(text) // 4)
67
68 @dataclass
69 class ThresholdContext:
70     events: list[dict]
71     current_size: int
72     target_size: int
73     min_turns_kept: int
74     store: ArtifactStore
75     llm: Any | None = None
76
77 @dataclass
78 class ThresholdResult:
79     removed_event_ids: list[str] = field(default_factory=list)
80     summary: str = ""
81     freed_estimate: int = 0
82     notes: dict = field(default_factory=dict)
83
84 class ThresholdStage(ABC):
85     name: str = "threshold_stage"
86     mode: str = "threshold"
87
88     def __init__(self, config: dict | None = None) -> None:
89         self.config: dict = config or {}
90         self.enabled: bool = bool(self.config.get("enabled", False))
91
92     @abstractmethod
93     async def apply_threshold(self, tctx: ThresholdContext) -> ThresholdResult:
94         """Decide what to remove (and optionally summarise)."""

```

## Anexa 8: src/compaction/stages/bulk.py

```

1 from __future__ import annotations
2 from typing import Any
3 from langchain_core.messages import HumanMessage, SystemMessage
4 from src.compaction.base import (
5     ThresholdContext,
6     ThresholdResult,
7     ThresholdStage,
8     approx_tokens,
9     stringify,
10 )
11 from ui.sessions import (
12     KIND_ASSISTANT_TEXT,
13     KIND_HUMAN,
14     KIND_SYSTEM_PROMPT,
15 )
16
17 class Bulk(ThresholdStage):
18     name = "bulk"
19     mode = "threshold"
20
21     def __init__(self, config: dict | None = None) -> None:
22         super().__init__(config)
23         strategy = (self.config.get("strategy") or "hybrid").lower()
24         if strategy not in ("trim", "summarize", "hybrid"):
25             strategy = "hybrid"
26         self.strategy: str = strategy
27         sumcfg = self.config.get("summarize") or {}
28         self.max_summary_tokens: int = int(sumcfg.get("max_summary_tokens"
29             , 1500))
30
31     async def apply_threshold(self, tctx: ThresholdContext) -> ThresholdResult:
32         effective = self.strategy
33         fallback_reason: str | None = None
34         if effective in ("summarize", "hybrid") and tctx.llm is None:
35             fallback_reason = f"{effective} requested but no LLM available"
36         effective = "trim"
37
38         turn_order: list[str] = []
39         events_by_turn: dict[str, list[dict]] = {}
40         for evt in tctx.events:
41             tid = evt.get("turn_id")
42             if not tid:
43                 continue
44             if tid not in events_by_turn:
45                 events_by_turn[tid] = []
46                 turn_order.append(tid)
47             events_by_turn[tid].append(evt)
48
49         if not turn_order:
50             return ThresholdResult(notes={"reason": "no turns yet"})
51         protected = set(turn_order[-tctx.min_turns_kept :]) if tctx.min_turns_kept > 0 else set()
52         droppable = [t for t in turn_order if t not in protected]
53
54         def _turn_size(tid: str) -> int:
55             return sum(
56                 approx_tokens(stringify(e.get("projected") or e.get("content")))
57                 for e in events_by_turn[tid]
58             )
59
60         size_remaining = tctx.current_size

```



```

60         drop: list[str] = []
61         for tid in droppable:
62             if size_remaining <= tctx.target_size:
63                 break
64             drop.append(tid)
65             size_remaining -= _turn_size(tid)
66
67         if not drop:
68             notes: dict[str, Any] = {"reason": "already under target", "
69 strategy": effective}
70             if fallback_reason:
71                 notes["fallback"] = fallback_reason
72             return ThresholdResult(notes=notes)
73
74         removed_ids = [
75             evt["id"]
76             for tid in drop
77             for evt in events_by_turn[tid]
78             if evt["kind"] != KIND_SYSTEM_PROMPT
79         ]
80
81         summary = ""
82         if effective in ("summarize", "hybrid"):
83             summary = await self._summarize_turns(
84                 tctx.llm, [events_by_turn[t] for t in drop]
85             )
86
87         notes = {
88             "strategy": effective,
89             "dropped_turns": len(drop),
90             "protected_turns": len(protected),
91         }
92         if fallback_reason:
93             notes["fallback"] = fallback_reason
94
95         return ThresholdResult(
96             removed_event_ids=removed_ids,
97             summary=summary,
98             notes=notes,
99         )
100
101     async def _summarize_turns(self, llm, turn_event_lists: list[list[dict]]) -> str:
102         """Ask the LLM to compress dropped turns into a short summary."""
103         lines: list[str] = []
104         for evts in turn_event_lists:
105             human = next((e for e in evts if e["kind"] == KIND_HUMAN),
106
107
108         if human is not None:
109             lines.append(
110                 f"User: {stringify(human.get('projected') or human.get('content'))}"
111             )
112             final_text = ""
113             for e in evts:
114                 if e["kind"] == KIND_ASSISTANT_TEXT:
115                     text = stringify(e.get("projected") or e.get("content"))
116                     if text.strip():
117                         final_text = text
118             if final_text:
119                 lines.append(f"Assistant: {final_text}")
120
121         if not lines:
122             return ""
123
124         transcript = "\n".join(lines)
125         instr = (
126             "Summarize the following conversation excerpt for a future-
127 self of "
128             "the assistant who must continue the conversation. Preserve
129 user "
130             "intent, key decisions, names, file paths, and any open
131 questions. "
132             f"Aim for under {self.max_summary_tokens} tokens.\n\n"
133             f"--- transcript ---\n"
134             f"{transcript}\n"
135             f"--- end transcript ---"
136         )
137         prompt = [
138             SystemMessage(content="You compress conversations losslessly
139 for the parts that matter."),
140             HumanMessage(content=instr),
141         ]
142         try:
143             response = await llm.ainvoke(prompt)
144         except Exception as exc:
145             # noqa: BLE001
146             return f"(summarization failed: {exc})"
147         text = response.content
148         if isinstance(text, list):
149             text = "".join(
150                 b.get("text", "") if isinstance(b, dict) else str(b) for b
151 in text
152             )
153         return str(text or "").strip()

```

## Anexa 9: src/compaction/stages/ref\_substitute.py

```

1 from __future__ import annotations
2 from src.compaction.base import Stage, approx_tokens, stringify
3 from ui.sessions import KIND_TOOL_RESULT
4
5 class RefSubstitute(Stage):
6     name = "ref_substitute"
7     mode = "eager"
8
9     def __init__(self, config: dict | None = None) -> None:
10         super().__init__(config)
11         self.min_ref_tokens: int = int(self.config.get("min_ref_tokens",
12 1500))
13         self.excerpt_tokens: int = int(self.config.get("excerpt_tokens",
14 200))
15
16     def apply_eager(self, event, *, store) -> bool:
17         if event.get("kind") != KIND_TOOL_RESULT:
18             return False
19         existing_meta = (event.get("meta") or {}).get("compaction") or {}
20         if "ref_substitute" in existing_meta:
21             return False
22
23         original_body = stringify(event.get("content"))
24         if approx_tokens(original_body) < self.min_ref_tokens:
25             return False
26         ref = store.put(original_body, kind="tool_result", ext="txt")
27         excerpt_chars = self.excerpt_tokens * 4
28         excerpt = original_body[:excerpt_chars]
29         projected = (
30             f"[ref_substitute] {ref.handle} ({ref.bytes_} bytes) - "
31             f"call fetch_artifact to retrieve.\n--- excerpt ---\n{excerpt}"
32         )
33
34         event["projected"] = projected
35         if event.get("meta") is None:
36             event["meta"] = {}
37         comp = event["meta"].setdefault("compaction", {})
38         comp["ref_substitute"] = {
39             "original_bytes": ref.bytes_,
40             "artifact": ref.handle,
41         }
42         return True

```

## Anexa 10: src/compaction/stages/tool\_truncate.py

```

1 from __future__ import annotations
2 from src.compaction.base import Stage
3 from src.compaction.stages.tool_helpers import _truncate_to_excerpt,
4 event_body
5
6 class ToolTruncate(Stage):
7     name = "tool_truncate"
8     mode = "eager"
9
10     def __init__(self, config: dict | None = None) -> None:
11         super().__init__(config)
12         self.max_tool_tokens: int = int(self.config.get("max_tool_tokens",
13 2000))
14         head_tail = self.config.get("head_tail") or [800, 400]
15         self.head_tokens: int = int(head_tail[0])
16         self.tail_tokens: int = int(head_tail[1])
17
18     def apply_eager(self, event, *, store) -> bool:
19         if event.get("kind") != KIND_TOOL_RESULT:
20             return False
21
22         body = event_body(event)
23         result = _truncate_to_excerpt(
24             body,
25             store=store,
26             max_tool_tokens=self.max_tool_tokens,
27             head_tokens=self.head_tokens,
28             tail_tokens=self.tail_tokens,
29             marker_label="tool_truncate",
30         )
31         if result is None:
32             return False
33
34         event["projected"] = result.projected
35         if event.get("meta") is None:
36             event["meta"] = {}
37         comp = event["meta"].setdefault("compaction", {})
38         comp["tool_truncate"] = {
39             "original_bytes": result.original_bytes,
40             "artifact": result.artifact_handle,
41         }
42         return True

```

## Anexa 11: src/main.py

```

1 import logging
2 import os
3 import re
4 import time
5 from pathlib import Path
6 from typing import Annotated, TypedDict
7
8 import yaml
9 from langchain_core.messages import HumanMessage, ToolMessage
10 from langchain_core.runnables import RunnableConfig
11 from langgraph.graph import StateGraph, START, END
12 from langgraph.graph.message import add_messages
13
14 from src.logging_setup import emit_event, get_logger, redact
15 from src.permissions import request_permission
16 from src.providers import get_provider
17 from src.tool_guards import check_preconditions, run_post_hooks
18 from src.tools import ALL_TOOLS, reset_tool_ctx, set_tool_ctx
19
20 def _resolve_env_vars(value):
21     if isinstance(value, str):
22         return re.sub(
23             r"\${(\\w+)}",
24             lambda m: os.environ.get(m.group(1), ""),
25             value,
26         )
27     if isinstance(value, dict):
28         return {k: _resolve_env_vars(v) for k, v in value.items()}
29     if isinstance(value, list):
30         return [_resolve_env_vars(v) for v in value]
31     return value
32
33 _bundled_config_path = Path(__file__).resolve().parent.parent / "config"
34 _config_yaml = "config.yaml"
35
36 from ui.config_init import ensure_user_config
37 from ui.paths import USER_CONFIG_PATH
38
39 ensure_user_config(_bundled_config_path, USER_CONFIG_PATH)
40 _config_path = USER_CONFIG_PATH
41
42 with open(_config_path) as f:
43     config = _resolve_env_vars(yaml.safe_load(f))
44
45 provider_configs: dict[str, dict] = config.setdefault("providers", {})
46 providers: list[str] = [
47     k for k, v in provider_configs.items() if isinstance(v, dict) and v.get("models")
48 ]
49
50 for _prov in providers:
51     if "model" not in provider_configs[_prov]:
52         provider_configs[_prov]["model"] = provider_configs[_prov]["models"][0]
53
54 class AgentState(TypedDict):
55     messages: Annotated[list, add_messages]
56
57
58 def get_model(provider: str):
59     cfg = provider_configs.get(provider)
60     if cfg is None:
61         raise ValueError(f"Unsupported model provider: {provider}")
62     return get_provider(provider).create_model(cfg)
63
64 _active_tools: list = list(ALL_TOOLS)
65
66 def get_active_tools() -> list:
67     return list(_active_tools)
68
69 def set_active_tools(tools: list):
70     _active_tools.clear()
71     _active_tools.extend(tools)
72
73 def _tool_by_name(name: str):
74     for t in _active_tools:
75         if getattr(t, "name", None) == name:
76             return t
77     return None
78
79 ATTACHMENT_ERROR_PHRASES = (
80     "image_url", "image input", "image content",
81     "vision", "multimodal", "modality",
82     "media type", "document", "pdf", "image",
83     "does not support", "unsupported content type",
84     "invalid content type", "no image in", "not accept",
85 )
86
87 def _classify_attachment_error(exc: BaseException) -> str | None:
88     msg = " ".join(str(arg) for arg in getattr(exc, "args", ())) or str(
89         exc
90     )
91     msg_low = msg.lower()
92     for attr in ("body", "response", "message"):
93         v = getattr(exc, attr, None)
94         if v:
95             msg_low += " " + str(v).lower()
96
97 if not any(p in msg_low for p in ATTACHMENT_ERROR_PHRASES):
98     return None
99
100 if "pdf" in msg_low or "document" in msg_low:
101     return "pdf"
102 if "image" in msg_low or "vision" in msg_low or "image_url" in msg_low:
103     return "image"
104 if "multimodal" in msg_low or "modality" in msg_low:
105     return "image"
106 return None
107
108 def _human_message_has_kind(messages: list, kind: str) -> bool:
109     for msg in reversed(messages):
110         if not isinstance(msg, HumanMessage):
111             continue
112         content = getattr(msg, "content", None)
113         if not isinstance(content, list):
114             return False
115         for block in content:
116             if not isinstance(block, dict):
117                 continue
118             btype = block.get("type", "")
119             if kind == "image" and btype in ("image", "image_url"):
120                 return True
121             if kind == "pdf" and (
122                 btype == "document"
123                 or block.get("kind") == "pdf"
124                 or "pdf" in str(block.get("media_type", "")).lower()
125             ):
126                 return True
127             return False
128     return False
129
130 async def chatbot_node(state: AgentState, config: RunnableConfig):
131     provider = config.get("configurable", {}).get("provider", "openai")
132     _module_provider_configs = globals().get("provider_configs") or {}
133     model_name = _module_provider_configs.get(provider, {}).get("model")
134     llm_log = get_logger("llm")
135
136     llm = get_model(provider)
137     llm_with_tools = llm.bind_tools(get_active_tools())
138
139     emit_event(
140         llm_log,
141         "llm.start",
142         provider=provider,
143         model=model_name,
144         message_count=len(state["messages"]),
145     )
146
147 t0 = time.perf_counter()
148 try:
149     response = await llm_with_tools.ainvoke(state["messages"])
150 except Exception as e:
151     kind = _classify_attachment_error(e)
152     if kind and model_name and _human_message_has_kind(state["messages"], kind):
153         try:
154             from src import capabilities as _caps
155             _caps.record_unsupported(
156                 provider, model_name, kind, reason=str(e)[:200]
157             )
158         except Exception:
159             pass
160     try:
161         setattr(e, "_vibe_attachment_kind_rejected", kind)
162     except Exception:
163         pass
164     emit_event(
165         llm_log,
166         "llm.end",
167         level=logging.ERROR,
168         provider=provider,
169         model=model_name,
170         ok=False,
171         error_type=type(e).__name__,
172         attachment_kind_rejected=kind,
173         duration_ms=(time.perf_counter() - t0) * 1000.0,
174         exc_info=True,
175     )
176     raise
177
178 um = getattr(response, "usage_metadata", None) or {}
179 input_token_details = um.get("input_token_details") or {}
180 tool_calls = getattr(response, "tool_calls", None) or []
181
182 emit_event(
183     llm_log,
184     "llm.end",
185     provider=provider,
186     model=model_name,
187     ok=True,
188     input_tokens=int(um.get("input_tokens") or 0),
189     output_tokens=int(um.get("output_tokens") or 0),
190     cache_read=int(input_token_details.get("cache_read") or 0),
191     cache_write=int(input_token_details.get("cache_creation") or 0),
192     tool_calls=len(tool_calls),
193     stop_reason=getattr(response, "response_metadata", {}).get("stop_reason"),
194     duration_ms=(time.perf_counter() - t0) * 1000.0,
195 )
196 return {"messages": [response]}
197
198 def _read_text_safe(path: str) -> str | None:
199     try:
200         p = Path(path).expanduser().resolve()
201         if not p.exists():
202             return ""
203         if not p.is_file():
204             return None
205         return p.read_text(errors="replace")
206     except Exception:
207         return None
208
209 def _tool_source(name: str) -> str:
210     return "mcp:" + name.split("___", 1)[0] if "___" in name else "builtin"
211
212 async def _run_one_tool_call(tc: dict, app_ctx: dict) -> ToolMessage:
213     from ui.ui import label_for_tool, tool_log_line
214
215     tool_log = get_logger("tool_call")
216     name = tc["name"]
217     args = tc.get("args", {}) or {}
218     call_id = tc["id"]
219
220     tool = _tool_by_name(name)
221     if tool is None:
222         emit_event(
223             tool_log,
224             "tool.unavailable",
225             level=logging.WARNING,
226             tool=name,
227             tool_call_id=call_id,
228             source=_tool_source(name),
229         )
230         return ToolMessage(
231             content=f"Tool '{name}' is not available.",
232             tool_call_id=call_id,
233             name=name,
234         )
235     )

```

```

233
234 emit_event(
235     tool_log,
236     "tool.start",
237     tool=name,
238     tool_call_id=call_id,
239     source=_tool_source(name),
240     args=redact(args),
241 )
242
243 decision = await request_permission(name, args, app_ctx)
244 if not decision.allow:
245     emit_event(
246         tool_log,
247         "tool.end",
248         tool=name,
249         tool_call_id=call_id,
250         source=_tool_source(name),
251         ok=False,
252         denied=True,
253         reason_len=len(decision.reason or ""),
254     )
255     return ToolMessage(
256         content=f"User denied this tool call. Reason: {decision.reason}",
257         tool_call_id=call_id,
258         name=name,
259     )
260
261 denied = check_preconditions(name, args, app_ctx)
262 if denied is not None:
263     guard_name, reason = denied
264     emit_event(
265         tool_log,
266         "tool.precondition.denied",
267         tool=name,
268         tool_call_id=call_id,
269         source=_tool_source(name),
270         guard=guard_name,
271         reason_len=len(reason),
272     )
273     app_ctx.setdefault("pending_precondition_denials", {})[call_id] = {
274         "guard": guard_name,
275         "reason": reason,
276     }
277     return ToolMessage(content=reason, tool_call_id=call_id, name=name)
278
279 _tool_t0 = time.perf_counter()
280 app_ctx["status_label"] = label_for_tool(name, args)
281
282 if name not in ("write_file", "edit_file"):
283     app_ctx.setdefault("pending_tool_logs", {})[call_id] =
284         tool_log_line(name, args)
285
286 before_content: str | None = None
287 if name in ("write_file", "edit_file"):
288     path_arg = args.get("path") if isinstance(args, dict) else None
289     if isinstance(path_arg, str) and path_arg:
290         before_content = _read_text_safe(path_arg)
291
292 ctx_token = set_tool_ctx(app_ctx)
293 tool_error: Exception | None = None
294 tool_call_envelope = {
295     "name": name,
296     "args": args,
297     "id": call_id,
298     "type": "tool_call",
299 }
300 try:
301     result = await tool.ainvoke(tool_call_envelope)
302 except Exception as e:
303     tool_error = e
304     result = f"Tool error: {e}"
305 finally:
306     reset_tool_ctx(ctx_token)
307
308 if isinstance(result, ToolMessage):
309     result = result.content
310 if not isinstance(result, str):
311     result = str(result)
312 emit_event(
313     tool_log,
314     "tool.end",
315     tool=name,
316     tool_call_id=call_id,
317     source=_tool_source(name),
318     ok=tool_error is None,
319     error_type=type(tool_error).__name__ if tool_error else None,
320     result_bytes=len(result),
321     duration_ms=(time.perf_counter() - _tool_t0) * 1000.0,
322 )
323
324 if tool_error is None:
325     run_post_hooks(name, args, result, app_ctx)
326
327 if (
328     name in ("write_file", "edit_file")
329     and before_content is not None
330     and (result.startswith("Written ") or result.startswith("Edited "
331 )):
332     path_arg = args.get("path") if isinstance(args, dict) else None
333     after_content = (
334         _read_text_safe(path_arg) if isinstance(path_arg, str) else
335         None
336     )
337     if after_content is not None:
338         app_ctx.setdefault("pending_diffs", {})[call_id] = {
339             "path": path_arg,
340             "before": before_content,
341             "after": after_content,
342         }
343
344 app_ctx["status_label"] = "thinking"
345 return ToolMessage(content=result, tool_call_id=call_id, name=name)
346
347 async def permissioned_tools_node(state: AgentState, config:
348
349     RunnableConfig):
350     import asyncio
351
352     last = state["messages"][-1]
353     tool_calls = getattr(last, "tool_calls", None) or []
354     app_ctx = config.get("configurable", {}).get("app_ctx", {})
355
356     sub_cfg = (app_ctx.get("subagents_config") or {})
357     max_parallel = max(1, int(sub_cfg.get("max_parallel", 3)))
358
359     results: dict[int, ToolMessage] = {}
360     i = 0
361     n = len(tool_calls)
362     while i < n:
363         tc = tool_calls[i]
364         if tc["name"] == "spawn_subagent":
365             batch_indices: list[int] = []
366             while (
367                 i < n
368                 and tool_calls[i]["name"] == "spawn_subagent"
369                 and len(batch_indices) < max_parallel
370             ):
371                 batch_indices.append(i)
372                 i += 1
373             batch_tcs = [tool_calls[idx] for idx in batch_indices]
374             app_ctx["status_label"] = (
375                 f"Agentx{len(batch_tcs)} running"
376                 if len(batch_tcs) > 1
377                 else app_ctx.get("status_label") or "thinking"
378             )
379             batch_results = await asyncio.gather(
380                 *(_run_one_tool_call(t, app_ctx) for t in batch_tcs),
381                 return_exceptions=False,
382             )
383             for idx, tm in zip(batch_indices, batch_results):
384                 results[idx] = tm
385             app_ctx["status_label"] = "thinking"
386             continue
387         results[i] = await _run_one_tool_call(tc, app_ctx)
388         i += 1
389
390 out_messages: list = [results[k] for k in sorted(results)]
391
392 injections = app_ctx.pop("pending_image_injections", None) or []
393 if injections:
394     synth_msg = _build_synthetic_image_human_message(injections)
395     if synth_msg is not None:
396         out_messages.append(synth_msg)
397         _record_synthetic_human_event(app_ctx, injections, synth_msg)
398     return {"messages": out_messages}
399
400 def _build_synthetic_image_human_message(injections: list[dict]):
401     blocks: list[dict] = []
402     summary_parts: list[str] = []
403     for inj in injections:
404         name = inj.get("pdf_name") or "<pdf>"
405         start = inj.get("page_start")
406         end = inj.get("page_end")
407         if start is not None and end is not None and start != end:
408             summary_parts.append(f"{name} pages {start}-{end}")
409         elif start is not None:
410             summary_parts.append(f"{name} page {start}")
411         else:
412             summary_parts.append(name)
413     blocks.extend(inj.get("image_blocks") or [])
414     if not blocks:
415         return None
416     preamble = "Rendered images: " + ", ".join(summary_parts) + ". "
417     return HumanMessage(content=[{"type": "text", "text": preamble}, *
418         blocks])
419
420 def _record_synthetic_human_event(app_ctx: dict, injections: list[dict],
421     msg) -> None:
422     sess = app_ctx.get("session")
423     if sess is None:
424         return
425     try:
426         from ui.sessions import KIND_HUMAN as _KIND_HUMAN
427         page_refs: list[dict] = []
428         for inj in injections:
429             for sha in inj.get("page_shas") or []:
430                 page_refs.append(
431                     {
432                         "type": "rendered_page",
433                         "pdf_name": inj.get("pdf_name") or "",
434                         "sha": sha,
435                     }
436                 )
437         content = list(getattr(msg, "content", []) or [])
438         compact = [b for b in content if isinstance(b, dict) and b.get("
439 type") == "text"]
440         compact.extend(page_refs)
441         meta = {
442             "synthesized": True,
443             "source": "read_pdf_pages",
444             "injections": [
445                 {
446                     "pdf_name": inj.get("pdf_name"),
447                     "page_start": inj.get("page_start"),
448                     "page_end": inj.get("page_end"),
449                     "page_count": len(inj.get("page_shas") or []),
450                 }
451                 for inj in injections
452             ],
453         }
454         sess.append_event(_KIND_HUMAN, compact, meta=meta)
455     except Exception:
456         pass
457
458 def should_continue(state: AgentState):
459     last_message = state["messages"][-1]
460     if hasattr(last_message, "tool_calls") and last_message.tool_calls:
461         return "tools"
462     return END

```

```

463 builder = StateGraph(AgentState)
464
465 builder.add_node("chatbot", chatbot_node)
466 builder.add_node("tools", permissioned_tools_node)
467
468 builder.add_edge(START, "chatbot")

```

```

469 builder.add_conditional_edges("chatbot", should_continue, {"tools": "tools",
470 builder.add_edge("tools", "chatbot")
471
472 graph = builder.compile()

```

## Anexa 12: src/multimodal.py

```

1 from __future__ import annotations
2 import base64
3 import logging
4 from typing import Any
5 from src.attachments import Attachment, fetch_bytes
6 logger = logging.getLogger(__name__)
7
8 class MultimodalEncodingError(Exception):
9     ...
10
11 def _b64(data: bytes) -> str:
12     return base64.b64encode(data).decode("ascii")
13
14 def _image_block_for(provider: str, att: Attachment, b64: str) -> dict:
15     if provider == "openai":
16         return {
17             "type": "image_url",
18             "image_url": {"url": f"data:{att.mime_type};base64,{b64}"},
19         }
20     if provider in ("claude", "bedrock"):
21         return {
22             "type": "image",
23             "source": {
24                 "type": "base64",
25                 "media_type": att.mime_type,
26                 "data": b64,
27             },
28         }
29     if provider == "gemini":
30         return {
31             "type": "image_url",
32             "image_url": f"data:{att.mime_type};base64,{b64}",
33         }
34     return {
35         "type": "image_url",
36         "image_url": {"url": f"data:{att.mime_type};base64,{b64}"},
37     }
38
39 def _text_block(text: str) -> dict:
40     return {"type": "text", "text": text}
41
42 def build_human_content(
43     text: str,
44     attachments: list[Attachment],
45     provider: str,
46     session_id: str,
47     capability_filter: Any = None,
48 ) -> str | list[dict]:
49     if not attachments:
50         return text
51
52     blocks: list[dict] = []
53     dropped_notes: list[str] = []
54
55     for att in attachments:
56         if att.kind != "text":
57             continue
58         if capability_filter and not capability_filter(att.kind):
59             dropped_notes.append(
60                 f"[attachment '{att.name}' omitted by capability filter]"
61             )
62         continue
63         body = att.text_content
64         if body is None:
65             data = fetch_bytes(session_id, att.sha)
66             if data is None:
67                 dropped_notes.append(
68                     f"[attachment '{att.name}' bytes missing from artifact
69                     store]"
70                 )
71             continue
72             body = data.decode("utf-8", errors="replace")
73             blocks.append(_text_block(f"# attachment: {att.name}\n\n{body}\n"))
74
75     image_ok = (capability_filter is None) or capability_filter("image")
76     for att in attachments:
77         if att.kind != "pdf":
78             continue
79         if capability_filter and not capability_filter(att.kind):
80             dropped_notes.append(
81                 f"[attachment '{att.name}' (PDF) omitted by capability
82                 filter]"
83             )
84             continue
85         if (
86             att.pdf_render_strategy == "full"
87             and att.rendered_page_shas
88             and image_ok
89         ):
90             header = (
91                 f"# attachment: {att.name} "
92                 f"({att.pdf_page_count} or len(att.rendered_page_shas))
93                 pages, "
94                 "full mode - text + page images inline)"
95             )
96             body = att.pdf_full_text or ""
97             blocks.append(_text_block(f"{header}\n\n{body}\n"))
98             for sha in att.rendered_page_shas:
99                 data = fetch_bytes(session_id, sha)
100                 if data is None:

```

```

100                 dropped_notes.append(
101                     f"[rendered page from '{att.name}' missing from
102                     artifact store]"
103                 )
104                 continue
105             try:
106                 b64 = _b64(data)
107             except Exception as exc:
108                 raise MultimodalEncodingError(
109                     f"failed to base64-encode page from {att.name}: {
110                     exc}"
111                 ) from exc
112             img_att = Attachment(
113                 path="",
114                 name=f"{att.name}#page",
115                 kind="image",
116                 mime_type="image/png",
117                 size=len(data),
118                 sha=sha,
119                 ext="png",
120             )
121             blocks.append(_image_block_for(provider, img_att, b64))
122             continue
123             manifest = att.pdf_manifest or f"# attachment: {att.name} (PDF)"
124             blocks.append(_text_block(manifest))
125
126     if text:
127         blocks.append(_text_block(text))
128
129     for att in attachments:
130         if att.kind != "image":
131             continue
132         if capability_filter and not capability_filter(att.kind):
133             dropped_notes.append(
134                 f"[image '{att.name}' omitted: active model doesn't accept
135                 image input]"
136             )
137             continue
138             data = fetch_bytes(session_id, att.sha)
139             if data is None:
140                 dropped_notes.append(
141                     f"[image '{att.name}' bytes missing from artifact store]"
142                 )
143             try:
144                 b64 = _b64(data)
145             except Exception as exc:
146                 raise MultimodalEncodingError(
147                     f"failed to base64-encode {att.name}: {exc}"
148                 ) from exc
149             blocks.append(_image_block_for(provider, att, b64))
150
151     if dropped_notes:
152         blocks.append(_text_block("\n".join(dropped_notes)))
153
154     if not blocks:
155         return text or "(no content)"
156
157     return blocks
158
159 def build_image_blocks_from_shas(
160     shas: list[str],
161     *,
162     provider: str,
163     session_id: str,
164     mime_type: str = "image/png",
165 ) -> list[dict]:
166     blocks: list[dict] = []
167     for sha in shas:
168         data = fetch_bytes(session_id, sha)
169         if data is None:
170             logger.info("multimodal.injection_missing sha=%s", sha[:12])
171             continue
172         try:
173             b64 = _b64(data)
174         except Exception as exc:
175             logger.info(
176                 "multimodal.injection_encode_failed sha=%s error=%s",
177                 sha[:12],
178                 exc,
179             )
180             continue
181             shell = Attachment(
182                 path="",
183                 name="rendered_page.png",
184                 kind="image",
185                 mime_type=mime_type,
186                 size=len(data),
187                 sha=sha,
188                 ext="png",
189             )
190             blocks.append(_image_block_for(provider, shell, b64))
191     return blocks
192
193 def event_blocks_for_attachments(attachments: list[Attachment]) -> list[
194     dict]:
195     return [att.to_block() for att in attachments]
196
197 def build_human_event_content(
198     text: str, attachments: list[Attachment]
199 ) -> str | list[dict]:
200     if not attachments:

```

```

199         return text
200     blocks: list[dict] = [{"type": "text", "text": text}] if text else []
201     blocks.extend(event_blocks_for_attachments(attachments))
202     return blocks
203
204 __all__ = [
205     "MultimodalEncodingError", "build_human_content",
206     "build_human_event_content", "build_image_blocks_from_shas",
207     "event_blocks_for_attachments",
208 ]

```

## Anexa 13: src/permissions.py

```

1 from __future__ import annotations
2 import json
3 from dataclasses import dataclass
4 from typing import Any
5 from prompt_toolkit import PromptSession
6 from prompt_toolkit.formatted_text import HTML
7 from prompt_toolkit.patch_stdout import patch_stdout
8 from rich.panel import Panel
9 from rich.text import Text
10 from src.logging_setup import emit_event, get_logger
11 from ui import preferences
12 from ui.ui import console
13
14 _perm_log = get_logger("permission")
15
16 @dataclass
17 class Decision:
18     allow: bool
19     reason: str = ""
20
21 _session_allow: set[str] = set()
22
23 def reset_session():
24     _session_allow.clear()
25
26 def session_allow(tool_name: str):
27     _session_allow.add(tool_name)
28
29 def is_session_allowed(tool_name: str) -> bool:
30     return tool_name in _session_allow
31
32 def _format_args(tool_args: Any) -> str:
33     try:
34         s = json.dumps(tool_args, indent=2, default=str)
35     except (TypeError, ValueError):
36         s = str(tool_args)
37     if len(s) > 1500:
38         s = s[:1500] + "\n... (truncated)"
39     return s
40
41 def _render_prompt_panel(tool_name: str, tool_args: Any):
42     body = Text()
43     body.append("Tool: ", style="muted")
44     body.append(f"{tool_name}\n\n", style="bold accent")
45     body.append("Arguments:\n", style="muted")
46     body.append(_format_args(tool_args))
47     body.append("\n\n")
48     body.append("1", style="bold accent")
49     body.append("Yes, run this call\n")
50     body.append("2", style="bold accent")
51     body.append("Yes, and don't ask again this session\n")
52     body.append("a", style="bold accent")
53     body.append("Always allow this tool (persist)\n")
54     body.append("3", style="bold accent")
55     body.append("No (provide a reason)")
56
57     console.print()
58     console.print(
59         Panel(
60             body,
61             title="[bold]Tool permission required[/bold]",
62             border_style="primary",
63             padding=(1, 2),
64         )
65     )
66
67 async def _prompt_choice(session: PromptSession | None) -> str:
68     prompt_msg = HTML('<style fg="#a78bfa"><b>permission </b></style>')
69     if session is None:
70         session = PromptSession()
71     with patch_stdout():
72         return (await session.prompt_async(prompt_msg)).strip().lower()
73
74 async def _prompt_reason(session: PromptSession | None) -> str:
75     prompt_msg = HTML('<style fg="#a78bfa"><b>reason </b></style>')
76     if session is None:
77         session = PromptSession()
78     with patch_stdout():
79         return (await session.prompt_async(prompt_msg)).strip()
80
81 async def request_permission(
82     tool_name: str,
83     tool_args: Any,
84     ctx: dict | None = None,
85 ) -> Decision:
86     ctx = ctx or {}
87
88     if ctx.get("skip_permissions"):
89         emit_event(
90             _perm_log, "permission.auto_allow", tool=tool_name, scope="skip_permissions"
91         )
92     return Decision(allow=True)
93
94 if tool_name in preferences.get_always_deny():
95     emit_event(
96         _perm_log, "permission.auto_deny", tool=tool_name, scope="always_deny"
97     )
98     return Decision(
99         allow=False,
100         reason="Tool is on the user's permanent deny list.",
101     )
102
103 if tool_name in preferences.get_always_allow():
104     emit_event(
105         _perm_log, "permission.auto_allow", tool=tool_name, scope="always_allow"
106     )
107     return Decision(allow=True)
108
109 if is_session_allowed(tool_name):
110     emit_event(
111         _perm_log, "permission.auto_allow", tool=tool_name, scope="session"
112     )
113     return Decision(allow=True)
114
115 session: PromptSession | None = ctx.get("pt_session")
116 status = ctx.get("status")
117 if status is not None:
118     try:
119         status.stop()
120     except Exception:
121         pass
122
123 emit_event(_perm_log, "permission.prompt", tool=tool_name)
124 try:
125     return await _interactive_prompt(tool_name, tool_args, session)
126 finally:
127     if status is not None:
128         try:
129             status.start()
130         except Exception:
131             pass
132
133 async def _interactive_prompt(
134     tool_name: str,
135     tool_args: Any,
136     session: PromptSession | None,
137 ) -> Decision:
138     while True:
139         _render_prompt_panel(tool_name, tool_args)
140         try:
141             choice = await _prompt_choice(session)
142         except (EOFError, KeyboardInterrupt):
143             emit_event(
144                 _perm_log,
145                 "permission.decision",
146                 tool=tool_name,
147                 allow=False,
148                 scope="cancelled",
149             )
150             return Decision(
151                 allow=False,
152                 reason="User cancelled the permission prompt.",
153             )
154
155     if choice in ("1", "y", "yes"):
156         emit_event(_perm_log, "permission.decision", tool=tool_name, allow=True, scope="once")
157         return Decision(allow=True)
158     if choice == "2":
159         session_allow(tool_name)
160         emit_event(_perm_log, "permission.decision", tool=tool_name, allow=True, scope="session")
161         return Decision(allow=True)
162     if choice == "a":
163         preferences.add_always_allow(tool_name)
164         emit_event(_perm_log, "permission.decision", tool=tool_name, allow=True, scope="always_allow")
165         return Decision(allow=True)
166     if choice in ("3", "n", "no"):
167         try:
168             reason = await _prompt_reason(session)
169         except (EOFError, KeyboardInterrupt):
170             reason = ""
171         emit_event(
172             _perm_log,
173             "permission.decision",
174             tool=tool_name,
175             allow=False,
176             scope="denied",
177             reason_len=len(reason),
178         )
179         return Decision(
180             allow=False,
181             reason=reason or "User denied this tool call without a reason.",
182         )
183
184     console.print("[muted]Please choose 1, 2, a, or 3.[/muted]")

```

## Anexa 14: src/providers.py

```

1 from __future__ import annotations
2
3 import json
4 import os
5 from typing import Any
6 from urllib.request import Request, urlopen
7
8 ProviderConfig = dict[str, Any]
9
10 class BaseProvider:
11     name: str
12     env_var: str | list[str] | None = None
13     validation_timeout: float = 5.0
14
15     def env_vars(self) -> list[str]:
16         if self.env_var is None:
17             return []
18         if isinstance(self.env_var, str):
19             return [self.env_var]
20         return list(self.env_var)
21
22     def preferred_env_var(self) -> str | None:
23         names = self.env_vars()
24         return names[0] if names else None
25
26     def resolve_env_key(self) -> str | None:
27         for name in self.env_vars():
28             value = os.environ.get(name)
29             if value:
30                 return value
31         return None
32
33     def requires_api_key(self) -> bool:
34         return bool(self.env_vars())
35
36     def base_url(self, cfg: ProviderConfig) -> str | None:
37         return None
38
39     def unavailable_reason(self) -> str:
40         if self.requires_api_key():
41             return "has no valid API key"
42         return "is not reachable"
43
44     def unreachable_help(self, cfg: ProviderConfig) -> tuple[str, str]:
45         return (
46             f"Could not reach {self.name}.",
47             "Try again or choose another provider.",
48         )
49
50     def require_api_key(self, cfg: ProviderConfig) -> str:
51         api_key = cfg.get("api_key")
52         if not api_key:
53             raise ValueError(
54                 f"API key for '{self.name}' is not set. "
55                 f"Please set the corresponding environment variable in "
56                 f"config.yml."
57             )
58         return str(api_key)
59
60     def validate(self, cfg: ProviderConfig) -> bool:
61         raise NotImplementedError
62
63     def create_model(self, cfg: ProviderConfig):
64         raise NotImplementedError
65
66 class OpenAIProvider(BaseProvider):
67     name = "openai"
68     env_var = "OPENAI_API_KEY"
69
70     def validate(self, cfg: ProviderConfig) -> bool:
71         api_key = cfg.get("api_key")
72         if not api_key:
73             return False
74
75         try:
76             import openai
77
78             client = openai.OpenAI(api_key=api_key)
79             client.models.list()
80             return True
81         except Exception:
82             return False
83
84     def create_model(self, cfg: ProviderConfig):
85         from langchain_openai import ChatOpenAI
86
87         return ChatOpenAI(
88             model=cfg["model"],
89             temperature=cfg["temperature"],
90             api_key=self.require_api_key(cfg),
91         )
92
93 class ClaudeProvider(BaseProvider):
94     name = "claude"
95     env_var = "ANTHROPIC_API_KEY"
96
97     def validate(self, cfg: ProviderConfig) -> bool:
98         api_key = cfg.get("api_key")
99         if not api_key:
100             return False
101
102         try:
103             import anthropic
104
105             client = anthropic.Anthropic(api_key=api_key)
106             client.models.list()
107             return True
108         except Exception:
109             return False
110
111     def create_model(self, cfg: ProviderConfig):
112         from langchain_anthropic import ChatAnthropic
113
114         return ChatAnthropic(
115             model=cfg["model"],
116             temperature=cfg["temperature"],
117             api_key=self.require_api_key(cfg),
118         )
119
120
121 class GeminiProvider(BaseProvider):
122     name = "gemini"
123     env_var = "GOOGLE_API_KEY"
124
125     def validate(self, cfg: ProviderConfig) -> bool:
126         api_key = cfg.get("api_key")
127         if not api_key:
128             return False
129
130         try:
131             import google.generativeai
132         except Exception:
133             return False
134
135         client = google.generativeai.Client(api_key=api_key)
136         delays = (0.0, 2.0, 6.0)
137         last_exc: Exception | None = None
138         for delay in delays:
139             if delay:
140                 import time
141                 time.sleep(delay)
142             try:
143                 list(client.models.list(config={"page_size": 1}))
144                 return True
145             except Exception as exc:
146                 last_exc = exc
147                 msg = str(exc).lower()
148                 if not any(s in msg for s in ("429", "quota", "rate", "exhausted")):
149                     return False
150                 _ = last_exc
151                 return False
152
153     def create_model(self, cfg: ProviderConfig):
154         from langchain_google_genai import ChatGoogleGenerativeAI
155
156         return ChatGoogleGenerativeAI(
157             model=cfg["model"],
158             temperature=cfg["temperature"],
159             google_api_key=self.require_api_key(cfg),
160         )
161
162 class OllamaProvider(BaseProvider):
163     name = "ollama"
164     default_base_url = "http://localhost:11434"
165     base_url_env_var = "OLLAMA_URL"
166     validation_timeout = 10.0
167
168     def base_url(self, cfg: ProviderConfig) -> str:
169         raw = cfg.get("base_url") or os.environ.get(self.base_url_env_var)
170         or self.default_base_url
171         return str(raw).rstrip("/")
172
173     def validate(self, cfg: ProviderConfig) -> bool:
174         base = self.base_url(cfg)
175         model = cfg.get("model")
176
177         try:
178             with urlopen(f"{base}/api/tags", timeout=2) as resp:
179                 if not (200 <= resp.status < 300):
180                     return False
181                 payload = json.loads(resp.read())
182             except Exception:
183                 return False
184
185             if model:
186                 available = {m.get("name", "") for m in payload.get("models", [])}
187                 if not any(
188                     m == model or m == f"{model}:latest" or m.startswith(f"{model}:")
189                     for m in available
190                 ):
191                     return False
192
193             try:
194                 req = Request(
195                     f"{base}/api/chat",
196                     data=json.dumps(
197                         {
198                             "model": model,
199                             "messages": [{"role": "user", "content": "."}],
200                             "stream": False,
201                             "keep_alive": 0,
202                             "options": {"num_predict": 1},
203                         }
204                     ).encode(),
205                     headers={"Content-Type": "application/json"},
206                     method="POST",
207                 )
208                 with urlopen(req, timeout=8) as resp:
209                     if not (200 <= resp.status < 300):
210                         return False
211             except Exception:
212                 return False
213
214             return True
215
216     def create_model(self, cfg: ProviderConfig):
217         from langchain_ollama import ChatOllama
218
219         return ChatOllama(
220             model=cfg["model"],
221             temperature=cfg["temperature"],
222             base_url=self.base_url(cfg),
223         )
224
225     def unreachable_help(self, cfg: ProviderConfig) -> tuple[str, str]:
226         return (
227             f"Could not reach ollama at {self.base_url(cfg)}.",
228             "Start it with 'ollama serve', then try again or choose "
229             "another provider.",
230         )
231
232 class BedrockProvider(BaseProvider):

```



```

234
235 name = "bedrock"
236 env_var = ["BEDROCK_API_KEY", "AWS_BEARER_TOKEN_BEDROCK"]
237 default_region = "us-east-1"
238
239 def _region(self, cfg: ProviderConfig) -> str:
240     return str(
241         cfg.get("region")
242         or os.environ.get("AWS_REGION")
243         or os.environ.get("AWS_DEFAULT_REGION")
244         or self.default_region
245     )
246
247 def validate(self, cfg: ProviderConfig) -> bool:
248     api_key = cfg.get("api_key")
249     if not api_key:
250         return False
251     os.environ["AWS_BEARER_TOKEN_BEDROCK"] = str(api_key)
252
253     try:
254         import boto3
255         import langchain_aws
256     except Exception:
257         return False
258
259     try:
260         client = boto3.client("bedrock", region_name=self._region(cfg))
261         client.list_foundation_models()
262         return True
263     except Exception:
264         return False
265
266 def create_model(self, cfg: ProviderConfig):
267     from langchain_aws import ChatBedrockConverse
268
269     api_key = self.require_api_key(cfg)
270     os.environ["AWS_BEARER_TOKEN_BEDROCK"] = api_key
271
272     kwargs: dict[str, object] = {
273         "model": cfg["model"],
274         "region_name": self._region(cfg),
275     }
276     if _bedrock_accepts_temperature(cfg["model"]):
277         kwargs["temperature"] = cfg["temperature"]
278     return ChatBedrockConverse(**kwargs)
279
280 def _bedrock_accepts_temperature(model: str) -> bool:
281     name = (model or "").lower()
282     if "claude-opus-4-" in name:
283         tail = name.split("claude-opus-4-", 1)[1]
284         digits = ""
285         for ch in tail:
286             if ch.isdigit():
287                 digits += ch
288             else:
289                 break
290         if digits and int(digits) >= 7:
291             return False
292     if any(s in name for s in ("claude-opus-5", "claude-opus-6", "claude-opus-7")):
293         return False
294     return True
295
296 PROVIDER_REGISTRY: dict[str, BaseProvider] = {
297     provider.name: provider
298     for provider in (
299         OllamaProvider(),
300         OpenAIProvider(),
301         ClaudeProvider(),
302         GeminiProvider(),
303         BedrockProvider(),
304     )
305 }
306
307 def get_provider(name: str) -> BaseProvider:
308     try:
309         return PROVIDER_REGISTRY[name]
310     except KeyError as exc:
311         raise ValueError(f"Unsupported model provider: {name}") from exc

```

## Anexa 15: src/sandbox.py

```

1 from __future__ import annotations
2 import os
3 import shutil
4 import subprocess
5 from dataclasses import dataclass
6 from pathlib import Path
7
8 PROJECT_MARKERS = (
9     ".git", "pyproject.toml", "package.json",
10     "Cargo.toml", "go.mod", "Gemfile",
11     "requirements.txt", "Makefile", ".vibe-cli",
12 )
13
14 _HARD_DENY_ABSOLUTE = {"/", "/tmp", "/var/tmp"}
15
16 _HOME_PERSONAL_SUBDIRS = {
17     "Desktop", "Documents", "Downloads",
18     "Movies", "Music", "Pictures", "Library",
19 }
20
21 _AMBIGUOUS_MAX_MB = 500
22 _AMBIGUOUS_MAX_FILES = 10_000
23
24 @dataclass
25 class PolicyResult:
26     allow: bool
27     reason: str
28     silent: bool = True
29
30 def _is_hard_denied(cwd: Path, home: Path) -> bool:
31     if str(cwd) in _HARD_DENY_ABSOLUTE:
32         return True
33     if cwd == home:
34         return True
35     parts = cwd.parts
36     if len(parts) == 3 and parts[0] == "/" and parts[1] in ("Users", "home"):
37         return True
38     if cwd.parent == home and cwd.name in _HOME_PERSONAL_SUBDIRS:
39         return True
40     return False
41
42 def _has_project_marker(cwd: Path) -> bool:
43     return any((cwd / m).exists() for m in PROJECT_MARKERS)
44
45 def _estimate_size(cwd: Path) -> tuple[int, int]:
46     mb = 0
47     file_count = 0
48     try:
49         r = subprocess.run(
50             ["du", "-sk", str(cwd)],
51             capture_output=True,
52             text=True,
53             timeout=2,
54         )
55     except Exception:
56         pass
57     try:
58         r = subprocess.run(
59             ["find", str(cwd), "-type", "f"],
60             capture_output=True,
61             text=True,
62             timeout=2,
63         )
64     except Exception:
65         pass
66     if r.returncode == 0 and r.stdout.strip():
67         kb = int(r.stdout.split()[0])
68         mb = kb // 1024
69
70     file_count = r.stdout.count("\n")
71     except Exception:
72         pass
73     return mb, file_count
74
75 def cwd_policy(
76     cwd: Path,
77     max_mb: int = _AMBIGUOUS_MAX_MB,
78     max_files: int = _AMBIGUOUS_MAX_FILES,
79 ) -> PolicyResult:
80     home = Path.home()
81
82     if os.environ.get("VIBE_SANDBOX_FORCE") == "1":
83         return PolicyResult(
84             allow=True,
85             reason="VIBE_SANDBOX_FORCE=1 - policy bypassed",
86             silent=False,
87         )
88
89     if _is_hard_denied(cwd, home):
90         return PolicyResult(
91             allow=False,
92             reason=(
93                 f"cwd {cwd} looks like a personal directory tree, not a "
94                 f"project. Falling back to host execution. "
95                 f"(Set VIBE_SANDBOX_FORCE=1 to override.)"
96             ),
97             silent=False,
98         )
99
100     if _has_project_marker(cwd):
101         return PolicyResult(
102             allow=True,
103             reason="project markers detected",
104             silent=True,
105         )
106
107     mb, files = _estimate_size(cwd)
108     if mb > max_mb or files > max_files:
109         return PolicyResult(
110             allow=False,
111             reason=(
112                 f"cwd {cwd} has no project markers and is large "
113                 f"({mb} MB, {files} files). Falling back to host execution "
114                 f"(Set VIBE_SANDBOX_FORCE=1 to override.)"
115             ),
116             silent=False,
117         )
118     return PolicyResult(
119         allow=True,
120         reason=(
121             f"Sandboxing {cwd} (no project markers detected, "
122             f"{mb} MB / {files} files)."
123         ),
124         silent=False,
125     )
126
127 def docker_available() -> tuple[bool, str]:
128     if shutil.which("docker") is None:
129         return False, "docker CLI not found in PATH"
130     try:
131         r = subprocess.run(
132             ["docker", "info"],
133             capture_output=True,
134             text=True,
135             timeout=2,
136         )

```

```

137     except subprocess.TimeoutExpired:
138         return False, "docker info timed out (daemon not responding)"
139     except Exception as e:
140         return False, f"docker info failed: {e}"
141     if r.returncode != 0:
142         first = (r.stderr or "").strip().splitlines()[1:]
143         return False, first[0] if first else "docker daemon unreachable"
144     return True, "ok"
145
146 @dataclass
147 class RunResult:
148     stdout: str
149     stderr: str
150     exit_code: int
151     timed_out: bool = False
152
153 class Sandbox:
154
155     def __init__(
156         self,
157         container_id: str,
158         workspace: Path,
159         image: str,
160         timeout_default: int,
161     ):
162         self.container_id = container_id
163         self.workspace = workspace
164         self.image = image
165         self.timeout_default = timeout_default
166         self._closed = False
167
168     @property
169     def short_id(self) -> str:
170         return self.container_id[:12]
171
172     @classmethod
173     def try_start(
174         cls, cfg: dict, workspace: Path
175     ) -> tuple["Sandbox | None", str]:
176         ok, why = docker_available()
177         if not ok:
178             return None, why
179
180         policy = cwd_policy(
181             workspace,
182             max_mb=int(cfg.get("ambiguous_max_mb") or _AMBIGUOUS_MAX_MB),
183             max_files=int(cfg.get("ambiguous_max_files") or
184                             _AMBIGUOUS_MAX_FILES),
185         )
186         if not policy.allow:
187             return None, policy.reason
188
189         image = cfg.get("image") or "python:3.13-slim"
190         network = cfg.get("network") or "bridge"
191         memory = str(cfg.get("memory") or "2g")
192         cpus = str(cfg.get("cpus") or 2)
193         timeout_default = int(cfg.get("timeout_default") or 30)
194
195         cmd = [
196             "docker", "run", "-d", "--rm", "-v",
197             f"{workspace.resolve()}/workspace:rw",
198             "-u", "/workspace", "--network",
199             str(network), "--memory", memory,
200             "--cpus", cpus, "--cap-drop", "ALL",
201             "--user", f"{os.getuid()}-{os.getgid()}", image,
202             "sleep", "infinity",
203         ]
204         try:
205             r = subprocess.run(cmd, capture_output=True, text=True,
206                               timeout=60)
207             except subprocess.TimeoutExpired:
208
209         return None, "docker run timed out (image pull may be in
210         progress)"
211     except Exception as e:
212         return None, f"docker run failed: {e}"
213
214     if r.returncode != 0:
215         err = (r.stderr or r.stdout or "").strip().splitlines()[1:]
216         return None, err[0] if err else "docker run failed (no output)"
217
218     cid = r.stdout.strip()
219     if not cid:
220         return None, "docker run returned no container id"
221
222     notice = policy.reason if not policy.silent else ""
223     return cls(cid, workspace, image, timeout_default), notice or "ok"
224
225 def run(self, command: str, timeout: int | None = None) -> RunResult:
226     if self._closed:
227         return RunResult(
228             stdout="",
229             stderr="sandbox is closed",
230             exit_code=-1,
231         )
232     t = int(timeout if timeout is not None else self.timeout_default)
233     try:
234         r = subprocess.run(
235             [
236                 "docker",
237                 "exec",
238                 "-w",
239                 "/workspace",
240                 self.container_id,
241                 "sh",
242                 "-c",
243                 command,
244             ],
245             capture_output=True,
246             text=True,
247             timeout=t,
248         )
249         return RunResult(
250             stdout=r.stdout or "",
251             stderr=r.stderr or "",
252             exit_code=r.returncode,
253         )
254     except subprocess.TimeoutExpired as e:
255         return RunResult(
256             stdout=e.stdout.decode(errors="replace") if e.stdout else
257             "",
258             stderr=e.stderr.decode(errors="replace") if e.stderr else
259             "",
260             exit_code=-1,
261             timed_out=True,
262         )
263     except Exception as e:
264         return RunResult(stdout="", stderr=f"sandbox exec error: {e}",
265                           exit_code=-1)
266
267 def close(self) -> None:
268     if self._closed:
269         return
270     self._closed = True
271     try:
272         subprocess.run(
273             ["docker", "kill", self.container_id],
274             capture_output=True,
275             timeout=5,
276         )
277     except Exception:
278         pass

```

## Anexa 16: src/subagents/base.py

```

1 from __future__ import annotations
2 import asyncio
3 import logging
4 import time
5 from abc import ABC, abstractmethod
6 from copy import deepcopy
7 from dataclasses import dataclass, field
8 from typing import Any
9 from langchain_core.messages import HumanMessage, SystemMessage,
10     ToolMessage
11
12 def _extract_text(response) -> str:
13     content = getattr(response, "content", None)
14     if isinstance(content, str):
15         return content
16     parts: list[str] = []
17     for block in content or []:
18         if isinstance(block, dict) and block.get("type") == "text":
19             parts.append(block.get("text", ""))
20         elif isinstance(block, str):
21             parts.append(block)
22     return "\n".join(p for p in parts if p)
23
24 @dataclass
25 class SubagentResult:
26     text: str
27     timed_out: bool = False
28     iterations: int = 0
29     tool_call_count: int = 0
30     error: str | None = None
31     duration_s: float = 0.0
32     usage: dict = field(default_factory=dict)
33
34 SUBAGENT_CONTRACT = """\
35 You are a subagent spawned by an orchestrator (the parent agent).
36 You MUST follow these rules; they take precedence over everything below.
37
38 1. SCOPE
39 Do ONLY what the orchestrator asked. Do not extend the task, do not
40 "be helpful" by tackling adjacent issues. If the request is ambiguous,
41 make the most conservative interpretation and note your assumption in
42 the final report.
43
44 2. NO UNAUTHORISED STATE CHANGES
45 Do NOT write files, edit files, run mutating shell commands, push
46 commits, install packages, send network requests with side effects,
47 or take any other action that modifies state UNLESS the orchestrator's
48 prompt explicitly told you to. When in doubt, read and report - let
49 the orchestrator be the one to act on your findings.
50
51 3. ALWAYS END WITH A FINAL REPORT
52 Your last message MUST be a text reply (no tool call) addressed to the
53 orchestrator. Lead with the answer or finding. Cite concrete file paths
54 ,
55 line ranges, commands, or sources where relevant. Keep it concise but
56 complete - the orchestrator only sees this text, not your intermediate
57 steps. If you couldn't finish, say so explicitly and report what you
58 did find. Never end silently.
59
60 4. NO QUESTIONS
61 You cannot ask the orchestrator clarifying questions - there is no
62 reply channel. Use your best judgement and state assumptions in the
63 report.
64
65 Type-specific guidance follows below.
66 ---
67 """
68
69 class Subagent(ABC):
70     name: str = "subagent"
71     description: str = ""
72     default_timeout_s: int = 120
73     max_iterations: int = 20
74     allowed_tools: list[str] | None = None

```



```

75     excluded_tools: tuple[str, ...] = ("spawn_subagent",)
76
77     def __init__(self, config: dict | None = None) -> None:
78         self.config = config or {}
79
80     @abstractmethod
81     def role_prompt(self, prompt: str, parent_ctx: dict) -> str:
82         ...
83
84     def system_prompt(self, prompt: str, parent_ctx: dict) -> str:
85         return SUBAGENT_CONTRACT + self.role_prompt(prompt, parent_ctx)
86
87     def tool_names(self, parent_ctx: dict) -> list[str] | None:
88         return self.allowed_tools
89
90     async def run(
91         self,
92         prompt: str,
93         parent_ctx: dict,
94         *,
95         provider: str,
96         model: str,
97         timeout_s: int,
98         parent_tool_call_id: str | None = None,
99     ) -> SubagentResult:
100         from src.logging_setup import emit_event, get_logger
101
102         log = get_logger("subagent")
103         t0 = time.perf_counter()
104         emit_event(
105             log,
106             "subagent.start",
107             subagent_type=self.name,
108             provider=provider,
109             model=model,
110             timeout_s=timeout_s,
111             depth=parent_ctx.get("subagent_depth", 0) + 1,
112         )
113         try:
114             result = await asyncio.wait_for(
115                 self._drive_loop(
116                     prompt, parent_ctx, provider, model,
117                     parent_tool_call_id,
118                     timeout=timeout_s,
119                 ),
120                 result.duration_s = time.perf_counter() - t0
121             )
122             emit_event(
123                 log,
124                 "subagent.end",
125                 subagent_type=self.name,
126                 provider=provider,
127                 model=model,
128                 ok=result.error is None,
129                 iterations=result.iterations,
130                 tool_calls=result.tool_call_count,
131                 duration_ms=result.duration_s * 1000.0,
132             )
133             return result
134         except asyncio.TimeoutError:
135             dur = time.perf_counter() - t0
136             emit_event(
137                 log,
138                 "subagent.end",
139                 level=logging.WARNING,
140                 subagent_type=self.name,
141                 provider=provider,
142                 model=model,
143                 ok=False,
144                 timed_out=True,
145                 duration_ms=dur * 1000.0,
146             )
147             return SubagentResult(
148                 text=f"[subagent '{self.name}' timed out after {timeout_s}s]",
149                 timed_out=True,
150                 duration_s=dur,
151             )
152         except Exception as exc:
153             dur = time.perf_counter() - t0
154             emit_event(
155                 log,
156                 "subagent.end",
157                 level=logging.ERROR,
158                 subagent_type=self.name,
159                 provider=provider,
160                 model=model,
161                 ok=False,
162                 error_type=type(exc).__name__,
163                 duration_ms=dur * 1000.0,
164             )
165             return SubagentResult(
166                 text=f"[subagent '{self.name}' error: {type(exc).__name__}: {exc}]",
167                 error=str(exc),
168                 duration_s=dur,
169             )
170
171     async def _drive_loop(
172         self,
173         prompt: str,
174         parent_ctx: dict,
175         provider: str,
176         model: str,
177         parent_tool_call_id: str | None = None,
178     ) -> SubagentResult:
179         """Run the child tool-calling loop until the model stops calling
180         tools."""
181         from src.main import get_active_tools, provider_configs
182         from src.providers import get_provider
183
184         prov_cfg = provider_configs.get(provider) or {}
185         cfg_copy = deepcopy(prov_cfg)
186         if model:
187             cfg_copy["model"] = model
188         llm = get_provider(provider).create_model(cfg_copy)
189
190         parent_tools = get_active_tools()
191         allowed = self.tool_names(parent_ctx)
192         excluded = set(self.excluded_tools or ())
193
194         if allowed is None:
195             tools = [t for t in parent_tools if t.name not in excluded]
196         else:
197             allowed_set = set(allowed)
198             tools = [
199                 t for t in parent_tools if t.name in allowed_set and t.name not in excluded
200             ]
201
202         llm_with_tools = llm.bind_tools(tools) if tools else llm
203
204         parent_sess = parent_ctx.get("session")
205         session_id_for_tracing = getattr(parent_sess, "id", None)
206
207         child_ctx: dict[str, Any] = dict(parent_ctx)
208         child_ctx["status_label"] = "thinking"
209         child_ctx["pending_tool_logs"] = {}
210         child_ctx["pending_diffs"] = {}
211         depth = parent_ctx.get("subagent_depth", 0) + 1
212         child_ctx["subagent_depth"] = depth
213         child_ctx["parent_tool_call_id"] = parent_tool_call_id
214         child_ctx.pop("messages", None)
215         child_ctx.pop("session", None)
216         child_ctx["provider"] = provider
217
218         if_handler = parent_ctx.get("langfuse_handler")
219         callbacks = [if_handler] if if_handler is not None else []
220         if metadata: dict = {
221             "langfuse_tags": [
222                 f"subagent:{self.name}",
223                 f"subagent_depth:{depth}",
224                 f"provider:{provider}",
225                 f"model:{model}",
226             ],
227         }
228         if session_id_for_tracing:
229             if metadata["langfuse_session_id"] = session_id_for_tracing
230         if parent_ctx.get("user_id"):
231             if metadata["langfuse_user_id"] = parent_ctx["user_id"]
232         child_run_config: dict[str, Any] = {
233             "callbacks": callbacks,
234             "metadata": if_metadata,
235             "run_name": f"subagent:{self.name}",
236         }
237         child_ctx["_run_config"] = child_run_config
238
239         messages: list = [
240             SystemMessage(content=self.system_prompt(prompt, parent_ctx)),
241             HumanMessage(content=prompt),
242         ]
243
244         tool_call_count = 0
245         iterations = 0
246         final_text = ""
247         usage_totals = {
248             "input": 0,
249             "output": 0,
250             "cache_read": 0,
251             "cache_write": 0,
252         }
253
254         while iterations < self.max_iterations:
255             iterations += 1
256             response = await llm_with_tools.ainvoke(messages, config=child_run_config)
257             messages.append(response)
258
259             um = getattr(response, "usage_metadata", None) or {}
260             itd = um.get("input_token_details") or {}
261             usage_totals["input"] += int(um.get("input_tokens") or 0)
262             usage_totals["output"] += int(um.get("output_tokens") or 0)
263             usage_totals["cache_read"] += int(itd.get("cache_read") or 0)
264             usage_totals["cache_write"] += int(itd.get("cache_creation") or 0)
265
266             tool_calls = getattr(response, "tool_calls", None) or []
267             if not tool_calls:
268                 # No tool calls - extract text and return.
269                 final_text = _extract_text(response)
270                 break
271
272             tool_call_count += len(tool_calls)
273
274             tool_results = await self._invoke_tools(
275                 tool_calls, tools, child_ctx
276             )
277             messages.extend(tool_results)
278             child_cfg = (
279                 parent_ctx.get("subagents_config") or {}
280             ).get("child_context") or {}
281             mtt = int(child_cfg.get("max_tool_tokens") or 0)
282             if mtt > 0:
283                 ht = child_cfg.get("head_tail") or [800, 400]
284                 from src.subagents.compaction import truncate_oversized_tool_results
285                 truncate_oversized_tool_results(
286                     messages,
287                     max_tool_tokens=mtt,
288                     head_tokens=int(ht[0]),
289                     tail_tokens=int(ht[1]),
290                 )
291             else:
292                 return SubagentResult(
293                     text=f"[subagent '{self.name}' stopped after {self.max_iterations} "
294                         "iterations without producing a final answer]",
295                     iterations=iterations,
296                     tool_call_count=tool_call_count,
297                     error="max_iterations",
298                     usage=usage_totals,
299                 )
300             if not final_text.strip() and iterations < self.max_iterations:
301                 iterations += 1
302                 messages.append(
303                     HumanMessage(
304                         content=(

```

```

305         "You ended without writing a final report. As per
306         "the system instructions, your last message MUST
307     be "    "a text reply to the orchestrator. Write that
308     report "    "now - even a one-sentence summary of what you did
309         "    "or couldn't do. Do not call any more tools."
310     )
311 )
312 )
313 try:
314     response = await llm_with_tools.ainvoke(
315         messages, config=child_run_config
316     )
317     messages.append(response)
318     um = getattr(response, "usage_metadata", None) or {}
319     itd = um.get("input_token_details") or {}
320     usage_totals["input"] += int(um.get("input_tokens") or 0)
321     usage_totals["output"] += int(um.get("output_tokens") or 0)
322 )
323     usage_totals["cache_read"] += int(itd.get("cache_read") or 0)
324     usage_totals["cache_write"] += int(itd.get("cache_creation
325 ") or 0)
326     final_text = _extract_text(response)
327 except Exception:
328     pass
329 if not final_text.strip():
330     final_text = (
331         f"[subagent '{self.name}' ended without a final report -
332         "ran the requested work but did not summarise its findings
333         "
334         "Treat any side-effects as unverified.]"
335     )
336 return SubagentResult(
337     text=final_text,
338     iterations=iterations,
339     tool_call_count=tool_call_count,
340     usage=usage_totals,
341 )
342
343 async def _invoke_tools(
344     self, tool_calls: list, tools: list, child_ctx: dict
345 ) -> list[ToolMessage]:
346     from src.logging_setup import emit_event, get_logger, redact
347     from src.permissions import request_permission
348     from src.tools import reset_tool_ctx, set_tool_ctx
349
350     tool_log = get_logger("tool_call")
351     by_name = {t.name: t for t in tools}
352     parent_tool_call_id = child_ctx.get("parent_tool_call_id")
353     out: list[ToolMessage] = []
354
355     def _tool_source(name: str) -> str:
356         return "mcp:" + name.split("__", 1)[0] if "__" in name else
357         "builtin"
358
359     for tc in tool_calls:
360         name = tc["name"]
361         args = tc.get("args", {}) or {}
362         call_id = tc["id"]
363
364         tool = by_name.get(name)
365         if tool is None:
366             emit_event(
367                 tool_log,
368                 "tool_unavailable",
369                 level=logging.WARNING,
370                 tool=name,
371                 tool_call_id=call_id,
372                 source=_tool_source(name),
373                 subagent_type=self.name,
374                 parent_tool_call_id=parent_tool_call_id,
375             )
376             out.append(
377                 ToolMessage(
378                     content=(
379                         f"Tool '{name}' is not available to subagent
380                         '{self.name}'."
381                     ),
382                     tool_call_id=call_id,
383                     name=name,
384                 )
385             )
386             continue
387
388         emit_event(
389             tool_log,
390             "tool_start",
391             tool=name,
392             tool_call_id=call_id,
393             source=_tool_source(name),
394             args=redact(args),
395             subagent_type=self.name,
396             parent_tool_call_id=parent_tool_call_id,
397         )
398
399         decision = await request_permission(name, args, child_ctx)
400         if not decision.allow:
401             emit_event(
402                 tool_log,
403                 "tool_end",
404                 tool=name,
405                 tool_call_id=call_id,
406                 source=_tool_source(name),
407                 ok=False,
408                 denied=True,
409                 reason_len=len(decision.reason or ""),
410                 subagent_type=self.name,
411                 parent_tool_call_id=parent_tool_call_id,
412             )
413             out.append(
414                 ToolMessage(
415                     content=(
416                         f"User denied this tool call. "
417                         f"Reason: {decision.reason}"
418                     ),
419                     tool_call_id=call_id,
420                     name=name,
421                 )
422             )
423             continue
424
425         t0 = time.perf_counter()
426         token = set_tool_ctx(child_ctx)
427         tool_error: Exception | None = None
428         child_run_config = child_ctx.get("_run_config")
429         tool_call_envelope = {
430             "name": name,
431             "args": args,
432             "id": call_id,
433             "type": "tool_call",
434         }
435         try:
436             if child_run_config is not None:
437                 result = await tool.ainvoke(
438                     tool_call_envelope, config=child_run_config
439                 )
440             else:
441                 result = await tool.ainvoke(tool_call_envelope)
442         except Exception as exc:
443             tool_error = exc
444             result = f"Tool error: {type(exc).__name__}: {exc}"
445         finally:
446             reset_tool_ctx(token)
447
448         if isinstance(result, ToolMessage):
449             result = result.content
450         if not isinstance(result, str):
451             result = str(result)
452
453         emit_event(
454             tool_log,
455             "tool_end",
456             tool=name,
457             tool_call_id=call_id,
458             source=_tool_source(name),
459             ok=tool_error is None,
460             error_type=type(tool_error).__name__ if tool_error else
461             None,
462             result_bytes=len(result),
463             duration_ms=(time.perf_counter() - t0) * 1000.0,
464             subagent_type=self.name,
465             parent_tool_call_id=parent_tool_call_id,
466         )
467         if tool_error is None:
468             run_post_hooks(name, args, result, child_ctx)
469             out.append(ToolMessage(content=result, tool_call_id=call_id,
470 name=name))
471             return out
472
473     async def run_subagent(
474         subagent_type: str,
475         prompt: str,
476         parent_ctx: dict,
477         *,
478         timeout_s: int | None = None,
479         parent_tool_call_id: str | None = None,
480     ) -> SubagentResult:
481         from src.subagents import get as _get_subagent
482
483         cls = _get_subagent(subagent_type)
484         if cls is None:
485             from src.subagents import list_types
486
487             avail = ", ".join(list_types()) or "(none)"
488             return SubagentResult(
489                 text=(
490                     f"Unknown subagent_type '{subagent_type}'. "
491                     f"Available: {avail}."
492                 ),
493                 error="unknown_type",
494             )
495
496         type_cfg = (parent_ctx.get("subagents_config") or {}).get("types", {})
497         .get(
498             subagent_type, {}
499         ) or {}
500
501         instance = cls(type_cfg)
502
503         provider, model = _resolve_subagent_model(subagent_type, parent_ctx,
504 type_cfg)
505
506         max_depth = int((parent_ctx.get("subagents_config") or {}).get("
507 max_depth", 2))
508         cur_depth = int(parent_ctx.get("subagent_depth", 0))
509         if cur_depth >= max_depth:
510             return SubagentResult(
511                 text=(
512                     f"[subagent depth limit reached ({cur_depth} >= {max_depth
513                     )]; "
514                     "refusing to spawn deeper subagents]"
515                 ),
516                 error="depth_limit",
517             )

```

```

528         )
529
530     cfg = parent_ctx.get("subagents_config") or {}
531     default_to = int(
532         type_cfg.get("default_timeout_s")
533         or cfg.get("default_timeout_s")
534         or instance.default_timeout_s
535     )
536     max_to = int(cfg.get("max_timeout_s", 600))
537     if timeout_s is None or timeout_s <= 0:
538         effective_to = default_to
539     else:
540         effective_to = min(int(timeout_s), max_to)
541
542     return await instance.run(
543         prompt,
544         parent_ctx,
545         provider=provider,
546         model=model,
547         timeout_s=effective_to,
548         parent_tool_call_id=parent_tool_call_id,
549     )
550
551 def _resolve_subagent_model(
552     subagent_type: str, parent_ctx: dict, type_cfg: dict
553
554 ) -> tuple[str, str]:
555     from src.main import provider_configs
556     from ui import preferences
557     try:
558         override = preferences.get_subagent_model(subagent_type)
559     except AttributeError:
560         override = None
561     if override and override.get("provider") and override.get("model"):
562         prov = override["provider"]
563         prov_cfg = provider_configs.get(prov)
564         if isinstance(prov_cfg, dict) and override["model"] in (
565             prov_cfg.get("models") or []
566         ):
567             return prov, override["model"]
568
569     prov = parent_ctx.get("provider")
570     if prov:
571         model = provider_configs.get(prov, {}).get("model")
572         if model:
573             return prov, model
574     for name, c in provider_configs.items():
575         models = (c or {}).get("models") or []
576         if models:
577             return name, models[0]
578     raise RuntimeError("No providers configured for subagent.")

```

## Anexa 17: src/subagents/types/web\_research.py

```

1 from __future__ import annotations
2 from src.subagents import register
3 from src.subagents.base import Subagent
4
5 @register
6 class WebResearchSubagent(Subagent):
7     name = "web_research"
8     description = (
9         "Web research subagent. Searches the web, reads results, and
10         returns "
11         "a synthesised brief with citations. No file system or shell
12         access."
13     )
14     default_timeout_s = 120
15     max_iterations = 18
16     allowed_tools = ["web_search"]
17
18     def role_prompt(self, prompt: str, parent_ctx: dict) -> str:
19         return (
20             "ROLE: web-research subagent.\n\n"
21             "Your only tool is 'web_search'. You cannot read or write the
22             "
23             "filesystem, run shell commands, or take any local action.\n\n"
24             "Approach:\n"
25             " - Issue 1-3 focused queries; do not fan out indefinitely.\n"
26             " - Cross-check claims across results when the topic is "
27             "factual or time-sensitive.\n"
28             " - Be explicit about uncertainty - if the web doesn't agree,
29             say so.\n\n"
30             "Final report format (Markdown):\n"
31             " - 1-3 paragraph synthesis (or bullets) answering the "
32             "orchestrator's request.\n"
33             " - A 'Sources' section listing the URLs you actually relied
34             on, "
35             "each with a one-line description."
36         )

```

## Anexa 18: src/tool\_guard.py

```

1 from __future__ import annotations
2 import hashlib
3 from collections.abc import Callable
4 from pathlib import Path
5 from typing import Any
6
7 Precondition = Callable[[dict, dict], str | None]
8 """Signature: (args, ctx) -> error message or None.
9
10 Return None to allow the tool to run. Return a string to short-circuit the
11 call; the string becomes the tool's 'ToolMessage.content'."""
12
13 PostHook = Callable[[dict, str, dict], None]
14 """Signature: (args, result, ctx) -> None.
15
16 Runs after a successful tool invocation to update tracked state (e.g.
17 remember a file's SHA after read_file)."""
18
19 PRECONDITIONS: dict[str, list[tuple[str, Precondition]]] = {}
20 POST_HOOKS: dict[str, list[tuple[str, PostHook]]] = {}
21
22 def precondition(tool_name: str, *, guard_name: str):
23     """Register 'fn' as a precondition for 'tool_name'.
24
25     'guard_name' identifies the rule in logs / web UI tooltips so users
26     can
27     tell why a tool was rejected without parsing the error string.
28     """
29     def deco(fn: Precondition) -> Precondition:
30         PRECONDITIONS.setdefault(tool_name, []).append((guard_name, fn))
31         return fn
32     return deco
33
34 def post_hook(tool_name: str, *, name: str):
35     """Register 'fn' to run after a successful invocation of 'tool_name'.
36
37     Used to update state that other guards read (e.g. recording a file's
38     SHA after read_file).
39     """
40     def deco(fn: PostHook) -> PostHook:
41         POST_HOOKS.setdefault(tool_name, []).append((name, fn))
42         return fn
43     return deco
44
45 def check_preconditions(
46     tool_name: str, args: dict, ctx: dict
47 ) -> tuple[str, str] | None:
48     """Run all preconditions for 'tool_name' in registration order.
49
50     Returns '(guard_name, error_message)' for the first guard that denies,
51     or None if all allow. Guards are skipped entirely when
52     'ctx.tool_guard.enabled' is False (master kill switch).
53     """
54     cfg = _config(ctx)
55     if not cfg.get("enabled", True):
56         return None
57     for guard_name, fn in PRECONDITIONS.get(tool_name, []):
58         try:
59             err = fn(args, ctx)
60             except Exception as exc: # pragma: no cover - guards should not
61                 throw
62             err = f"[tool-guard '{guard_name}'] raised {type(exc).__name__}: {exc}"
63             if err:
64                 return (guard_name, err)
65     return None
66
67 def run_post_hooks(tool_name: str, args: dict, result: str, ctx: dict) -> None:
68     """Run all post-hooks for 'tool_name'. Best-effort - exceptions
69     swallowed."""
70     cfg = _config(ctx)
71     if not cfg.get("enabled", True):
72         return
73     for _name, fn in POST_HOOKS.get(tool_name, []):
74         try:
75             fn(args, result, ctx)
76         except Exception:
77             pass
78
79 def _config(ctx: dict) -> dict:
80     """Return the tool_guard config block (always a dict; empty by
81     default)."""
82     cfg = ctx.get("tool_guard_config")
83     return cfg if isinstance(cfg, dict) else {}
84
85 def _guard_config(ctx: dict, guard_name: str) -> dict:
86     """Per-guard config block under 'tool_guard_config'."""
87     raw = _config(ctx).get(guard_name)
88     return raw if isinstance(raw, dict) else {}
89
90 def _resolved_path(raw: Any) -> Path | None:
91     if not isinstance(raw, str) or not raw:
92         return None
93     try:
94         return Path(raw).expanduser().resolve()
95     except Exception:
96         return None
97
98 def _sha(text: str) -> str:

```

```

99     return hashlib.sha256(text.encode("utf-8", errors="replace")).
100         hexdigest()
101 def _file_reads(ctx: dict) -> dict[str, str]:
102     """Get-or-create the per-session read tracker."""
103     fr = ctx.get("file_reads")
104     if not isinstance(fr, dict):
105         fr = {}
106     ctx["file_reads"] = fr
107     return fr
108
109 @post_hook("read_file", name="track_read")
110 def _track_read(args: dict, result: str, ctx: dict) -> None:
111     """Remember the SHA of files we've read so write/edit can verify
112     freshness.
113     """
114     p = _resolved_path(args.get("path"))
115     if p is None or not p.exists() or not p.is_file():
116         return
117     try:
118         content = p.read_text(errors="replace")
119     except Exception:
120         return
121     _file_reads(ctx)[str(p)] = _sha(content)
122
123 @post_hook("write_file", name="track_write")
124 @post_hook("edit_file", name="track_edit")
125 def _track_mutation(args: dict, result: str, ctx: dict) -> None:
126     """After a successful write/edit, refresh the tracked SHA so
127     subsequent
128     writes within the same session don't trip the freshness check on the
129     very content we just produced."""
130     if not isinstance(result, str):
131         return
132     if not (result.startswith("Written ") or result.startswith("Edited ")):
133         return
134     p = _resolved_path(args.get("path"))
135     if p is None or not p.exists() or not p.is_file():
136         return
137     try:
138         content = p.read_text(errors="replace")
139     except Exception:
140         return
141     _file_reads(ctx)[str(p)] = _sha(content)
142
143 def _require_read_before_mutate(
144     args: dict, ctx: dict, *, action: str
145 ) -> str | None:
146     guard_cfg = _guard_config(ctx, "read_before_write")
147     if guard_cfg.get("enabled") is False:
148         return None
149     p = _resolved_path(args.get("path"))
150     if p is None or not p.exists() or not p.is_file():
151         return None # creating a new file - no read needed.
152
153     tracked_sha = _file_reads(ctx).get(str(p))
154     if tracked_sha is None:
155         return (
156             f"Refusing to {action} '{args.get('path')}': you haven't read
157             this "
158             f"file in the current session. Call read_file('{{path': "
159             f"'{args.get('path')}}'}) first so you know what you're
160             overwriting, "
161             f"then retry the {action}."
162         )
163     if guard_cfg.get("check_freshness", True):
164         try:
165             current = p.read_text(errors="replace")
166         except Exception:
167             return None
168         if _sha(current) != tracked_sha:
169             return (
170                 f"Refusing to {action} '{args.get('path')}': the file has
171                 "
172                 f"changed on disk since you last read it (external edit or
173                 a "
174                 f"prior write you didn't track). Call read_file('{{path': "
175                 f"'{args.get('path')}}'}) again to see the current content
176                 , "
177                 f"then retry."
178             )
179     return None
180
181 @precondition("write_file", guard_name="read_before_write")
182 def _write_requires_read(args: dict, ctx: dict) -> str | None:
183     return _require_read_before_mutate(args, ctx, action="write")
184
185 @precondition("edit_file", guard_name="read_before_write")
186 def _edit_requires_read(args: dict, ctx: dict) -> str | None:
187     return _require_read_before_mutate(args, ctx, action="edit")
188
189 __all__ = [
190     "PRECONDITIONS", "POST_HOOKS", "precondition",
191     "post_hook", "check_preconditions", "run_post_hooks",
192 ]

```

## Anexa 19: src/tools.py

```

1 from __future__ import annotations
2 import contextvars
3 import subprocess
4 from pathlib import Path
5 from langchain_core.tools import InjectedToolCallId, tool
6 from typing_extensions import Annotated
7
8 _tool_ctx: contextvars.ContextVar["dict | None"] = contextvars.ContextVar(
9     "vibe_tool_ctx", default=None)
10
11 def set_tool_ctx(ctx: dict | None):
12     return _tool_ctx.set(ctx)
13
14 def reset_tool_ctx(token) -> None:
15     _tool_ctx.reset(token)
16
17 def _ctx() -> dict:
18     return _tool_ctx.get() or {}
19
20 @tool
21 def run_skill(skill_name: str) -> str:
22     """Load the full instructions for a previously-listed skill.
23
24     The system prompt lists available skills with name + description +
25     when-to-use. Call this tool with a skill's 'name' to retrieve its
26     full instructions, then follow them for the rest of the turn.
27
28     Args:
29         skill_name: Name of an enabled skill (e.g. "pr-review").
30     """
31     import logging as _logging
32
33     from src.logging_setup import emit_event as _emit, get_logger as
34     _get_logger
35
36     _log = _get_logger("skill")
37
38     index = _ctx().get("skills_index")
39     if index is None:
40         _emit(
41             _log,
42             "skill.run",
43             level=_logging.DEBUG,
44             skill=skill_name,
45             ok=False,
46             reason="no_index",
47         )
48         return "No skills are loaded in this session."
49
50     name = (skill_name or "").strip()
51     if not name:
52         _emit(
53             _log,
54             "skill.run",
55             level=_logging.DEBUG,
56             skill=name,
57             ok=False,
58             reason="missing_name",
59         )
60
61     skill = index.get(name) if hasattr(index, "get") else None
62     if skill is None:
63         _emit(
64             _log,
65             "skill.run",
66             level=_logging.DEBUG,
67             skill=name,
68             ok=False,
69             reason="not_found",
70         )
71
72     available = ", ".join(s.name for s in get_attr(index, "active", []))
73
74     return (
75         f"Skill '{name}' not found. "
76         f"Available enabled skills: {available or '(none)}'."
77     )
78
79 if not skill.enabled:
80     _emit(
81         _log,
82         "skill.run",
83         level=_logging.DEBUG,
84         skill=name,
85         ok=False,
86         reason="disabled",
87     )
88     return f"Skill '{name}' is disabled. Enable it via /skills enable
89     {name}."
90
91 if not index.is_visible_to_model(name):
92     _emit(
93         _log,
94         "skill.run",
95         level=_logging.DEBUG,
96         skill=name,
97         ok=False,
98         reason="dropped",
99     )
100
101     return (
102         f"Skill '{name}' is enabled but over the active cap "
103         f"({index.max_active}); it has been dropped from this session.
104         "
105         f"Ask the user to raise 'skills.max_active' or disable another
106         skill."
107     )
108
109 if not skill.body:
110     _emit(
111         _log,
112         "skill.run",
113         level=_logging.DEBUG,
114         skill=name,
115         ok=False,
116         reason="empty_body",
117     )
118
119     return f"Skill '{name}' has no instructions in its body."

```

```

114     _emit(
115         _log,
116         "skill.run",
117         level=_logging.DEBUG,
118         skill=name,
119         ok=True,
120         body_bytes=len(skill.body),
121     )
122     return (
123         f"=== Skill: {name} ===\n"
124         "Apply the following instructions for the REST OF THIS TURN, then
125         "write your reply to the user's most recent message. Do not call
126         "another tool unless the instructions below tell you to.\n\n"
127         f"{skill.body}\n\n"
128         "=== End of skill ===\n"
129         "Now produce your user-facing reply, applying the instructions
130         above."
131     )
132
133 @tool
134 def remember(scope: str, content: str) -> str:
135     """Save a short, durable learning to VIBE.md so it persists across
136     sessions.
137
138     Use this when you learn something the next conversation should know:
139
140     - scope="global" -> a user-wide preference (tone, naming style, "
141       always do
142       X", tools they prefer). Saved to ~/.vibe-cli/VIBE.md.
143     - scope="project" -> a non-obvious fact about THIS repo (an
144       architectural
145       decision, owner, gotcha, convention). Saved to ~/.vibe/VIBE.md.
146
147     Do NOT use this for ephemeral task state, recent diffs, anything
148     derivable
149     from git/code, or secrets. Keep entries to one short paragraph or
150     bullet.
151
152     Args:
153         scope: Either "global" or "project".
154         content: One short paragraph or bullet. Be specific.
155     """
156     import logging as _logging
157
158     from src.logging_setup import emit_event as _emit, get_logger as
159     _get_logger
160
161     from ui.vibe_md import (
162         DEFAULT_MAX_BYTES,
163         VibeWriteError,
164         append_global,
165         append_project,
166     )
167
168     _log = _get_logger("vibe_md")
169
170     s = (scope or "").strip().lower()
171     if s not in ("global", "project"):
172         return (
173             "Invalid scope. Use 'global' for user-wide preferences or "
174             "'project' for facts about THIS repo."
175         )
176
177     body = (content or "").strip()
178     if not body:
179         return "Refusing to save empty content."
180
181     ctx = _ctx()
182     cap = int(
183         (ctx.get("vibe_md_config") or {}).get("max_bytes_per_file") or
184         DEFAULT_MAX_BYTES
185     )
186
187     try:
188         if s == "global":
189             written = append_global(body, max_bytes=cap)
190             target = "~/.vibe-cli/VIBE.md"
191         else:
192             written = append_project(body, max_bytes=cap)
193             target = "~/.vibe/VIBE.md"
194     except VibeWriteError as e:
195         _emit(
196             _log,
197             "vibe_md.remember",
198             level=_logging.WARNING,
199             scope=s,
200             ok=False,
201             reason=str(e),
202         )
203         return (
204             f"Could not save to {target}: {e}. "
205             "Consolidate existing entries by hand, then try again."
206         )
207
208     _emit(_log, "vibe_md.remember", scope=s, ok=True, bytes=written)
209     return f"Saved to {target} (+{written} bytes)."
```

```

224         lines.append(f"    {r['href']}")
225         lines.append(f"    {r['body']}")
226         lines.append("")
227         return "\n".join(lines)
228
229     except ImportError:
230         return "Web search unavailable: install 'duckduckgo-search'
231         package."
232     except Exception as e:
233         return f"Search error: {e}"
234
235 @tool
236 def read_file(path: str, offset: int = 1, limit: int = 500) -> str:
237     """Read a slice of a file by line range.
238
239     Returns the requested lines prefixed with a header showing the slice
240     and
241     the file's total line count, e.g. '=== path (lines 1-500 of 1843)
242     ==='.
243
244     Use 'offset' + 'limit' to page through large files; widen 'limit' only
245     when you actually need more context - pulling thousands of lines
246     wastes
247     the context window.
248
249     Args:
250         path: Path to the file to read.
251         offset: 1-indexed first line to return (default 1).
252         limit: Maximum number of lines to return (default 500, max 2000).
253     """
254     try:
255         p = Path(path).expanduser().resolve()
256         if not p.exists():
257             return f"File not found: {path}"
258         if not p.is_file():
259             return f"Not a file: {path}"
260
261         offset = max(1, int(offset))
262         limit = max(1, min(int(limit), 2000))
263
264         with p.open("r", errors="replace") as f:
265             lines = f.readlines()
266             total = len(lines)
267
268             if offset > total:
269                 return f"=== {path} (empty slice: offset {offset} > {total}
270                 lines) ==="
271
272             end = min(offset + limit - 1, total)
273             body = "".join(lines[offset - 1 : end])
274             if body and not body.endswith("\n"):
275                 body += "\n"
276             header = f"=== {path} (lines {offset}-{end} of {total}) ===\n"
277             return header + body
278     except PermissionError:
279         return f"Permission denied: {path}"
280     except Exception as e:
281         return f"Error reading file: {e}"
282
283 @tool
284 def write_file(path: str, content: str) -> str:
285     """Write content to a file, replacing it entirely. Creates parent
286     directories
287     if needed.
288
289     Use for **new files** or **complete rewrites**. For surgical changes
290     to an
291     existing file, prefer 'edit_file' - rewriting a whole file just to
292     change a
293     few lines wastes tokens and risks dropping unrelated content.
294
295     Args:
296         path: Path to the file to write.
297         content: The full content to write.
298     """
299     try:
300         p = Path(path).expanduser().resolve()
301         p.parent.mkdir(parents=True, exist_ok=True)
302         p.write_text(content)
303         return f"Written {len(content)} characters to {path}"
304     except PermissionError:
305         return f"Permission denied: {path}"
306     except Exception as e:
307         return f"Error writing file: {e}"
308
309 @tool
310 def edit_file(path: str, old_string: str, new_string: str) -> str:
311     """Replace an exact string in a file with a new string.
312
313     'old_string' must match **exactly once** in the file (including
314     whitespace
315     and indentation). If it matches zero or multiple times the edit is
316     rejected - widen 'old_string' with surrounding context to make it
317     unique,
318     then retry. Pass an empty 'new_string' to delete the matched text.
319
320     Prefer this over 'write_file' whenever you're modifying an existing
321     file:
322     it's safer (the rest of the file can't drift) and far cheaper in
323     tokens.
324
325     Args:
326         path: Path to the file to edit.
327         old_string: Exact text to find. Must appear exactly once.
328         new_string: Replacement text (empty string deletes).
329     """
330     try:
331         p = Path(path).expanduser().resolve()
332         if not p.exists():
333             return f"File not found: {path}"
334         if not p.is_file():
335             return f"Not a file: {path}"
336         if old_string == new_string:
337             return "Error: old_string and new_string are identical."
338         if not old_string:
339             return "Error: old_string is empty. Use write_file to create a
340             new file."
341
342         text = p.read_text(errors="replace")

```

```

331     count = text.count(old_string)
332     if count == 0:
333         return (
334             f"Error: old_string not found in {path}. "
335             "Check whitespace/indentation, or use read_file to re-
inspect."
336         )
337     if count > 1:
338         return (
339             f"Error: old_string matched {count} times in {path}. "
340             "Add surrounding context to make it unique."
341         )
342     new_text = text.replace(old_string, new_string, 1)
343     p.write_text(new_text)
344     delta = len(new_text) - len(text)
345     sign = "+" if delta >= 0 else "-"
346     return f"Edited {path} ({sign}{delta} chars)"
347 except PermissionError:
348     return f"Permission denied: {path}"
349 except Exception as e:
350     return f"Error editing file: {e}"
351
352
353
354 @tool
355 def list_directory(path: str = ".", show_hidden: bool = False) -> str:
356     """List files and directories at the given path.
357
358     Args:
359         path: Directory path to list (default: current directory).
360         show_hidden: Whether to include hidden files (default: false).
361     """
362     try:
363         p = Path(path).expanduser().resolve()
364         if not p.exists():
365             return f"Directory not found: {path}"
366         if not p.is_dir():
367             return f"Not a directory: {path}"
368
369         entries = sorted(p.iterdir(), key=lambda e: (not e.is_dir(), e.
name.lower()))
370         lines = [f"Contents of {p}:", ""]
371         for entry in entries:
372             if not show_hidden and entry.name.startswith("."):
373                 continue
374             prefix = "[dir] " if entry.is_dir() else "[doc] "
375             size = ""
376             if entry.is_file():
377                 s = entry.stat().st_size
378                 if s < 1024:
379                     size = f"({s}B)"
380                 elif s < 1_048_576:
381                     size = f"({s / 1024:.1f}KB)"
382                 else:
383                     size = f"({s / 1_048_576:.1f}MB)"
384             lines.append(f" {prefix}{entry.name}{size}")
385
386         if len(lines) == 2:
387             lines.append(" (empty)")
388         return "\n".join(lines)
389     except PermissionError:
390         return f"Permission denied: {path}"
391     except Exception as e:
392         return f"Error listing directory: {e}"
393
394 def _format_run_output(
395     stdout: str,
396     stderr: str,
397     exit_code: int,
398     timed_out: bool,
399     timeout: int,
400 ) -> str:
401     if timed_out:
402         return f"Command timed out after {timeout}s"
403     parts: list[str] = []
404     if stdout:
405         parts.append(stdout)
406     if stderr:
407         parts.append(f"[stderr]\n[stderr]")
408     if exit_code != 0:
409         parts.append(f"[exit code: {exit_code}]")
410     output = "\n".join(parts).strip()
411     if len(output) > 10_000:
412         output = output[:10_000] + "\n... (truncated)"
413     return output if output else "(no output)"
414
415
416 def _run_on_host(command: str, timeout: int) -> str:
417     try:
418         r = subprocess.run(
419             command,
420             shell=True,
421             capture_output=True,
422             text=True,
423             timeout=timeout,
424         )
425         return _format_run_output(
426             r.stdout or "",
427             r.stderr or "",
428             r.returncode,
429             timed_out=False,
430             timeout=timeout,
431         )
432     except subprocess.TimeoutExpired:
433         return f"Command timed out after {timeout}s"
434     except Exception as e:
435         return f"Error running command: {e}"
436
437
438 @tool
439 def run_command(command: str, timeout: int = 30) -> str:
440     """Execute a shell command and return its output.
441
442     When sandboxing is enabled, runs inside the per-session Docker
443     container at
444     /workspace (which is bind-mounted from your host cwd). Filesystem
445     writes
446     inside /workspace are visible on the host immediately. Writes outside
447     /workspace are ephemeral.
448
449     If you genuinely need host-only access (paths outside cwd, host-only
450     binaries), use 'run_command_host' instead - it requires explicit user
451     approval per call.
452
453     **Keep outputs small.** Output is truncated at 10k chars, which loses
454     the
455     tail. If you expect a large result, narrow it at the source rather
456     than
457     relying on truncation:
458     - 'grep -n PATTERN file' instead of dumping the file
459     - 'head -n 50' / 'tail -n 50' to bound size
460     - 'wc -l file' to count before reading
461     - 'find ... | head' instead of unbounded 'find'
462     - 'ls -la dir | head -n 30' for big directories
463     - pipe through 'awk' / 'sed' / 'jq' to extract just the fields you
464     need
465
466     For repetitive or one-off data work, drive Python from the shell:
467     'python3 -c '...' for short snippets, or 'python3 path/to/script.py'
468     for
469     longer logic. Same truncation rules apply - print only what you need.
470
471     Args:
472         command: The shell command to execute.
473         timeout: Maximum execution time in seconds (default 30).
474     """
475     sandbox = _ctx().get("sandbox")
476     if sandbox is not None:
477         result = sandbox.run(command, timeout=timeout)
478         return _format_run_output(
479             result.stdout,
480             result.stderr,
481             result.exit_code,
482             timed_out=result.timed_out,
483             timeout=timeout,
484         )
485     return _run_on_host(command, timeout)
486
487 @tool
488 def run_command_host(command: str, timeout: int = 30) -> str:
489     """Execute a shell command on the HOST (outside the sandbox).
490
491     Use only when you need access to files or tools that aren't available
492     inside the sandbox container - for example, reading a sibling repo, or
493     invoking a host-only binary. The user is prompted for permission per
494     call; expect denials.
495
496     Args:
497         command: The shell command to execute on the host.
498         timeout: Maximum execution time in seconds (default 30).
499     """
500     return _run_on_host(command, timeout)
501
502
503 @tool
504 async def spawn_subagent(
505     subagent_type: str,
506     prompt: str,
507     timeout_s: int | None = None,
508     tool_call_id: Annotated[str, InjectedToolCallId] = "",
509 ) -> str:
510     """Spawn a specialised subagent to handle a focused sub-task in
511     isolation.
512
513     The subagent runs its own tool-calling loop with a filtered toolset
514     and a
515     dedicated system prompt, then returns a single final report. You only
516     see
517     that report - the subagent's intermediate steps are not surfaced. Use
518     this to parallelise independent work or to delegate a deep dive
519     without
520     polluting your own context.
521
522     Pick a 'subagent_type' that matches the work:
523     - 'general' - open-ended research / multi-step tasks (full
524     toolset)
525     - 'code_search' - read-only code lookups (paths, symbols, snippets)
526     - 'web_research' - web search + synthesis (no file/shell access)
527
528     Call this tool multiple times in one assistant turn to spawn subagents
529     in parallel - they run independently.
530
531     Args:
532         subagent_type: One of the registered types above.
533         prompt: The task description for the subagent. Be specific: it has
534         none of your conversation context.
535         timeout_s: Optional hard timeout in seconds. Defaults to the type's
536         s
537         built-in default; capped by config.
538     """
539     from src.subagents import run_subagent
540
541     parent_ctx = _ctx()
542     result = await run_subagent(
543         subagent_type=(subagent_type or "").strip(),
544         prompt=(prompt or "").strip(),
545         parent_ctx=parent_ctx,
546         timeout_s=timeout_s,
547         parent_tool_call_id=tool_call_id or None,
548     )
549     if tool_call_id:
550         parent_ctx.setdefault("pending_subagent_results", {})[tool_call_id]
551         = {
552             "subagent_type": (subagent_type or "").strip(),
553             "prompt": (prompt or "").strip(),
554             "text": result.text,
555             "iterations": result.iterations,
556             "tool_call_count": result.tool_call_count,
557             "timed_out": result.timed_out,
558             "error": result.error,
559             "duration_s": result.duration_s,
560             "usage": result.usage,
561         }
562     return result.text
563
564
565 _PDF_PAGE_RANGE_CAP = 20
566
567 @tool
568 def read_pdf_pages(
569     name: str,

```



```

556 start: int,
557 end: int | None = None,
558 as_image: bool = False,
559 ) -> str:
560     """Read specific pages from a previously-attached PDF.
561
562     At attach time you got a *manifest* of the PDF (page count, TOC,
563     short preview per page). Use it to pick a narrow range, then call
564     this tool to read the content.
565
566     Args:
567         name: Filename of the PDF as it appeared in the manifest (e.g. "
568             report.pdf").
569         start: First page (1-indexed, inclusive).
570         end: Last page (1-indexed, inclusive). Defaults to 'start'.
571             Max 20 pages per call regardless of 'as_image'.
572         as_image: When True, render each page to a PNG. The rendered
573             images are delivered to you in the NEXT message - this tool
574             call returns only a short confirmation, then a synthesized
575             user-style message follows with the actual image blocks.
576             Only works on image-capable models; refused otherwise.
577             Use for figures, diagrams, tables, scanned content. Use
578             'as_image=False' (default) for plain text fidelity, which
579             is cheaper.
580
581     """
582     from src import attachments as _atts
583
584     ctx = _ctx()
585     registry = ctx.get("session_attachments") or {}
586     att = registry.get(name)
587     if att is None:
588         for key, val in registry.items():
589             if Path(key).name == Path(name).name:
590                 att = val
591                 break
592     if att is None:
593         attached = ", ".join(sorted(registry.keys())) or "(none)"
594         return (
595             f"Error: no PDF named {name!r} is attached to this session. "
596             f"Currently attached: {attached}"
597         )
598     if getattr(att, "kind", None) != "pdf":
599         return f"Error: attachment {name!r} is not a PDF (kind={att.kind})"
600
601     page_count = getattr(att, "pdf_page_count", None) or 0
602     if page_count <= 0:
603         return f"Error: attachment {name!r} has no readable page count."
604
605     if end is None:
606         end = start
607     if start < 1 or end < 1 or start > page_count or end > page_count or
608         start > end:
609         return (
610             f"Error: page range [{start}, {end}] is invalid "
611             f"('{name!r}' has {page_count} pages, 1-indexed)."
612         )
613     if end - start + 1 > _PDF_PAGE_RANGE_CAP:
614         return (
615             f"Error: range too wide ({end - start + 1} pages). "
616             f"Limit is {_PDF_PAGE_RANGE_CAP} pages per call - narrow the
617             range "
618             "and call again."
619         )
620
621     session_id = ctx.get("session").id if ctx.get("session") else ""
622     if not session_id:
623         return "Error: no active session - cannot fetch PDF bytes."
624
625     data = _atts.fetch_bytes(session_id, att.sha)
626     if data is None:
627         return f"Error: PDF bytes for {name!r} are no longer in the
628             artifact store."
629
630     if as_image:
631         provider = ctx.get("provider") or ""
632         model = ctx.get("model") or ""
633         if not model:
634             try:
635                 from src.main import provider_configs as _pc
636
637                 model = _pc.get(provider, {}).get("model", "") or ""
638             except Exception:
639                 pass
640         try:
641             from src import capabilities as _caps
642
643             if model and not _caps.get_capabilities(provider, model):
644                 supports("image"):
645                     return (
646                         f"Error: as_image=True needs an image-capable model,
647                         but "
648                         f"{model} is recorded as not supporting image input.
649                         "Use as_image=False to get text instead."
650                     )
651             except Exception:
652                 pass
653         try:
654             shas = _render_pdf_pages_for_injection(
655                 data,
656                 start=start,
657                 end=end,
658                 session_id=session_id,
659                 dpi=int(
660                     (ctx.get("multimodal_config") or {})
661                     .get("pdf", {})
662                     .get("render_dpi", 144)
663                 ),
664             )
665         except Exception as exc:
666             return f"Error rendering PDF pages: {exc}"
667
668     try:
669         from src.multimodal import build_image_blocks_from_shas
670
671         image_blocks = build_image_blocks_from_shas(
672             shas, provider=provider, session_id=session_id, mime_type=
673             "image/png"
674         )
675     except Exception as exc:
676         return f"Error encoding rendered pages: {exc}"
677
678     ctx.setdefault("pending_image_injections", []).append(
679         {
680             "source": "read_pdf_pages",
681             "pdf_name": att.name,
682             "page_start": start,
683             "page_end": end,
684             "page_shas": shas,
685             "image_blocks": image_blocks,
686         }
687     )
688     return (
689         f"Rendered pages {start}-{end} of {att.name} ({len(shas)}
690         image"
691         f"{'s' if len(shas) != 1 else ''}). Images delivered in the
692         next "
693         "message."
694     )
695
696 try:
697     from pypdf import PdfReader # type: ignore[import-not-found]
698 except ImportError:
699     return (
700         "Error: text extraction needs 'pypdf' (base dep - refresh with
701         "
702         "'make setup' or 'uv pip install pypdf')."
703     )
704
705 import io
706
707 reader = PdfReader(io.BytesIO(data))
708 out: list[str] = [f"# {name} - pages {start}-{end} of {page_count}"]
709 for i in range(start, end + 1):
710     try:
711         txt = reader.pages[i - 1].extract_text() or ""
712     except Exception as exc:
713         txt = f"(failed to extract: {exc})"
714     out.append(f"\n--- page {i} ---\n{txt}")
715 return "\n".join(out)
716
717 def _render_pdf_pages_for_injection(
718     data: bytes,
719     *,
720     start: int,
721     end: int,
722     session_id: str,
723     dpi: int,
724 ) -> list[str]:
725     try:
726         import fitz
727     except ImportError as exc:
728         raise RuntimeError(
729             "PyMuPDF ('pymupdf') is needed for as_image=True. "
730             "Install with: uv pip install 'vibe-cli[multimodal]'"
731         ) from exc
732
733     from src.artifacts import ArtifactStore
734
735     store = ArtifactStore(session_id)
736
737     doc = fitz.open(stream=data, filetype="pdf")
738     try:
739         shas: list[str] = []
740         for i in range(start, end + 1):
741             page = doc.load_page(i - 1)
742             pix = page.get_pixmap(dpi=dpi)
743             png = pix.tobytes("png")
744             ref = store.put(png, kind="attachment.image", ext="png")
745             shas.append(ref.sha256)
746     finally:
747         doc.close()
748     return shas
749
750 ALL_TOOLS = [
751     web_search, read_file, write_file,
752     edit_file, list_directory, run_command, run_skill,
753     remember, spawn_subagent, read_pdf_pages,
754 ]

```

## Anexa 20: src/tracing.py

```

1 from __future__ import annotations
2 import os
3 import sys
4 from src.main import config
5 from ui import preferences
6 _active_backend: str | None = None
7 _last_resolved: dict = {}
8
9 LANGFUSE_DEFAULT_HOST = "https://cloud.langfuse.com"
10
11 def resolve_tracing_settings() -> dict:
12     cfg = config.get("tracing") or {}
13     prefs = preferences.get_tracing()
14
15     def pick(field: str, default: str = "") -> str:
16         v = prefs.get(field)

```

```

17     if v not in (None, ""):
18         return str(v)
19     v = cfg.get(field)
20     if v not in (None, ""):
21         return str(v)
22     return default
23
24     pref_enabled = prefs.get("enabled")
25     cfg_enabled = bool(cfg.get("enabled", False))
26     if pref_enabled is None:
27         enabled = cfg_enabled
28     else:
29         enabled = bool(pref_enabled)
30
31     return {
32         "enabled": enabled,
33         "project": pick("project"),
34         "public_key": pick("public_key") or os.environ.get("
35             LANGFUSE_PUBLIC_KEY", ""),
36         "secret_key": pick("secret_key") or os.environ.get("
37             LANGFUSE_SECRET_KEY", ""),
38         "host": pick("host") or os.environ.get("LANGFUSE_HOST", "")
39         or LANGFUSE_DEFAULT_HOST,
40     }
41
42 def setup_tracing() -> dict:
43     cfg = config.get("tracing" or {})
44     legacy_backend = (cfg.get("backend") or "").lower()
45     if legacy_backend and legacy_backend != "langfuse":
46         print(
47             f"[tracing] ignoring legacy 'backend: {legacy_backend}' - only
48             'LangFuse is supported. Remove the field from config.yml.'",
49             file=sys.stderr,
50         )
51
52     settings = resolve_tracing_settings()
53
54     global _last_resolved
55     _last_resolved = settings
56
57     if not settings["enabled"]:
58         return {}
59
60     return _setup_langfuse(settings)
61
62 def _setup_langfuse(settings: dict) -> dict:
63     public_key = settings.get("public_key") or ""
64     secret_key = settings.get("secret_key") or ""
65     host = settings.get("host") or LANGFUSE_DEFAULT_HOST
66
67     if not public_key or not secret_key:
68         print(
69             "[tracing] LangFuse public_key/secret_key missing - run "
70             "'uv pip install 'vibe-cli[tracing]'"
71             "'\nOriginal error: {e}'",
72             file=sys.stderr,
73         )
74     return {}
75
76     os.environ["LANGFUSE_PUBLIC_KEY"] = public_key
77     os.environ["LANGFUSE_SECRET_KEY"] = secret_key
78     os.environ["LANGFUSE_HOST"] = host
79
80     try:
81         from langfuse.langchain import CallbackHandler as LangfuseHandler
82     except ImportError as e:
83         print(
84             "[tracing] LangFuse LangChain integration failed to import. "
85             "Install the tracing extras with "
86             "'uv pip install 'vibe-cli[tracing]'"
87             "'\nOriginal error: {e}'",
88             file=sys.stderr,
89         )
90     return {}
91
92     handler = LangfuseHandler()
93     except Exception as e:
94         print(f"[tracing] failed to construct LangFuse handler: {e}", file=
95             sys.stderr)
96         return {}
97
98     global _active_backend
99     _active_backend = "langfuse"
100     return {"config": {"callbacks": [handler]}}
101
102 def get_tracing_info() -> str | None:
103     if _active_backend is None:
104         return None
105
106     project = _last_resolved.get("project") or ""
107     if project:
108         return f"{{_active_backend}}:{{project}}"
109     return _active_backend
110
111 def last_resolved_settings() -> dict:
112     """Return the last-resolved tracing settings (for '/tracing' status)."""
113     return dict(_last_resolved)
114
115 def _get_client():
116     if _active_backend != "langfuse":
117         return None
118     try:
119         from langfuse import get_client
120     except ImportError:
121         return None
122     try:
123         return get_client()
124     except Exception:
125         return None
126
127 def post_score(
128     trace_id: str,
129     name: str,
130     value: float,
131     comment: str | None = None,
132 ) -> tuple[bool, str]:
133     if not trace_id:
134         return False, "no recent trace to score (send a message first)"
135
136     client = _get_client()
137     if client is None:
138         return False, "tracing is off - enable LangFuse in config.yml"
139
140     try:
141         client.create_score(
142             trace_id=trace_id,
143             name=name,
144             value=value,
145             comment=comment,
146         )
147     except Exception:
148         pass
149     return True, f"scored: {name}={value}" + (f" - {comment}" if
150         comment else "")
151     except Exception as e:
152         return False, f"could not post score: {e}"
153
154 def add_dataset_item(
155     dataset_name: str,
156     input_payload: dict,
157     expected_output: dict | None = None,
158     metadata: dict | None = None,
159 ) -> tuple[bool, str]:
160     client = _get_client()
161     if client is None:
162         return False, "tracing is off - enable LangFuse in config.yml"
163
164     try:
165         client.create_dataset(name=dataset_name)
166         client.create_dataset_item(
167             dataset_name=dataset_name,
168             input=input_payload,
169             expected_output=expected_output,
170             metadata=metadata or {},
171         )
172     except Exception:
173         pass
174     return True, f"added to dataset: {dataset_name}"
175     except Exception as e:
176         return False, f"could not add to dataset: {e}"
177
178 def list_datasets() -> tuple[bool, list[dict] | str]:
179     client = _get_client()
180     if client is None:
181         return False, "tracing is off - enable LangFuse in config.yml"
182
183     try:
184         page = client.api.datasets.list()
185     except Exception as e:
186         return False, f"could not list datasets: {e}"
187
188     items = []
189     for ds in getattr(page, "data", []) or []:
190         items.append(
191             {
192                 "name": getattr(ds, "name", ""),
193                 "description": getattr(ds, "description", "") or "",
194                 "item_count": getattr(ds, "item_count", None),
195             }
196         )
197     return True, items

```

## Anexa 21: ui/skills.py

```

1 from __future__ import annotations
2 import os
3 import re
4 import shutil
5 from dataclasses import dataclass, field
6 from pathlib import Path
7 from typing import Iterable
8 from ui.paths import SKILLS_DIR, project_skills_dir
9
10 DEFAULT_MAX_ACTIVE = 10
11
12 FRONTMATTER_RE = re.compile(
13     r"\A---\s*\n(?:<frontmatter>.*?)\n---\s*\n(?:<body>.*?)\Z",
14     re.DOTALL,
15 )
16
17 @dataclass
18 class Skill:
19     path: Path
20     name: str
21     description: str
22     when_to_use: str
23     enabled: bool
24     body: str
25     scope: str = "global"
26
27 @property
28 def is_active(self) -> bool:
29     return self.enabled

```



```

31
32 def _parse_frontmatter_block(block: str) -> dict[str, str | bool]:
33     out: dict[str, str | bool] = {}
34     for raw in block.splitlines():
35         line = raw.strip()
36         if not line or line.lstrip().startswith("#"):
37             continue
38         if ":" not in line:
39             continue
40         key, _, value = line.partition(":")
41         key = key.strip()
42         value = value.strip()
43         if (value.startswith('"') and value.endswith('"')) or (
44             value.startswith("'") and value.endswith("'"))
45         ):
46             value = value[1:-1]
47         if key == "enabled":
48             out[key] = value.lower() in ("true", "yes", "on", "1")
49         else:
50             out[key] = value
51     return out
52
53
54 def _parse_skill_file(path: Path) -> Skill | None:
55     try:
56         text = path.read_text(encoding="utf-8")
57     except OSError:
58         return None
59
60     m = _FRONTMATTER_RE.match(text)
61     if not m:
62         return None
63     if not m:
64         return None
65     return Skill(
66         path=path,
67         name=path.stem,
68         description="",
69         when_to_use="",
70         enabled=False,
71         body=m.group("body"),
72     )
73
74     fm = _parse_frontmatter_block(m.group("frontmatter"))
75     body = m.group("body").strip()
76
77     name = path.stem
78     return Skill(
79         path=path,
80         name=name,
81         description=str(fm.get("description") or ""),
82         when_to_use=str(fm.get("when_to_use") or ""),
83         enabled=bool(fm.get("enabled", False)),
84         body=body,
85     )
86
87 def _serialize_skill(skill: Skill) -> str:
88     fm_lines = [
89         "----",
90         f"name: {skill.name}",
91         f"description: {skill.description}",
92     ]
93     if skill.when_to_use:
94         fm_lines.append(f"when_to_use: {skill.when_to_use}")
95     fm_lines.append(f"enabled: {'true' if skill.enabled else 'false'}")
96     fm_lines.append("----")
97     return (
98         "\n".join(fm_lines) + "\n" + (skill.body.rstrip() + "\n" if skill.body else "")
99     )
100
101 def _atomic_write(path: Path, text: str) -> None:
102     path.parent.mkdir(parents=True, exist_ok=True)
103     tmp = path.with_suffix(path.suffix + ".tmp")
104     tmp.write_text(text, encoding="utf-8")
105     os.replace(tmp, path)
106
107 @dataclass
108 class SkillsIndex:
109
110     active: list[Skill] = field(default_factory=list)
111     dropped: list[Skill] = field(default_factory=list)
112     disabled: list[Skill] = field(default_factory=list)
113     max_active: int = DEFAULT_MAX_ACTIVE
114
115     @property
116     def all(self) -> list[Skill]:
117         return sorted(
118             self.active + self.dropped + self.disabled,
119             key=lambda s: s.name,
120         )
121
122     @property
123     def by_name(self) -> dict[str, Skill]:
124         return {s.name: s for s in self.all}
125
126     def get(self, name: str) -> Skill | None:
127         return self.by_name.get(name)
128
129     def is_visible_to_model(self, name: str) -> bool:
130         return any(s.name == name for s in self.active)
131
132 def _load_from_dir(directory: Path, scope: str) -> list[Skill]:
133     if not directory.is_dir():
134         return []
135     parsed: list[Skill] = []
136     for entry in sorted(directory.iterdir(), key=lambda p: p.name.lower()):
137         if not entry.is_file() or entry.suffix.lower() != ".md":
138             continue
139         skill = _parse_skill_file(entry)
140         if skill is not None:
141             skill.scope = scope
142             parsed.append(skill)
143     return parsed
144
145 def load_skills(
146     max_active: int = DEFAULT_MAX_ACTIVE,
147     *,
148     cwd: Path | str | None = None,
149 ) -> SkillsIndex:
150     from src.logging_setup import emit_event, get_logger
151
152     SKILLS_DIR.mkdir(parents=True, exist_ok=True)
153     global_skills = _load_from_dir(SKILLS_DIR, scope="global")
154     project_skills = _load_from_dir(project_skills_dir(cwd), scope="project")
155
156     merged: dict[str, Skill] = {s.name: s for s in global_skills}
157     for s in project_skills:
158         merged[s.name] = s
159     parsed = list(merged.values())
160
161     enabled = sorted([s for s in parsed if s.enabled], key=lambda s: s.name)
162     disabled = sorted([s for s in parsed if not s.enabled], key=lambda s: s.name)
163
164     if max_active is None or max_active < 0:
165         max_active = DEFAULT_MAX_ACTIVE
166
167     active = enabled[:max_active]
168     dropped = enabled[max_active:]
169
170     emit_event(
171         get_logger("skill"),
172         "skills.loaded",
173         active=len(active),
174         dropped=len(dropped),
175         disabled=len(disabled),
176         max_active=max_active,
177         project_skills=len(project_skills),
178         global_skills=len(global_skills),
179     )
180
181     return SkillsIndex(
182         active=active,
183         dropped=dropped,
184         disabled=disabled,
185         max_active=max_active,
186     )
187
188 def set_enabled(
189     name: str,
190     enabled: bool,
191     *,
192     cwd: Path | str | None = None,
193 ) -> Skill | None:
194     from src.logging_setup import emit_event, get_logger
195
196     project_path = project_skills_dir(cwd) / f"{name}.md"
197     global_path = SKILLS_DIR / f"{name}.md"
198     if project_path.is_file():
199         path = project_path
200         scope = "project"
201     elif global_path.is_file():
202         path = global_path
203         scope = "global"
204     else:
205         return None
206
207     skill = _parse_skill_file(path)
208     if skill is None:
209         return None
210     skill.scope = scope
211     if skill.enabled == enabled:
212         return skill
213     skill.enabled = enabled
214     _atomic_write(path, _serialize_skill(skill))
215     emit_event(
216         get_logger("skill"),
217         "skill.toggle",
218         skill=name,
219         enabled=enabled,
220         scope=scope,
221     )
222     return skill
223
224 def seed_examples_if_empty(source_dir: Path) -> int:
225     SKILLS_DIR.mkdir(parents=True, exist_ok=True)
226     existing = [p for p in SKILLS_DIR.iterdir() if p.suffix.lower() == ".md"]
227     if existing:
228         return 0
229     if not source_dir.is_dir():
230         return 0
231     count = 0
232     for src in source_dir.iterdir():
233         if src.is_file() and src.suffix.lower() == ".md":
234             shutil.copy2(src, SKILLS_DIR / src.name)
235             count += 1
236     return count
237
238 def render_skills_directory(skills: Iterable[Skill]) -> str:
239     items = list(skills)
240     if not items:
241         return ""
242     lines = [
243         "You have these skills available. To use one, call the 'run_skill'",
244         "tool with its name; the tool will return the full instructions",
245         "for",
246         "the skill. Calling 'run_skill' is a SETUP step - after the tool",
247         "returns, you MUST write a user-facing reply that applies those",
248         "instructions to the user's request. Do not stop after the tool",
249         "call.",
250         "",
251     ]
252     for s in items:
253         lines.append(f"- {s.name} - {s.description or '(no description)'}")
254     if s.when_to_use:
255         lines.append(f"  When to use: {s.when_to_use}")
256     return "\n".join(lines)

```

## Anexa 22: ui/vibe\_md.py

```
1 from __future__ import annotations
2 from pathlib import Path
3 from ui.paths import GLOBAL_VIBE_MD, project_vibe_dir, project_vibe_md
4 from ui.skills import _atomic_write
5
6 DEFAULT_MAX_BYTES = 8192
7
8 def _read(path: Path) -> str:
9     try:
10         return path.read_text(encoding="utf-8").strip()
11     except OSError:
12         return ""
13
14 def read_global_vibe() -> str:
15     return _read(GLOBAL_VIBE_MD)
16
17 def read_project_vibe(cwd: Path | str | None = None) -> str:
18     return _read(project_vibe_md(cwd))
19
20 def render_vibe_block(global_text: str, project_text: str) -> str:
21     sections: list[str] = []
22     if global_text:
23         sections.append(
24             "# Persistent preferences (~/.vibe-cli/VIBE.md)\n"
25             "Apply these throughout the conversation - they capture how
26             the "
27             "user prefers to work across every project.\n\n"
28             f"{global_text}"
29         )
30     if project_text:
31         sections.append(
32             "# Project notes (./.vibe/VIBE.md)\n"
33             "Project-specific facts and conventions for THIS repo. Treat
34             as "
35             "ground truth alongside the codebase itself.\n\n"
36             f"{project_text}"
37         )
38     return "\n\n".join(sections)
39
40 class VibeWriteError(Exception):
41     ...
42
43 def _append(path: Path, content: str, *, max_bytes: int) -> int:
44     body = content.strip()
45     if not body:
46         raise VibeWriteError("content is empty")
47
48     existing = _read(path)
49     if existing:
50         new_text = f"{existing}\n\n- {body}\n"
51     else:
52         new_text = f"- {body}\n"
53
54     encoded = new_text.encode("utf-8")
55     if len(encoded) > max_bytes:
56         raise VibeWriteError(
57             f"file would exceed cap ({len(encoded)} > {max_bytes} bytes);
58             "
59             "consolidate existing entries first"
60         )
61     path.parent.mkdir(parents=True, exist_ok=True)
62     _atomic_write(path, new_text)
63     return len(body.encode("utf-8"))
64
65 def append_global(content: str, *, max_bytes: int = DEFAULT_MAX_BYTES) ->
66     int:
67     return _append(GLOBAL_VIBE_MD, content, max_bytes=max_bytes)
68
69 def append_project(
70     content: str,
71     *,
72     cwd: Path | str | None = None,
73     max_bytes: int = DEFAULT_MAX_BYTES,
74 ) -> int:
75     project_vibe_dir(cwd).mkdir(parents=True, exist_ok=True)
76     return _append(project_vibe_md(cwd), content, max_bytes=max_bytes)
```